```
# 1. Theta exact equality

theta_equal = np.array_equal(state_preserved.theta, state_zero.theta)

results["theta_equal"] = theta_equal



# 2. Coherence_acc exact equality and both zero

coh_p_zero = np.array_equal(state_preserved.coherence_acc,
np.zeros_like(state_preserved.coherence_acc))

coh_z_zero = np.array_equal(state_zero.coherence_acc, np.zeros_like(state_zero.coherence_acc))

coh_equal = np.array_equal(state_preserved.coherence_acc, state_zero.coherence_acc)

results["coherence_acc_preserved_is_zero"] = coh_p_zero

results["coherence_acc_zero_is_zero"] = coh_z_zero

results["coherence_acc_equal"] = coh_equal



# 3. B checks - preserved equals split B, zero is exactly zero matrix

B_preserved_equal_split = np.array_equal(state_preserved.B, B_at_split)

B_zero_is_zero = np.array_equal(state_zero.B, np.zeros((N, N)))

results["B_preserved_equal_split"] = B_preserved_equal_split
```

```
results["B_zero_is_zero"] = B_zero_is_zero



# 4. B properties

try:

validate_B_matrix(state_preserved.B, N)

results["B_preserved_valid"] = True

except ValueError:

results["B_preserved_valid"] = False



try:

validate_B_matrix(state_zero.B, N)

results["B_zero_valid"] = True

except ValueError:

results["B_zero_valid"] = False



# 5. History: length equal and every entry equals theta

history_len_equal = len(state_preserved.history) == len(state_zero.history)
```

```
results["history_length_equal"] = history_len_equal



history_equal = True

history_matches_theta = True

if history_len_equal:

for hp, hz in zip(state_preserved.history, state_zero.history):

if not np.array_equal(hp, hz):

history_equal = False

if not np.array_equal(hp, state_preserved.theta):

history_matches_theta = False

results["history_equal"] = history_equal

results["history_matches_theta"] = history_matches_theta

else:

results["history_equal"] = False

results["history_matches_theta"] = False



# 6. Array hashes for all matched arrays
```

```
results["theta_hash_preserved"] = array_sha256(state_preserved.theta)

results["theta_hash_zero"] = array_sha256(state_zero.theta)

results["B_hash_preserved"] = array_sha256(state_preserved.B)

results["B_hash_zero"] = array_sha256(state_zero.B)

results["coherence_hash_preserved"] = array_sha256(state_preserved.coherence_acc)

results["coherence_hash_zero"] = array_sha256(state_zero.coherence_acc)



# History hash - stack deque entries

if history_len_equal and len(state_preserved.history) &gt; 0:

hist_p_array = np.stack(tuple(state_preserved.history))

hist_z_array = np.stack(tuple(state_zero.history))

results["history_hash_preserved"] = array_sha256(hist_p_array)

results["history_hash_zero"] = array_sha256(hist_z_array)

else:

results["history_hash_preserved"] = None

results["history_hash_zero"] = None
```

```
# Overall pass - ALL checks must pass
all_checks = [
theta_equal,
coh_p_zero,
coh_z_zero,
coh_equal,
B_preserved_equal_split,
B_zero_is_zero,
results.get("B_preserved_valid", False),
results.get("B_zero_valid", False),
history_len_equal,
history_equal,
history_matches_theta,
rng_state_equal,
]
results["pass"] = all(all_checks)
```

return results

==
Frozen Anchor Validation (MANDATORY)
==

def validate_frozen_anchor(

text

new_result: dict[str, Any],

frozen_data: dict[str, np.ndarray],

seed: str,

branch_label: str,

matrix_keys: list[str],

) -> dict[str, Any]:

text

"""Validate new result against frozen data for anchor branch.

Uses EXACT equality (np.array_equal). Missing keys are fatal.

Returns dict with validation results.

"""

results: dict[str, Any] = {

"seed": seed,

"branch": branch_label,

"pass": True,

}

for key in matrix_keys:

frozen_key = f"{branch_label}_seed_{seed}_{key}"

if frozen_key not in frozen_data:

results[f"{key}_missing_in_frozen"] = True

results["pass"] = False

continue

if key not in new_result:

results[f"{key}_missing_in_new"] = True

results["pass"] = False

continue

```
frozen_arr = np.asarray(frozen_data[frozen_key])

new_arr = np.asarray(new_result[key])


# EXACT equality

exact_equal = np.array_equal(frozen_arr, new_arr)

results[f"{key}_exact"] = exact_equal


if not exact_equal:

# Record max diff for diagnostics

max_diff = float(np.max(np.abs(frozen_arr - new_arr)))

results[f"{key}_max_diff"] = max_diff

results["pass"] = False



return results

================================================================================
Model Constant Validation
================================================================================
```

```
def validate_model_constants(
text
model: dict[str, Any],
manifest_model: dict[str, Any],
) -> None:
text
"""Validate that model constants match frozen manifest. Raises ValueError on mismatch."""
# Get constants from both sources
frozen_constants = {
"N": int(manifest_model["N"]),
"dt": float(manifest_model["dt"]),
"T_window": float(manifest_model["T_window"]),
}



# Get from production module
prod_constants = {
"N": int(model["N"]),
```

"dt": float(model["dt"]),

"T_window": float(model["T_window"]),

}

Compare

for key in frozen_constants:

if frozen_constants[key] != prod_constants[key]:

raise ValueError(

f"Model constant mismatch: {key} frozen={frozen_constants[key]}, "

f"prod={prod_constants[key]}"

)

==========
Recovery Outcome Computation
==========

def compute_recovery_outcomes(

text

result: dict[str, Any],

continue_result: dict[str, Any] | None,

```
seed: str,

branch_id: str,

protocol: dict[str, Any],

) -> dict[str, Any]:

text

"""Compute recovery outcomes using validated helpers from recovery module.



Uses relative thresholds (90% of continue reference) not absolute values.

Uses np.maximum(0, ...) for deficit calculations.

Creates explicit binary recovery indicators.

"""

outcomes: dict[str, Any] = {}



# Copy all production scalars

for key in prod.BRANCH_SCALAR_KEYS:

if key in result:

outcomes[key] = result[key]
```

```
# Get reference values from continue branch

if continue_result is not None:

reference_final_density = continue_result.get("D_B_final", np.nan)

reference_late_similarity = continue_result.get("Weighted_similarity_late", np.nan)

window_time = float(protocol.get("window_steps", 0)) * float(protocol.get("dt", 0.01))

else:

reference_final_density = np.nan

reference_late_similarity = np.nan

window_time = 10.0



sample_times = result.get("sample_times")



# Similarity trajectory outcomes

if "similarity_trajectory" in result:

sim_traj = np.asarray(result["similarity_trajectory"], dtype=float)
```

```
# Handle NaN by mapping to 0 for recovery calculations only

sim_traj_clean = np.where(np.isnan(sim_traj), 0.0, sim_traj)


if sample_times is not None and RECOVERY_AVAILABLE:

# Use relative threshold: 90% of continue late similarity

if not np.isnan(reference_late_similarity):

similarity_threshold = 0.9 * reference_late_similarity

else:

similarity_threshold = 0.9 # Fallback only


# time_to_similarity_90

time_to_sim = recovery.time_to_at_least(

sample_times, sim_traj_clean, similarity_threshold

)

outcomes["time_to_similarity_90"] = time_to_sim


# sustained_similarity_90_rest

sustained_sim = recovery.first_time_sustained_for_rest(

sample_times, sim_traj_clean, similarity_threshold
```

```
)
outcomes["sustained_similarity_90_rest"] = sustained_sim

# held_similarity_90_for_1Twindow
held_sim = recovery.first_time_held_for_duration(
sample_times, sim_traj_clean, similarity_threshold, window_time
)
outcomes["held_similarity_90_for_1Twindow"] = held_sim

# Explicit binary recovery indicators
outcomes["sustained_similarity_90_recovered"] = bool(np.isfinite(sustained_sim))
outcomes["held_similarity_90_for_1Twindow_recovered"] = bool(np.isfinite(held_sim))

# AUC similarity deficit vs continue
if continue_result is not None and "similarity_trajectory" in continue_result:
cont_traj = np.asarray(continue_result["similarity_trajectory"], dtype=float)
cont_traj_clean = np.where(np.isnan(cont_traj), 0.0, cont_traj)

# Use np.maximum(0, ...) to avoid negative deficits
```

```
similarity_deficit = np.maximum(0.0, cont_traj_clean - sim_traj_clean)


outcomes["AUC_similarity_deficit_vs_continue"] = trapezoid_integral(

similarity_deficit, sample_times

)

outcomes["Mean_similarity_deficit_vs_continue"] = float(

np.mean(similarity_deficit)

)

else:

# Fallback if recovery module not available

outcomes["time_to_similarity_90"] = np.nan

outcomes["sustained_similarity_90_rest"] = np.nan

outcomes["held_similarity_90_for_1Twindow"] = np.nan

outcomes["sustained_similarity_90_recovered"] = False

outcomes["held_similarity_90_for_1Twindow_recovered"] = False

if continue_result is not None and "similarity_trajectory" in continue_result:

cont_traj = np.asarray(continue_result["similarity_trajectory"], dtype=float)
```

```
cont_traj_clean = np.where(np.isnan(cont_traj), 0.0, cont_traj)

similarity_deficit = np.maximum(0.0, cont_traj_clean - sim_traj_clean)

if sample_times is not None:

outcomes["AUC_similarity_deficit_vs_continue"] = trapezoid_integral(

similarity_deficit, sample_times

)

else:

outcomes["AUC_similarity_deficit_vs_continue"] = np.nan

outcomes["Mean_similarity_deficit_vs_continue"] = float(

np.mean(similarity_deficit)

)



# Density trajectory outcomes

if "density_trajectory" in result:

dens_traj = np.asarray(result["density_trajectory"], dtype=float)


if sample_times is not None and RECOVERY_AVAILABLE:

# Use relative threshold: 90% of continue final density
```

```
if not np.isnan(reference_final_density):

density_threshold = 0.9 * reference_final_density

else:

density_threshold = 0.9 # Fallback only


# time_to_density_90

time_to_dens = recovery.time_to_at_least(

sample_times, dens_traj, density_threshold

)

outcomes["time_to_density_90"] = time_to_dens


# sustained_density_90_rest

sustained_dens = recovery.first_time_sustained_for_rest(

sample_times, dens_traj, density_threshold

)

outcomes["sustained_density_90_rest"] = sustained_dens


# Explicit binary recovery indicator
```

```
outcomes["sustained_density_90_recovered"] = bool(np.isfinite(sustained_dens))


# AUC density deficit vs continue

if continue_result is not None and "density_trajectory" in continue_result:

cont_dens = np.asarray(continue_result["density_trajectory"], dtype=float)

density_deficit = np.maximum(0.0, cont_dens - dens_traj)


outcomes["AUC_density_deficit_vs_continue"] = trapezoid_integral(

density_deficit, sample_times

)

else:

outcomes["time_to_density_90"] = np.nan

outcomes["sustained_density_90_rest"] = np.nan

outcomes["sustained_density_90_recovered"] = False

if continue_result is not None and "density_trajectory" in continue_result:

cont_dens = np.asarray(continue_result["density_trajectory"], dtype=float)

density_deficit = np.maximum(0.0, cont_dens - dens_traj)

if sample_times is not None:
```

```
outcomes["AUC_density_deficit_vs_continue"] = trapezoid_integral(

density_deficit, sample_times

)

else:

outcomes["AUC_density_deficit_vs_continue"] = np.nan



return outcomes

# ============================================================================
# Contrast Computation
# ============================================================================

def compute_paired_contrasts(

text

outcomes: dict[str, dict[str, Any]],

) -> dict[str, dict[str, float]]:

text

"""Compute paired contrasts: Y(B=preserved) - Y(B=zero) for each reset source.



Returns: reset_source -&gt; outcome -&gt; contrast
```

Only computes contrasts for explicitly declared CONTRAST_OUTCOMES.

"""

contrasts: dict[str, dict[str, float]] = {}

reset_sources = ["carrier_reset", "full_reset"]

for reset_source in reset_sources:

contrasts[reset_source] = {}

preserved_key = f"{reset_source}__B_preserved"

zero_key = f"{reset_source}__B_zero"

if preserved_key not in outcomes or zero_key not in outcomes:

continue

pres = outcomes[preserved_key]

zero = outcomes[zero_key]

```
# Only compute for explicitly declared contrast outcomes

for key in CONTRAST_OUTCOMES:

if key in pres and key in zero:

pres_val = pres[key]

zero_val = zero[key]

# Only compute for numeric scalars

if isinstance(pres_val, (int, float, np.number)) and isinstance(zero_val, (int, float, np.number)):

try:

contrast = float(pres_val) - float(zero_val)

contrasts[reset_source][key] = contrast

except (TypeError, ValueError):

pass



return contrasts

def compute_seed_level_contrasts(

text

paired_contrasts: dict[str, dict[str, float]],
```

) -> dict[str, float]:

text

"""Compute seed-level average and interaction contrasts.

Δ̄_s(Y) = 0.5 * [Δ_{s,carrier}(Y) + Δ_{s,full}(Y)]

I_s(Y) = Δ_{s,carrier}(Y) - Δ_{s,full}(Y)

Returns: outcome -> avg or interaction value

"""

seed_contrasts: dict[str, float] = {}

all_outcomes = set()

for reset_source in paired_contrasts:

all_outcomes.update(paired_contrasts[reset_source].keys())

for outcome in all_outcomes:

carrier_val = paired_contrasts.get("carrier_reset", {}).get(outcome, np.nan)

```
full_val = paired_contrasts.get("full_reset", {}).get(outcome, np.nan)


# Seed-level average

if not np.isnan(carrier_val) and not np.isnan(full_val):

seed_contrasts[f"avg_{outcome}"] = 0.5 * (carrier_val + full_val)

else:

seed_contrasts[f"avg_{outcome}"] = np.nan


# Interaction

if not np.isnan(carrier_val) and not np.isnan(full_val):

seed_contrasts[f"interaction_{outcome}"] = carrier_val - full_val

else:

seed_contrasts[f"interaction_{outcome}"] = np.nan



return seed_contrasts

============================================================================
====
Main Analysis Function
============================================================================
====

def run_matched_phase_ablation(
```

```
text

npz_path: str,

manifest_path: str,

scalar_path: str,

output_dir: Path,

seed_filter: list[str] | None = None,

max_seeds: int | None = None,

bootstrap_repetitions: int = 1000,

bootstrap_seed: int = 42,

verify_anchors: bool = True,

overwrite: bool = False,

) -> dict[str, Any]:

text

"""Main analysis function."""



# FIX: Check for stale outputs

output_dir.mkdir(parents=True, exist_ok=True)
```

```
# Check for existing outputs

existing_outputs = [

"matched_phase_scalar_summary.csv",

"matched_phase_paired_contrasts.csv",

"matched_phase_seed_level_contrasts.csv",

"matched_phase_recovery_summary.csv",

"matched_initial_state_audit.csv",

"frozen_anchor_validation.csv",

"matched_phase_branch_diagnostics.npz",

"matched_phase_report.txt",

"analysis_manifest.json",

]



if not overwrite:

existing_files = [f for f in existing_outputs if (output_dir / f).exists()]

if existing_files:
```

```
raise RuntimeError(
f"Output directory {output_dir} contains existing files: {existing_files}. "
f"Use --overwrite to replace, or use an empty directory."
)


# --- Load manifest ---
with open(manifest_path, "r") as f:
manifest = json.load(f)


# Validate manifest structure - CORRECTED PATHS
if "protocol" not in manifest or "branches" not in manifest["protocol"]:
raise ValueError("Manifest missing protocol.branches")
if "randomization" not in manifest or "paired_seeds" not in manifest["randomization"]:
raise ValueError("Manifest missing randomization.paired_seeds")


model = manifest["model"]
protocol = manifest["protocol"]
```

```
# FIX: Validate model constants match production

validate_model_constants(model, manifest["model"])



# Get branch order from correct path

branch_order = tuple(manifest["protocol"]["branches"])



# Validate branch order matches production

if not hasattr(prod, "BRANCHES"):

raise ValueError("Production module missing BRANCHES attribute")

if branch_order != tuple(prod.BRANCHES):

raise ValueError(

f"Branch order mismatch: manifest={branch_order}, "

f"prod.BRANCHES={tuple(prod.BRANCHES)}"

)
```

```
# Get seeds from correct path
all_seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]
if not all_seeds:
raise ValueError("No seeds found in manifest")


# Apply filter
if seed_filter:
seeds = [s for s in all_seeds if s in seed_filter]
else:
seeds = all_seeds


# FIX: Reject empty seed filter
if not seeds:
raise ValueError("No seeds selected - check seed filter")


# FIX: Handle max_seeds=0
if max_seeds is not None:
```

```
if max_seeds == 0:

raise ValueError("--max-seeds must be positive or None")

seeds = seeds[:max_seeds]



# FIX: Validate bootstrap_repetitions &gt; 0 for production

if bootstrap_repetitions &lt;= 0:

raise ValueError("--bootstrap-repetitions must be positive")



# FIX: Recovery dependency is MANDATORY for production

if not RECOVERY_AVAILABLE:

raise RuntimeError(

f"Validated recovery dependency unavailable: {RECOVERY_ERROR}"

)



print(f"Processing {len(seeds)} seeds...")



# Get model constants
```

N = int(model["N"])

dt = float(model["dt"])

T_window = float(model["T_window"])

Get matrix keys for validation

matrix_keys = getattr(prod, "BRANCH_MATRIX_KEYS", [])

if not matrix_keys:

raise ValueError("prod.BRANCH_MATRIX_KEYS is empty or not defined")

--- Load frozen data ---

with np.load(npz_path, allow_pickle=False) as frozen_npz:

frozen_data = {key: frozen_npz[key] for key in frozen_npz.files}

Load scalar CSV for hash verification

scalar_sha256 = None

if os.path.exists(scalar_path):

scalar_sha256 = file_sha256(scalar_path)

```
# --- Initialize storage ---

all_results: dict[str, dict[str, Any]] = {}

all_contrasts: dict[str, dict[str, dict[str, float]]] = {}

all_seed_contrasts: dict[str, dict[str, float]] = {}

audit_rows: list[dict[str, Any]] = []

anchor_validation_rows: list[dict[str, Any]] = []

seed_status: dict[str, str] = {}



# Determine run scope

full_cohort = (seeds == all_seeds)



# --- Process each seed ---

for seed in seeds:

seed_str = str(seed)

print(f" Seed: {seed_str}")


try:
```

```
# --- Load split state and prefix ---

split_state, prefix = load_split_and_prefix(frozen_data, seed_str, N)

B_at_split = split_state.B.copy()


# FIX: Validate split state

validate_split_state(split_state, N, seed_str)


# FIX: Create snapshot for immutability check

split_snapshot = create_split_snapshot(split_state)


# --- Reconstruct ALL branch RNG seeds from manifest ---

branch_rng_seeds = reconstruct_branch_rng_seeds(seed, list(branch_order))


# Get seeds for our reset blocks

carrier_reset_seed = branch_rng_seeds.get("B_carrier_only")

if carrier_reset_seed is None:

raise ValueError(f"B_carrier_only not in branch_rng_seeds for seed {seed_str}")


full_reset_seed = branch_rng_seeds.get("full_reset")
```

```
if full_reset_seed is None:

raise ValueError(f"full_reset not in branch_rng_seeds for seed {seed_str}")


# FIX: Require continue RNG seed

if "continue" not in branch_rng_seeds:

raise RuntimeError(f"continue RNG seed missing for seed {seed_str}")

continue_seed = branch_rng_seeds["continue"]


# --- RESET BLOCK A: carrier_reset seed ---

print(f" Reset block A (carrier_reset, seed={carrier_reset_seed})")


# CORRECTED: Use actual production API

# carrier_reset__B_preserved: reproduces B_carrier_only

result_carrier_preserved = prod.simulate_future(

copy.deepcopy(split_state),

prefix,

"B_carrier_only",

np.random.default_rng(carrier_reset_seed),

)
```

```
# carrier_reset__B_zero: new, B=0, same reset seed
result_carrier_zero = prod.simulate_future(
copy.deepcopy(split_state),
prefix,
"full_reset",
np.random.default_rng(carrier_reset_seed),
)

# --- Initial matching audit for Block A ---
# FIX: Compare RNG states directly after intervention
state_preserved_A = copy.deepcopy(split_state)
state_zero_A = copy.deepcopy(split_state)

rng_audit_p_A = np.random.default_rng(carrier_reset_seed)
rng_audit_z_A = np.random.default_rng(carrier_reset_seed)

prod.apply_intervention(state_preserved_A, "B_carrier_only", rng_audit_p_A)
prod.apply_intervention(state_zero_A, "full_reset", rng_audit_z_A)
```

```
rng_state_equal_A = (rng_audit_p_A.bit_generator.state == rng_audit_z_A.bit_generator.state)

audit_A = audit_initial_matching(
state_preserved_A, state_zero_A, B_at_split, N,
carrier_reset_seed, "carrier_reset", rng_state_equal_A
)
audit_A["seed"] = seed_str
audit_rows.append(audit_A)

# MANDATORY: Stop on matching failure
if not audit_A["pass"]:
raise RuntimeError(
f"Initial matching FAILED for seed={seed_str}, "
f"block=carrier_reset. Check matched_initial_state_audit.csv"
)

# --- RESET BLOCK B: full_reset seed ---
print(f" Reset block B (full_reset, seed={full_reset_seed})")
```

```
# full_reset__B_preserved: new, B=Bsplit
result_full_preserved = prod.simulate_future(
copy.deepcopy(split_state),
prefix,
"B_carrier_only",
np.random.default_rng(full_reset_seed),
)

# full_reset__B_zero: reproduces full_reset
result_full_zero = prod.simulate_future(
copy.deepcopy(split_state),
prefix,
"full_reset",
np.random.default_rng(full_reset_seed),
)

# --- Initial matching audit for Block B ---
```

```
state_preserved_B = copy.deepcopy(split_state)

state_zero_B = copy.deepcopy(split_state)


rng_audit_p_B = np.random.default_rng(full_reset_seed)

rng_audit_z_B = np.random.default_rng(full_reset_seed)


prod.apply_intervention(state_preserved_B, "B_carrier_only", rng_audit_p_B)

prod.apply_intervention(state_zero_B, "full_reset", rng_audit_z_B)


rng_state_equal_B = (rng_audit_p_B.bit_generator.state == rng_audit_z_B.bit_generator.state)


audit_B = audit_initial_matching(

state_preserved_B, state_zero_B, B_at_split, N,

full_reset_seed, "full_reset", rng_state_equal_B

)

audit_B["seed"] = seed_str

audit_rows.append(audit_B)


# MANDATORY: Stop on matching failure

if not audit_B["pass"]:
```

```
raise RuntimeError(
f"Initial matching FAILED for seed={seed_str}, "
f"block=full_reset. Check matched_initial_state_audit.csv"
)

# --- CONTINUE branch ---
print(f" Continue branch (seed={continue_seed})")
result_continue = prod.simulate_future(
copy.deepcopy(split_state),
prefix,
"continue",
np.random.default_rng(continue_seed),
)

# --- FIX: Check split state immutability ---
immutability_ok = check_split_state_immutability(
split_state, split_snapshot, N
)
```

```
if not immutability_ok:
raise RuntimeError(
f"Split state was mutated for seed={seed_str}"
)

# --- Frozen anchor validation (MANDATORY) ---
if verify_anchors:
# Validate carrier_reset__B_preserved against frozen B_carrier_only
anchor_A = validate_frozen_anchor(
result_carrier_preserved, frozen_data, seed_str,
"B_carrier_only", matrix_keys
)
anchor_A["production_branch"] = "B_carrier_only"
anchor_A["matched_branch"] = "carrier_reset__B_preserved"
anchor_validation_rows.append(anchor_A)

# Validate full_reset__B_zero against frozen full_reset
anchor_B = validate_frozen_anchor(
```

```
result_full_zero, frozen_data, seed_str,
"full_reset", matrix_keys
)
anchor_B["production_branch"] = "full_reset"
anchor_B["matched_branch"] = "full_reset__B_zero"
anchor_validation_rows.append(anchor_B)

# Validate continue against frozen continue
anchor_C = validate_frozen_anchor(
result_continue, frozen_data, seed_str,
"continue", matrix_keys
)
anchor_C["production_branch"] = "continue"
anchor_C["matched_branch"] = "continue"
anchor_validation_rows.append(anchor_C)

# MANDATORY: Stop on any anchor failure
seed_anchors = [r for r in anchor_validation_rows if r["seed"] == seed_str]
```

```
if any(not r.get("pass", True) for r in seed_anchors):
raise RuntimeError(
f"Anchor validation FAILED for seed={seed_str}. "
f"Check frozen_anchor_validation.csv for details."
)

# --- FIX: Store raw branch results for NPZ serialization ---
branch_results = {
"carrier_reset__B_preserved": result_carrier_preserved,
"carrier_reset__B_zero": result_carrier_zero,
"full_reset__B_preserved": result_full_preserved,
"full_reset__B_zero": result_full_zero,
"continue": result_continue,
}

# --- Compute outcomes ---
outcomes: dict[str, dict[str, Any]] = {}
```

```
for branch_id in BRANCH_IDS:

result = branch_results[branch_id]

meta = BRANCH_META[branch_id]


# Compute recovery outcomes

if branch_id == "continue":

# FIX: For continue, use its own result as reference

branch_outcomes = compute_recovery_outcomes(

result, result, seed_str, branch_id, protocol

)

else:

branch_outcomes = compute_recovery_outcomes(

result, result_continue, seed_str, branch_id, protocol

)


# Add metadata

branch_outcomes.update({

"seed": seed_str,
```

```
"branch": branch_id,

"reset_source": meta["reset_source"],

"B_status": meta["B_status"],

"production_intervention_label": meta["production_intervention_label"],

"frozen_anchor_branch": meta["frozen_anchor_branch"],

})


# Add reset seed

if meta["reset_source"] == "carrier_reset":

branch_outcomes["reset_seed_uint64"] = carrier_reset_seed

elif meta["reset_source"] == "full_reset":

branch_outcomes["reset_seed_uint64"] = full_reset_seed

else:

branch_outcomes["reset_seed_uint64"] = continue_seed


outcomes[branch_id] = branch_outcomes


# --- Compute contrasts ---

paired = compute_paired_contrasts(outcomes)
```

```
seed_level = compute_seed_level_contrasts(paired)

# Commit seed results (only after all validation passes)
seed_results: dict[str, Any] = {
"branches": branch_results, # FIX: Store raw results
"outcomes": outcomes,
"paired_contrasts": paired,
"seed_level_contrasts": seed_level,
}

all_results[seed_str] = seed_results
all_contrasts[seed_str] = paired
all_seed_contrasts[seed_str] = seed_level

seed_status[seed_str] = "success"
print(f" Status: SUCCESS")

except Exception as e:
seed_status[seed_str] = f"failed: {str(e)}"
```

```
print(f" Status: FAILED - {e}")

import traceback

traceback.print_exc()

# FIX: Abort on ANY seed failure for production run

raise



# --- Check for failures ---

failed_seeds = [s for s, status in seed_status.items() if status != "success"]

initial_match_failed = any(not r.get("pass", False) for r in audit_rows)

anchor_failed = any(not r.get("pass", True) for r in anchor_validation_rows) if anchor_validation_rows
else False



if failed_seeds:

print(f"\nFAILED seeds: {failed_seeds}")

if initial_match_failed:

print(f"Initial matching FAILED for some cases")

if anchor_failed:
```

print(f"Anchor validation FAILED for some cases")

FIX: If there are any validation failures, do NOT produce scientific outputs

has_fatal_failure = initial_match_failed or anchor_failed or len(failed_seeds) > 0

if has_fatal_failure and len(seeds) > 0:

Write validation-only outputs

if audit_rows:

audit_df = pd.DataFrame(audit_rows)

atomic_write_csv(output_dir / "matched_initial_state_audit.csv", audit_df)

if anchor_validation_rows:

anchor_df = pd.DataFrame(anchor_validation_rows)

atomic_write_csv(output_dir / "frozen_anchor_validation.csv", anchor_df)

Write failure manifest

failure_manifest = {

"status": "FAILED",

"error": "Validation failures detected",

```
"initial_match_failed": initial_match_failed,

"anchor_failed": anchor_failed,

"failed_seeds": failed_seeds,

"seeds_requested": seeds,

"seed_status": seed_status,

"run_scope": "partial",

"full_cohort": False,

"freezeable": False,

}

atomic_write_json(output_dir / "analysis_manifest.json", failure_manifest)


raise RuntimeError(

"Matched-phase run failed validation; scientific outputs were not produced"

)



# --- Save outputs ---
```

```
# 1. Scalar summary - include ALL production scalars

scalar_rows = []

for seed_str, results in all_results.items():

for branch_id, branch_outcomes in results["outcomes"].items():

row = dict(branch_outcomes) # Includes all production scalars + derived

scalar_rows.append(row)



if scalar_rows:

scalar_df = pd.DataFrame(scalar_rows)

atomic_write_csv(output_dir / "matched_phase_scalar_summary.csv", scalar_df)

print(f" Saved: matched_phase_scalar_summary.csv")



# 2. Paired contrasts

contrast_rows = []

for seed_str, paired in all_contrasts.items():

for reset_source, outcomes in paired.items():

for outcome, contrast in outcomes.items():
```

```
contrast_rows.append({

"seed": seed_str,

"reset_source": reset_source,

"outcome": outcome,

"contrast": contrast,

})



if contrast_rows:

contrast_df = pd.DataFrame(contrast_rows)

atomic_write_csv(output_dir / "matched_phase_paired_contrasts.csv", contrast_df)

print(f" Saved: matched_phase_paired_contrasts.csv")



# 3. Seed-level contrasts

seed_contrast_rows = []

for seed_str, seed_level in all_seed_contrasts.items():

for outcome, value in seed_level.items():

seed_contrast_rows.append({
```

```
"seed": seed_str,
"outcome": outcome,
"value": value,
})

if seed_contrast_rows:
seed_contrast_df = pd.DataFrame(seed_contrast_rows)
atomic_write_csv(output_dir / "matched_phase_seed_level_contrasts.csv", seed_contrast_df)
print(f" Saved: matched_phase_seed_level_contrasts.csv")

# 4. Recovery summary
recovery_outcomes = [
"sustained_similarity_90_rest",
"sustained_density_90_rest",
"held_similarity_90_for_1Twindow",
"sustained_similarity_90_recovered",
"sustained_density_90_recovered",
```

```
"held_similarity_90_for_1Twindow_recovered",

"time_to_similarity_90",

"time_to_density_90",

"AUC_similarity_deficit_vs_continue",

"Mean_similarity_deficit_vs_continue",

"AUC_density_deficit_vs_continue",

] + list(prod.BRANCH_SCALAR_KEYS)



recovery_rows = []

for seed_str, results in all_results.items():

for branch_id, branch_outcomes in results["outcomes"].items():

row = {

"seed": seed_str,

"branch": branch_id,

"reset_source": branch_outcomes.get("reset_source"),

"B_status": branch_outcomes.get("B_status"),

}
```

```
for key in recovery_outcomes:

if key in branch_outcomes:

row[key] = branch_outcomes[key]

recovery_rows.append(row)



if recovery_rows:

recovery_df = pd.DataFrame(recovery_rows)

atomic_write_csv(output_dir / "matched_phase_recovery_summary.csv", recovery_df)

print(f" Saved: matched_phase_recovery_summary.csv")



# 5. Initial state audit

if audit_rows:

audit_df = pd.DataFrame(audit_rows)

atomic_write_csv(output_dir / "matched_initial_state_audit.csv", audit_df)

print(f" Saved: matched_initial_state_audit.csv")



# 6. Frozen anchor validation
```

```
if anchor_validation_rows:

anchor_df = pd.DataFrame(anchor_validation_rows)

atomic_write_csv(output_dir / "frozen_anchor_validation.csv", anchor_df)

print(f" Saved: frozen_anchor_validation.csv")



# 7. Branch diagnostics NPZ - FIX: Use stored branch_results

npz_output = {}

for seed_str, results in all_results.items():

for branch_id, branch_result in results["branches"].items():

if isinstance(branch_result, dict):

for key, value in branch_result.items():

if isinstance(value, np.ndarray):

npz_key = f"{branch_id}_seed_{seed_str}_{key}"

npz_output[npz_key] = value



if npz_output:

atomic_write_npz(output_dir / "matched_phase_branch_diagnostics.npz", **npz_output)
```

print(f" Saved: matched_phase_branch_diagnostics.npz")

--- Compute aggregate statistics ---

FIX: Only use CONTINUOUS_SUMMARY_OUTCOMES for continuous bootstrap

continuous_bootstrap_stats: dict[str, dict[str, Any]] = {}

for outcome in CONTINUOUS_SUMMARY_OUTCOMES:

Only compute for avg_* outcomes

avg_key = f"avg_{outcome}"

values = []

for seed_str, seed_level in all_seed_contrasts.items():

if avg_key in seed_level and not np.isnan(seed_level[avg_key]):

values.append(seed_level[avg_key])

if len(values) >= 2:

mean_val, ci_lower, ci_upper = bootstrap_ci(

np.array(values), bootstrap_repetitions, bootstrap_seed

)

```
n = len(values)

values_arr = np.array(values)

n_pos = int(np.sum(values_arr &gt; 0))

n_zero = int(np.sum(values_arr == 0))

n_neg = int(np.sum(values_arr &lt; 0))


continuous_bootstrap_stats[outcome] = {

"mean": mean_val,

"ci_lower": ci_lower,

"ci_upper": ci_upper,

"median": float(np.median(values)),

"std": float(np.std(values, ddof=1)), # Sample SD

"n": n,

"n_positive": n_pos,

"n_zero": n_zero,

"n_negative": n_neg,

}
```

```
# For binary outcomes: recovery probability per reset_source and B_status

binary_stats: dict[str, dict[str, Any]] = {}

for outcome in BINARY_OUTCOMES:

# Count per reset_source and B_status

counts: dict[str, dict[str, dict[str, int]]] = {

"carrier_reset": {"preserved": {"success": 0, "total": 0}, "zero": {"success": 0, "total": 0}},

"full_reset": {"preserved": {"success": 0, "total": 0}, "zero": {"success": 0, "total": 0}},

}


for seed_str, results in all_results.items():

for branch_id, branch_outcomes in results["outcomes"].items():

reset_source = branch_outcomes.get("reset_source")

b_status = branch_outcomes.get("B_status")


if reset_source not in ["carrier_reset", "full_reset"]:

continue

if b_status not in ["preserved", "zero"]:
```

```
continue


if outcome in branch_outcomes:

val = branch_outcomes[outcome]

# FIX: Include all seeds in denominator, check for truth value

counts[reset_source][b_status]["total"] += 1

if val:

counts[reset_source][b_status]["success"] += 1


binary_stats[outcome] = {}

for reset_source in ["carrier_reset", "full_reset"]:

for b_status in ["preserved", "zero"]:

s = counts[reset_source][b_status]["success"]

t = counts[reset_source][b_status]["total"]

key_base = f"{reset_source}_{b_status}"

if t &gt; 0:

p, ci_l, ci_u = wilson_ci(s, t)

binary_stats[outcome][f"{key_base}_probability"] = p
```

```
binary_stats[outcome][f"{key_base}_ci_lower"] = ci_l

binary_stats[outcome][f"{key_base}_ci_upper"] = ci_u

binary_stats[outcome][f"{key_base}_count"] = t

else:

binary_stats[outcome][f"{key_base}_probability"] = np.nan

binary_stats[outcome][f"{key_base}_ci_lower"] = np.nan

binary_stats[outcome][f"{key_base}_ci_upper"] = np.nan

binary_stats[outcome][f"{key_base}_count"] = 0


# Discordant counts for this reset_source

discordant = {"both": 0, "only_preserved": 0, "only_zero": 0, "neither": 0}

for seed_str, results in all_results.items():

pres_val = results["outcomes"].get(f"{reset_source}__B_preserved", {}).get(outcome)

zero_val = results["outcomes"].get(f"{reset_source}__B_zero", {}).get(outcome)


# FIX: Include all seeds, handle None/NaN properly

if pres_val is None or zero_val is None:

continue
```

```
if pres_val and zero_val:

discordant["both"] += 1

elif pres_val and not zero_val:

discordant["only_preserved"] += 1

elif not pres_val and zero_val:

discordant["only_zero"] += 1

else:

discordant["neither"] += 1


for k, v in discordant.items():

binary_stats[outcome][f"{reset_source}_discordant_{k}"] = v


# FIX: Paired probability difference (McNemar-style)

n_only_pres = discordant["only_preserved"]

n_only_zero = discordant["only_zero"]

n_total = discordant["both"] + n_only_pres + n_only_zero + discordant["neither"]

if n_total &gt; 0:

paired_diff = (n_only_pres - n_only_zero) / n_total
```

```
binary_stats[outcome][f"{reset_source}_paired_probability_diff"] = paired_diff

else:

binary_stats[outcome][f"{reset_source}_paired_probability_diff"] = np.nan



# For censored time outcomes: recovery probability and restricted mean

censored_stats: dict[str, dict[str, Any]] = {}



# FIX: Use correct horizon - end of observation from continue branch

# Get horizon from first successful seed's continue result

horizon = None

for seed_str, results in all_results.items():

if "continue" in results["branches"]:

cont_result = results["branches"]["continue"]

if "sample_times" in cont_result and len(cont_result["sample_times"]) &gt; 0:

horizon = float(cont_result["sample_times"][-1])

break
```

```
if horizon is None:

# Fallback to protocol value

horizon = float(protocol.get("recovery_end", 100)) * dt



for outcome in CENSORED_TIME_OUTCOMES:

censored_stats[outcome] = {}


for reset_source in ["carrier_reset", "full_reset"]:

for b_status in ["preserved", "zero"]:

finite_counts = 0

total_counts = 0

restricted_times: list[float] = []


for seed_str, results in all_results.items():

branch_id = f"{reset_source}__B_{b_status}"

if branch_id in results["outcomes"]:

val = results["outcomes"][branch_id].get(outcome)

# FIX: Include ALL seeds in denominator
```

```
total_counts += 1

if val is not None and not np.isnan(val) and np.isfinite(val):

finite_counts += 1

restricted_times.append(min(float(val), horizon))

else:

# Unrecovered: use horizon

restricted_times.append(horizon)


key_base = f"{reset_source}_{b_status}"

if total_counts &gt; 0:

censored_stats[outcome][f"{key_base}_recovery_probability"] = finite_counts / total_counts

else:

censored_stats[outcome][f"{key_base}_recovery_probability"] = np.nan


# Restricted Mean Time (not RMST - no KM estimator)

if restricted_times:

censored_stats[outcome][f"{key_base}_restricted_mean_time"] = float(np.mean(restricted_times))

else:
```

```
censored_stats[outcome][f"{key_base}_restricted_mean_time"] = np.nan



# FIX: Seed-level two-block binary contrast

# Bootstrap seed-level binary contrasts across 40 seeds

binary_contrast_stats: dict[str, dict[str, Any]] = {}

for outcome in BINARY_OUTCOMES:

# Get seed-level average contrasts for binary outcomes

values = []

for seed_str, seed_level in all_seed_contrasts.items():

avg_key = f"avg_{outcome}"

if avg_key in seed_level and not np.isnan(seed_level[avg_key]):

values.append(seed_level[avg_key])


if len(values) &gt;= 2:

mean_val, ci_lower, ci_upper = bootstrap_ci(

np.array(values), bootstrap_repetitions, bootstrap_seed

)
```

```
n = len(values)

values_arr = np.array(values)

n_pos = int(np.sum(values_arr &gt; 0))

n_zero = int(np.sum(values_arr == 0))

n_neg = int(np.sum(values_arr &lt; 0))


binary_contrast_stats[outcome] = {

"mean": mean_val,

"ci_lower": ci_lower,

"ci_upper": ci_upper,

"median": float(np.median(values)),

"std": float(np.std(values, ddof=1)),

"n": n,

"n_positive": n_pos,

"n_zero": n_zero,

"n_negative": n_neg,

}
```

```
# --- FIX: Summarize reset-realization interaction ---

interaction_stats: dict[str, dict[str, Any]] = {}

for outcome in CONTINUOUS_SUMMARY_OUTCOMES:

interaction_key = f"interaction_{outcome}"

values = []

for seed_str, seed_level in all_seed_contrasts.items():

if interaction_key in seed_level and not np.isnan(seed_level[interaction_key]):

values.append(seed_level[interaction_key])


if len(values) &gt;= 2:

mean_val, ci_lower, ci_upper = bootstrap_ci(

np.array(values), bootstrap_repetitions, bootstrap_seed

)

values_arr = np.array(values)

n_pos = int(np.sum(values_arr &gt; 0))

n_zero = int(np.sum(values_arr == 0))

n_neg = int(np.sum(values_arr &lt; 0))
```

```
interaction_stats[outcome] = {

"mean": mean_val,

"ci_lower": ci_lower,

"ci_upper": ci_upper,

"median": float(np.median(values)),

"std": float(np.std(values, ddof=1)),

"n": len(values),

"n_positive": n_pos,

"n_zero": n_zero,

"n_negative": n_neg,

}



# --- Generate report ---

report_lines = [

"=" * 70,

"AIEF-1.4 Matched Phase Memory Ablation",

"=" * 70,
```

```
"",
"PURPOSE:",
"Correct the unmatched phase/history randomization in the original",
"B_carrier_only vs full_reset comparison by constructing matched",
"phase/history reset blocks.",
"",
"SCIENTIFIC ESTIMAND:",
"For each frozen seed/graph, estimate the effect of preserving mature B,",
"conditional on the same randomized phase vector, history deque,",
"zeroed complex EMA state, omega, and future dynamics.",
"",
"-" * 70,
"RUN SUMMARY",
"-" * 70,
f"Total seeds: {len(seeds)}",
f"Seeds succeeded: {len([s for s in seed_status.values() if s == 'success'])}",
f"Seeds failed: {len(failed_seeds)}",
```

```
f"Initial matching passed: {len([r for r in audit_rows if r.get('pass', False)])}/{len(audit_rows) if audit_rows else 0}",

f"Anchor validation passed: {len([r for r in anchor_validation_rows if r.get('pass', True)])}/{len(anchor_validation_rows) if anchor_validation_rows else 0}",

f"Censoring horizon: {horizon:.4f}",

f"Run scope: {'full_cohort' if full_cohort else 'partial'}",

f"Freezeable: {not has_fatal_failure and verify_anchors and full_cohort}",

"",

]



# Continuous outcomes with bootstrap CI

if continuous_bootstrap_stats:

report_lines.extend([

"-" * 70,

"CONTINUOUS OUTCOMES - Seed-level Average Contrasts",

"Delta_bar_s(Y) = 0.5 * [Delta_{s,carrier}(Y) + Delta_{s,full}(Y)]",

"with bootstrap 95% CI",

"-" * 70,

"",
```

```
])

for outcome, s in continuous_bootstrap_stats.items():

report_lines.append(f" {outcome}:")

report_lines.append(

f" mean = {s['mean']:.6g} "

f"[CI: {s['ci_lower']:.6g}, {s['ci_upper']:.6g}]"

)

report_lines.append(

f" median = {s['median']:.6g}, std = {s['std']:.6g}"

)

report_lines.append(

f" n = {s['n']}, + = {s['n_positive']}, "

f"0 = {s['n_zero']}, - = {s['n_negative']}"

)

report_lines.append("")



# Binary outcomes - per reset source
```

```
if binary_stats:
report_lines.extend([
"-" * 70,
"BINARY OUTCOMES - Recovery Probability by Reset Source",
"-" * 70,
"",
])
for outcome, s in binary_stats.items():
report_lines.append(f" {outcome}:")
for reset_source in ["carrier_reset", "full_reset"]:
report_lines.append(f" Reset source: {reset_source}")
for b_status in ["preserved", "zero"]:
p_key = f"{reset_source}_{b_status}_probability"
ci_l_key = f"{reset_source}_{b_status}_ci_lower"
ci_u_key = f"{reset_source}_{b_status}_ci_upper"
n_key = f"{reset_source}_{b_status}_count"
p = s.get(p_key, np.nan)
```

```
ci_l = s.get(ci_l_key, np.nan)

ci_u = s.get(ci_u_key, np.nan)

n = s.get(n_key, 0)

if not pd.isna(p):

report_lines.append(

f" B={b_status}: p={p:.4f} [CI: {ci_l:.4f}, {ci_u:.4f}], n={n}"

)

# Discordant counts

for k in ["both", "only_preserved", "only_zero", "neither"]:

v = s.get(f"{reset_source}_discordant_{k}", 0)

report_lines.append(f" Discordant {k}: {v}")

# Paired probability difference

p_diff = s.get(f"{reset_source}_paired_probability_diff", np.nan)

if not pd.isna(p_diff):

report_lines.append(

f" Paired probability difference: {p_diff:.4f}"

)

report_lines.append("")
```

```
# Seed-level two-block binary contrast

if binary_contrast_stats:

report_lines.extend([

"-" * 70,

"SEED-LEVEL TWO-BLOCK BINARY CONTRAST",

"Delta^binary_s = 0.5 * [(I^B_{s,carrier} - I^0_{s,carrier}) + (I^B_{s,full} - I^0_{s,full})]",

"Bootstrap CI across 40 seeds (values in {-1, -1/2, 0, 1/2, 1})",

"-" * 70,

"",

])

for outcome, s in binary_contrast_stats.items():

report_lines.append(f" {outcome}:")

report_lines.append(

f" mean = {s['mean']:.6g} "

f"[CI: {s['ci_lower']:.6g}, {s['ci_upper']:.6g}]"

)
```

```
report_lines.append(
f" median = {s['median']:.6g}, std = {s['std']:.6g}"
)
report_lines.append(
f" n = {s['n']}, + = {s['n_positive']}, "
f"0 = {s['n_zero']}, - = {s['n_negative']}"
)
report_lines.append("")


# Censored time outcomes - per reset source
if censored_stats:
report_lines.extend([
"-" * 70,
"CENSORED TIME OUTCOMES - Recovery over Horizon",
f"Horizon: {horizon:.4f} (from continue branch sample_times[-1])",
"All seeds included in denominators; unrecovered = horizon",
"-" * 70,
```

```
"",
])
for outcome, s in censored_stats.items():
report_lines.append(f" {outcome}:")
for reset_source in ["carrier_reset", "full_reset"]:
report_lines.append(f" Reset source: {reset_source}")
for b_status in ["preserved", "zero"]:
p_key = f"{reset_source}_{b_status}_recovery_probability"
rmst_key = f"{reset_source}_{b_status}_restricted_mean_time"
p = s.get(p_key, np.nan)
rmst = s.get(rmst_key, np.nan)
if not pd.isna(p):
report_lines.append(
f" B={b_status}: recovery_prob={p:.4f}, restricted_mean_time={rmst:.4f}"
)
report_lines.append("")
```

```
# Reset-realization interaction summary

if interaction_stats:

report_lines.extend([

"-" * 70,

"RESET-REALIZATION INTERACTION - I_s(Y) = Delta_{s,carrier} - Delta_{s,full}",

"Quantifies sensitivity of B effect to reset realization",

"-" * 70,

"",

])

for outcome, s in interaction_stats.items():

report_lines.append(f" {outcome}:")

report_lines.append(

f" mean = {s['mean']:.6g} "

f"[CI: {s['ci_lower']:.6g}, {s['ci_upper']:.6g}]"

)

report_lines.append(

f" median = {s['median']:.6g}, std = {s['std']:.6g}"
```

```
)
report_lines.append(
f" n = {s['n']}, + = {s['n_positive']}, "
f"0 = {s['n_zero']}, - = {s['n_negative']}"
)
report_lines.append("")


report_lines.extend([
"-" * 70,
"INTERPRETATION CONSTRAINTS",
"-" * 70,
"",
"The experiment estimates the conditional effect of preserving mature B",
"under matched randomized phase/history and reset EMA.",
"",
"This is a controlled counterfactual, NOT a test that B is the",
"unique memory carrier.",
```

```
"",
"The seed/graph is the inferential unit (40 seeds).",
"The 80 reset-block contrasts are NOT independent observations.",
"",
])


report = "\n".join(report_lines)
atomic_write_text(output_dir / "matched_phase_report.txt", report)
print(f" Saved: matched_phase_report.txt")


# --- Build manifest ---
manifest_output = {
"analysis": "AIEF-1.4 matched phase memory ablation",
"version": "1.0",
"source": {
"path": str(Path(__file__).resolve()),
"sha256": file_sha256(Path(__file__).resolve()),
```

```
},
"frozen_authorities": {
"manifest": {
"path": str(Path(manifest_path).resolve()),
"sha256": file_sha256(manifest_path),
},
"npz": {
"path": str(Path(npz_path).resolve()),
"sha256": file_sha256(npz_path),
},
"scalar_csv": {
"path": str(Path(scalar_path).resolve()) if os.path.exists(scalar_path) else None,
"sha256": scalar_sha256,
"note": "Hashed for provenance but not used computationally",
},
},
"dependencies": {
"AIEF_1_4_memory_ablation_updated": {
```

"path": str(Path(prod.__file__).resolve()),

"sha256": compute_module_sha256(prod),

},

"AIEF_1_4_trajectory_recovery_equivalence": {

"path": str(Path(recovery.__file__).resolve()),

"sha256": compute_module_sha256(recovery),

},

},

"environment": {

"python_version": sys.version,

"numpy_version": np.__version__,

},

"model_constants": {

"N": N,

"dt": dt,

"T_window": T_window,

"alpha": math.exp(-dt / T_window),

"scale_epsilon": 1e-12,

},

"branch_order": list(branch_order),

"branch_rng_algorithm": (

"_, branch_root = np.random.SeedSequence(seed).spawn(2); "

"branch_sequences = branch_root.spawn(len(branch_order)); "

"branch_rng_seeds = {branch: int(sequence.generate_state(1, dtype=np.uint64)[0]) "

"for branch, sequence in zip(branch_order, branch_sequences)}"

),

"matched_reset_block_design": (

"Reset block A (carrier_reset seed from B_carrier_only): "

"carrier_reset__B_preserved (reproduces B_carrier_only), "

"carrier_reset__B_zero (new, B=0). "

"Reset block B (full_reset seed from full_reset): "

"full_reset__B_preserved (new, B=Bsplit), "

"full_reset__B_zero (reproduces full_reset). "

"Continue branch: reproduces continue."

),

"run_metadata": {

"run_scope": "full_cohort" if full_cohort else "partial",

"requested_seed_count": len(seeds),

"all_seeds_count": len(all_seeds),

"full_cohort": full_cohort,

},

"seeds": {

"all": all_seeds,

"requested": seeds,

"status": seed_status,

},

"validation": {

"initial_match_pass_count": len([r for r in audit_rows if r.get("pass", False)]),

"initial_match_total_count": len(audit_rows),

"anchor_validation_pass_count": len([r for r in anchor_validation_rows if r.get("pass", True)]),

"anchor_validation_total_count": len(anchor_validation_rows),

"verify_anchors": verify_anchors,

"split_state_immutability": all(

check_split_state_immutability(

all_results[s]["branches"]["carrier_reset__B_preserved"] if "carrier_reset__B_preserved" in all_results[s]["branches"] else None,

Note: We checked immutability during execution, this is just for manifest

)

for s in all_results

) if all_results else True,

},

"censoring": {

"horizon": horizon,

"source": "sample_times[-1] from continue branch (first seed with data)",

},

"recovery_definitions": {

"similarity_threshold": "0.9 * continue_Weighted_similarity_late",

"density_threshold": "0.9 * continue_D_B_final",

"deficit_calculation": "np.maximum(0, continue_value - value)",

"binary_recovery": "np.isfinite(time_to_event)",

"restricted_mean_time": "mean of (min(time, horizon) for all seeds)",

},

"command": sys.argv,

"success_count": len([s for s in seed_status.values() if s == "success"]),

"failure_count": len(failed_seeds),

"failed_seeds": failed_seeds,

"freezeable": not has_fatal_failure and verify_anchors and full_cohort,

"outputs": [

"matched_phase_scalar_summary.csv",

"matched_phase_paired_contrasts.csv",

"matched_phase_seed_level_contrasts.csv",

"matched_phase_recovery_summary.csv",

"matched_initial_state_audit.csv",

"frozen_anchor_validation.csv",

"matched_phase_branch_diagnostics.npz",

"matched_phase_report.txt",

"analysis_manifest.json",

],

"interpretation_constraints": [

"The experiment estimates the conditional effect of preserving mature B under matched randomized phase/history and reset EMA.",

"This is a controlled counterfactual, not a test that B is the unique memory carrier.",

"The seed/graph is the inferential unit (40 seeds).",

"The 80 reset-block contrasts are not independent observations.",

],

}

atomic_write_json(output_dir / "analysis_manifest.json", manifest_output)

print(f" Saved: analysis_manifest.json")

return {

"success_count": len([s for s in seed_status.values() if s == "success"]),

"failure_count": len(failed_seeds),

"failed_seeds": failed_seeds,

"initial_match_failed": initial_match_failed,

```
"anchor_failed": anchor_failed,

"has_fatal_failure": has_fatal_failure,

"full_cohort": full_cohort,

}

================================================================================
Self-Test
================================================================================

def run_self_test() -> dict[str, Any]:

text

"""Run self-test without frozen inputs."""

print("Running self-test...")



results: dict[str, Any] = {}



# FIX: Self-test REQUIRES production and recovery modules

if not PROD_AVAILABLE:

print(f" ERROR: Production module not available: {PROD_ERROR}")

results["all_passed"] = False
```

```
return results



if not RECOVERY_AVAILABLE:

print(f" ERROR: Recovery module not available: {RECOVERY_ERROR}")

results["all_passed"] = False

return results



# Test 1: Same reset seed produces equal theta/history/EMA

N = 12

np.random.seed(42)

mock_theta = np.random.randn(N)

mock_B = np.random.rand(N, N)

mock_B = 0.5 * (mock_B + mock_B.T)

np.fill_diagonal(mock_B, 0.0)

mock_coherence = (np.random.randn(N, N) + 1j * np.random.randn(N, N))

mock_history = deque([np.random.randn(N) for _ in range(10)])
```

```
# CORRECTED: Construct DynamicState properly
mock_state = prod.DynamicState(
theta=mock_theta.copy(),
B=mock_B.copy(),
coherence_acc=mock_coherence.copy(),
history=copy.deepcopy(mock_history),
)

test_seed = 12345

# Apply same intervention with same seed
state1 = copy.deepcopy(mock_state)
state2 = copy.deepcopy(mock_state)
rng1 = np.random.default_rng(test_seed)
rng2 = np.random.default_rng(test_seed)

prod.apply_intervention(state1, "full_reset", rng1)
```

```
prod.apply_intervention(state2, "full_reset", rng2)



test1 = (

np.array_equal(state1.theta, state2.theta) and

np.array_equal(state1.coherence_acc, state2.coherence_acc) and

len(state1.history) == len(state2.history) and

all(np.array_equal(h1, h2) for h1, h2 in zip(state1.history, state2.history))

)

results["test1_same_seed_equals"] = test1

print(f" Test 1 (same seed -&gt; same state): {'PASS' if test1 else 'FAIL'}")



# Test 2: Only B differs between preserved and zero

state_p = copy.deepcopy(mock_state)

state_z = copy.deepcopy(mock_state)

rng_p = np.random.default_rng(test_seed)

rng_z = np.random.default_rng(test_seed)
```

```
prod.apply_intervention(state_p, "B_carrier_only", rng_p)

prod.apply_intervention(state_z, "full_reset", rng_z)



test2 = (

np.array_equal(state_p.theta, state_z.theta) and

np.array_equal(state_p.coherence_acc, state_z.coherence_acc) and

np.array_equal(state_p.coherence_acc, np.zeros_like(state_p.coherence_acc)) and

np.array_equal(state_z.coherence_acc, np.zeros_like(state_z.coherence_acc)) and

not np.array_equal(state_p.B, state_z.B) and

np.array_equal(state_z.B, np.zeros((N, N)))

)

results["test2_only_B_differs"] = test2

print(f" Test 2 (only B differs): {'PASS' if test2 else 'FAIL'}")



# Test 3: History equality between preserved and zero

test3 = (

len(state_p.history) == len(state_z.history) and
```

```
all(np.array_equal(hp, hz) for hp, hz in zip(state_p.history, state_z.history))

)

results["test3_history_equals"] = test3

print(f" Test 3 (history equality): {'PASS' if test3 else 'FAIL'}")



# Test 4: RNG state equality after intervention

state_p2 = copy.deepcopy(mock_state)

state_z2 = copy.deepcopy(mock_state)

rng_p2 = np.random.default_rng(test_seed)

rng_z2 = np.random.default_rng(test_seed)



prod.apply_intervention(state_p2, "B_carrier_only", rng_p2)

prod.apply_intervention(state_z2, "full_reset", rng_z2)



rng_state_equal = (rng_p2.bit_generator.state == rng_z2.bit_generator.state)

results["test4_rng_state_equals"] = rng_state_equal

print(f" Test 4 (RNG state equality): {'PASS' if rng_state_equal else 'FAIL'}")
```

```
# Test 5: Array hashing - FIXED
arr = np.array([1.0, 2.0, 3.0])
h1 = array_sha256(arr)
h2 = array_sha256(arr)
h3 = array_sha256(np.array([1.0, 2.0, 4.0]))
test5 = (h1 == h2) and (h1 != h3)
results["test5_array_hashing"] = test5
print(f" Test 5 (array hashing): {'PASS' if test5 else 'FAIL'}")


# Test 6: Coherence hash changes with dtype
arr_float = np.array([1.0, 2.0, 3.0])
arr_complex = np.array([1.0+0j, 2.0+0j, 3.0+0j])
h_float = array_sha256(arr_float)
h_complex = array_sha256(arr_complex)
test6 = (h_float != h_complex)
results["test6_dtype_hash_differs"] = test6
```

```
print(f" Test 6 (dtype affects hash): {'PASS' if test6 else 'FAIL'}")



# Test 7: Contrast construction - FIX: Use declared outcome

mock_outcomes = {

"carrier_reset__B_preserved": {"P_mean_late": 1.0},

"carrier_reset__B_zero": {"P_mean_late": 0.5},

}

contrasts = compute_paired_contrasts(mock_outcomes)

test7 = contrasts.get("carrier_reset", {}).get("P_mean_late", np.nan) == 0.5

results["test7_contrast_construction"] = test7

print(f" Test 7 (contrast construction): {'PASS' if test7 else 'FAIL'}")



# Test 8: Seed-level averaging - FIX: Use declared outcome

paired = {

"carrier_reset": {"P_mean_late": 1.0},

"full_reset": {"P_mean_late": 0.6},

}
```

```
seed_level = compute_seed_level_contrasts(paired)
test8 = (
abs(seed_level.get("avg_P_mean_late", np.nan) - 0.8) &lt; 1e-10 and
abs(seed_level.get("interaction_P_mean_late", np.nan) - 0.4) &lt; 1e-10
)
results["test8_seed_level_averaging"] = test8
print(f" Test 8 (seed-level averaging): {'PASS' if test8 else 'FAIL'}")


# Test 9: Atomic writes
tmp_dir = Path(tempfile.mkdtemp())
try:
test_df = pd.DataFrame({"a": [1, 2, 3]})
atomic_write_csv(tmp_dir / "test.csv", test_df)
test9 = (tmp_dir / "test.csv").exists()
print(f" Test 9 (atomic CSV write): {'PASS' if test9 else 'FAIL'}")
results["test9_atomic_writes"] = test9
finally:
```

```
shutil.rmtree(tmp_dir, ignore_errors=True)



# Test 10: Bootstrap CI

values = np.random.randn(100)

mean_val, ci_l, ci_u = bootstrap_ci(values, 100, 42)

test10 = (

not np.isnan(mean_val) and

not np.isnan(ci_l) and

not np.isnan(ci_u) and

ci_l &lt;= mean_val &lt;= ci_u

)

results["test10_bootstrap_ci"] = test10

print(f" Test 10 (bootstrap CI): {'PASS' if test10 else 'FAIL'}")



# Test 11: Wilson CI

p, ci_l, ci_u = wilson_ci(80, 100)

test11 = (
```

0.0 <= p <= 1.0 and

0.0 <= ci_l <= p <= ci_u <= 1.0

)

results["test11_wilson_ci"] = test11

print(f" Test 11 (Wilson CI): {'PASS' if test11 else 'FAIL'}")

Test 12: Binary recovery indicator

finite_time = 5.0

nan_time = np.nan

test12a = bool(np.isfinite(finite_time))

test12b = not bool(np.isfinite(nan_time))

test12 = test12a and test12b

results["test12_binary_recovery_indicator"] = test12

print(f" Test 12 (binary recovery indicator): {'PASS' if test12 else 'FAIL'}")

Test 13: Censored seed in denominator

total = 3

```
finite = 2

recovery_prob = finite / total

test13 = abs(recovery_prob - 2/3) &lt; 1e-10

results["test13_censored_in_denominator"] = test13

print(f" Test 13 (censored in denominator): {'PASS' if test13 else 'FAIL'}")



# Test 14: Split state immutability

state_before = copy.deepcopy(mock_state)

state_after = copy.deepcopy(mock_state)

# Simulate mutation check

snapshot = create_split_snapshot(state_before)

immutability_ok = check_split_state_immutability(state_before, snapshot, N)

results["test14_split_state_immutability"] = immutability_ok

print(f" Test 14 (split state immutability check): {'PASS' if immutability_ok else 'FAIL'}")



all_pass = all(results.values())

print(f"\nSelf-test: {'ALL PASSED' if all_pass else 'SOME FAILED'}")
```

```
results["all_passed"] = all_pass



return results

# ============================================================================
# CLI
# ============================================================================

def parse_arguments() -> argparse.Namespace:

text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

description="AIEF-1.4 Matched Phase Memory Ablation"

)



# Required inputs

parser.add_argument(

"--npz",

default=DEFAULT_NPZ,
```

```
help="Frozen NPZ path (default: %(default)s)",

)

parser.add_argument(

"--manifest",

default=DEFAULT_MANIFEST,

help="Frozen manifest path (default: %(default)s)",

)

parser.add_argument(

"--scalar",

default=DEFAULT_SCALAR,

help="Frozen scalar CSV path (default: %(default)s)",

)



# Output

parser.add_argument(

"--output-dir",

default=DEFAULT_OUTPUT_DIR,
```

```
help="Output directory (default: %(default)s)",
)
parser.add_argument(
"--overwrite",
action="store_true",
default=False,
help="Overwrite existing outputs (default: fail if outputs exist)",
)


# Seed control
parser.add_argument(
"--max-seeds",
type=parse_nonnegative_int,
default=None,
help="Max seeds to process (default: all)",
)
parser.add_argument(
```

```
"--seed-filter",

type=parse_seed_list,

default=None,

help="Comma-separated seed filter (default: all)",

)



# Bootstrap

parser.add_argument(

"--bootstrap-repetitions",

type=parse_positive_int,

default=1000,

help="Bootstrap repetitions (default: %(default)s)",

)

parser.add_argument(

"--bootstrap-seed",

type=int,

default=42,
```

```
help="Bootstrap RNG seed (default: %(default)s)",

)



# Options

parser.add_argument(

"--verify-anchors",

action="store_true",

default=True,

help="Verify frozen anchors (default: on)",

)

parser.add_argument(

"--no-verify-anchors",

action="store_false",

dest="verify_anchors",

help="Skip anchor verification (debug only)",

)

parser.add_argument(
```

```
"--self-test",

action="store_true",

help="Run self-test and exit",

)



return parser.parse_args()

def main() -> None:

text

"""Main entry point."""

args = parse_arguments()



# Self-test mode - REQUIRES production and recovery modules

if args.self_test:

test_results = run_self_test()

if not test_results.get("all_passed", False):

print("Self-test FAILED")

sys.exit(1)
```

```
print("Self-test PASSED")

sys.exit(0)



# Check production module

if not PROD_AVAILABLE:

print(f"ERROR: Production module not available: {PROD_ERROR}")

sys.exit(1)



# Check recovery module

if not RECOVERY_AVAILABLE:

print(f"ERROR: Recovery module not available: {RECOVERY_ERROR}")

sys.exit(1)



# Validate inputs

for path in [args.npz, args.manifest]:

if not os.path.exists(path):

print(f"ERROR: Input file not found: {path}")
```

```
sys.exit(1)



if not os.path.exists(args.scalar):

print(f"WARNING: Scalar CSV not found: {args.scalar}")



output_dir = Path(args.output_dir)



# Run analysis

print("Starting matched phase memory ablation analysis...")

try:

results = run_matched_phase_ablation(

args.npz,

args.manifest,

args.scalar,

output_dir,

seed_filter=args.seed_filter,

max_seeds=args.max_seeds,
```

```
bootstrap_repetitions=args.bootstrap_repetitions,

bootstrap_seed=args.bootstrap_seed,

verify_anchors=args.verify_anchors,

overwrite=args.overwrite,

)

except RuntimeError as e:

print(f"\nFATAL ERROR: {e}")

sys.exit(1)



# Final report

print("\n" + "=" * 70)

print("ANALYSIS COMPLETE")

print("=" * 70)

print(f"Seeds succeeded: {results['success_count']}")

print(f"Seeds failed: {results['failure_count']}")

if results['failed_seeds']:

print(f"Failed seeds: {results['failed_seeds']}")
```

print(f"Initial matching failed: {results['initial_match_failed']}")

print(f"Anchor validation failed: {results['anchor_failed']}")

if results.get('has_fatal_failure'):

print("WARNING: Fatal validation failures detected. Scientific outputs may be invalid.")

print(f"Full cohort: {results.get('full_cohort', False)}")

print(f"Freezeable: {not results.get('has_fatal_failure', True) and args.verify_anchors and results.get('full_cohort', False)}")

print(f"\nOutput directory: {output_dir.resolve()}")

if name == "main":

text

main()

This version is very close in scientific design, but it still contains three concrete execution failures that a self-test/smoke test should expose immediately. Do not run 40 seeds yet.
Critical execution failures
1. Atomic writes call fsync after closing the file

Every atomic helper has this pattern:

Python

with os.fdopen(fd, "w") as f:
...
os.fsync(fd)

Exiting the with block closes fd, so:

Python

os.fsync(fd)

will raise:

text

OSError: Bad file descriptor

This affects CSV, JSON, text, and NPZ outputs. The self-test’s atomic-write test should fail here.

Use:

Python

```
def atomic_write_csv(
path: Path,
frame: pd.DataFrame,
) -> None:
path.parent.mkdir(
parents=True,
exist_ok=True,
)

fd, temporary = tempfile.mkstemp(
dir=path.parent,
prefix=f".{path.name}.",
suffix=".tmp",
)

try:
with os.fdopen(
fd,
"w",
encoding="utf-8",
newline="",
) as handle:
```

frame.to_csv(
handle,
index=False,
)
handle.flush()
os.fsync(handle.fileno())

os.replace(temporary, path)
except Exception:
try:
os.unlink(temporary)
except FileNotFoundError:
pass
raise

Use the same pattern for JSON/text.

For NPZ:

Python

with os.fdopen(fd, "wb") as handle:
np.savez(handle, **arrays)
handle.flush()
os.fsync(handle.fileno())

then replace after leaving the block.
2. Manifest construction calls the immutability helper incorrectly

This cannot run:

Python

check_split_state_immutability(
all_results[s]["branches"][
"carrier_reset__B_preserved"
],
missing split_snapshot and N

)

Problems:

the first argument is a branch result dictionary, not a state;
split_snapshot is missing;
N is missing.

The run will finish every simulation and then crash while constructing the manifest.

Store the already calculated Boolean:

Python

immutability_by_seed: dict[str, bool] = {}

Inside the seed loop:

Python

immutability_by_seed[seed_str] = (
immutability_ok
)

Then in the manifest:

Python

"split_state_immutability": {
"all_passed": bool(
all(
immutability_by_seed.values()
)
),
"by_seed": immutability_by_seed,
},

Do not attempt to reconstruct this check from branch output dictionaries.
3. Model-constant validation compares the manifest to itself

This call:

Python

```
validate_model_constants(
model,
manifest["model"],
)
```

passes the same dictionary twice. It cannot detect production-source drift.

The function must compare the frozen manifest against imported production constants:

Python

```
def validate_model_constants(
frozen_model: dict[str, Any],
) -> None:
exact_scalars = {
"N": (
int(frozen_model["N"]),
int(prod.N),
),
"dt": (
float(frozen_model["dt"]),
float(prod.DT),
),
"tau": (
float(frozen_model["tau"]),
float(prod.TAU),
),
"T_window": (
float(frozen_model["T_window"]),
float(prod.T_WINDOW),
),
"eta": (
```

```
float(frozen_model["eta"]),
float(prod.ETA),
),
"mu": (
float(frozen_model["mu"]),
float(prod.MU),
),
"gamma": (
float(frozen_model["gamma"]),
float(prod.GAMMA),
),
"B_c": (
float(frozen_model["B_c"]),
float(prod.B_C),
),
}


for name, (
frozen_value,
source_value,
) in exact_scalars.items():
if frozen_value != source_value:
raise RuntimeError(
f"{name} mismatch: "
f"frozen={frozen_value}, "
f"source={source_value}"
)


if not np.array_equal(
np.asarray(
frozen_model["omega"],
dtype=float,
),
np.asarray(
prod.OMEGA,
dtype=float,
),
):
raise RuntimeError(
"omega mismatch"
)
```

Also validate:

text

alpha
capacity
future_steps
window_steps
recovery_end
diagnostic_stride
fixed epsilon

Then call:

Python

validate_model_constants(model)

Failure handling remains internally inconsistent
4. Immediate re-raise prevents writing failure diagnostics

Inside the seed loop:

Python

except Exception:
...
raise

This exits the function immediately. The later block intended to write:

text

matched_initial_state_audit.csv
frozen_anchor_validation.csv
failure analysis_manifest.json

is unreachable.

Use:

Python

```
fatal_exception = None

for seed in seeds:
try:
...
except Exception as exc:
seed_status[seed_str] = (
f"failed: {type(exc).__name__}: "
f"{exc}"
)
fatal_exception = exc
break
```

After the loop:

Python

```
if fatal_exception is not None:
write_validation_only_outputs(...)
write_failure_manifest(...)
raise RuntimeError(
"matched-phase production run "
"failed validation"
) from fatal_exception
```

This preserves diagnostics without producing scientific summaries.
Stale-output protection is incomplete
5. --overwrite does not remove old outputs before execution

When overwrite=True, the code permits existing outputs but leaves them in place. If the run fails before rewriting all files, stale files survive.

At startup:

Python

```
if overwrite:
for filename in existing_outputs:
path = output_dir / filename
if path.exists():
path.unlink()
```

Without --overwrite, continue to fail on any known existing output.

For maximum safety, use a fresh output directory for the smoke and production runs.
Remaining statistical refinement
6. Add restricted-time outcomes to the per-seed contrast table

The marginal censored summaries now correctly:

include every seed;
replace unrecovered times by the observation horizon;
use the sampled future endpoint.

But the primary paired seed-level effect on recovery time is still absent.

After determining each branch horizon, create:

Python

```
restricted_time_to_similarity_90
restricted_time_to_density_90
```

per branch:

Python

```
restricted = (
float(event_time)
```

if np.isfinite(event_time)
else horizon
)

Add these to CONTINUOUS_SUMMARY_OUTCOMES and CONTRAST_OUTCOMES.

Then the primary two-reset seed average can be bootstrapped across all 40 seeds without complete-case selection.

Keep raw uncensored event times in the detailed recovery CSV, but do not aggregate their ordinary differences.
7. Raw censored times remain in CONTRAST_OUTCOMES

These are still included:

Python

time_to_similarity_90
time_to_density_90

Their contrast becomes NaN if either branch is censored. That is acceptable for a diagnostic CSV, but it should be explicitly labeled:

text

complete-case raw time contrast

The restricted-time contrast should be primary.
Self-test quality
8. Test 14 is tautological

This:

Python

snapshot = create_split_snapshot(
state_before

```
)
immutability_ok = (
check_split_state_immutability(
state_before,
snapshot,
N,
)
)
```

only checks a state against a snapshot made from itself. It does not test whether branch execution preserves the split state.

A meaningful unit test would call a lightweight branch simulation, but that is expensive without production constants. At minimum, mutate a copy and verify the helper detects it:

Python

```
snapshot = create_split_snapshot(
state_before
)

unchanged_pass = (
check_split_state_immutability(
state_before,
snapshot,
N,
)
)

state_before.B[0, 1] += 1e-6
state_before.B[1, 0] += 1e-6

mutation_detected = not (
check_split_state_immutability(
state_before,
snapshot,
N,
)
)
```

test14 = (
unchanged_pass
and mutation_detected
)

9. Add a raw-branch NPZ schema test

The previous revision’s largest failure was discovered only at serialization. Add a synthetic all_results structure with:

Python

```
{
"seed": {
"branches": {
branch_id: {
"sample_times": array,
"B_final": array,
}
}
}
}
```

Run the same NPZ-flattening loop and assert the expected keys exist.
10. The atomic-write self-test should test all formats

After fixing fsync, test:

CSV;
JSON;
text;
NPZ.

Read every output back and verify contents.
Smaller issues
11. Full-cohort detection is order-sensitive

Python

full_cohort = seeds == all_seeds

is acceptable because filtering currently preserves order. More explicit is:

Python

full_cohort = (
len(seeds) == len(all_seeds)
and set(seeds) == set(all_seeds)
)

Also ensure no duplicate seeds.
12. scipy is a new dependency for Wilson intervals

wilson_ci imports:

Python

from scipy import stats

Record SciPy in the environment/dependency manifest, or avoid it using:

Python

from statistics import NormalDist

z = NormalDist().inv_cdf(
1.0 - (1.0 - confidence) / 2.0
)

The standard-library option is preferable.
13. Protocol dt fallback remains implicit

compute_recovery_outcomes calculates:

Python

protocol.get("dt", 0.01)

but dt is in the model, not the protocol.

Pass dt explicitly into the function:

Python

compute_recovery_outcomes(
...,
dt=dt,
)

Then:

Python

window_time = (
protocol["window_steps"] * dt
)

14. Add a seed-status output

The failure manifest contains statuses, but a successful run should also save:

text

matched_phase_seed_status.csv

with:

text

seed
status
carrier_reset_seed
full_reset_seed
continue_seed
initial_match_pass
anchor_pass
split_immutable

This is useful for lineage and direct auditing.
What is now solid

Matched reset design;
exact RNG reconstruction;
exact intervention matching;
two historical branch anchors;
continue anchor;
raw branch result storage;
correct relative recovery thresholds;
explicit binary recovery indicators;
block-specific paired binary summaries;
seed-level two-block binary contrast;
all-seed censoring denominator;
sampled-horizon restricted times;
reset-realization interaction summaries;
full-cohort freezeability rule;
no silent partial scientific aggregation.

Updated rating
Area Rating
Experimental design 9/10
Production API use 9/10
Exact validation architecture 9/10
Recovery/censoring design 8/10
Reproducibility structure 8/10
Current execution correctness 6/10
Overall current version: 8/10 design, 6.5/10 executable implementation

The remaining issues are conventional programming defects rather than scientific-design failures. That is an important improvement.
Next gate

Fix the three execution failures:

fsync placement;
manifest immutability construction;
actual production constant validation.

Then run only:

Bash

```
python -m py_compile \
AIEF_1_4_matched_phase_memory_ablation.py

python AIEF_1_4_matched_phase_memory_ablation.py \
--self-test
```

Do not review another static rewrite before seeing the actual self-test output. Once self-test passes, run the one-seed smoke test in a fresh directory.

This version was updated by the agent that completed all the work after V4 two channel script, a different agent then the one who created the code so far:

```
#!/usr/bin/env python3

"""

AIEF-1.4 Matched-Phase Memory Correction

Fixes the confound in the original memory-ablation experiment:

B_carrier_only and full_reset used DIFFERENT randomized phase resets.

This driver:

For each seed, generates ONE shared randomized reset (theta, H, C=0)
```

Clones into two branches differing ONLY in initial B:

B_carrier: B = mature B_split, C=0, theta/H = shared reset

full_reset: B = 0, C=0, theta/H = shared reset

Runs both branches forward under production dynamics

Compares paired seed-level recovery metrics

Estimand: effect of preserving mature B, conditional on identical

text

phase/history/EMA reset and unchanged future dynamics.

Evidence class: controlled intervention experiment.

"""

```
from future import annotations

import argparse, copy, hashlib, json, math, os, re, sys, tempfile, time

from pathlib import Path

import numpy as np

import pandas as pd

DEFAULT_RUN_HASH = "5a9f093a2fe0"

DEFAULT_NPZ = f"runs/{DEFAULT_RUN_HASH}/AIEF_1_4_{DEFAULT_RUN_HASH}
_branch_diagnostics.npz"
```

DEFAULT_MANIFEST = f"runs/{DEFAULT_RUN_HASH}/manifest.json"

DEFAULT_OUTPUT_DIR = "aief_1_4_matched_phase_memory"

DEFAULT_DT = 0.01

DEFAULT_SCALE_EPSILON = 1e-12

DEFAULT_MAX_SEEDS = 40

DEFAULT_RNG_SEED = 20260803
── utilities
──

def file_sha256(path):

text

d = hashlib.sha256()

with open(path, "rb") as h:

while b := h.read(1024*1024): d.update(b)

return d.hexdigest()

def compute_module_sha256(m):

text

if m is None or not hasattr(m, "__file__") or m.__file__ is None:

return "not_available"

return file_sha256(Path(m.__file__).resolve())

def load_production_modules():

text

import AIEF_1_4_stationarity_perturbation_analysis_updated as base

import AIEF_1_4_component_controller_audit as ca

import AIEF_1_4_augmented_tangent_cocycle_analysis as aug

return base, ca, aug

── atomic writes (FIXED: fsync inside with block) ──────────────────────

def safe_atomic_write_text(path, content):

text

path.parent.mkdir(parents=True, exist_ok=True)

fd, tp = tempfile.mkstemp(dir=path.parent, prefix=f"{path.name}_", suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as f:

f.write(content)

f.flush()

os.fsync(f.fileno())

os.replace(tp, path)

except Exception:

if os.path.exists(tp): os.remove(tp)

raise

def safe_atomic_write_csv(path, df):

text

path.parent.mkdir(parents=True, exist_ok=True)

fd, tp = tempfile.mkstemp(dir=path.parent, prefix=f"{path.name}_", suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8", newline="") as f:

df.to_csv(f, index=False)

f.flush()

os.fsync(f.fileno())

os.replace(tp, path)

except Exception:

if os.path.exists(tp): os.remove(tp)

raise

def safe_atomic_write_json(path, data):

```
text

path.parent.mkdir(parents=True, exist_ok=True)

fd, tp = tempfile.mkstemp(dir=path.parent, prefix=f"{[path.name](http://path.name)}_", suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as f:

json.dump(data, f, indent=2, sort_keys=True)

f.write("\n")

f.flush()

os.fsync(f.fileno())

os.replace(tp, path)

except Exception:

if os.path.exists(tp): os.remove(tp)

raise

── Seed RNG derivation ─────────────────────────────────────────────

def make_seed_rng(master_seed, seed_id):

text

digest = hashlib.sha256(f"{master_seed}|{seed_id}".encode()).digest()
```

```
child_seed = int.from_bytes(digest[:8], "little", signed=False)

return np.random.default_rng(child_seed)

def generate_shared_reset(state, rng):

text

N = len(state.B)

theta_reset = rng.uniform(0, 2*np.pi, size=N).astype(float)

depth = state.history_depth if hasattr(state, 'history_depth') else len(state.history)

history_reset = []

for _ in range(depth):

history_reset.append(rng.uniform(0, 2*np.pi, size=N).astype(float))

coherence_reset = np.zeros((N, N), dtype=complex)

return {"theta": theta_reset, "history": history_reset, "coherence_acc": coherence_reset}

── Forward simulation
──────────────────────────────────────────────────────────────────────

def run_branch(state, model, caps, A, edges_mat, eps_sc, future_steps, aug):

text

snapshots = {}

sample_steps = {0, future_steps} | set(range(50, future_steps, 50))
```

```
for step in range(future_steps):

inter = aug.base_intermediates(state, model, caps, A, edges_mat, eps_sc)

if step in sample_steps:

snapshots[step] = {

"B": state.B.copy(),

"D_B": *compute*DB(state.B),

"effective_edges": *count*effective(state.B, model),

"total_edge_weight": float(np.sum(state.B)) * 0.5,

}

aug.advance_base_state(state, inter)

snapshots[future_steps] = {

"B": state.B.copy(),

"D_B": *compute*DB(state.B),

"effective_edges": *count*effective(state.B, model),

"total_edge_weight": float(np.sum(state.B)) * 0.5,

}

return state, snapshots

def computeDB(B):
```

```
text

N = len(B)

s = 0.0

for i in range(N):

for j in range(i+1, N):

s += B[i,j]

return s

def counteffective(B, model):

text

try: B_c = float(model["B_c"])

except: B_c = 0.12

return int(np.sum(B &gt; B_c)) // 2

── Recovery metrics ──────────────────────────────────────────────────────

def compute_recovery_metrics(snap_carrier, snap_reset, B_target, model):

text

fs = max(snap_carrier.keys())

B_carrier_final = snap_carrier[fs]["B"]
```

```
B_reset_final = snap_reset[fs]["B"]

def frob_sim(B_a, B_b):

na = np.linalg.norm(B_a, 'fro')

nb = np.linalg.norm(B_b, 'fro')

if na &lt; 1e-30 and nb &lt; 1e-30: return 1.0

return 1.0 - np.linalg.norm(B_a - B_b, 'fro') / (na + nb + 1e-30)

sim_carrier = frob_sim(B_carrier_final, B_target)

sim_reset = frob_sim(B_reset_final, B_target)

DB_carrier = snap_carrier[fs]["D_B"]

DB_reset = snap_reset[fs]["D_B"]

DB_target = *compute*DB(B_target)

return {

"similarity_carrier": float(sim_carrier),

"similarity_reset": float(sim_reset),

"similarity_delta": float(sim_carrier - sim_reset),

"DB_carrier": float(DB_carrier),

"DB_reset": float(DB_reset),
```

"DB_target": float(DB_target),

"DB_ratio_carrier": float(DB_carrier / max(DB_target, 1e-30)),

"DB_ratio_reset": float(DB_reset / max(DB_target, 1e-30)),

"ee_carrier": int(snap_carrier[fs]["effective_edges"]),

"ee_reset": int(snap_reset[fs]["effective_edges"]),

"ee_target": *count*effective(B_target, model),

"total_weight_carrier": float(snap_carrier[fs]["total_edge_weight"]),

"total_weight_reset": float(snap_reset[fs]["total_edge_weight"]),

}

── CLI ──

def parse_args():

text

p = argparse.ArgumentParser(description=__doc__)

p.add_argument("--npz", default=DEFAULT_NPZ)

p.add_argument("--manifest", default=DEFAULT_MANIFEST)

p.add_argument("--output-dir", default=DEFAULT_OUTPUT_DIR)

p.add_argument("--dt", type=float, default=DEFAULT_DT)

```
p.add_argument("--scale-epsilon", type=float, default=DEFAULT_SCALE_EPSILON)

p.add_argument("--max-seeds", type=int, default=DEFAULT_MAX_SEEDS)

p.add_argument("--rng-seed", type=int, default=DEFAULT_RNG_SEED)

return p.parse_args()

── Seed discovery ──────────────────────────────────────────────────────────

def discover_seeds(npz_path):

text

with np.load(npz_path, allow_pickle=False) as data:

all_keys = list(data.keys())

pattern = re.compile(r"^continue_seed_(\d+)_B_final$")

return sorted({[m.group](http://m.group)(1) for k in all_keys if (m := pattern.match(k))}, key=int)

── Model constant validation (FIXED: compares frozen manifest to production) ──

def validate_model_constants(frozen_model, prod_module):

text

mismatches = []

checks = {

"N": (int(frozen_model["N"]), int(prod_module.N)),
```

"dt": (float(frozen_model["dt"]), float(prod_module.DT)),

"T_window": (float(frozen_model["T_window"]), float(prod_module.T_WINDOW)),

}

for name, (frozen_val, prod_val) in checks.items():

if frozen_val != prod_val:

mismatches.append(f"{name}: frozen={frozen_val}, prod={prod_val}")

omega array

frozen_omega = np.asarray(frozen_model["omega"], dtype=float)

prod_omega = np.asarray(prod_module.OMEGA, dtype=float)

if not np.array_equal(frozen_omega, prod_omega):

mismatches.append("omega arrays differ")

if mismatches:

raise RuntimeError("Model constant mismatch: " + "; ".join(mismatches))

── main ──────────────

```
args = parse_args()

for p in (args.npz, args.manifest):

if not os.path.isfile(p): raise FileNotFoundError(f"Missing: {p}")

base, ca, aug = load_production_modules()

with open(args.manifest) as h: manifest = json.load(h)

model = manifest["model"]

dt = float(model["dt"])

cap = (float(model["a"])-float(model["delta"]))/float(model["q"])

caps = np.full(int(model["N"]), cap)

N = int(model["N"])

A, edges_mat = base.incidence_matrix(N)

fs = int(manifest["protocol"]["future_steps"])

eps_sc = args.scale_epsilon

# FIXED: validate model constants against actual production module

validate_model_constants(model, base)

seed_keys = discover_seeds(args.npz)

n_available = len(seed_keys)

n_seeds = min(n_available, args.max_seeds)
```

```
print(f"Found {n_available} seeds. Running {n_seeds} pairs.", flush=True)

results = []

seed_status = {}

immutability_by_seed = {}

for si, seed_id in enumerate(seed_keys[:n_seeds]):

t0 = time.time()

try:

with np.load(args.npz, allow_pickle=False) as data:

split_state = base.load_split_state(data, seed_id)

B_target = split_state.B.copy()

split_snapshot = {

"theta": split_state.theta.copy(),

"B": split_state.B.copy(),

"coherence": split_state.coherence_acc.copy(),

"history": [h.copy() for h in split_state.history],

}

# Shared reset
```

```
rng = make_seed_rng(args.rng_seed, seed_id)

reset = generate_shared_reset(split_state, rng)

# B_carrier branch

state_carrier = copy.deepcopy(split_state)

state_carrier.theta[:] = reset["theta"]

state_carrier.coherence_acc[:] = reset["coherence_acc"]

state_carrier.history.clear()

for entry in reset["history"]:

state_carrier.history.append(entry.copy())

# full_reset branch

state_reset = copy.deepcopy(split_state)

state_reset.theta[:] = reset["theta"]

state_reset.coherence_acc[:] = reset["coherence_acc"]

state_reset.B[:] = 0.0

np.fill_diagonal(state_reset.B, 0.0)

state_reset.history.clear()

for entry in reset["history"]:

state_reset.history.append(entry.copy())
```

```
# Run branches
state_carrier, snap_c = run_branch(state_carrier, model, caps, A,
edges_mat, eps_sc, fs, aug)
state_reset, snap_r = run_branch(state_reset, model, caps, A,
edges_mat, eps_sc, fs, aug)
# FIXED: check split state immutability and store per-seed
immut_ok = (
np.array_equal(split_state.theta, split_snapshot["theta"])
and np.array_equal(split_state.B, split_snapshot["B"])
and np.array_equal(split_state.coherence_acc, split_snapshot["coherence"])
and len(split_state.history) == len(split_snapshot["history"])
and all(np.array_equal(h, sh)
for h, sh in zip(split_state.history, split_snapshot["history"]))
)
immutability_by_seed[seed_id] = immut_ok
metrics = compute_recovery_metrics(snap_c, snap_r, B_target, model)
metrics["seed"] = seed_id
```

```
metrics["elapsed_s"] = time.time() - t0

metrics["immutable"] = immut_ok

results.append(metrics)

seed_status[seed_id] = "success"

print(f" [{si+1:&amp;gt;2}/{n_seeds}] {seed_id}: "

f"sim_c={metrics['similarity_carrier']:.4f} "

f"sim_r={metrics['similarity_reset']:.4f} "

f"Δ={metrics['similarity_delta']:+.4f} "

f"immut={'OK' if immut_ok else 'FAIL'} "

f"({metrics['elapsed_s']:.0f}s)", flush=True)

except Exception as e:

seed_status[seed_id] = f"failed: {type(e).__name__}: {e}"

print(f" [{si+1:&amp;gt;2}/{n_seeds}] {seed_id}: FAILED — "

f"{type(e).__name__}: {e}", flush=True)

# ── Output ──

od = Path(args.output_dir); od.mkdir(parents=True, exist_ok=True)

results_df = pd.DataFrame(results)

# Seed status CSV
```

status_rows = [{"seed": s, "status": st} for s, st in seed_status.items()]

if status_rows:

safe_atomic_write_csv(od/"matched_phase_seed_status.csv", pd.DataFrame(status_rows))

if len(results_df) > 0:

safe_atomic_write_csv(od/"matched_phase_recovery.csv", results_df)

── Report ──

n = len(results_df)

n_immut = int(results_df["immutable"].sum()) if n > 0 and "immutable" in results_df.columns else 0

if n > 0:

delta_col = results_df["similarity_delta"]

n_pos = int((delta_col > 0.001).sum())

n_neg = int((delta_col < -0.001).sum())

n_tie = n - n_pos - n_neg

mean_delta = float(delta_col.mean())

median_delta = float(delta_col.median())

lines = [

"="*70,

```
"AIEF-1.4 Matched-Phase Memory Correction",
"="*70,
f"Seeds run: {n}",
f"Split-state immutability: {n_immut}/{n}",
"",
"Branches share identical randomized phase/history/EMA reset.",
"They differ ONLY in initial B:",
" B_carrier: B = mature split-state weights",
" full_reset: B = 0",
"",
"─"*70,
"Similarity to Mature Target (final step):",
"─"*70,
]
if n &gt; 0:
lines.extend([
f" B_carrier mean: {float(results_df['similarity_carrier'].mean()):.4f}",
```

```
f" full_reset mean: {float(results_df['similarity_reset'].mean()):.4f}",

f" Δ (carrier − reset) mean: {mean_delta:+.4f}",

f" Δ median: {median_delta:+.4f}",

f" Δ &gt; 0: {n_pos} Δ ≈ 0: {n_tie} Δ &lt; 0: {n_neg}",

"",

])

if n_pos &gt; n * 0.75:

lines.append(" → Mature B provides a robust conditional recovery advantage.")

elif n_pos &gt; n * 0.5:

lines.append(" → Mature B provides a modest conditional advantage.")

elif n_tie &gt; n * 0.75:

lines.append(" → Preserving mature B has negligible effect under this")

lines.append(" matched-reset protocol.")

else:

lines.append(" → Mixed results. B effect varies across seeds.")

lines.extend([

"",

"─"*70,
```

```
"Capacity Density (D_B) Relative to Target:",
"─"*70,
f" B_carrier mean ratio: {float(results_df['DB_ratio_carrier'].mean()):.3f}",
f" full_reset mean ratio: {float(results_df['DB_ratio_reset'].mean()):.3f}",
"",
"─"*70,
"Effective Edges Relative to Target:",
"─"*70,
f" target: {int(results_df['ee_target'].iloc[0]) if n&gt;0 else 'N/A'}",
f" carrier: {int(results_df['ee_carrier'].mean()) if n&gt;0 else 'N/A'} (mean)",
f" reset: {int(results_df['ee_reset'].mean()) if n&gt;0 else 'N/A'} (mean)",
"",
])
lines.extend([
"─"*70,
"Interpretation:",
"─"*70,
```

```
"If B_carrier outperforms full_reset:",

" Mature B provides a conditional recovery advantage when phase",

" and EMA are matched-reset. This does NOT imply B is the unique",

" memory carrier — only that it contributes under these conditions.",

"",

"If branches are comparable:",

" Preserving mature B alone is not sufficient to materially alter",

" recovery under this protocol. Other state variables (phase",

" alignment, EMA coherence) dominate the reconstruction dynamics.",

"",

"Evidence class: controlled intervention experiment.",

"Paired seed-level design. Components within seeds are not independent.",

])

report = "\n".join(lines)

safe_atomic_write_text(od/"matched_phase_report.txt", report)

# FIXED: manifest with correctly-stored immutability

mf = {

"analysis": "AIEF-1.4 matched-phase memory correction",
```

"evidence_class": "controlled intervention experiment",

"design": "paired seed-level, shared randomized reset, B-only difference",

"n_seeds": n,

"n_immutable": n_immut,

"rng_seed": args.rng_seed,

"parameters": {"dt": dt, "scale_epsilon": eps_sc,

"future_steps": fs, "T_window": model["T_window"]},

"source_hashes": {

Path(__file__).name: file_sha256(Path(__file__).resolve()),

"base": compute_module_sha256(base),

},

"inputs": {

"npz": {"path": str(Path(args.npz).resolve()),

"sha256": file_sha256(args.npz)},

"manifest": {"path": str(Path(args.manifest).resolve()),

"sha256": file_sha256(args.manifest)},

},

"split_state_immutability": {

"all_passed": bool(n_immut == n and n > 0),

"by_seed": immutability_by_seed,

},

"seed_status": seed_status,

"outputs": ["matched_phase_recovery.csv", "matched_phase_seed_status.csv",

"matched_phase_report.txt", "analysis_manifest.json"],

}

if n > 0:

mf["summary"] = {

"mean_similarity_carrier": float(results_df["similarity_carrier"].mean()),

"mean_similarity_reset": float(results_df["similarity_reset"].mean()),

"mean_delta": mean_delta,

"median_delta": median_delta,

"n_B_carrier_better": n_pos,

"n_tie": n_tie,

"n_full_reset_better": n_neg,

}

safe_atomic_write_json(od/"analysis_manifest.json", mf)

print(report, flush=True)

print(f"\nDone. → {od.resolve()}", flush=True)

if name == "main":

text

main()

This is a substantial regression from the previous near-final matched-phase design. It is shorter, but it no longer performs the specified correction, uses incorrect outcome definitions, has no frozen anchors, and likely fails before processing the first seed.

I would reject this version rather than patch it.
1. Fatal runtime error

The script validates production constants against:

Python

base

where base is:

Python

AIEF_1_4_stationarity_perturbation_analysis_updated

That module does not define:

text

N
DT
T_WINDOW
OMEGA

Therefore this call:

Python

validate_model_constants(model, base)

will raise an AttributeError before any seed runs.

The production constants belong to:

Python

AIEF_1_4_memory_ablation_updated

which this script does not import.

This alone means the code has not passed even a basic smoke test.
2. It does not reproduce the original reset protocol

The original intervention does:

Python

theta = rng.random(N) * 2*pi

history = deque(
(
theta.copy()
for _ in range(DELAY_STEPS + 1)
),

maxlen=DELAY_STEPS + 1,
)

Every history entry equals the randomized theta.

The new code instead independently randomizes every history entry:

Python

for _ in range(depth):
history_reset.append(
rng.uniform(
0,
2*pi,
size=N,
)
)

This is a new phase/history intervention. It is not the matched correction to the original B_carrier_only/full_reset comparison.

Independent random history entries can create a very different initial delayed phase field. The script therefore estimates:

Effect of preserving BB under a newly invented random-history intervention.

It does not estimate the predeclared matched version of the original experiment.

If a new intervention were desired, it would need separate scientific justification and versioning.
3. It discards the two anchored reset blocks

The previous design used:

the original B_carrier_only reset draw;
the original full_reset reset draw.

This provided:

two matched reset contexts;
exact reproduction of both historical branches;
sensitivity to reset realization;
direct anchor validation.

This version uses one new hash-derived random reset per seed:

Python

```
make_seed_rng(
master_seed,
seed_id,
)
```

It has no historical anchor and no reset-realization sensitivity estimate.

That is a weaker design and does not directly correct the frozen comparison.
4. No frozen branch reproduces exactly

The script never checks:

text

```
continue
original B_carrier_only
original full_reset
```

against the frozen NPZ.

There is no validation of:

source-code drift;
update-order consistency;
RNG reconstruction;
final arrays;
sampled trajectories;

split-state loading.

The exact anchor gate was one of the strongest parts of the previous design. Removing it substantially weakens reproducibility.
5. D_B is defined incorrectly

This function:

Python

def compute_DB(B):
return sum(
B[i, j]
for i < j
)

computes total undirected edge weight.

It does not compute DBDB.

The production definition is threshold density:
DB=
DB=

\frac{
#{e:B_e>B_c}
}{
\binom N2
}.
]

Use:

Python

weights = B[prod.EDGES]

D_B = float(
np.count_nonzero(
weights > prod.B_C
)
/ prod.N_EDGES
)

The report’s “capacity density” ratios are currently total-weight ratios mislabeled as DBDB.
6. “Effective edges” is also defined incorrectly

The function:

Python

count_effective

counts edges above BcBc.

That is threshold edge count, not the established effective-number statistic.

The production effective number is approximately:
Neff=
Neff=

\frac{
(\sum_e w_e)^2
}{
\sum_e w_e^2
}.
]

The current report conflates:

threshold edge count;
edge density;
effective edge number;
total weight.

Those must remain separate.
7. The similarity metric is new and incompatible

The script invents:
$1-\|B-B_{target}\|_F/\|B\|_F+\|B_{target}\|_F$.
$1-\|B\|_F+\|B_{target}\|_F/\|B-B_{target}\|_F$.

The frozen experiment uses edgewise cosine similarity through the validated production helper.

This new metric is not inherently invalid, but it cannot be called the corrected version of the original outcome without separate labeling and comparison.

Use the original:

Python

```
prod.cosine_edges(
B_target,
B_final,
)
```

and retain normalized Frobenius distance only as a secondary outcome if desired.
8. It does not measure recovery

Despite the filename and report, the script records only sparse snapshots and final-state quantities.

It does not calculate:

Pmean,latePmean,late;
weighted late similarity;
retention;
emergence;
first recovery;
sustained recovery;
held recovery;
AUC deficit versus continue;
censoring-aware recovery probability;
restricted recovery time.

There is also no continue branch.

This is a final-state similarity experiment, not a memory-recovery correction.
9. It silently changes the randomization protocol

The prompt required reconstruction of original branch RNG seeds. Instead the script introduces:

Python

DEFAULT_RNG_SEED = 20260803

and derives new seeds by hashing:

Python

sha256(
f"{master_seed}|{seed_id}"
)

This is deterministic, but it is not the frozen randomization scheme. It prevents direct anchors and changes the intervention.

Again, that could define a separate experiment, but not the requested correction.
10. Initial matching is asserted, not audited

The branches do appear to share the same generated reset dictionary, but the script does not save or validate:

exact theta equality;
exact history equality;
exact zero EMA;
“only BB differs”;
array hashes;
RNG state;
reset-state schema.

The report simply states:

Branches share identical randomized phase/history/EMA reset.

This should be an exact machine-readable audit, not report prose.
11. Failure handling allows partial scientific output

Failures are caught per seed:

Python

except Exception:
seed_status[...] = "failed"

and the script continues. It then reports only successful seeds as:

text

Seeds run: n

There is no requirement that all requested seeds complete.

A 35/40 partial result could be reported without a clear nonfreezeable status. The manifest lacks:

text

requested count
success count
failure count
freezeable

in a sufficient production-gate form.
12. No paired uncertainty analysis

The report gives:

mean difference;
median difference;
sign counts.

There is no:

paired seed-level bootstrap interval;
exact paired binary count;
reset-realization interaction;
recovery probability;
censoring treatment.

The threshold:

Python

delta > 0.001

used to define positive/tie/negative is arbitrary and not recorded as a parameter.
13. Report prose overstates results

This branch:

Python

if n_pos > 0.75 * n:
"Mature B provides a robust conditional recovery advantage."

hardcodes an interpretation based only on sign count and an arbitrary 0.0010.001 threshold.

No uncertainty interval or effect-size criterion is used.

Likewise:

Other state variables dominate the reconstruction dynamics.

does not follow if preserving BB has little effect. Similar outcomes could arise because:

both branches reconstruct;
both fail;
the intervention overwhelms BB;
the attractor is redundant;
the new random-history protocol changes the estimand.

Do not hardcode either conclusion.
14. No trajectory or branch diagnostics are saved

The code stores snapshots in memory but outputs only one scalar CSV.

It does not save:

branch BB arrays;
sampled trajectories;
reset states;
matching audit;
branch diagnostics NPZ;
continue reference;
anchor checks.

This severely limits later reinterpretation.
15. Model validation is incomplete even if pointed at the right module

It checks only:

text

N
dt
T_window
omega

It must also check:

text

tau
eta
mu
gamma
capacity
B_c
alpha
future_steps
diagnostic_stride
delay steps
fixed safeguard

16. Useful parts

There are some good implementation choices:

deterministic per-seed reset generation;
exact shared reset dictionary;
split-state immutability check;
corrected atomic fsync placement;
simple paired branches;
explicit seed-status CSV;
clear interpretation warning that BB is not unique memory;
no spectral detour.

But these do not compensate for changing the intervention and redefining all major outcomes.
Comparison with the previous near-final version
Feature Previous near-final This version
Exact production intervention Yes No
Original RNG reconstruction Yes No
Two anchored reset blocks Yes No
Frozen anchor validation Yes No
Continue reference Yes No
Existing recovery definitions Yes No
Correct DBDB Yes No
Correct effective edges Yes No
Existing similarity metric Yes No
Censoring-aware outcomes Nearly complete No
Matched phase/history Yes Yes

Simplicity Moderate High

This version is simpler because it removed most of the scientific requirements.
Recommendation

Do not merge this version into the previous script.

Return to the previous near-final implementation and fix its remaining conventional bugs:

atomic fsync placement;
manifest immutability construction;
model-constant validation;
continuous/binary/censored outcome separation;
stale-output protection;
one-seed smoke testing.

Those are much smaller corrections than rebuilding the missing scientific design here.
Rating
Coding simplicity: 7/10
Experimental fidelity: 3/10
Outcome validity: 2/10
Reproducibility relative to frozen run: 3/10
Overall: 3.5/10

The script is a valid prototype for a different, newly randomized final-state BB-retention experiment. It is not the requested matched-phase correction to the frozen AIEF-1.4 memory-ablation experiment.

This is the V4 Coding agent version:

#!/usr/bin/env python3

"""AIEF-1.4 Matched Phase Memory Ablation.

Corrected comparison: B_carrier_only versus full_reset with matched phase/history.

PURPOSE:

The original frozen run compared B_carrier_only vs full_reset, but both branches

used different branch-specific random draws for phase/history randomization.

This confounded:

preservation vs removal of mature B, and

different randomized phase/history initial conditions.

This script constructs matched phase/history reset blocks so that within each

seed, the two branches differ ONLY in initial B (preserved vs zero).

SCIENTIFIC ESTIMAND:

For each frozen seed/graph, estimate the effect of preserving mature B,

conditional on:

the same randomized phase vector

the same randomized history deque

the same zeroed complex EMA state

the same omega

the same future dynamics and safeguards

Primary contrast: Y(B=Bsplit) - Y(B=0) under otherwise identical reset state.

This is a controlled counterfactual about the contribution of mature B.

It is NOT a test that B is the unique memory carrier.

FROZEN SOURCES:

runs/5a9f093a2fe0/manifest.json

runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz

runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv

DEPENDENCIES:

AIEF_1_4_memory_ablation_updated (imported as prod, DO NOT MODIFY)

AIEF_1_4_trajectory_recovery_equivalence (REQUIRED for validated recovery helpers)

INTERPRETATION CONSTRAINTS:

Allowed: "The experiment estimates the conditional effect of preserving mature B

text

under matched randomized phase/history and reset EMA."

Forbidden: "B is the unique memory carrier."

text

"B stores the attractor by itself."

"No other state variable matters."

"Matched reset proves universal causality."

The seed/graph is the inferential unit (40 seeds).

The 80 reset-block contrasts are NOT independent observations.

"""

from future import annotations

import argparse

import copy

import hashlib

import json

import math

import os

import shutil

import sys

import tempfile

from collections import deque

from pathlib import Path

from statistics import NormalDist

from typing import Any

import numpy as np

import pandas as pd

```
================================================================================
IMPORTS - Production dependencies (DO NOT MODIFY these modules)
================================================================================

try:

text

import AIEF_1_4_memory_ablation_updated as prod

PROD_AVAILABLE = True

except ImportError as e:

text

PROD_AVAILABLE = False

PROD_ERROR = str(e)

try:

text

import AIEF_1_4_trajectory_recovery_equivalence as recovery

RECOVERY_AVAILABLE = True

except ImportError as e:

text

RECOVERY_AVAILABLE = False
```

```
RECOVERY_ERROR = str(e)

# ============================================================================
# Constants
# ============================================================================

DEFAULT_RUN_HASH = "5a9f093a2fe0"

DEFAULT_NPZ = (

text

f"runs/{DEFAULT_RUN_HASH}/AIEF_1_4_{DEFAULT_RUN_HASH}_branch_diagnostics.npz"

)

DEFAULT_MANIFEST = f"runs/{DEFAULT_RUN_HASH}/manifest.json"

DEFAULT_SCALAR = (

text

f"runs/{DEFAULT_RUN_HASH}/AIEF_1_4_{DEFAULT_RUN_HASH}_scalar_summary.csv"

)

DEFAULT_OUTPUT_DIR = "aief_1_4_matched_phase_memory_ablation"
Branch IDs for output

BRANCH_IDS = [

text

"carrier_reset__B_preserved",
```

"carrier_reset__B_zero",

"full_reset__B_preserved",

"full_reset__B_zero",

"continue",

]
Branch metadata - production_intervention_label is what gets passed to simulate_future

BRANCH_META = {

text

"carrier_reset__B_preserved": {

"reset_source": "carrier_reset",

"B_status": "preserved",

"production_intervention_label": "B_carrier_only",

"frozen_anchor_branch": "B_carrier_only",

},

"carrier_reset__B_zero": {

"reset_source": "carrier_reset",

"B_status": "zero",

"production_intervention_label": "full_reset",

"frozen_anchor_branch": None,

},

"full_reset__B_preserved": {

"reset_source": "full_reset",

"B_status": "preserved",

"production_intervention_label": "B_carrier_only",

"frozen_anchor_branch": None,

},

"full_reset__B_zero": {

"reset_source": "full_reset",

"B_status": "zero",

"production_intervention_label": "full_reset",

"frozen_anchor_branch": "full_reset",

},

"continue": {

"reset_source": "continue",

"B_status": "continue",

"production_intervention_label": "continue",

"frozen_anchor_branch": "continue",

},

}

Outcome sets - FIX: Separate continuous, binary, censored

CONTINUOUS_SUMMARY_OUTCOMES = [

text

"P_mean_late",

"Weighted_similarity_late",

"Weighted_similarity_final",

"Retention_selected_late",

"Emergence_unselected_late",

"D_B_final",

"Total_edge_weight",

"Effective_edges",

"Nonzero_edge_fraction",

"Minimum_similarity",

"AUC_similarity_deficit_vs_continue",

"Mean_similarity_deficit_vs_continue",

"AUC_density_deficit_vs_continue",

FIX: Add restricted-time outcomes

"restricted_time_to_similarity_90",

"restricted_time_to_density_90",

]

BINARY_OUTCOMES = [

text

"sustained_similarity_90_recovered",

"sustained_density_90_recovered",

"held_similarity_90_for_1Twindow_recovered",

]

CENSORED_TIME_OUTCOMES = [

text

"time_to_similarity_90",

"time_to_density_90",

]
All outcomes for paired contrasts (includes continuous, binary, censored for diagnostics)

CONTRAST_OUTCOMES = (

text

CONTINUOUS_SUMMARY_OUTCOMES

+ BINARY_OUTCOMES

+ CENSORED_TIME_OUTCOMES

)
==========
Utility Functions
==========

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file."""

digest = hashlib.sha256()

with open(path, "rb") as handle:

while True:

block = handle.read(1024 * 1024)

if not block:

break

digest.update(block)

```
return digest.hexdigest()

def array_sha256(arr: np.ndarray) -> str:

text

"""Compute SHA-256 hash of a numpy array (shape, dtype, bytes)."""

digest = hashlib.sha256()

shape_bytes = np.asarray(arr.shape, dtype=np.int64).tobytes()

digest.update(shape_bytes)

digest.update(arr.dtype.str.encode())

contiguous = np.ascontiguousarray(arr)

if contiguous.size &gt; 0:

digest.update(contiguous.tobytes())

return digest.hexdigest()

def compute_module_sha256(module: Any) -> str:

text

"""Compute SHA-256 hash of a module's source file."""

path = Path(module.__file__).resolve()

return file_sha256(path)
```

```
def parse_seed_list(value: str) -> list[str]:

text

"""Parse comma-separated seed strings."""

return [s.strip() for s in value.split(",") if s.strip()]

def parse_positive_int(value: str) -> int:

text

"""Parse positive integer."""

val = int(value)

if val &lt;= 0:

raise argparse.ArgumentTypeError("value must be positive")

return val

def parse_nonnegative_int(value: str) -> int:

text

"""Parse nonnegative integer."""

val = int(value)

if val &lt; 0:

raise argparse.ArgumentTypeError("value must be nonnegative")

return val
```

def atomic_write_csv(path: Path, df: pd.DataFrame) -> None:

text

"""Write CSV atomically using tempfile with proper fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=path.parent, prefix=f".{path.name}.", suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8", newline="") as handle:

df.to_csv(handle, index=False)

handle.flush()

os.fsync(handle.fileno())

os.replace(tmp_path, str(path))

except Exception:

try:

os.unlink(tmp_path)

except FileNotFoundError:

pass

raise

```
def atomic_write_json(path: Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically using tempfile with proper fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=path.parent, prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as handle:

json.dump(data, handle, indent=2, sort_keys=True)

handle.write("\n")

handle.flush()

os.fsync(handle.fileno())

os.replace(tmp_path, str(path))

except Exception:

try:

os.unlink(tmp_path)

except FileNotFoundError:

pass
```

raise

def atomic_write_text(path: Path, text: str) -> None:

text

"""Write text atomically using tempfile with proper fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=path.parent, prefix=f".{path.name}.", suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(tmp_path, str(path))

except Exception:

try:

os.unlink(tmp_path)

except FileNotFoundError:

pass

```
raise

def atomic_write_npz(path: Path, **kwargs: Any) -> None:

text

"""Write NPZ atomically using tempfile with proper fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=path.parent, prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp.npz")

try:

with os.fdopen(fd, "wb") as handle:

np.savez(handle, **kwargs)

handle.flush()

os.fsync(handle.fileno())

os.replace(tmp_path, str(path))

except Exception:

try:

os.unlink(tmp_path)

except FileNotFoundError:

pass
```

raise

def trapezoid_integral(y: np.ndarray, x: np.ndarray) -> float:

text

"""Compute trapezoidal integral, using np.trapezoid when available."""

if hasattr(np, 'trapezoid'):

return float(np.trapezoid(y, x))

else:

return float(np.trapz(y, x))

==========
Bootstrap Functions
==========

def bootstrap_ci(

text

values: np.ndarray,

repetitions: int = 1000,

seed: int = 42,

confidence: float = 0.95,

) -> tuple[float, float, float]:

text

"""Compute bootstrap confidence interval for the mean.

Returns: (mean, ci_lower, ci_upper)

"""

if len(values) < 2:

return (float(np.nan), float(np.nan), float(np.nan))

values = np.asarray(values, dtype=float)

valid = ~np.isnan(values)

if np.sum(valid) < 2:

return (float(np.nan), float(np.nan), float(np.nan))

values = values[valid]

n = len(values)

mean_val = float(np.mean(values))

```
rng = np.random.default_rng(seed)

bootstrap_means = np.zeros(repetitions)



for i in range(repetitions):

indices = rng.integers(0, n, size=n)

sample = values[indices]

bootstrap_means[i] = np.mean(sample)



alpha = 1.0 - confidence

ci_lower = float(np.percentile(bootstrap_means, 100 * alpha / 2))

ci_upper = float(np.percentile(bootstrap_means, 100 * (1 - alpha / 2)))



return (mean_val, ci_lower, ci_upper)

def wilson_ci(

text

successes: int,
```

```
trials: int,

confidence: float = 0.95,

) -> tuple[float, float, float]:

text

"""Compute Wilson score confidence interval for a proportion.



Returns: (proportion, ci_lower, ci_upper)

FIX: Use statistics.NormalDist instead of scipy

"""

if trials == 0:

return (float(np.nan), float(np.nan), float(np.nan))



p = successes / trials

# FIX: Use statistics.NormalDist instead of scipy

norm = NormalDist()

z = norm.inv_cdf(1.0 - (1.0 - confidence) / 2.0)
```

```
denominator = 1 + z**2 / trials

centre = p + z**2 / (2 * trials)

adjustment = z * math.sqrt(

(p *(1 - p) + z**2 / (4 * trials)) / trials

)



ci_lower = (centre - adjustment) / denominator

ci_upper = (centre + adjustment) / denominator



return (p, max(0.0, float(ci_lower)), min(1.0, float(ci_upper)))

================================================================================
Reconstruct Branch RNG Seeds
================================================================================

def reconstruct_branch_rng_seeds(

text

seed: int | str,

branch_order: list[str],

) -> dict[str, int]:
```

text

"""Reconstruct original branch RNG seeds from seed and branch_order.

Uses the algorithm from the frozen run:

*, branch*root = np.random.SeedSequence(seed).spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

branch_rng_seeds = {

branch: int(sequence.generate_state(1, dtype=np.uint64)[0])

for branch, sequence in zip(branch_order, branch_sequences)

}

"""

seed_int = int(seed)

*, branch*root = np.random.SeedSequence(seed_int).spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

return {

branch: int(sequence.generate_state(1, dtype=np.uint64)[0])

for branch, sequence in zip(branch_order, branch_sequences)

}

==
State Construction and Loading
==

def load_split_and_prefix(

text

data: dict[str, np.ndarray],

seed: str,

N: int,

) -> tuple[Any, dict[str, np.ndarray]]:

text

"""Load and construct split state and prefix from NPZ data.

Constructs DynamicState directly with theta, B, coherence_acc, history.

Preserves original dtypes in prefix arrays.

"""

prefix: dict[str, np.ndarray] = {}

```
# Load prefix keys - PRESERVE ORIGINAL DTYPES

for key in prod.PREFIX_KEYS:

prefix_key = f"prefix_seed_{seed}_{key}"

if prefix_key in data:

prefix[key] = np.asarray(data[prefix_key]).copy()

else:

raise KeyError(f"Missing prefix key: {prefix_key}")



# Load state components

theta = np.asarray(data[f"prefix_seed_{seed}_theta_at_split"], dtype=float).copy()

B = np.asarray(data[f"prefix_seed_{seed}_B_at_split"], dtype=float).copy()

coherence_acc = np.asarray(data[f"prefix_seed_{seed}_coherence_acc_at_split"],
dtype=complex).copy()



history_array = np.asarray(data[f"prefix_seed_{seed}_history_at_split"], dtype=float)

history = deque((row.copy() for row in history_array), maxlen=len(history_array))
```

```
# Construct state

state = prod.DynamicState(

theta=theta,

B=B,

coherence_acc=coherence_acc,

history=history,

)



return state, prefix

def validate_split_state(

text

state: Any,

N: int,

seed: str,

) -> None:

text

"""Validate split state after loading. Raises ValueError on failure."""
```

```
# Check theta shape
if state.theta.shape != (N,):
raise ValueError(
f"theta shape {state.theta.shape} != (N,) for seed {seed}"
)

# Check B shape
if state.B.shape != (N, N):
raise ValueError(
f"B shape {state.B.shape} != (N, N) for seed {seed}"
)

# Check coherence_acc shape
if state.coherence_acc.shape != (N, N):
raise ValueError(
f"coherence_acc shape {state.coherence_acc.shape} != (N, N) for seed {seed}"
)
```

```
# Check history length

expected_history_len = getattr(prod, "DELAY_STEPS", 0) + 1

if len(state.history) != expected_history_len:

raise ValueError(

f"history length {len(state.history)} != {expected_history_len} for seed {seed}"

)



# Validate B matrix

validate_B_matrix(state.B, N, seed)

def validate_B_matrix(B: np.ndarray, N: int, seed: str | None = None) -> None:

text

"""Validate B matrix: symmetric, nonnegative, zero diagonal. Raises ValueError on failure."""

msg = f" for seed {seed}" if seed else ""

if B.shape != (N, N):

raise ValueError(f"B shape {B.shape} != (N, N){msg}")

if not np.array_equal(B, B.T):
```

raise ValueError(f"B not symmetric{msg}")

FIX: Use exact equality for zero diagonal

if not np.array_equal(np.diag(B), np.zeros(N, dtype=B.dtype)):

raise ValueError(f"B diagonal not exactly zero{msg}")

if np.any(B < 0):

raise ValueError(f"B has negative entries{msg}")

==========
Split-State Immutability Check
==========

def check_split_state_immutability(

text

split_state: Any,

split_snapshot: dict[str, Any],

N: int,

) -> bool:

text

"""Check that split state was not mutated during branch execution."""

```
checks = [
np.array_equal(split_state.theta, split_snapshot["theta"]),
np.array_equal(split_state.B, split_snapshot["B"]),
np.array_equal(split_state.coherence_acc, split_snapshot["coherence"]),
]


# Check history
if len(split_state.history) == len(split_snapshot["history"]):
history_ok = all(
np.array_equal(h, snap_h)
for h, snap_h in zip(split_state.history, split_snapshot["history"])
)
checks.append(history_ok)
else:
checks.append(False)


return all(checks)
```

```
def create_split_snapshot(split_state: Any) -> dict[str, Any]:

text

"""Create a snapshot of split state for immutability checking."""

return {

"theta": split_state.theta.copy(),

"B": split_state.B.copy(),

"coherence": split_state.coherence_acc.copy(),

"history": [h.copy() for h in split_state.history],

}

================================================================================
Initial State Matching Audit (MANDATORY)
================================================================================

def audit_initial_matching(

text

state_preserved: Any,

state_zero: Any,

B_at_split: np.ndarray,

N: int,
```

```
reset_seed: int,

reset_block: str,

rng_state_equal: bool,

) -> dict[str, Any]:

text

"""Perform MANDATORY initial state matching audit with EXACT equality.



Returns dict with validation results. All checks must use exact equality.

"""

results: dict[str, Any] = {

"reset_block": reset_block,

"reset_seed_uint64": reset_seed,

"rng_state_equal": rng_state_equal,

}



# 1. Theta exact equality

theta_equal = np.array_equal(state_preserved.theta, state_zero.theta)
```

```
results["theta_equal"] = theta_equal



# 2. Coherence_acc exact equality and both zero

coh_p_zero = np.array_equal(state_preserved.coherence_acc,
np.zeros_like(state_preserved.coherence_acc))

coh_z_zero = np.array_equal(state_zero.coherence_acc, np.zeros_like(state_zero.coherence_acc))

coh_equal = np.array_equal(state_preserved.coherence_acc, state_zero.coherence_acc)

results["coherence_acc_preserved_is_zero"] = coh_p_zero

results["coherence_acc_zero_is_zero"] = coh_z_zero

results["coherence_acc_equal"] = coh_equal



# 3. B checks - preserved equals split B, zero is exactly zero matrix

B_preserved_equal_split = np.array_equal(state_preserved.B, B_at_split)

B_zero_is_zero = np.array_equal(state_zero.B, np.zeros((N, N)))

results["B_preserved_equal_split"] = B_preserved_equal_split

results["B_zero_is_zero"] = B_zero_is_zero



# 4. B properties
```

```
try:
validate_B_matrix(state_preserved.B, N)
results["B_preserved_valid"] = True
except ValueError:
results["B_preserved_valid"] = False

try:
validate_B_matrix(state_zero.B, N)
results["B_zero_valid"] = True
except ValueError:
results["B_zero_valid"] = False

# 5. History: length equal and every entry equals theta
history_len_equal = len(state_preserved.history) == len(state_zero.history)
results["history_length_equal"] = history_len_equal

history_equal = True
```

```
history_matches_theta = True

if history_len_equal:

for hp, hz in zip(state_preserved.history, state_zero.history):

if not np.array_equal(hp, hz):

history_equal = False

if not np.array_equal(hp, state_preserved.theta):

history_matches_theta = False

results["history_equal"] = history_equal

results["history_matches_theta"] = history_matches_theta

else:

results["history_equal"] = False

results["history_matches_theta"] = False



# 6. Array hashes for all matched arrays

results["theta_hash_preserved"] = array_sha256(state_preserved.theta)

results["theta_hash_zero"] = array_sha256(state_zero.theta)

results["B_hash_preserved"] = array_sha256(state_preserved.B)
```

```
results["B_hash_zero"] = array_sha256(state_zero.B)

results["coherence_hash_preserved"] = array_sha256(state_preserved.coherence_acc)

results["coherence_hash_zero"] = array_sha256(state_zero.coherence_acc)



# History hash - stack deque entries

if history_len_equal and len(state_preserved.history) &gt; 0:

hist_p_array = np.stack(tuple(state_preserved.history))

hist_z_array = np.stack(tuple(state_zero.history))

results["history_hash_preserved"] = array_sha256(hist_p_array)

results["history_hash_zero"] = array_sha256(hist_z_array)

else:

results["history_hash_preserved"] = None

results["history_hash_zero"] = None



# Overall pass - ALL checks must pass

all_checks = [

theta_equal,
```

```
coh_p_zero,

coh_z_zero,

coh_equal,

B_preserved_equal_split,

B_zero_is_zero,

results.get("B_preserved_valid", False),

results.get("B_zero_valid", False),

history_len_equal,

history_equal,

history_matches_theta,

rng_state_equal,

]

results["pass"] = all(all_checks)



return results

================================================================================
Frozen Anchor Validation (MANDATORY)
```

```
================================================================================
====

def validate_frozen_anchor(

text

new_result: dict[str, Any],

frozen_data: dict[str, np.ndarray],

seed: str,

branch_label: str,

matrix_keys: list[str],

) -> dict[str, Any]:

text

"""Validate new result against frozen data for anchor branch.



Uses EXACT equality for matrix/scalar keys, TOLERANCE-based equality for trajectory arrays.

Missing keys are fatal.

Returns dict with validation results.



FIX: Use np.allclose() for trajectory arrays (similarity_trajectory, density_trajectory, sample_times)
```

to handle floating-point precision differences across NumPy versions/BLAS implementations.

Matrix keys (B_final, incidence_matrix_final, coherence_matrix_final, etc.) use exact equality.

"""

results: dict[str, Any] = {

"seed": seed,

"branch": branch_label,

"pass": True,

}

for key in matrix_keys:

frozen_key = f"{branch_label}_seed_{seed}_{key}"

if frozen_key not in frozen_data:

results[f"{key}_missing_in_frozen"] = True

results["pass"] = False

continue

if key not in new_result:

```
results[f"{key}_missing_in_new"] = True

results["pass"] = False

continue


frozen_arr = np.asarray(frozen_data[frozen_key])

new_arr = np.asarray(new_result[key])


# Determine comparison method based on key name and dtype

# Use tolerance for trajectory/time-series data (floating-point precision sensitive)

# Use exact equality for matrix data (should be deterministic)

key_lower = key.lower()


# Keys that are known to have FP precision issues

is_trajectory = any(

term in key_lower

for term in ['trajectory', 'times']

)


# Matrix keys that should use exact equality
```

```
is_matrix_key = any(

pattern in key_lower

for pattern in ['_final', '_matrix', 'incidence', 'coherence', 'B_']

)


# Use tolerance for trajectory keys, exact for matrix keys

if is_trajectory and not is_matrix_key:

# Tolerance-based comparison for trajectories

exact_equal = np.allclose(frozen_arr, new_arr, rtol=1e-14, atol=1e-14)

elif is_matrix_key:

# EXACT equality for matrix keys (B_final, incidence_matrix_final, coherence_matrix_final, etc.)

exact_equal = np.array_equal(frozen_arr, new_arr)

else:

# For other keys, use tolerance if float/complex, exact if integer

if np.issubdtype(frozen_arr.dtype, np.floating) or np.issubdtype(frozen_arr.dtype, np.complexfloating):

exact_equal = np.allclose(frozen_arr, new_arr, rtol=1e-14, atol=1e-14)

else:
```

exact_equal = np.array_equal(frozen_arr, new_arr)

results[f"{key}_exact"] = exact_equal

if not exact_equal:

Record max diff for diagnostics

max_diff = float(np.max(np.abs(frozen_arr - new_arr)))

results[f"{key}_max_diff"] = max_diff

results["pass"] = False

return results

==========
Model Constant Validation
==========

def validate_model_constants(frozen_model: dict[str, Any]) -> None:

text

"""Validate that frozen manifest model constants match production module constants.

Raises RuntimeError on mismatch.

FIX: Compare frozen manifest against prod module constants, not itself.

"""

Get constants from frozen manifest

frozen_N = int(frozen_model["N"])

frozen_dt = float(frozen_model["dt"])

frozen_T_window = float(frozen_model["T_window"])

frozen_omega = np.asarray(frozen_model["omega"], dtype=float)

Get from production module

These should be attributes of the prod module

prod_N = int(prod.N) if hasattr(prod, 'N') else int(getattr(prod, 'N_NODES', frozen_N))

prod_dt = float(prod.DT) if hasattr(prod, 'DT') else float(getattr(prod, 'dt', frozen_dt))

prod_T_window = float(prod.T_WINDOW) if hasattr(prod, 'T_WINDOW') else float(getattr(prod, 'T_window', frozen_T_window))

prod_omega = np.asarray(getattr(prod, 'OMEGA', frozen_omega), dtype=float)

Compare

if frozen_N != prod_N:

```
raise RuntimeError(
f"N mismatch: frozen={frozen_N}, production={prod_N}"
)
if not math.isclose(frozen_dt, prod_dt, rel_tol=1e-15):
raise RuntimeError(
f"dt mismatch: frozen={frozen_dt}, production={prod_dt}"
)
if not math.isclose(frozen_T_window, prod_T_window, rel_tol=1e-15):
raise RuntimeError(
f"T_window mismatch: frozen={frozen_T_window}, production={prod_T_window}"
)
if not np.array_equal(frozen_omega, prod_omega):
raise RuntimeError(
f"omega mismatch: frozen={frozen_omega}, production={prod_omega}"
)


# Also validate other important constants if available
```

```
if hasattr(prod, 'ALPHA'):

frozen_alpha = math.exp(-frozen_dt / frozen_T_window)

prod_alpha = float(prod.ALPHA)

if not math.isclose(frozen_alpha, prod_alpha, rel_tol=1e-15):

raise RuntimeError(

f"alpha mismatch: frozen={frozen_alpha}, production={prod_alpha}"

)



if hasattr(prod, 'SCALE_EPSILON'):

frozen_epsilon = 1e-12

prod_epsilon = float(prod.SCALE_EPSILON)

if not math.isclose(frozen_epsilon, prod_epsilon, rel_tol=1e-15):

raise RuntimeError(

f"scale_epsilon mismatch: frozen={frozen_epsilon}, production={prod_epsilon}"

)

================================================================================
Recovery Outcome Computation
================================================================================
```

```
def compute_recovery_outcomes(
text
result: dict[str, Any],
continue_result: dict[str, Any] | None,
seed: str,
branch_id: str,
protocol: dict[str, Any],
dt: float,
) -> dict[str, Any]:
text
"""Compute recovery outcomes using validated helpers from recovery module.


Uses relative thresholds (90% of continue reference) not absolute values.
Uses np.maximum(0, ...) for deficit calculations.
Creates explicit binary recovery indicators.


FIX: Pass dt explicitly instead of using protocol.get("dt", 0.01)
```

"""

outcomes: dict[str, Any] = {}

Copy all production scalars

for key in prod.BRANCH_SCALAR_KEYS:

if key in result:

outcomes[key] = result[key]

Get reference values from continue branch

if continue_result is not None:

reference_final_density = continue_result.get("D_B_final", np.nan)

reference_late_similarity = continue_result.get("Weighted_similarity_late", np.nan)

window_time = float(protocol.get("window_steps", 0)) * dt

else:

reference_final_density = np.nan

reference_late_similarity = np.nan

window_time = 10.0

```
sample_times = result.get("sample_times")



# Similarity trajectory outcomes

if "similarity_trajectory" in result:

sim_traj = np.asarray(result["similarity_trajectory"], dtype=float)


# Handle NaN by mapping to 0 for recovery calculations only

sim_traj_clean = np.where(np.isnan(sim_traj), 0.0, sim_traj)


if sample_times is not None and RECOVERY_AVAILABLE:

# Use relative threshold: 90% of continue late similarity

if not np.isnan(reference_late_similarity):

similarity_threshold = 0.9 * reference_late_similarity

else:

similarity_threshold = 0.9 # Fallback only


# time_to_similarity_90
```

```
time_to_sim = recovery.time_to_at_least(
sample_times, sim_traj_clean, similarity_threshold
)
outcomes["time_to_similarity_90"] = time_to_sim

# sustained_similarity_90_rest
sustained_sim = recovery.first_time_sustained_for_rest(
sample_times, sim_traj_clean, similarity_threshold
)
outcomes["sustained_similarity_90_rest"] = sustained_sim

# held_similarity_90_for_1Twindow
held_sim = recovery.first_time_held_for_duration(
sample_times, sim_traj_clean, similarity_threshold, window_time
)
outcomes["held_similarity_90_for_1Twindow"] = held_sim

# Explicit binary recovery indicators
outcomes["sustained_similarity_90_recovered"] = bool(np.isfinite(sustained_sim))
```

```
outcomes["held_similarity_90_for_1Twindow_recovered"] = bool(np.isfinite(held_sim))


# FIX: Add restricted-time outcomes

horizon = sample_times[-1] if len(sample_times) &gt; 0 else window_time

restricted_time_to_sim = min(float(time_to_sim), horizon) if np.isfinite(time_to_sim) else horizon

outcomes["restricted_time_to_similarity_90"] = restricted_time_to_sim


# AUC similarity deficit vs continue

if continue_result is not None and "similarity_trajectory" in continue_result:

cont_traj = np.asarray(continue_result["similarity_trajectory"], dtype=float)

cont_traj_clean = np.where(np.isnan(cont_traj), 0.0, cont_traj)


# Use np.maximum(0, ...) to avoid negative deficits

similarity_deficit = np.maximum(0.0, cont_traj_clean - sim_traj_clean)


outcomes["AUC_similarity_deficit_vs_continue"] = trapezoid_integral(

similarity_deficit, sample_times

)

outcomes["Mean_similarity_deficit_vs_continue"] = float(
```

```
np.mean(similarity_deficit)

)

else:

# Fallback if recovery module not available

outcomes["time_to_similarity_90"] = np.nan

outcomes["sustained_similarity_90_rest"] = np.nan

outcomes["held_similarity_90_for_1Twindow"] = np.nan

outcomes["sustained_similarity_90_recovered"] = False

outcomes["held_similarity_90_for_1Twindow_recovered"] = False

outcomes["restricted_time_to_similarity_90"] = np.nan

if continue_result is not None and "similarity_trajectory" in continue_result:

cont_traj = np.asarray(continue_result["similarity_trajectory"], dtype=float)

cont_traj_clean = np.where(np.isnan(cont_traj), 0.0, cont_traj)

similarity_deficit = np.maximum(0.0, cont_traj_clean - sim_traj_clean)

if sample_times is not None:

outcomes["AUC_similarity_deficit_vs_continue"] = trapezoid_integral(

similarity_deficit, sample_times
```

```
)

else:

outcomes["AUC_similarity_deficit_vs_continue"] = np.nan

outcomes["Mean_similarity_deficit_vs_continue"] = float(

np.mean(similarity_deficit)

)



# Density trajectory outcomes

if "density_trajectory" in result:

dens_traj = np.asarray(result["density_trajectory"], dtype=float)



if sample_times is not None and RECOVERY_AVAILABLE:

# Use relative threshold: 90% of continue final density

if not np.isnan(reference_final_density):

density_threshold = 0.9 * reference_final_density

else:

density_threshold = 0.9 # Fallback only
```

```
# time_to_density_90

time_to_dens = recovery.time_to_at_least(

sample_times, dens_traj, density_threshold

)

outcomes["time_to_density_90"] = time_to_dens


# sustained_density_90_rest

sustained_dens = recovery.first_time_sustained_for_rest(

sample_times, dens_traj, density_threshold

)

outcomes["sustained_density_90_rest"] = sustained_dens


# Explicit binary recovery indicator

outcomes["sustained_density_90_recovered"] = bool(np.isfinite(sustained_dens))


# FIX: Add restricted-time outcomes

horizon = sample_times[-1] if len(sample_times) &gt; 0 else window_time

restricted_time_to_dens = min(float(time_to_dens), horizon) if np.isfinite(time_to_dens) else horizon

outcomes["restricted_time_to_density_90"] = restricted_time_to_dens
```

```
# AUC density deficit vs continue

if continue_result is not None and "density_trajectory" in continue_result:

cont_dens = np.asarray(continue_result["density_trajectory"], dtype=float)

density_deficit = np.maximum(0.0, cont_dens - dens_traj)


outcomes["AUC_density_deficit_vs_continue"] = trapezoid_integral(

density_deficit, sample_times

)

else:

outcomes["time_to_density_90"] = np.nan

outcomes["sustained_density_90_rest"] = np.nan

outcomes["sustained_density_90_recovered"] = False

outcomes["restricted_time_to_density_90"] = np.nan

if continue_result is not None and "density_trajectory" in continue_result:

cont_dens = np.asarray(continue_result["density_trajectory"], dtype=float)

density_deficit = np.maximum(0.0, cont_dens - dens_traj)

if sample_times is not None:
```

```
outcomes["AUC_density_deficit_vs_continue"] = trapezoid_integral(

density_deficit, sample_times

)

else:

outcomes["AUC_density_deficit_vs_continue"] = np.nan



return outcomes

# ============================================================================
Contrast Computation
# ============================================================================

def compute_paired_contrasts(

text

outcomes: dict[str, dict[str, Any]],

) -> dict[str, dict[str, float]]:

text

"""Compute paired contrasts: Y(B=preserved) - Y(B=zero) for each reset source.
```

Returns: reset_source -> outcome -> contrast

Only computes contrasts for explicitly declared CONTRAST_OUTCOMES.

"""

contrasts: dict[str, dict[str, float]] = {}

reset_sources = ["carrier_reset", "full_reset"]

for reset_source in reset_sources:

contrasts[reset_source] = {}

preserved_key = f"{reset_source}__B_preserved"

zero_key = f"{reset_source}__B_zero"

if preserved_key not in outcomes or zero_key not in outcomes:

continue

pres = outcomes[preserved_key]

zero = outcomes[zero_key]

```
# Only compute for explicitly declared contrast outcomes

for key in CONTRAST_OUTCOMES:

if key in pres and key in zero:

pres_val = pres[key]

zero_val = zero[key]

# Only compute for numeric scalars

if isinstance(pres_val, (int, float, np.number)) and isinstance(zero_val, (int, float, np.number)):

try:

contrast = float(pres_val) - float(zero_val)

contrasts[reset_source][key] = contrast

except (TypeError, ValueError):

pass



return contrasts

def compute_seed_level_contrasts(

text

paired_contrasts: dict[str, dict[str, float]],
```

) -> dict[str, float]:

text

"""Compute seed-level average and interaction contrasts.

Δ̄_s(Y) = 0.5 * [Δ_{s,carrier}(Y) + Δ_{s,full}(Y)]

I_s(Y) = Δ_{s,carrier}(Y) - Δ_{s,full}(Y)

Returns: outcome -> avg or interaction value

"""

seed_contrasts: dict[str, float] = {}

all_outcomes = set()

for reset_source in paired_contrasts:

all_outcomes.update(paired_contrasts[reset_source].keys())

for outcome in all_outcomes:

carrier_val = paired_contrasts.get("carrier_reset", {}).get(outcome, np.nan)

```
full_val = paired_contrasts.get("full_reset", {}).get(outcome, np.nan)


# Seed-level average

if not np.isnan(carrier_val) and not np.isnan(full_val):

seed_contrasts[f"avg_{outcome}"] = 0.5 * (carrier_val + full_val)

else:

seed_contrasts[f"avg_{outcome}"] = np.nan


# Interaction

if not np.isnan(carrier_val) and not np.isnan(full_val):

seed_contrasts[f"interaction_{outcome}"] = carrier_val - full_val

else:

seed_contrasts[f"interaction_{outcome}"] = np.nan



return seed_contrasts

============================================================================
====
Main Analysis Function
============================================================================
====

def run_matched_phase_ablation(
```

```
text

npz_path: str,

manifest_path: str,

scalar_path: str,

output_dir: Path,

seed_filter: list[str] | None = None,

max_seeds: int | None = None,

bootstrap_repetitions: int = 1000,

bootstrap_seed: int = 42,

verify_anchors: bool = True,

overwrite: bool = False,

) -> dict[str, Any]:

text

"""Main analysis function."""



output_dir.mkdir(parents=True, exist_ok=True)
```

```
# FIX: Remove existing outputs if overwrite=True
existing_outputs = [
"matched_phase_scalar_summary.csv",
"matched_phase_paired_contrasts.csv",
"matched_phase_seed_level_contrasts.csv",
"matched_phase_recovery_summary.csv",
"matched_initial_state_audit.csv",
"frozen_anchor_validation.csv",
"matched_phase_branch_diagnostics.npz",
"matched_phase_report.txt",
"analysis_manifest.json",
"matched_phase_seed_status.csv",
]

if overwrite:
for filename in existing_outputs:
path = output_dir / filename
```

```
if path.exists():

path.unlink()

else:

existing_files = [f for f in existing_outputs if (output_dir / f).exists()]

if existing_files:

raise RuntimeError(

f"Output directory {output_dir} contains existing files: {existing_files}. "

f"Use --overwrite to replace, or use an empty directory."

)



# --- Load manifest ---

with open(manifest_path, "r") as f:

manifest = json.load(f)



# Validate manifest structure - CORRECTED PATHS

if "protocol" not in manifest or "branches" not in manifest["protocol"]:

raise ValueError("Manifest missing protocol.branches")
```

```
if "randomization" not in manifest or "paired_seeds" not in manifest["randomization"]:

raise ValueError("Manifest missing randomization.paired_seeds")



model = manifest["model"]

protocol = manifest["protocol"]



# FIX: Validate model constants match production

validate_model_constants(model)



# Get branch order from correct path

branch_order = tuple(manifest["protocol"]["branches"])



# Validate branch order matches production

if not hasattr(prod, "BRANCHES"):

raise ValueError("Production module missing BRANCHES attribute")

if branch_order != tuple(prod.BRANCHES):
```

```
raise ValueError(
f"Branch order mismatch: manifest={branch_order}, "
f"prod.BRANCHES={tuple(prod.BRANCHES)}"
)


# Get seeds from correct path
all_seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]
if not all_seeds:
raise ValueError("No seeds found in manifest")


# Apply filter
if seed_filter:
seeds = [s for s in all_seeds if s in seed_filter]
else:
seeds = all_seeds


# FIX: Reject empty seed filter
```

```
if not seeds:

raise ValueError("No seeds selected - check seed filter")



# FIX: Handle max_seeds=0

if max_seeds is not None:

if max_seeds == 0:

raise ValueError("--max-seeds must be positive or None")

seeds = seeds[:max_seeds]



# FIX: Validate bootstrap_repetitions &gt; 0 for production

if bootstrap_repetitions &lt;= 0:

raise ValueError("--bootstrap-repetitions must be positive")



# FIX: Recovery dependency is MANDATORY for production

if not RECOVERY_AVAILABLE:

raise RuntimeError(

f"Validated recovery dependency unavailable: {RECOVERY_ERROR}"
```

)

print(f"Processing {len(seeds)} seeds...")

Get model constants

N = int(model["N"])

dt = float(model["dt"])

T_window = float(model["T_window"])

Get matrix keys for validation

matrix_keys = getattr(prod, "BRANCH_MATRIX_KEYS", [])

if not matrix_keys:

raise ValueError("prod.BRANCH_MATRIX_KEYS is empty or not defined")

--- Load frozen data ---

with np.load(npz_path, allow_pickle=False) as frozen_npz:

frozen_data = {key: frozen_npz[key] for key in frozen_npz.files}

```
# Load scalar CSV for hash verification

scalar_sha256 = None

if os.path.exists(scalar_path):

scalar_sha256 = file_sha256(scalar_path)



# --- Initialize storage ---

all_results: dict[str, dict[str, Any]] = {}

all_contrasts: dict[str, dict[str, dict[str, float]]] = {}

all_seed_contrasts: dict[str, dict[str, float]] = {}

audit_rows: list[dict[str, Any]] = []

anchor_validation_rows: list[dict[str, Any]] = []

seed_status: dict[str, str] = {}

immutability_by_seed: dict[str, bool] = {} # FIX: Store immutability results



# Determine run scope

# FIX: Use set comparison for order-insensitive check
```

```
full_cohort = (set(seeds) == set(all_seeds) and len(seeds) == len(all_seeds))



# FIX: Collect fatal exception instead of immediate re-raise

fatal_exception = None



# --- Process each seed ---

for seed in seeds:

seed_str = str(seed)

print(f" Seed: {seed_str}")



try:

# --- Load split state and prefix ---

split_state, prefix = load_split_and_prefix(frozen_data, seed_str, N)

B_at_split = split_state.B.copy()



# FIX: Validate split state

validate_split_state(split_state, N, seed_str)



# FIX: Create snapshot for immutability check
```

```
split_snapshot = create_split_snapshot(split_state)


# --- Reconstruct ALL branch RNG seeds from manifest ---

branch_rng_seeds = reconstruct_branch_rng_seeds(seed, list(branch_order))


# Get seeds for our reset blocks

carrier_reset_seed = branch_rng_seeds.get("B_carrier_only")

if carrier_reset_seed is None:

raise ValueError(f"B_carrier_only not in branch_rng_seeds for seed {seed_str}")


full_reset_seed = branch_rng_seeds.get("full_reset")

if full_reset_seed is None:

raise ValueError(f"full_reset not in branch_rng_seeds for seed {seed_str}")


# FIX: Require continue RNG seed

if "continue" not in branch_rng_seeds:

raise RuntimeError(f"continue RNG seed missing for seed {seed_str}")

continue_seed = branch_rng_seeds["continue"]
```

```
# --- RESET BLOCK A: carrier_reset seed ---

print(f" Reset block A (carrier_reset, seed={carrier_reset_seed})")


# CORRECTED: Use actual production API

# carrier_reset__B_preserved: reproduces B_carrier_only

result_carrier_preserved = prod.simulate_future(

copy.deepcopy(split_state),

prefix,

"B_carrier_only",

np.random.default_rng(carrier_reset_seed),

)


# carrier_reset__B_zero: new, B=0, same reset seed

result_carrier_zero = prod.simulate_future(

copy.deepcopy(split_state),

prefix,

"full_reset",

np.random.default_rng(carrier_reset_seed),
```

```
)

# --- Initial matching audit for Block A ---
# FIX: Compare RNG states directly after intervention
state_preserved_A = copy.deepcopy(split_state)
state_zero_A = copy.deepcopy(split_state)

rng_audit_p_A = np.random.default_rng(carrier_reset_seed)
rng_audit_z_A = np.random.default_rng(carrier_reset_seed)

prod.apply_intervention(state_preserved_A, "B_carrier_only", rng_audit_p_A)
prod.apply_intervention(state_zero_A, "full_reset", rng_audit_z_A)

rng_state_equal_A = (rng_audit_p_A.bit_generator.state == rng_audit_z_A.bit_generator.state)

audit_A = audit_initial_matching(
state_preserved_A, state_zero_A, B_at_split, N,
carrier_reset_seed, "carrier_reset", rng_state_equal_A
)
audit_A["seed"] = seed_str
```

```
audit_rows.append(audit_A)


# MANDATORY: Stop on matching failure
if not audit_A["pass"]:
raise RuntimeError(
f"Initial matching FAILED for seed={seed_str}, "
f"block=carrier_reset. Check matched_initial_state_audit.csv"
)


# --- RESET BLOCK B: full_reset seed ---
print(f" Reset block B (full_reset, seed={full_reset_seed})")


# full_reset__B_preserved: new, B=Bsplit
result_full_preserved = prod.simulate_future(
copy.deepcopy(split_state),
prefix,
"B_carrier_only",
np.random.default_rng(full_reset_seed),
```

```
)

# full_reset__B_zero: reproduces full_reset
result_full_zero = prod.simulate_future(
copy.deepcopy(split_state),
prefix,
"full_reset",
np.random.default_rng(full_reset_seed),
)

# --- Initial matching audit for Block B ---
state_preserved_B = copy.deepcopy(split_state)
state_zero_B = copy.deepcopy(split_state)

rng_audit_p_B = np.random.default_rng(full_reset_seed)
rng_audit_z_B = np.random.default_rng(full_reset_seed)

prod.apply_intervention(state_preserved_B, "B_carrier_only", rng_audit_p_B)
prod.apply_intervention(state_zero_B, "full_reset", rng_audit_z_B)
```

```
rng_state_equal_B = (rng_audit_p_B.bit_generator.state == rng_audit_z_B.bit_generator.state)


audit_B = audit_initial_matching(

state_preserved_B, state_zero_B, B_at_split, N,

full_reset_seed, "full_reset", rng_state_equal_B

)

audit_B["seed"] = seed_str

audit_rows.append(audit_B)


# MANDATORY: Stop on matching failure

if not audit_B["pass"]:

raise RuntimeError(

f"Initial matching FAILED for seed={seed_str}, "

f"block=full_reset. Check matched_initial_state_audit.csv"

)


# --- CONTINUE branch ---

print(f" Continue branch (seed={continue_seed})")
```

```
result_continue = prod.simulate_future(
copy.deepcopy(split_state),
prefix,
"continue",
np.random.default_rng(continue_seed),
)

# --- FIX: Check split state immutability ---
immutability_ok = check_split_state_immutability(
split_state, split_snapshot, N
)
immutability_by_seed[seed_str] = immutability_ok

if not immutability_ok:
raise RuntimeError(
f"Split state was mutated for seed={seed_str}"
)

# --- Frozen anchor validation (MANDATORY) ---
```

```
if verify_anchors:
# Validate carrier_reset__B_preserved against frozen B_carrier_only
anchor_A = validate_frozen_anchor(
result_carrier_preserved, frozen_data, seed_str,
"B_carrier_only", matrix_keys
)
anchor_A["production_branch"] = "B_carrier_only"
anchor_A["matched_branch"] = "carrier_reset__B_preserved"
anchor_validation_rows.append(anchor_A)

# Validate full_reset__B_zero against frozen full_reset
anchor_B = validate_frozen_anchor(
result_full_zero, frozen_data, seed_str,
"full_reset", matrix_keys
)
anchor_B["production_branch"] = "full_reset"
anchor_B["matched_branch"] = "full_reset__B_zero"
```

```
anchor_validation_rows.append(anchor_B)


# Validate continue against frozen continue
anchor_C = validate_frozen_anchor(
result_continue, frozen_data, seed_str,
"continue", matrix_keys
)
anchor_C["production_branch"] = "continue"
anchor_C["matched_branch"] = "continue"
anchor_validation_rows.append(anchor_C)


# MANDATORY: Stop on any anchor failure
seed_anchors = [r for r in anchor_validation_rows if r["seed"] == seed_str]
if any(not r.get("pass", True) for r in seed_anchors):
raise RuntimeError(
f"Anchor validation FAILED for seed={seed_str}. "
f"Check frozen_anchor_validation.csv for details."
)
```

```
# --- FIX: Store raw branch results for NPZ serialization ---

branch_results = {

"carrier_reset__B_preserved": result_carrier_preserved,

"carrier_reset__B_zero": result_carrier_zero,

"full_reset__B_preserved": result_full_preserved,

"full_reset__B_zero": result_full_zero,

"continue": result_continue,

}


# --- Compute outcomes ---

outcomes: dict[str, dict[str, Any]] = {}


for branch_id in BRANCH_IDS:

result = branch_results[branch_id]

meta = BRANCH_META[branch_id]


# Compute recovery outcomes - FIX: Pass dt explicitly

if branch_id == "continue":
```

```
# FIX: For continue, use its own result as reference
branch_outcomes = compute_recovery_outcomes(
result, result, seed_str, branch_id, protocol, dt
)
else:
branch_outcomes = compute_recovery_outcomes(
result, result_continue, seed_str, branch_id, protocol, dt
)

# Add metadata
branch_outcomes.update({
"seed": seed_str,
"branch": branch_id,
"reset_source": meta["reset_source"],
"B_status": meta["B_status"],
"production_intervention_label": meta["production_intervention_label"],
"frozen_anchor_branch": meta["frozen_anchor_branch"],
"reset_seed_uint64": meta["reset_source"] == "carrier_reset" and carrier_reset_seed
```

```
or meta["reset_source"] == "full_reset" and full_reset_seed

or continue_seed,

})


outcomes[branch_id] = branch_outcomes


# --- Compute contrasts ---

paired = compute_paired_contrasts(outcomes)

seed_level = compute_seed_level_contrasts(paired)


# Commit seed results (only after all validation passes)

seed_results: dict[str, Any] = {

"branches": branch_results, # FIX: Store raw results

"outcomes": outcomes,

"paired_contrasts": paired,

"seed_level_contrasts": seed_level,

}


all_results[seed_str] = seed_results
```

```
all_contrasts[seed_str] = paired

all_seed_contrasts[seed_str] = seed_level


seed_status[seed_str] = "success"

print(f" Status: SUCCESS")


except Exception as exc:

seed_status[seed_str] = f"failed: {type(exc).__name__}: {exc}"

print(f" Status: FAILED - {type(exc).__name__}: {exc}")

import traceback

traceback.print_exc()

# FIX: Collect exception instead of immediate re-raise

if fatal_exception is None:

fatal_exception = exc

# Continue to next seed to collect all failures

# But for production, we want to stop on first failure

# So we'll break after collecting this one

break
```

```
# --- Check for failures ---

failed_seeds = [s for s, status in seed_status.items() if status != "success"]

initial_match_failed = any(not r.get("pass", False) for r in audit_rows)

anchor_failed = any(not r.get("pass", True) for r in anchor_validation_rows) if anchor_validation_rows else False



if failed_seeds:

print(f"\nFAILED seeds: {failed_seeds}")

if initial_match_failed:

print(f"Initial matching FAILED for some cases")

if anchor_failed:

print(f"Anchor validation FAILED for some cases")



# FIX: Write validation outputs and failure manifest if there was a fatal exception

if fatal_exception is not None:

# Write validation-only outputs

if audit_rows:
```

```
audit_df = pd.DataFrame(audit_rows)

atomic_write_csv(output_dir / "matched_initial_state_audit.csv", audit_df)

if anchor_validation_rows:

anchor_df = pd.DataFrame(anchor_validation_rows)

atomic_write_csv(output_dir / "frozen_anchor_validation.csv", anchor_df)


# Write failure manifest

failure_manifest = {

"status": "FAILED",

"error": str(fatal_exception),

"initial_match_failed": initial_match_failed,

"anchor_failed": anchor_failed,

"failed_seeds": failed_seeds,

"seeds_requested": seeds,

"seed_status": seed_status,

"run_scope": "partial",

"full_cohort": False,
```

```
"freezeable": False,

}

atomic_write_json(output_dir / "analysis_manifest.json", failure_manifest)


raise RuntimeError(

"Matched-phase run failed validation; scientific outputs were not produced"

) from fatal_exception



# --- Save outputs ---



# 1. Scalar summary - include ALL production scalars

scalar_rows = []

for seed_str, results in all_results.items():

for branch_id, branch_outcomes in results["outcomes"].items():

row = dict(branch_outcomes) # Includes all production scalars + derived

scalar_rows.append(row)



if scalar_rows:
```

```
scalar_df = pd.DataFrame(scalar_rows)

atomic_write_csv(output_dir / "matched_phase_scalar_summary.csv", scalar_df)

print(f" Saved: matched_phase_scalar_summary.csv")



# 2. Paired contrasts

contrast_rows = []

for seed_str, paired in all_contrasts.items():

for reset_source, outcomes in paired.items():

for outcome, contrast in outcomes.items():

contrast_rows.append({

"seed": seed_str,

"reset_source": reset_source,

"outcome": outcome,

"contrast": contrast,

})



if contrast_rows:
```

```
contrast_df = pd.DataFrame(contrast_rows)

atomic_write_csv(output_dir / "matched_phase_paired_contrasts.csv", contrast_df)

print(f" Saved: matched_phase_paired_contrasts.csv")



# 3. Seed-level contrasts

seed_contrast_rows = []

for seed_str, seed_level in all_seed_contrasts.items():

for outcome, value in seed_level.items():

seed_contrast_rows.append({

"seed": seed_str,

"outcome": outcome,

"value": value,

})



if seed_contrast_rows:

seed_contrast_df = pd.DataFrame(seed_contrast_rows)

atomic_write_csv(output_dir / "matched_phase_seed_level_contrasts.csv", seed_contrast_df)
```

```
print(f" Saved: matched_phase_seed_level_contrasts.csv")



# 4. Recovery summary

recovery_outcomes = [

"sustained_similarity_90_rest",

"sustained_density_90_rest",

"held_similarity_90_for_1Twindow",

"sustained_similarity_90_recovered",

"sustained_density_90_recovered",

"held_similarity_90_for_1Twindow_recovered",

"time_to_similarity_90",

"time_to_density_90",

"restricted_time_to_similarity_90",

"restricted_time_to_density_90",

"AUC_similarity_deficit_vs_continue",

"Mean_similarity_deficit_vs_continue",

"AUC_density_deficit_vs_continue",
```

```
] + list(prod.BRANCH_SCALAR_KEYS)



recovery_rows = []

for seed_str, results in all_results.items():

for branch_id, branch_outcomes in results["outcomes"].items():

row = {

"seed": seed_str,

"branch": branch_id,

"reset_source": branch_outcomes.get("reset_source"),

"B_status": branch_outcomes.get("B_status"),

}

for key in recovery_outcomes:

if key in branch_outcomes:

row[key] = branch_outcomes[key]

recovery_rows.append(row)



if recovery_rows:
```

```
recovery_df = pd.DataFrame(recovery_rows)

atomic_write_csv(output_dir / "matched_phase_recovery_summary.csv", recovery_df)

print(f" Saved: matched_phase_recovery_summary.csv")



# 5. Initial state audit

if audit_rows:

audit_df = pd.DataFrame(audit_rows)

atomic_write_csv(output_dir / "matched_initial_state_audit.csv", audit_df)

print(f" Saved: matched_initial_state_audit.csv")



# 6. Frozen anchor validation

if anchor_validation_rows:

anchor_df = pd.DataFrame(anchor_validation_rows)

atomic_write_csv(output_dir / "frozen_anchor_validation.csv", anchor_df)

print(f" Saved: frozen_anchor_validation.csv")



# 7. Branch diagnostics NPZ - FIX: Use stored branch_results
```

```
npz_output = {}

for seed_str, results in all_results.items():

for branch_id, branch_result in results["branches"].items():

if isinstance(branch_result, dict):

for key, value in branch_result.items():

if isinstance(value, np.ndarray):

npz_key = f"{branch_id}_seed_{seed_str}_{key}"

npz_output[npz_key] = value



if npz_output:

atomic_write_npz(output_dir / "matched_phase_branch_diagnostics.npz", **npz_output)

print(f" Saved: matched_phase_branch_diagnostics.npz")



# FIX: 8. Add seed-status output

seed_status_rows = []

for seed_str, status in seed_status.items():

row = {"seed": seed_str, "status": status}
```

```
# Add reset seeds

if seed_str in all_results:

results = all_results[seed_str]

if "branches" in results:

for branch_id, branch_result in results["branches"].items():

meta = BRANCH_META.get(branch_id, {})

if meta.get("reset_source") == "carrier_reset":

row["carrier_reset_seed"] = meta.get("reset_seed_uint64")

elif meta.get("reset_source") == "full_reset":

row["full_reset_seed"] = meta.get("reset_seed_uint64")

elif meta.get("reset_source") == "continue":

row["continue_seed"] = meta.get("reset_seed_uint64")


# Add validation results

row["initial_match_pass"] = any(

r.get("seed") == seed_str and r.get("pass", False)

for r in audit_rows
```

```
) if audit_rows else False

row["anchor_pass"] = any(

r.get("seed") == seed_str and r.get("pass", True)

for r in anchor_validation_rows

) if anchor_validation_rows else True

row["split_immutable"] = immutability_by_seed.get(seed_str, False)


seed_status_rows.append(row)



if seed_status_rows:

seed_status_df = pd.DataFrame(seed_status_rows)

atomic_write_csv(output_dir / "matched_phase_seed_status.csv", seed_status_df)

print(f" Saved: matched_phase_seed_status.csv")



# --- Compute aggregate statistics ---



# FIX: Only use CONTINUOUS_SUMMARY_OUTCOMES for continuous bootstrap
```

```
continuous_bootstrap_stats: dict[str, dict[str, Any]] = {}

for outcome in CONTINUOUS_SUMMARY_OUTCOMES:

# Only compute for avg_* outcomes

avg_key = f"avg_{outcome}"

values = []

for seed_str, seed_level in all_seed_contrasts.items():

if avg_key in seed_level and not np.isnan(seed_level[avg_key]):

values.append(seed_level[avg_key])


if len(values) &gt;= 2:

mean_val, ci_lower, ci_upper = bootstrap_ci(

np.array(values), bootstrap_repetitions, bootstrap_seed

)

n = len(values)

values_arr = np.array(values)

n_pos = int(np.sum(values_arr &gt; 0))

n_zero = int(np.sum(values_arr == 0))

n_neg = int(np.sum(values_arr &lt; 0))
```

```
continuous_bootstrap_stats[outcome] = {

"mean": mean_val,

"ci_lower": ci_lower,

"ci_upper": ci_upper,

"median": float(np.median(values)),

"std": float(np.std(values, ddof=1)), # Sample SD

"n": n,

"n_positive": n_pos,

"n_zero": n_zero,

"n_negative": n_neg,

}



# For binary outcomes: recovery probability per reset_source and B_status

binary_stats: dict[str, dict[str, Any]] = {}

for outcome in BINARY_OUTCOMES:

# Count per reset_source and B_status

counts: dict[str, dict[str, dict[str, int]]] = {
```

"carrier_reset": {"preserved": {"success": 0, "total": 0}, "zero": {"success": 0, "total": 0}},

"full_reset": {"preserved": {"success": 0, "total": 0}, "zero": {"success": 0, "total": 0}},

}

for seed_str, results in all_results.items():

for branch_id, branch_outcomes in results["outcomes"].items():

reset_source = branch_outcomes.get("reset_source")

b_status = branch_outcomes.get("B_status")

if reset_source not in ["carrier_reset", "full_reset"]:

continue

if b_status not in ["preserved", "zero"]:

continue

if outcome in branch_outcomes:

val = branch_outcomes[outcome]

FIX: Include all seeds in denominator, check for truth value

counts[reset_source][b_status]["total"] += 1

```
if val:

counts[reset_source][b_status]["success"] += 1


binary_stats[outcome] = {}

for reset_source in ["carrier_reset", "full_reset"]:

for b_status in ["preserved", "zero"]:

s = counts[reset_source][b_status]["success"]

t = counts[reset_source][b_status]["total"]

key_base = f"{reset_source}_{b_status}"

if t &gt; 0:

p, ci_l, ci_u = wilson_ci(s, t)

binary_stats[outcome][f"{key_base}_probability"] = p

binary_stats[outcome][f"{key_base}_ci_lower"] = ci_l

binary_stats[outcome][f"{key_base}_ci_upper"] = ci_u

binary_stats[outcome][f"{key_base}_count"] = t

else:

binary_stats[outcome][f"{key_base}_probability"] = np.nan

binary_stats[outcome][f"{key_base}_ci_lower"] = np.nan
```

```
binary_stats[outcome][f"{key_base}_ci_upper"] = np.nan

binary_stats[outcome][f"{key_base}_count"] = 0


# Discordant counts for this reset_source

discordant = {"both": 0, "only_preserved": 0, "only_zero": 0, "neither": 0}

for seed_str, results in all_results.items():

pres_val = results["outcomes"].get(f"{reset_source}__B_preserved", {}).get(outcome)

zero_val = results["outcomes"].get(f"{reset_source}__B_zero", {}).get(outcome)


# FIX: Include all seeds, handle None/NaN properly

if pres_val is None or zero_val is None:

continue


if pres_val and zero_val:

discordant["both"] += 1

elif pres_val and not zero_val:

discordant["only_preserved"] += 1

elif not pres_val and zero_val:

discordant["only_zero"] += 1
```

```
else:

discordant["neither"] += 1


for k, v in discordant.items():

binary_stats[outcome][f"{reset_source}_discordant_{k}"] = v


# FIX: Paired probability difference (McNemar-style)

n_only_pres = discordant["only_preserved"]

n_only_zero = discordant["only_zero"]

n_total = discordant["both"] + n_only_pres + n_only_zero + discordant["neither"]

if n_total &gt; 0:

paired_diff = (n_only_pres - n_only_zero) / n_total

binary_stats[outcome][f"{reset_source}_paired_probability_diff"] = paired_diff

else:

binary_stats[outcome][f"{reset_source}_paired_probability_diff"] = np.nan



# For censored time outcomes: recovery probability and restricted mean

censored_stats: dict[str, dict[str, Any]] = {}
```

```
# FIX: Use correct horizon - end of observation from continue branch

# Get horizon from first successful seed's continue result

horizon = None

for seed_str, results in all_results.items():

if "continue" in results["branches"]:

cont_result = results["branches"]["continue"]

if "sample_times" in cont_result and len(cont_result["sample_times"]) &gt; 0:

horizon = float(cont_result["sample_times"][-1])

break



if horizon is None:

# Fallback to protocol value

horizon = float(protocol.get("recovery_end", 100)) * dt



for outcome in CENSORED_TIME_OUTCOMES:

censored_stats[outcome] = {}
```

```
for reset_source in ["carrier_reset", "full_reset"]:

for b_status in ["preserved", "zero"]:

finite_counts = 0

total_counts = 0

restricted_times: list[float] = []


for seed_str, results in all_results.items():

branch_id = f"{reset_source}__B_{b_status}"

if branch_id in results["outcomes"]:

val = results["outcomes"][branch_id].get(outcome)

# FIX: Include ALL seeds in denominator

total_counts += 1

if val is not None and not np.isnan(val) and np.isfinite(val):

finite_counts += 1

restricted_times.append(min(float(val), horizon))

else:

# Unrecovered: use horizon
```

restricted_times.append(horizon)

key_base = f"{reset_source}_{b_status}"

if total_counts > 0:

censored_stats[outcome][f"{key_base}_recovery_probability"] = finite_counts / total_counts

else:

censored_stats[outcome][f"{key_base}_recovery_probability"] = np.nan

Restricted Mean Time (not RMST - no KM estimator)

if restricted_times:

censored_stats[outcome][f"{key_base}_restricted_mean_time"] = float(np.mean(restricted_times))

else:

censored_stats[outcome][f"{key_base}_restricted_mean_time"] = np.nan

FIX: Seed-level two-block binary contrast

Bootstrap seed-level binary contrasts across 40 seeds

binary_contrast_stats: dict[str, dict[str, Any]] = {}

for outcome in BINARY_OUTCOMES:

```
# Get seed-level average contrasts for binary outcomes

values = []

for seed_str, seed_level in all_seed_contrasts.items():

avg_key = f"avg_{outcome}"

if avg_key in seed_level and not np.isnan(seed_level[avg_key]):

values.append(seed_level[avg_key])


if len(values) &gt;= 2:

mean_val, ci_lower, ci_upper = bootstrap_ci(

np.array(values), bootstrap_repetitions, bootstrap_seed

)

n = len(values)

values_arr = np.array(values)

n_pos = int(np.sum(values_arr &gt; 0))

n_zero = int(np.sum(values_arr == 0))

n_neg = int(np.sum(values_arr &lt; 0))


binary_contrast_stats[outcome] = {
```

```
"mean": mean_val,

"ci_lower": ci_lower,

"ci_upper": ci_upper,

"median": float(np.median(values)),

"std": float(np.std(values, ddof=1)),

"n": n,

"n_positive": n_pos,

"n_zero": n_zero,

"n_negative": n_neg,

}



# --- FIX: Summarize reset-realization interaction ---

interaction_stats: dict[str, dict[str, Any]] = {}

for outcome in CONTINUOUS_SUMMARY_OUTCOMES:

interaction_key = f"interaction_{outcome}"

values = []

for seed_str, seed_level in all_seed_contrasts.items():
```

if interaction_key in seed_level and not np.isnan(seed_level[interaction_key]):

values.append(seed_level[interaction_key])

if len(values) >= 2:

mean_val, ci_lower, ci_upper = bootstrap_ci(

np.array(values), bootstrap_repetitions, bootstrap_seed

)

values_arr = np.array(values)

n_pos = int(np.sum(values_arr > 0))

n_zero = int(np.sum(values_arr == 0))

n_neg = int(np.sum(values_arr < 0))

interaction_stats[outcome] = {

"mean": mean_val,

"ci_lower": ci_lower,

"ci_upper": ci_upper,

"median": float(np.median(values)),

"std": float(np.std(values, ddof=1)),

```
"n": len(values),
"n_positive": n_pos,
"n_zero": n_zero,
"n_negative": n_neg,
}


# --- Generate report ---
report_lines = [
"=" * 70,
"AIEF-1.4 Matched Phase Memory Ablation",
"=" * 70,
"",
"PURPOSE:",
"Correct the unmatched phase/history randomization in the original",
"B_carrier_only vs full_reset comparison by constructing matched",
"phase/history reset blocks.",
"",
```

```
"SCIENTIFIC ESTIMAND:",

"For each frozen seed/graph, estimate the effect of preserving mature B,",

"conditional on the same randomized phase vector, history deque,",

"zeroed complex EMA state, omega, and future dynamics.",

"",

"-" * 70,

"RUN SUMMARY",

"-" * 70,

f"Total seeds: {len(seeds)}",

f"Seeds succeeded: {len([s for s in seed_status.values() if s == 'success'])}",

f"Seeds failed: {len(failed_seeds)}",

f"Initial matching passed: {len([r for r in audit_rows if r.get('pass', False)])}/{len(audit_rows) if
audit_rows else 0}",

f"Anchor validation passed: {len([r for r in anchor_validation_rows if r.get('pass',
True)])}/{len(anchor_validation_rows) if anchor_validation_rows else 0}",

f"Censoring horizon: {horizon:.4f}",

f"Run scope: {'full_cohort' if full_cohort else 'partial'}",

f"Freezeable: {not bool(failed_seeds) and verify_anchors and full_cohort}",
```

```
"",
]


# Continuous outcomes with bootstrap CI
if continuous_bootstrap_stats:
report_lines.extend([
"-" * 70,
"CONTINUOUS OUTCOMES - Seed-level Average Contrasts",
"Delta_bar_s(Y) = 0.5 * [Delta_{s,carrier}(Y) + Delta_{s,full}(Y)]",
"with bootstrap 95% CI",
"-" * 70,
"",
])
for outcome, s in continuous_bootstrap_stats.items():
report_lines.append(f" {outcome}:")
report_lines.append(
f" mean = {s['mean']:.6g} "
```

```
f"[CI: {s['ci_lower']:.6g}, {s['ci_upper']:.6g}]"

)

report_lines.append(

f" median = {s['median']:.6g}, std = {s['std']:.6g}"

)

report_lines.append(

f" n = {s['n']}, + = {s['n_positive']}, "

f"0 = {s['n_zero']}, - = {s['n_negative']}"

)

report_lines.append("")



# Binary outcomes - per reset source

if binary_stats:

report_lines.extend([

"-" * 70,

"BINARY OUTCOMES - Recovery Probability by Reset Source",

"-" * 70,
```

```
"",
])
for outcome, s in binary_stats.items():
report_lines.append(f" {outcome}:")
for reset_source in ["carrier_reset", "full_reset"]:
report_lines.append(f" Reset source: {reset_source}")
for b_status in ["preserved", "zero"]:
p_key = f"{reset_source}_{b_status}_probability"
ci_l_key = f"{reset_source}_{b_status}_ci_lower"
ci_u_key = f"{reset_source}_{b_status}_ci_upper"
n_key = f"{reset_source}_{b_status}_count"
p = s.get(p_key, np.nan)
ci_l = s.get(ci_l_key, np.nan)
ci_u = s.get(ci_u_key, np.nan)
n = s.get(n_key, 0)
if not pd.isna(p):
report_lines.append(
f" B={b_status}: p={p:.4f} [CI: {ci_l:.4f}, {ci_u:.4f}], n={n}"
```

```
)

# Discordant counts

for k in ["both", "only_preserved", "only_zero", "neither"]:

v = s.get(f"{reset_source}_discordant_{k}", 0)

report_lines.append(f" Discordant {k}: {v}")

# Paired probability difference

p_diff = s.get(f"{reset_source}_paired_probability_diff", np.nan)

if not pd.isna(p_diff):

report_lines.append(

f" Paired probability difference: {p_diff:.4f}"

)

report_lines.append("")



# Seed-level two-block binary contrast

if binary_contrast_stats:

report_lines.extend([

"-" * 70,
```

```
"SEED-LEVEL TWO-BLOCK BINARY CONTRAST",

"Delta^binary_s = 0.5 * [(I^B_{s,carrier} - I^0_{s,carrier}) + (I^B_{s,full} - I^0_{s,full})]",

"Bootstrap CI across 40 seeds (values in {-1, -1/2, 0, 1/2, 1})",

"-" * 70,

"",

])

for outcome, s in binary_contrast_stats.items():

report_lines.append(f" {outcome}:")

report_lines.append(

f" mean = {s['mean']:.6g} "

f"[CI: {s['ci_lower']:.6g}, {s['ci_upper']:.6g}]"

)

report_lines.append(

f" median = {s['median']:.6g}, std = {s['std']:.6g}"

)

report_lines.append(

f" n = {s['n']}, + = {s['n_positive']}, "
```

```
f"0 = {s['n_zero']}, - = {s['n_negative']}"

)

report_lines.append("")



# Censored time outcomes - per reset source

if censored_stats:

report_lines.extend([

"-" * 70,

"CENSORED TIME OUTCOMES - Recovery over Horizon",

f"Horizon: {horizon:.4f} (from continue branch sample_times[-1])",

"All seeds included in denominators; unrecovered = horizon",

"-" * 70,

"",

])

for outcome, s in censored_stats.items():

report_lines.append(f" {outcome}:")

for reset_source in ["carrier_reset", "full_reset"]:
```

```
report_lines.append(f" Reset source: {reset_source}")

for b_status in ["preserved", "zero"]:

p_key = f"{reset_source}_{b_status}_recovery_probability"

rmst_key = f"{reset_source}_{b_status}_restricted_mean_time"

p = s.get(p_key, np.nan)

rmst = s.get(rmst_key, np.nan)

if not pd.isna(p):

report_lines.append(

f" B={b_status}: recovery_prob={p:.4f}, restricted_mean_time={rmst:.4f}"

)

report_lines.append("")



# Reset-realization interaction summary

if interaction_stats:

report_lines.extend([

"-" * 70,

"RESET-REALIZATION INTERACTION - I_s(Y) = Delta_{s,carrier} - Delta_{s,full}",
```

```
"Quantifies sensitivity of B effect to reset realization",

"-" * 70,

"",

])

for outcome, s in interaction_stats.items():

report_lines.append(f" {outcome}:")

report_lines.append(

f" mean = {s['mean']:.6g} "

f"[CI: {s['ci_lower']:.6g}, {s['ci_upper']:.6g}]"

)

report_lines.append(

f" median = {s['median']:.6g}, std = {s['std']:.6g}"

)

report_lines.append(

f" n = {s['n']}, + = {s['n_positive']}, "

f"0 = {s['n_zero']}, - = {s['n_negative']}"

)

report_lines.append("")
```

```
report_lines.extend([
"-" * 70,
"INTERPRETATION CONSTRAINTS",
"-" * 70,
"",
"The experiment estimates the conditional effect of preserving mature B",
"under matched randomized phase/history and reset EMA.",
"",
"This is a controlled counterfactual, NOT a test that B is the",
"unique memory carrier.",
"",
"The seed/graph is the inferential unit (40 seeds).",
"The 80 reset-block contrasts are NOT independent observations.",
"",
])
```

```
report = "\n".join(report_lines)

atomic_write_text(output_dir / "matched_phase_report.txt", report)

print(f" Saved: matched_phase_report.txt")



# --- Build manifest ---

# FIX: Use stored immutability_by_seed, not reconstruct

manifest_output = {

"analysis": "AIEF-1.4 matched phase memory ablation",

"version": "1.0",

"source": {

"path": str(Path(__file__).resolve()),

"sha256": file_sha256(Path(__file__).resolve()),

},

"frozen_authorities": {

"manifest": {

"path": str(Path(manifest_path).resolve()),

"sha256": file_sha256(manifest_path),
```

},

"npz": {

"path": str(Path(npz_path).resolve()),

"sha256": file_sha256(npz_path),

},

"scalar_csv": {

"path": str(Path(scalar_path).resolve()) if os.path.exists(scalar_path) else None,

"sha256": scalar_sha256,

"note": "Hashed for provenance but not used computationally",

},

},

"dependencies": {

"AIEF_1_4_memory_ablation_updated": {

"path": str(Path(prod.__file__).resolve()),

"sha256": compute_module_sha256(prod),

},

"AIEF_1_4_trajectory_recovery_equivalence": {

"path": str(Path(recovery.__file__).resolve()),

"sha256": compute_module_sha256(recovery),

},

},

"environment": {

"python_version": sys.version,

"numpy_version": np.__version__,

},

"model_constants": {

"N": N,

"dt": dt,

"T_window": T_window,

"alpha": math.exp(-dt / T_window),

"scale_epsilon": 1e-12,

},

"branch_order": list(branch_order),

"branch_rng_algorithm": (

"_, branch_root = np.random.SeedSequence(seed).spawn(2); "

"branch_sequences = branch_root.spawn(len(branch_order)); "

"branch_rng_seeds = {branch: int(sequence.generate_state(1, dtype=np.uint64)[0]) "

"for branch, sequence in zip(branch_order, branch_sequences)}"

),

"matched_reset_block_design": (

"Reset block A (carrier_reset seed from B_carrier_only): "

"carrier_reset__B_preserved (reproduces B_carrier_only), "

"carrier_reset__B_zero (new, B=0). "

"Reset block B (full_reset seed from full_reset): "

"full_reset__B_preserved (new, B=Bsplit), "

"full_reset__B_zero (reproduces full_reset). "

"Continue branch: reproduces continue."

),

"run_metadata": {

"run_scope": "full_cohort" if full_cohort else "partial",

"requested_seed_count": len(seeds),

"all_seeds_count": len(all_seeds),

```
"full_cohort": full_cohort,

},

"seeds": {

"all": all_seeds,

"requested": seeds,

"status": seed_status,

},

"validation": {

"initial_match_pass_count": len([r for r in audit_rows if r.get("pass", False)]),

"initial_match_total_count": len(audit_rows),

"anchor_validation_pass_count": len([r for r in anchor_validation_rows if r.get("pass", True)]),

"anchor_validation_total_count": len(anchor_validation_rows),

"verify_anchors": verify_anchors,

"split_state_immutability": {

"all_passed": all(immutability_by_seed.values()) if immutability_by_seed else True,

"by_seed": immutability_by_seed,

},

},
```

"censoring": {

"horizon": horizon,

"source": "sample_times[-1] from continue branch (first seed with data)",

},

"recovery_definitions": {

"similarity_threshold": "0.9 * continue_Weighted_similarity_late",

"density_threshold": "0.9 * continue_D_B_final",

"deficit_calculation": "np.maximum(0, continue_value - value)",

"binary_recovery": "np.isfinite(time_to_event)",

"restricted_mean_time": "mean of (min(time, horizon) for all seeds)",

},

"command": sys.argv,

"success_count": len([s for s in seed_status.values() if s == "success"]),

"failure_count": len(failed_seeds),

"failed_seeds": failed_seeds,

"freezeable": not bool(failed_seeds) and verify_anchors and full_cohort,

"outputs": [

"matched_phase_scalar_summary.csv",

"matched_phase_paired_contrasts.csv",

"matched_phase_seed_level_contrasts.csv",

"matched_phase_recovery_summary.csv",

"matched_initial_state_audit.csv",

"frozen_anchor_validation.csv",

"matched_phase_branch_diagnostics.npz",

"matched_phase_report.txt",

"analysis_manifest.json",

"matched_phase_seed_status.csv",

],

"interpretation_constraints": [

"The experiment estimates the conditional effect of preserving mature B under matched randomized phase/history and reset EMA.",

"This is a controlled counterfactual, not a test that B is the unique memory carrier.",

"The seed/graph is the inferential unit (40 seeds).",

"The 80 reset-block contrasts are not independent observations.",

],

```
}



atomic_write_json(output_dir / "analysis_manifest.json", manifest_output)

print(f" Saved: analysis_manifest.json")



return {

"success_count": len([s for s in seed_status.values() if s == "success"]),

"failure_count": len(failed_seeds),

"failed_seeds": failed_seeds,

"initial_match_failed": initial_match_failed,

"anchor_failed": anchor_failed,

"has_fatal_failure": bool(fatal_exception),

"full_cohort": full_cohort,

}

================================================================================
Self-Test
================================================================================

def run_self_test() -> dict[str, Any]:
```

```
text

"""Run self-test without frozen inputs."""

print("Running self-test...")



results: dict[str, Any] = {}



# FIX: Self-test REQUIRES production and recovery modules

if not PROD_AVAILABLE:

print(f" ERROR: Production module not available: {PROD_ERROR}")

results["all_passed"] = False

return results



if not RECOVERY_AVAILABLE:

print(f" ERROR: Recovery module not available: {RECOVERY_ERROR}")

results["all_passed"] = False

return results
```

```
# Test 1: Same reset seed produces equal theta/history/EMA

N = 12

np.random.seed(42)

mock_theta = np.random.randn(N)

mock_B = np.random.rand(N, N)

mock_B = 0.5 * (mock_B + mock_B.T)

np.fill_diagonal(mock_B, 0.0)

mock_coherence = (np.random.randn(N, N) + 1j * np.random.randn(N, N))

mock_history = deque([np.random.randn(N) for _ in range(10)])



# CORRECTED: Construct DynamicState properly

mock_state = prod.DynamicState(

theta=mock_theta.copy(),

B=mock_B.copy(),

coherence_acc=mock_coherence.copy(),

history=copy.deepcopy(mock_history),

)
```

```
test_seed = 12345


# Apply same intervention with same seed
state1 = copy.deepcopy(mock_state)
state2 = copy.deepcopy(mock_state)
rng1 = np.random.default_rng(test_seed)
rng2 = np.random.default_rng(test_seed)

prod.apply_intervention(state1, "full_reset", rng1)
prod.apply_intervention(state2, "full_reset", rng2)

test1 = (
np.array_equal(state1.theta, state2.theta) and
np.array_equal(state1.coherence_acc, state2.coherence_acc) and
len(state1.history) == len(state2.history) and
```

```
all(np.array_equal(h1, h2) for h1, h2 in zip(state1.history, state2.history))

)

results["test1_same_seed_equals"] = test1

print(f" Test 1 (same seed -&gt; same state): {'PASS' if test1 else 'FAIL'}")



# Test 2: Only B differs between preserved and zero

state_p = copy.deepcopy(mock_state)

state_z = copy.deepcopy(mock_state)

rng_p = np.random.default_rng(test_seed)

rng_z = np.random.default_rng(test_seed)



prod.apply_intervention(state_p, "B_carrier_only", rng_p)

prod.apply_intervention(state_z, "full_reset", rng_z)



test2 = (

np.array_equal(state_p.theta, state_z.theta) and

np.array_equal(state_p.coherence_acc, state_z.coherence_acc) and
```

```
np.array_equal(state_p.coherence_acc, np.zeros_like(state_p.coherence_acc)) and

np.array_equal(state_z.coherence_acc, np.zeros_like(state_z.coherence_acc)) and

not np.array_equal(state_p.B, state_z.B) and

np.array_equal(state_z.B, np.zeros((N, N)))

)

results["test2_only_B_differs"] = test2

print(f" Test 2 (only B differs): {'PASS' if test2 else 'FAIL'}")



# Test 3: History equality between preserved and zero

test3 = (

len(state_p.history) == len(state_z.history) and

all(np.array_equal(hp, hz) for hp, hz in zip(state_p.history, state_z.history))

)

results["test3_history_equals"] = test3

print(f" Test 3 (history equality): {'PASS' if test3 else 'FAIL'}")



# Test 4: RNG state equality after intervention
```

```
state_p2 = copy.deepcopy(mock_state)

state_z2 = copy.deepcopy(mock_state)

rng_p2 = np.random.default_rng(test_seed)

rng_z2 = np.random.default_rng(test_seed)



prod.apply_intervention(state_p2, "B_carrier_only", rng_p2)

prod.apply_intervention(state_z2, "full_reset", rng_z2)



rng_state_equal = (rng_p2.bit_generator.state == rng_z2.bit_generator.state)

results["test4_rng_state_equals"] = rng_state_equal

print(f" Test 4 (RNG state equality): {'PASS' if rng_state_equal else 'FAIL'}")



# Test 5: Array hashing - FIXED

arr = np.array([1.0, 2.0, 3.0])

h1 = array_sha256(arr)

h2 = array_sha256(arr)

h3 = array_sha256(np.array([1.0, 2.0, 4.0]))
```

```
test5 = (h1 == h2) and (h1 != h3)

results["test5_array_hashing"] = test5

print(f" Test 5 (array hashing): {'PASS' if test5 else 'FAIL'}")



# Test 6: Coherence hash changes with dtype

arr_float = np.array([1.0, 2.0, 3.0])

arr_complex = np.array([1.0+0j, 2.0+0j, 3.0+0j])

h_float = array_sha256(arr_float)

h_complex = array_sha256(arr_complex)

test6 = (h_float != h_complex)

results["test6_dtype_hash_differs"] = test6

print(f" Test 6 (dtype affects hash): {'PASS' if test6 else 'FAIL'}")



# Test 7: Contrast construction - FIX: Use declared outcome

mock_outcomes = {

"carrier_reset__B_preserved": {"P_mean_late": 1.0},

"carrier_reset__B_zero": {"P_mean_late": 0.5},
```

```
}

contrasts = compute_paired_contrasts(mock_outcomes)

test7 = contrasts.get("carrier_reset", {}).get("P_mean_late", np.nan) == 0.5

results["test7_contrast_construction"] = test7

print(f" Test 7 (contrast construction): {'PASS' if test7 else 'FAIL'}")



# Test 8: Seed-level averaging - FIX: Use declared outcome

paired = {

"carrier_reset": {"P_mean_late": 1.0},

"full_reset": {"P_mean_late": 0.6},

}

seed_level = compute_seed_level_contrasts(paired)

test8 = (

abs(seed_level.get("avg_P_mean_late", np.nan) - 0.8) &lt; 1e-10 and

abs(seed_level.get("interaction_P_mean_late", np.nan) - 0.4) &lt; 1e-10

)

results["test8_seed_level_averaging"] = test8
```

```
print(f" Test 8 (seed-level averaging): {'PASS' if test8 else 'FAIL'}")



# Test 9: Atomic writes - FIX: Test all formats

tmp_dir = Path(tempfile.mkdtemp())

try:

# Test CSV

test_df = pd.DataFrame({"a": [1, 2, 3]})

atomic_write_csv(tmp_dir / "test.csv", test_df)

test9_csv = (tmp_dir / "test.csv").exists()


# Test JSON

test_data = {"key": "value"}

atomic_write_json(tmp_dir / "test.json", test_data)

test9_json = (tmp_dir / "test.json").exists()


# Test text

atomic_write_text(tmp_dir / "test.txt", "test content")

test9_text = (tmp_dir / "test.txt").exists()
```

```
# Test NPZ

test_arr = np.array([1, 2, 3])

atomic_write_npz(tmp_dir / "test.npz", test_array=test_arr)

test9_npz = (tmp_dir / "test.npz").exists()


test9 = test9_csv and test9_json and test9_text and test9_npz

results["test9_atomic_writes_all_formats"] = test9

print(f" Test 9 (atomic writes all formats): {'PASS' if test9 else 'FAIL'}")

finally:

shutil.rmtree(tmp_dir, ignore_errors=True)



# Test 10: Bootstrap CI

values = np.random.randn(100)

mean_val, ci_l, ci_u = bootstrap_ci(values, 100, 42)

test10 = (

not np.isnan(mean_val) and

not np.isnan(ci_l) and
```

```
not np.isnan(ci_u) and

ci_l &lt;= mean_val &lt;= ci_u

)

results["test10_bootstrap_ci"] = test10

print(f" Test 10 (bootstrap CI): {'PASS' if test10 else 'FAIL'}")



# Test 11: Wilson CI

p, ci_l, ci_u = wilson_ci(80, 100)

test11 = (

0.0 &lt;= p &lt;= 1.0 and

0.0 &lt;= ci_l &lt;= p &lt;= ci_u &lt;= 1.0

)

results["test11_wilson_ci"] = test11

print(f" Test 11 (Wilson CI): {'PASS' if test11 else 'FAIL'}")



# Test 12: Binary recovery indicator

finite_time = 5.0
```

```
nan_time = np.nan

test12a = bool(np.isfinite(finite_time))

test12b = not bool(np.isfinite(nan_time))

test12 = test12a and test12b

results["test12_binary_recovery_indicator"] = test12

print(f" Test 12 (binary recovery indicator): {'PASS' if test12 else 'FAIL'}")



# Test 13: Censored seed in denominator

total = 3

finite = 2

recovery_prob = finite / total

test13 = abs(recovery_prob - 2/3) &lt; 1e-10

results["test13_censored_in_denominator"] = test13

print(f" Test 13 (censored in denominator): {'PASS' if test13 else 'FAIL'}")



# Test 14: Split state immutability - FIX: Actually test mutation detection

state_before = copy.deepcopy(mock_state)
```

```
snapshot = create_split_snapshot(state_before)



# Unchanged should pass

unchanged_pass = check_split_state_immutability(state_before, snapshot, N)



# Mutate and check detection

state_mutated = copy.deepcopy(mock_state)

state_mutated.B[0, 1] += 1e-6

state_mutated.B[1, 0] += 1e-6

mutation_detected = not check_split_state_immutability(state_mutated, snapshot, N)



test14 = unchanged_pass and mutation_detected

results["test14_split_state_immutability"] = test14

print(f" Test 14 (split state immutability): {'PASS' if test14 else 'FAIL'}")



# Test 15: NPZ serialization schema

# Create synthetic all_results structure and test serialization
```

```
try:
synthetic_results = {
"seed_0": {
"branches": {
"carrier_reset__B_preserved": {
"sample_times": np.array([0.0, 1.0, 2.0]),
"B_final": np.array([[0.1, 0.2], [0.2, 0.1]]),
},
"carrier_reset__B_zero": {
"sample_times": np.array([0.0, 1.0, 2.0]),
"B_final": np.array([[0.0, 0.0], [0.0, 0.0]]),
},
}
}
}

# Test the same NPZ-flattening loop
npz_test_output = {}
```

```
for seed_str, results in synthetic_results.items():

for branch_id, branch_result in results["branches"].items():

if isinstance(branch_result, dict):

for key, value in branch_result.items():

if isinstance(value, np.ndarray):

npz_key = f"{branch_id}_seed_{seed_str}_{key}"

npz_test_output[npz_key] = value


# Verify expected keys exist

test15 = (

"carrier_reset__B_preserved_seed_seed_0_sample_times" in npz_test_output and

"carrier_reset__B_preserved_seed_seed_0_B_final" in npz_test_output

)

results["test15_npz_serialization_schema"] = test15

print(f" Test 15 (NPZ serialization schema): {'PASS' if test15 else 'FAIL'}")

except Exception as e:

results["test15_npz_serialization_schema"] = False

print(f" Test 15 (NPZ serialization schema): FAIL - {e}")
```

```
all_pass = all(results.values())

print(f"\nSelf-test: {'ALL PASSED' if all_pass else 'SOME FAILED'}")

results["all_passed"] = all_pass



return results

# ============================================================================
# CLI
# ============================================================================

def parse_arguments() -> argparse.Namespace:

text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

description="AIEF-1.4 Matched Phase Memory Ablation"

)



# Required inputs
```

```
parser.add_argument(
"--npz",
default=DEFAULT_NPZ,
help="Frozen NPZ path (default: %(default)s)",
)
parser.add_argument(
"--manifest",
default=DEFAULT_MANIFEST,
help="Frozen manifest path (default: %(default)s)",
)
parser.add_argument(
"--scalar",
default=DEFAULT_SCALAR,
help="Frozen scalar CSV path (default: %(default)s)",
)


# Output
```

```
parser.add_argument(
"--output-dir",
default=DEFAULT_OUTPUT_DIR,
help="Output directory (default: %(default)s)",
)
parser.add_argument(
"--overwrite",
action="store_true",
default=False,
help="Overwrite existing outputs (default: fail if outputs exist)",
)



# Seed control
parser.add_argument(
"--max-seeds",
type=parse_nonnegative_int,
default=None,
```

```
help="Max seeds to process (default: all)",
)
parser.add_argument(
"--seed-filter",
type=parse_seed_list,
default=None,
help="Comma-separated seed filter (default: all)",
)


# Bootstrap
parser.add_argument(
"--bootstrap-repetitions",
type=parse_positive_int,
default=1000,
help="Bootstrap repetitions (default: %(default)s)",
)
parser.add_argument(
```

```
"--bootstrap-seed",
type=int,
default=42,
help="Bootstrap RNG seed (default: %(default)s)",
)


# Options
parser.add_argument(
"--verify-anchors",
action="store_true",
default=True,
help="Verify frozen anchors (default: on)",
)
parser.add_argument(
"--no-verify-anchors",
action="store_false",
dest="verify_anchors",
```

```
help="Skip anchor verification (debug only)",

)

parser.add_argument(

"--self-test",

action="store_true",

help="Run self-test and exit",

)



return parser.parse_args()

def main() -> None:

text

"""Main entry point."""

args = parse_arguments()



# Self-test mode - REQUIRES production and recovery modules

if args.self_test:

test_results = run_self_test()
```

```
if not test_results.get("all_passed", False):

print("Self-test FAILED")

sys.exit(1)

print("Self-test PASSED")

sys.exit(0)



# Check production module

if not PROD_AVAILABLE:

print(f"ERROR: Production module not available: {PROD_ERROR}")

sys.exit(1)



# Check recovery module

if not RECOVERY_AVAILABLE:

print(f"ERROR: Recovery module not available: {RECOVERY_ERROR}")

sys.exit(1)



# Validate inputs
```

```
for path in [args.npz, args.manifest]:

if not os.path.exists(path):

print(f"ERROR: Input file not found: {path}")

sys.exit(1)



if not os.path.exists(args.scalar):

print(f"WARNING: Scalar CSV not found: {args.scalar}")



output_dir = Path(args.output_dir)



# Run analysis

print("Starting matched phase memory ablation analysis...")

try:

results = run_matched_phase_ablation(

args.npz,

args.manifest,

args.scalar,
```

```
output_dir,

seed_filter=args.seed_filter,

max_seeds=args.max_seeds,

bootstrap_repetitions=args.bootstrap_repetitions,

bootstrap_seed=args.bootstrap_seed,

verify_anchors=args.verify_anchors,

overwrite=args.overwrite,

)

except RuntimeError as e:

print(f"\nFATAL ERROR: {e}")

sys.exit(1)



# Final report

print("\n" + "=" * 70)

print("ANALYSIS COMPLETE")

print("=" * 70)

print(f"Seeds succeeded: {results['success_count']}")
```

print(f"Seeds failed: {results['failure_count']}")

if results['failed_seeds']:

print(f"Failed seeds: {results['failed_seeds']}")

print(f"Initial matching failed: {results['initial_match_failed']}")

print(f"Anchor validation failed: {results['anchor_failed']}")

if results.get('has_fatal_failure'):

print("WARNING: Fatal validation failures detected. Scientific outputs may be invalid.")

print(f"Full cohort: {results.get('full_cohort', False)}")

print(f"Freezeable: {not results.get('has_fatal_failure', True) and args.verify_anchors and results.get('full_cohort', False)}")

print(f"\nOutput directory: {output_dir.resolve()}")

if name == "main":

text

main()

This is the strongest version so far. The major design, API, censoring, output, and validation problems have been corrected. It is now genuinely close to a one-seed smoke test.

One issue will probably cause the smoke test to fail even when replay is correct: the full-reset similarity trajectory contains matching NaN values, but the anchor comparison does not treat aligned NaNs as equal.
1. Mandatory anchor correction: compare NaNs by position

The current trajectory comparison uses:

Python

```
np.allclose(
frozen_arr,
new_arr,
rtol=1e-14,
atol=1e-14,
)
```

with the default:

Python

```
equal_nan=False
```

The full_reset branch has an exactly zero BB vector early, making cosine similarity undefined. Both the frozen and replayed trajectories can therefore contain NaN at exactly the same positions.

np.allclose will return false even when those trajectories are otherwise identical.

Use a NaN-aware exact comparison:

Python

```
def exact_array_equal_with_nan(
left: np.ndarray,
right: np.ndarray,
) -> bool:
left = np.asarray(left)
right = np.asarray(right)

if left.shape != right.shape:
return False

if left.dtype != right.dtype:
return False
```

```
if (
np.issubdtype(
left.dtype,
np.floating,
)
or np.issubdtype(
left.dtype,
np.complexfloating,
)
):
left_nan = np.isnan(left)
right_nan = np.isnan(right)

if not np.array_equal(
left_nan,
right_nan,
):
return False

finite = ~left_nan

return np.array_equal(
left[finite],
right[finite],
)

return np.array_equal(
left,
right,
)
```

Then use it for every frozen matrix/trajectory key.

Since the same production source, split state, intervention seed, and future dynamics are being replayed, exact equality of finite entries is the correct anchor criterion. Do not weaken the historical anchor to an arbitrary tolerance.

If a tolerance comparison is retained as a secondary diagnostic, call the result:

text

tolerance_match

not:

text

exact

Current matrix-pattern classification is unreliable

This condition:

Python

```
pattern in key_lower
for pattern in [
"_final",
"_matrix",
"incidence",
"coherence",
"B_",
]
```

contains uppercase "B_" while key_lower is lowercase, so that pattern never matches. B_mean_late therefore receives a tolerance comparison.

The simplest correction is to remove key-name classification entirely and use the NaN-aware exact helper for all prod.BRANCH_MATRIX_KEYS.
2. Model validation should cover all frozen constants

The current production comparison is real now, but incomplete. It checks:

NN;
dtdt;

TwindowTwindow;
ωω;
possibly αα.

Add direct comparisons for:

text

tau
eta
mu
gamma
a
q
delta
B_c
B_init_scale
future_steps
window_steps
recovery_end
diagnostic_stride
delay_steps
capacity

The production module exposes:

Python

prod.TAU
prod.ETA
prod.MU
prod.GAMMA
prod.A
prod.Q
prod.DELTA
prod.B_C
prod.B_INIT_SCALE
prod.FUTURE_STEPS
prod.WINDOW_STEPS
prod.RECOVERY_END

prod.DIAGNOSTIC_STRIDE
prod.DELAY_STEPS
prod.CAP
prod.ZERO_TOL

The fixed safeguard is prod.ZERO_TOL, not necessarily an attribute named SCALE_EPSILON.

Avoid fallback-to-frozen behavior such as:

Python

```
getattr(prod, "N_NODES", frozen_N)
```

for required constants. If a required production constant is absent, validation should fail rather than silently comparing the frozen value to itself.
3. Seed-status reset seeds are currently written as None

This block uses:

Python

```
meta.get("reset_seed_uint64")
```

but BRANCH_META contains no reset-seed values.

The actual reset seeds were added to each branch’s outcomes. Read them from there:

Python

```
outcomes = results["outcomes"]
```

```
row["carrier_reset_seed"] = (
outcomes[
"carrier_reset__B_preserved"
]["reset_seed_uint64"]
)
```

row["full_reset_seed"] = (
outcomes[
"full_reset__B_zero"
]["reset_seed_uint64"]
)

row["continue_seed"] = (
outcomes["continue"][
"reset_seed_uint64"
]
)

Also change:

Python

initial_match_pass = any(...)
anchor_pass = any(...)

to require all expected rows:

Python

seed_audits = [
row
for row in audit_rows
if row["seed"] == seed_str
]

initial_match_pass = (
len(seed_audits) == 2
and all(
row["pass"]
for row in seed_audits
)
)

seed_anchors = [

```
row
for row in anchor_validation_rows
if row["seed"] == seed_str
]

anchor_pass = (
not verify_anchors
or (
len(seed_anchors) == 3
and all(
row["pass"]
for row in seed_anchors
)
)
)
```

The current any formulation could mark a seed passed if only one of two reset blocks or one of three anchors passed. Runtime failures already stop this, but the saved status should be intrinsically correct.
4. Mean similarity deficit should match the validated convention

The existing trajectory-recovery analysis defines mean deficit as:
∫deficit(t) dttmax−tmin.
tmax−tmin∫deficit(t)dt.

The current code uses a sample mean:

Python

```
np.mean(similarity_deficit)
```

Because sampling is regular, these are close but not identical.

Use:

Python

```
auc_similarity_deficit = (
trapezoid_integral(
```

similarity_deficit,
sample_times,
)
)

duration = float(
sample_times[-1]
- sample_times[0]
)

outcomes[
"AUC_similarity_deficit_vs_continue"
] = auc_similarity_deficit

outcomes[
"Mean_similarity_deficit_vs_continue"
] = (
auc_similarity_deficit / duration
if duration > 0.0
else np.nan
)

This preserves direct comparability with the frozen recovery analysis.
5. The reset-seed metadata expression is unnecessarily fragile

This expression:

Python

meta["reset_source"] == "carrier_reset"
and carrier_reset_seed
or meta["reset_source"] == "full_reset"
and full_reset_seed
or continue_seed

fails if a generated seed is zero because zero is falsey.

Use explicit branching:

Python

```
if meta["reset_source"] == "carrier_reset":
reset_seed_value = carrier_reset_seed
elif meta["reset_source"] == "full_reset":
reset_seed_value = full_reset_seed
else:
reset_seed_value = continue_seed
```

Then store reset_seed_value.
6. Self-test NPZ key uses a duplicated seed_ prefix

The synthetic seed is:

Python

```
"seed_0"
```

and the serializer adds:

Python

```
"_seed_{seed_str}_"
```

producing:

text

```
..._seed_seed_0_...
```

The test expects that duplicated form, so it passes without checking the production schema.

Use:

Python

synthetic_results = {
"0": {
...
}
}

and expect:

text

carrier_reset__B_preserved_seed_0_B_final

Also read the NPZ back after writing it, rather than merely checking file existence.
7. Failure diagnostic flow is now correct

The fatal_exception approach now:

records the failing seed;
stops further simulation;
writes audit outputs;
writes a failure manifest;
does not write scientific summaries;
re-raises a production failure.

That is appropriate.

One refinement: record unprocessed seeds separately in the failure manifest:

text

failed_seeds
unprocessed_seeds
successful_before_failure

Do not describe unprocessed seeds as failures.
8. Full-cohort freezeability is now correct

The code now distinguishes:

text

full_cohort
partial

and requires:

full cohort;
no failures;
anchor verification.

That is the correct policy.

Also verify uniqueness:

Python

if len(set(all_seeds)) != len(all_seeds):
raise RuntimeError(
"duplicate seeds in frozen manifest"
)

9. Recovery and inferential structure are now sound

The important corrections are present:

continuous outcomes separated from binary outcomes;
explicit binary recovery indicators;
block-specific probabilities;
block-specific discordant counts;
two-block seed-level binary contrast;
all seeds included in censoring denominators;
observation-end censoring horizon;
restricted-time outcomes;

reset-realization interaction;
seed-level bootstrap.

This now matches the intended inferential design.

The remaining raw time contrasts in CONTRAST_OUTCOMES are acceptable as diagnostic complete-case values because the restricted-time contrasts are now included and used as primary continuous summaries. Label the raw time contrasts accordingly in the CSV schema or manifest.
10. One small reporting issue

The report line:

text

Bootstrap CI across 40 seeds

will be wrong for a smoke or filtered run.

Use:

Python

f"Bootstrap CI across {len(seeds)} seeds"

The manifest correctly marks partial runs nonfreezeable, but report text should also adapt.
11. Self-test should include NaN-aware anchor equality

Given the likely full-reset trajectory behavior, add:

Python

a = np.array(
[1.0, np.nan, 2.0]
)
b = np.array(
[1.0, np.nan, 2.0]
)

```
c = np.array(
[1.0, np.nan, 2.1]
)

assert exact_array_equal_with_nan(a, b)
assert not exact_array_equal_with_nan(a, c)
```

This directly guards the most likely smoke-test failure.
Overall assessment
Experimental design: 9.5/10

The two historical reset blocks and exact anchors are excellent.
Production API use: 9/10

Correct and faithful.
Exact validation architecture: 9/10

Strong, after the NaN-aware anchor adjustment.
Recovery and censoring: 9/10

Now aligned with the validated conventions.
Reproducibility: 8.5/10

Good atomic writes, manifests, hashes, full-cohort freezeability, and failure handling.
Current executable version: 8/10

The main remaining likely failure is NaN handling in frozen trajectory anchors. Everything else is now in conventional cleanup territory.
Next gate

Make the NaN-aware exact anchor correction first. Then stop static editing and run:

Bash

```
python -m py_compile \
AIEF_1_4_matched_phase_memory_ablation.py
```

python AIEF_1_4_matched_phase_memory_ablation.py \
--self-test

If both pass, run one seed in a fresh directory:

Bash

python AIEF_1_4_matched_phase_memory_ablation.py \
--max-seeds 1 \
--output-dir \
aief_1_4_matched_phase_memory_ablation_smoke

Required smoke result:

text

2/2 initial matching blocks pass
3/3 frozen anchors pass
split-state immutability passes
1/1 seed succeeds
run_scope = partial
freezeable = false

If the smoke test passes, the script is ready for the full 40-seed matched-phase correction.

==========

AIEF-1.4 Matched Phase Memory Ablation

==========

PURPOSE:

Correct the unmatched phase/history randomization in the original

B_carrier_only vs full_reset comparison by constructing matched

phase/history reset blocks.

SCIENTIFIC ESTIMAND:

For each frozen seed/graph, estimate the effect of preserving mature B,

conditional on the same randomized phase vector, history deque,

zeroed complex EMA state, omega, and future dynamics.

RUN SUMMARY

Total seeds: 40

Seeds succeeded: 40

Seeds failed: 0

Initial matching passed: 80/80

Anchor validation passed: 120/120

Censoring horizon: 319.5100

Run scope: full_cohort

Freezeable: True

CONTINUOUS OUTCOMES - Seed-level Average Contrasts

Delta_bar_s(Y) = 0.5 * [Delta_{s,carrier}(Y) + Delta_{s,full}(Y)]

with bootstrap 95% CI

P_mean_late:

text

mean = -0.0176095 [CI: -0.0267873, -0.00882897]

median = -0.0226069, std = 0.029556

n = 40, + = 12, 0 = 3, - = 25

Weighted_similarity_late:

text

mean = 0.0641537 [CI: 0.0405751, 0.0889752]

median = 0.0533283, std = 0.0780827

n = 40, + = 31, 0 = 0, - = 9

Weighted_similarity_final:

text

mean = 0.0638799 [CI: 0.0402338, 0.0887457]

median = 0.0533071, std = 0.0786846

n = 40, + = 31, 0 = 0, - = 9

Retention_selected_late:

text

mean = 0.00728708 [CI: -0.0168658, 0.0316646]

median = 0.000610839, std = 0.0761301

n = 40, + = 24, 0 = 5, - = 11

Emergence_unselected_late:

text

mean = -0.0270618 [CI: -0.03626, -0.0178676]

median = -0.0269769, std = 0.0321918

n = 40, + = 6, 0 = 11, - = 23

D_B_final:

text

mean = -0.0179924 [CI: -0.0274621, -0.00890152]

median = -0.0227273, std = 0.030578

n = 40, + = 7, 0 = 8, - = 25

Total_edge_weight:

text

mean = -0.0829795 [CI: -0.13534, -0.0293594]

median = -0.158333, std = 0.184053

n = 40, + = 17, 0 = 0, - = 23

Effective_edges:

text

mean = -0.826419 [CI: -1.45916, -0.206972]

median = -1.17301, std = 2.04911

n = 40, + = 15, 0 = 0, - = 25

Nonzero_edge_fraction:

text

mean = -0.0179924 [CI: -0.0272727, -0.00946496]

median = -0.0227273, std = 0.029798

n = 40, + = 7, 0 = 8, - = 25

Minimum_similarity:

text

mean = 0.100776 [CI: 0.080042, 0.119867]

median = 0.11923, std = 0.0653776

n = 40, + = 37, 0 = 0, - = 3

AUC_similarity_deficit_vs_continue:

text

mean = -31.7407 [CI: -38.8781, -25.0103]

median = -28.8697, std = 22.7023

n = 40, + = 2, 0 = 0, - = 38

Mean_similarity_deficit_vs_continue:

text

mean = -0.0993451 [CI: -0.121684, -0.0782796]

median = -0.090359, std = 0.0710556

n = 40, + = 2, 0 = 0, - = 38

AUC_density_deficit_vs_continue:

text

mean = -0.296733 [CI: -1.64105, 1.0472]

median = -1.13636, std = 4.3163

n = 40, + = 14, 0 = 0, - = 26

restricted_time_to_similarity_90:

text

mean = -154.412 [CI: -188.927, -121.143]

median = -176, std = 108.565

n = 40, + = 0, 0 = 0, - = 40

restricted_time_to_density_90:

text

mean = -86.8438 [CI: -117.125, -61.6853]

median = -50.25, std = 88.6504

n = 40, + = 0, 0 = 0, - = 40

BINARY OUTCOMES - Recovery Probability by Reset Source

sustained_similarity_90_recovered:

text

Reset source: carrier_reset

B=preserved: p=0.8500 [CI: 0.7093, 0.9294], n=40

B=zero: p=0.6000 [CI: 0.4460, 0.7365], n=40

Discordant both: 22

Discordant only_preserved: 12

Discordant only_zero: 2

Discordant neither: 4

Paired probability difference: 0.2500

Reset source: full_reset

B=preserved: p=0.8500 [CI: 0.7093, 0.9294], n=40

B=zero: p=0.6250 [CI: 0.4703, 0.7578], n=40

Discordant both: 24

Discordant only_preserved: 10

Discordant only_zero: 1

Discordant neither: 5

Paired probability difference: 0.2250

sustained_density_90_recovered:

text

Reset source: carrier_reset

B=preserved: p=0.6750 [CI: 0.5202, 0.7992], n=40

B=zero: p=0.7250 [CI: 0.5717, 0.8389], n=40

Discordant both: 24

Discordant only_preserved: 3

Discordant only_zero: 5

Discordant neither: 8

Paired probability difference: -0.0500

Reset source: full_reset

B=preserved: p=0.7250 [CI: 0.5717, 0.8389], n=40

B=zero: p=0.8250 [CI: 0.6805, 0.9125], n=40

Discordant both: 29

Discordant only_preserved: 0

Discordant only_zero: 4

Discordant neither: 7

Paired probability difference: -0.1000

held_similarity_90_for_1Twindow_recovered:

text

Reset source: carrier_reset

B=preserved: p=0.8500 [CI: 0.7093, 0.9294], n=40

B=zero: p=0.6000 [CI: 0.4460, 0.7365], n=40

Discordant both: 22

Discordant only_preserved: 12

Discordant only_zero: 2

Discordant neither: 4

Paired probability difference: 0.2500

Reset source: full_reset

B=preserved: p=0.8750 [CI: 0.7389, 0.9454], n=40

B=zero: p=0.6250 [CI: 0.4703, 0.7578], n=40

Discordant both: 25

Discordant only_preserved: 10

Discordant only_zero: 0

Discordant neither: 5

Paired probability difference: 0.2500

SEED-LEVEL TWO-BLOCK BINARY CONTRAST

Delta^binary_s = 0.5 * [(I^B_{s,carrier} - I^0_{s,carrier}) + (I^B_{s,full} - I^0_{s,full})]

Bootstrap CI across 40 seeds (values in {-1, -1/2, 0, 1/2, 1})

sustained_similarity_90_recovered:

text

mean = 0.2375 [CI: 0.1125, 0.375]

median = 0, std = 0.408052

n = 40, + = 17, 0 = 20, - = 3

sustained_density_90_recovered:

text

mean = -0.075 [CI: -0.15, 0]

median = 0, std = 0.241523

n = 40, + = 2, 0 = 30, - = 8

held_similarity_90_for_1Twindow_recovered:

text

mean = 0.25 [CI: 0.125, 0.375312]

median = 0, std = 0.392232

n = 40, + = 17, 0 = 21, - = 2

CENSORED TIME OUTCOMES - Recovery over Horizon

Horizon: 319.5100 (from continue branch sample_times[-1])

All seeds included in denominators; unrecovered = horizon

time_to_similarity_90:

text

Reset source: carrier_reset

B=preserved: recovery_prob=1.0000, restricted_mean_time=0.0100

B=zero: recovery_prob=0.6000, restricted_mean_time=157.9350

Reset source: full_reset

B=preserved: recovery_prob=1.0000, restricted_mean_time=0.0100

B=zero: recovery_prob=0.6250, restricted_mean_time=150.9100

time_to_density_90:

text

Reset source: carrier_reset

B=preserved: recovery_prob=1.0000, restricted_mean_time=0.0100

B=zero: recovery_prob=0.8250, restricted_mean_time=83.5350

Reset source: full_reset

B=preserved: recovery_prob=1.0000, restricted_mean_time=0.0100

B=zero: recovery_prob=0.8500, restricted_mean_time=90.1725

RESET-REALIZATION INTERACTION - I_s(Y) = Delta_{s,carrier} - Delta_{s,full}

Quantifies sensitivity of B effect to reset realization

P_mean_late:

text

mean = -0.00259013 [CI: -0.0174614, 0.0116628]

median = 0, std = 0.0474496

n = 40, + = 18, 0 = 4, - = 18

Weighted_similarity_late:

text

mean = 0.00905696 [CI: -0.0221536, 0.0432124]

median = 6.55796e-06, std = 0.104895

n = 40, + = 20, 0 = 0, - = 20

Weighted_similarity_final:

text

mean = 0.00970767 [CI: -0.0220446, 0.0451179]

median = 2.01813e-10, std = 0.106514

n = 40, + = 20, 0 = 0, - = 20

Retention_selected_late:

text

mean = -0.00906109 [CI: -0.0549588, 0.037099]

median = 0.000201471, std = 0.147952

n = 40, + = 20, 0 = 5, - = 15

Emergence_unselected_late:

text

mean = -0.000269277 [CI: -0.0124115, 0.0136845]

median = 0, std = 0.0419482

n = 40, + = 10, 0 = 18, - = 12

D_B_final:

text

mean = -0.00189394 [CI: -0.0170549, 0.0128883]

median = 0, std = 0.0492686

n = 40, + = 14, 0 = 10, - = 16

Total_edge_weight:

text

mean = -0.0398742 [CI: -0.117244, 0.0354086]

median = -2.1973e-08, std = 0.248204

n = 40, + = 16, 0 = 0, - = 24

Effective_edges:

text

mean = -0.0497241 [CI: -1.01703, 0.938136]

median = -1.50363e-08, std = 3.16736

n = 40, + = 19, 0 = 0, - = 21

Nonzero_edge_fraction:

text

mean = -0.00189394 [CI: -0.0170455, 0.0128788]

median = 0, std = 0.0487883

n = 40, + = 14, 0 = 10, - = 16

Minimum_similarity:

text

mean = 0.0163262 [CI: -0.00631165, 0.0403827]

median = -0.00166276, std = 0.0720205

n = 40, + = 19, 0 = 0, - = 21

AUC_similarity_deficit_vs_continue:

text

mean = -2.76462 [CI: -11.8987, 5.81503]

median = 0.371016, std = 29.0681

n = 40, + = 21, 0 = 0, - = 19

Mean_similarity_deficit_vs_continue:

text

mean = -0.00865296 [CI: -0.0372418, 0.0182004]

median = 0.00116124, std = 0.09098

n = 40, + = 21, 0 = 0, - = 19

AUC_density_deficit_vs_continue:

text

mean = 1.89214 [CI: -1.30471, 5.01771]

median = 0.734848, std = 10.3772

n = 40, + = 24, 0 = 0, - = 16

restricted_time_to_similarity_90:

text

mean = -7.025 [CI: -54.0197, 40.7959]

median = 0, std = 155.361

n = 40, + = 15, 0 = 9, - = 16

restricted_time_to_density_90:

text

mean = 6.6375 [CI: -29.2631, 42.6031]

median = 0.5, std = 119.575

n = 40, + = 23, 0 = 7, - = 10

INTERPRETATION CONSTRAINTS

The experiment estimates the conditional effect of preserving mature B

under matched randomized phase/history and reset EMA.

This is a controlled counterfactual, NOT a test that B is the

unique memory carrier.

The seed/graph is the inferential unit (40 seeds).

The 80 reset-block contrasts are NOT independent observations.

This successfully closes the matched-phase correction. The validation gates are unusually strong:

40/40 seeds completed;
80/80 matched initial-state audits passed;
120/120 historical anchors passed;
full cohort;
freezeable output.

The two original reset realizations also give nearly identical estimates of the BB effect, so the result is not being driven by one favorable phase/history reset.
Primary result

Under matched randomized phase/history and zero EMA, preserving mature BB provides a clear conditional advantage in recovery of the original continuous-weight pattern.
Continuous-weight similarity
ΔSimilaritylate=
ΔSimilaritylate=

0.06415
]

with bootstrap interval
[0.04058, 0.08898].
[0.04058, 0.08898].

Likewise,

ΔSimilarityfinal=
ΔSimilarityfinal=

0.06388
]

with interval
[0.04023, 0.08875].
[0.04023, 0.08875].

The effect is positive in 31/40 seeds.

The minimum future similarity also improves:
ΔMinimum similarity=
ΔMinimum similarity=

0.10078
]

with interval
[0.08004, 0.11987].
[0.08004, 0.11987].

So mature BB acts as a robust scaffold for retaining or recovering the mature weight arrangement after the other state channels are reset.
Sustained similarity recovery

The average two-block effect on sustained 90%90% similarity recovery is:
\boxed{ \Delta p_{\rm sustained} =

0.2375
}
]

with bootstrap interval
[0.1125, 0.375].
[0.1125, 0.375].

For the held-for-one-TwindowTwindow criterion:
\boxed{ \Delta p_{\rm held} =

0.25
}
]

with interval
[0.125, 0.3753].
[0.125, 0.3753].

The matched block-specific results are consistent:
reset sourcep(B preserved)p(B=0)paired differencecarrier reset0.8500.6000.250full-reset reset0.8500.6250.225
reset sourcecarrier resetfull-reset resetp(B preserved)0.8500.850p(B=0)0.6000.625paired difference0.2500.225

This is strong evidence for a conditional contribution of mature BB.
The effect is specificity, not increased density

Preserving BB does not produce more topology. It produces a somewhat sparser but more target-aligned result.

Preserving BB decreases:
ΔPmean,late=
ΔPmean,late=

-0.01761,
]
ΔDB,final=
ΔDB,final=

-0.01799,
]
Δtotal weight=
Δtotal weight=

-0.08298,

]
ΔNeff=
ΔNeff=

-0.826.
]

It also decreases emergence of previously unselected edges:
\boxed{ \Delta\text{Emergence} =

-0.02706
}
]

with interval
[−0.03626, −0.01787].
[−0.03626, −0.01787].

Meanwhile, the selected-edge retention contrast is small and uncertain:
ΔRetention=
ΔRetention=

0.00729,
]

with interval crossing zero.

The best interpretation is therefore:

Mature BB biases reconstruction toward the previous continuous-weight organization and suppresses excess edge emergence, rather than increasing overall edge density or total weight.

This is an important separation between:

topology amount;
threshold persistence;
edge identity;
continuous-weight similarity.

Density recovery is not improved

The seed-level two-block effect on sustained density recovery is
−0.075
−0.075

with interval
[−0.15, 0].
[−0.15, 0].

Thus preserving mature BB improves mature-pattern similarity while slightly reducing or not improving recovery of the continue branch’s total density.

This reinforces that density recovery is not a proxy for topology identity.
Recovery-time effects

For similarity recovery, the restricted-time contrast is
−154.4
−154.4

time units, with interval
[−188.9, −121.1].
[−188.9, −121.1].

For density recovery:
−86.8
−86.8

with interval
[−117.1, −61.7].
[−117.1, −61.7].

Negative values mean the BB-preserved branches reach the target earlier or avoid censoring more often.

However, the first-crossing value of 0.010.01 for BB-preserved branches should be interpreted carefully. Those branches begin with mature BB, so they start close to the target by intervention design.

It is more accurate to call this immediate retention plus subsequent recovery, not reacquisition from the same initial distance.

The sustained, held-duration, minimum-similarity, and AUC outcomes are more informative.
Reset-realization interaction is small

For the main continuous outcomes, all reset-interaction intervals include zero:
Is(Y)=
Is(Y)=
\Delta_{s,\rm carrier}

\Delta_{s,\rm full}.
]

For late similarity:
meanI=
meanI=
0.0091,
\qquad
95%\text{ interval}

[-0.0222,\ 0.0432].
]

For final similarity:
0.0097,[−0.0220, 0.0451].
0.0097,[−0.0220, 0.0451].

For late persistence:
−0.0026,[−0.0175, 0.0117].
−0.0026,[−0.0175, 0.0117].

Thus the estimated mature-BB effect is reasonably stable across the two original randomized reset contexts. The original unmatched comparison was not being driven primarily by a lucky choice of phase/history reset.
Relationship to the original memory conclusions

The corrected result gives a nuanced picture:

reset_B with mature phase/history and EMA reconstructs nearly exactly.
When phase/history are randomized and EMA is zeroed, preserving mature BB materially improves recovery of the mature continuous-weight pattern.
Therefore BB is neither:
the unique memory register; nor
irrelevant.
It is a redundant but operationally useful scaffold whose importance increases when the other mature state channels are removed.

This preserves the earlier conclusion that memory is distributed across state channels while resolving the unmatched-reset caveat.
Recommended frozen claim

A manuscript-quality statement would be:

In a 40-seed matched-reset correction, preserving mature BB while holding randomized phase/history and zero EMA identical increased late continuous-weight similarity by 0.0640.064 on average (seed-level bootstrap 95% interval 0.0410.041–0.0890.089) and increased sustained 90%90% similarity recovery probability by 0.2380.238 (interval 0.1130.113–0.3750.375). Preserving BB simultaneously reduced edge emergence and final edge density. Thus mature BB provides a conditional scaffold for reconstructing the prior weight organization, but it is not a unique memory register.

Next steps

Freeze this run and its manifest.
Add the matched-phase result to the analysis lineage.
Replace the former caveat

B_carrier_only and full_reset used unmatched randomized resets
with the matched correction and its quantitative result.

Add a claim distinguishing:
continuous-weight recovery;
density recovery;
selected-edge retention;
emergence suppression.
Do not run another memory-ablation cohort.
Proceed next to the separately versioned, fully regenerated TwindowTwindow sensitivity study.

The matched-phase memory-ablation question is now adequately resolved.

Can you outline the specific SCIENTIFIC ESTIMAND, for the Proceed next to the separately versioned, fully regenerated T_ window sensitivity study for the coding agent?

Is it possible or prudent to outline and instruct this agent to turn the memory-ablation module, or create from scratch a Script that could act as a scaffolding that could easalies be codified to run a multitude of different studies on this data? Or we could just continue to create new scripts for each study?

Yes. This is the right point to introduce reusable scaffolding, but not as one universal “run any study” script.

The prudent architecture is:
validated parameterized simulation engine+separately versioned study drivers
validated parameterized simulation engine+separately versioned study drivers

Do not mutate the frozen memory-ablation module, and do not keep duplicating the entire simulator into every new study.
1. Scientific estimand for the TwindowTwindow study

Let
$T*=25\omega^{-}=17.5$
$T*=\omega^{-}25=17.5$

be the production value, and define the dimensionless ratio
$\rho=\text{Twindow}T*.$
$\rho=T*\text{Twindow}.$

A reasonable predeclared grid is:
$\rho\in\{0.5, 0.75, 1, 1.5, 2\}$
$\rho\in\{0.5, 0.75, 1, 1.5, 2\}$

corresponding to:
$\text{Twindow}\in\{8.75, 13.125, 17.5, 26.25, 35.0\}.$
$\text{Twindow}\in\{8.75, 13.125, 17.5, 26.25, 35.0\}.$

The EMA multiplier must be regenerated as

α(ρ)=
α(ρ)=

\exp\left(
-\frac{dt}{\rho T_*}
\right).
]
Primary estimand A — mature-regime sensitivity

For each paired seed ss, condition ρρ, and outcome YY, define
Δs,ρregime(Y)=
Δs,ρregime(Y)=
Y_{s,\rho,\mathrm{continue}}

Y_{s,1,\mathrm{continue}}.
]

This estimates:

The effect of changing the EMA memory timescale on the mature unablated AIEF regime, holding the initial seed/graph and all non-EMA model parameters fixed.

Primary mature-regime outcomes should be:

text

D_B_final
P_mean_late on fixed physical windows
Total_edge_weight
Effective_edges
Nonzero_edge_fraction
Weight_gini
Weight_entropy
Mean_row_sum
Max_row_sum
Corr_Rpast_Plate
adaptive stationarity diagnostics
support-component composition

Also compare each regenerated continue state with the baseline-ρ=1ρ=1 continue state from the same seed:

text

cross_T_weight_cosine
cross_T_threshold_Jaccard
cross_T_normalized_weight_distance
cross_T_component_signature_match

This asks whether changing TwindowTwindow preserves the same mature organization, not merely the same edge density.
Primary estimand B — sensitivity of core ablation effects

For a predeclared branch bb, define the within-condition branch effect
Cs,ρ,b(Y)=
Cs,ρ,b(Y)=
Y_{s,\rho,b}

Y_{s,\rho,\mathrm{continue}}.
]

Then define the difference-in-differences:
\boxed{ I_{s,\rho,b}(Y) =
C_{s,\rho,b}(Y)

C_{s,1,b}(Y).
}
]

This estimates:

How changing TwindowTwindow changes the effect of a specific memory intervention, beyond any effect it has on the unablated mature regime.

The first study should use only the core branches:

text

continue
reset_ema
reset_B
future_shuffle

These answer distinct questions:

reset_ema: sensitivity of EMA reconstruction and recovery;
reset_B: robustness of weight reconstruction;
future_shuffle: robustness of the edge-identity maintenance requirement.

Do not rerun every historical branch merely because the engine can.
Primary estimand C — recovery scaling in physical and EMA-normalized time

For recovery time trectrec, report both:
trec
trec

in physical time and
trecTwindow
Twindowtrec

in EMA-memory units.

This distinguishes:

recovery slowing merely because the memory timescale is longer;
a genuine change in recovery measured relative to that timescale.

Use:

text

recovery probability
first crossing
sustained 90% recovery

held 90% for one T_window
restricted mean recovery time
AUC deficit versus condition-specific continue

Censoring must include every seed.
2. Regeneration requirement

A changed TwindowTwindow changes the full EMA history from the start. Therefore, for every pair (s,ρ),
(s,ρ),

the script must regenerate:

initialization;
burn-in;
historical window;
split state;
future continuation and intervention branches.

Do not reuse:

text

B_at_split
theta_at_split
history_at_split
coherence_acc_at_split

from the ρ=1ρ=1 run at any other ρρ.

Only these may be reused:

the same 40 seed IDs;
the initial RNG protocol;
fixed model parameters other than TwindowTwindow;
branch-reset RNG identities.

For a given seed, the random initialization must be identical across ρρ. The only intended difference before dynamics begin is TwindowTwindow, hence αα.
3. Fixed versus TwindowTwindow-normalized horizons

Changing TwindowTwindow also changes what “one memory window” means. Do not silently change the outcome window and then attribute the difference to dynamics.

Report two horizon systems.
Primary — fixed physical horizons

Use the baseline AIEF-1.4 physical boundaries for every ρρ:

text

immediate: 0–17.5
recovery: 17.5–87.5
late: 87.5–320

These compare conditions over identical elapsed physical time.
Secondary — EMA-normalized horizons

Also report:

text

immediate: 0–1 T_window
recovery: 1–5 T_window
late: after 5 T_window

provided the interval lies within the common future horizon.

Label every outcome with:

text

horizon_system = fixed_physical | normalized_EMA

Do not combine them in one column.
4. Statistical unit and summaries

The 40 seed/graphs are the inferential units.

For each ρρ, report seed-level paired contrasts against ρ=1ρ=1:

mean;
median;
sample standard deviation;
deterministic seed-level bootstrap interval;
positive/zero/negative counts.

For the branch difference-in-differences, bootstrap:
Is,ρ,b
Is,ρ,b

across the 40 seeds.

Do not treat:

edges;
components;
branches;
parameter levels;
repeated reset contexts

as independent replicates.

Parameter levels are repeated conditions within each seed.
5. Prespecified scientific questions

The coding agent should treat these as the primary questions:
Q1. Regime robustness

Does the sparse mature regime persist over
0.5T∗≤Twindow≤2T∗?
0.5T∗≤Twindow≤2T∗?

Q2. Topology identity

Does each seed reconstruct approximately the same edge-weight organization as at ρ=1ρ=1, or only a similar density?
Q3. EMA recovery scaling

Does reset_ema recovery time scale approximately with TwindowTwindow, so that
trec/Twindow
trec/Twindow

is roughly stable?
Q4. Weight reconstruction robustness

Does reset_B continue to reconstruct the mature topology across EMA timescales?
Q5. Maintenance identity

Does future_shuffle remain destructive across all ρρ, supporting the requirement for future edge-specific driver identity?
6. Secondary mechanism estimands

Only after the primary regime checks pass, estimate:
rEMA(ρ)=1Twindow.
rEMA(ρ)=Twindow1.

Then compare late observed rates:
rΔR,rΔB,rdefect
rΔR,rΔB,rdefect

against rEMArEMA.

This analysis must be conditional on:

adequate signal above the numerical floor;
comparable branch/support geometry;
no unreported winner-map changes.

Do not require the exceptional K3 branch to persist at every ρρ. If it does not, report that as a structural outcome rather than selecting only surviving cases without qualification.

7. Recommended software architecture
Do not modify the frozen production driver

Keep:

text

AIEF_1_4_memory_ablation_updated.py

immutable as the reference implementation.
Create a parameterized engine

A suitable structure is:

text

aief_experiment_engine_v1.py
AIEF_1_4_Twindow_sensitivity.py
AIEF_1_4_Twindow_sensitivity_analysis.py

aief_experiment_engine_v1.py

Contains only reusable mechanics:

Python

```
@dataclass(frozen=True)
class ModelConfig:
N: int
dt: float
tau: float
omega: np.ndarray
a: float
q: float
delta: float
eta: float
mu: float
gamma: float
```

T_window: float
B_c: float
B_init_scale: float
scale_epsilon: float

@property
def alpha(self) -> float:
return math.exp(
-self.dt / self.T_window
)

@property
def capacity(self) -> float:
return (
self.a - self.delta
) / self.q

Also:

text

DynamicState
initialize_state
advance_state
simulate_prefix
apply_intervention
simulate_future
metric helpers
checkpoint serialization
manifest/hash helpers

Every model-changing quantity must come from ModelConfig; do not leave hidden module globals.
Study driver

AIEF_1_4_Twindow_sensitivity.py defines:

the fixed parameter grid;
paired seeds;
branches;

horizon systems;
checkpointing;
output schema;
exact estimand.

Analysis driver

AIEF_1_4_Twindow_sensitivity_analysis.py calculates:

paired contrasts;
difference-in-differences;
recovery/censoring;
bootstrap intervals;
tables and report.

This separation prevents the production simulator from accumulating study-specific statistical logic.
8. Engine validation gate

A reusable engine is prudent only if it exactly reproduces the original production workflow at the baseline configuration.

Before any nonbaseline TwindowTwindow run:
Baseline equivalence test

Set:
Twindow=17.5.
Twindow=17.5.

For all 40 seeds, require exact reproduction of the frozen run for:

text

split theta
split B
split coherence_acc
split history
continue B_final
continue P_late

continue sampled trajectories
all original branch anchors selected for validation

Record exact equality and maximum absolute differences.

If baseline equivalence fails, stop. Do not run the sensitivity grid.
Unit tests

The engine must test:

delay indexing;
EMA update;
symmetrization;
positivity;
global gate;
endpoint scaling;
fixed 10−1210−12 safeguard;
branch intervention matching;
named RNG streams;
state cloning and immutability;
checkpoint resume;
atomic outputs.

9. Generic engine versus one script per study

The best policy is not either extreme.
Avoid one giant universal script

A universal CLI such as:

text

--dt
--epsilon
--T-window
--tau
--branches
--horizons

--model

encourages accidental incompatible comparisons and weakens provenance.
Avoid copying the whole simulator repeatedly

Repeated copies lead to:

drift in update ordering;
inconsistent fixes;
different metric definitions;
difficult baseline validation.

Use a validated shared kernel plus thin versioned drivers

That gives:

code reuse;
exact study-specific estimands;
separate manifests;
explicit regeneration rules;
reduced risk of silent parameter changes.

Every new study should still get a new driver and output namespace, even when it uses the common engine.
10. Limits on generality

The first engine version should parameterize only what is immediately required:

text

T_window

while keeping all other model values frozen.

After baseline and TwindowTwindow validation pass, extend the config for the later timestep/safeguard study in a new engine version.

Do not attempt to anticipate every future experiment in v1.
11. Required outputs for the TwindowTwindow study

At minimum:

text

twindow_condition_manifest.csv
twindow_seed_status.csv
twindow_scalar_summary.csv
twindow_cross_condition_topology.csv
twindow_recovery_summary.csv
twindow_paired_contrasts.csv
twindow_branch_interactions.csv
twindow_branch_diagnostics.npz
twindow_sensitivity_report.txt
analysis_manifest.json

Every row should include:

text

seed
T_window
rho
alpha
branch
horizon_system
run_status

12. Stop conditions

Stop the study if:

baseline engine equivalence fails;
any seed is missing;
initialization differs across ρρ;
a condition reuses a split state from another ρρ;

model constants other than TwindowTwindow differ;
output windows are ambiguous;
a branch or condition fails without an explicit status row;
the fixed safeguard or update ordering changes;
fewer than all 40 seeds complete for a production/freezeable run.

Recommendation

Yes, instruct the agent to build reusable scaffolding now—but narrowly.

The next coding task should be:

Build a parameterized AIEF experiment engine that first reproduces the frozen baseline exactly, then implement a thin fully regenerated TwindowTwindow sensitivity driver with a predeclared five-level grid, fixed physical primary horizons, EMA-normalized secondary horizons, and seed-level paired difference-in-differences.

Continue to use separate study drivers afterward. This gives reusable code without turning the scientific program into an uncontrolled generic parameter sweep.

Evaluate these scripts:

"""AIEF Experiment Engine v1 - Parameterized simulation kernel.

This is a reusable, validated simulation engine that can reproduce the frozen

AIEF-1.4 baseline and support parameterized studies.

Architecture:

ModelConfig: frozen dataclass for all model parameters

DynamicState: state container

Core functions: initialize, advance, simulate, interventions

Validation: baseline equivalence test

Serialization: checkpoint I/O

DO NOT MODIFY: This is the reference implementation.

"""

from future import annotations

import copy

import hashlib

import json

import math

import os

import tempfile

import time

from dataclasses import dataclass, field

from pathlib import Path

from typing import Any

import numpy as np
==
====
Model Configuration (Frozen Dataclass)
==
====

@dataclass(frozen=True)

class ModelConfig:

text

"""Immutable model configuration.

All model-changing quantities must come from here. No hidden module globals.

"""

Core parameters

N: int

dt: float

tau: float

omega: np.ndarray

Coupling parameters

a: float

q: float

delta: float

Dynamics parameters

eta: float

mu: float

gamma: float

EMA parameters

T_window: float

B_c: float

B_init_scale: float

scale_epsilon: float = 1e-12

Derived properties

@property

def alpha(self) -> float:

"""EMA smoothing factor: exp(-dt / T_window)"""

return math.exp(-self.dt / self.T_window)

```
@property

def capacity(self) -&gt; float:

"""Projection capacity: (a - delta) / q"""

return (self.a - [self.delta](http://self.delta)) / self.q



@property

def delay_steps(self) -&gt; int:

"""Delay steps: round(tau / dt)"""

return int(round(self.tau / self.dt))



def validate(self) -&gt; None:

"""Validate model configuration."""

if self.N &lt;= 0:

raise ValueError("N must be positive")

if self.dt &lt;= 0:

raise ValueError("dt must be positive")
```

if self.tau <= 0:

raise ValueError("tau must be positive")

if len(self.omega) != self.N:

raise ValueError(f"omega must have {self.N} elements")

if self.a <= 0 or self.q <= 0 or self.delta < 0:

raise ValueError("a, q must be positive; delta must be nonnegative")

if self.eta < 0 or self.mu < 0 or self.gamma < 0:

raise ValueError("eta, mu, gamma must be nonnegative")

if self.T_window <= 0:

raise ValueError("T_window must be positive")

if self.B_c < 0:

raise ValueError("B_c must be nonnegative")

if self.B_init_scale < 0:

raise ValueError("B_init_scale must be nonnegative")

if self.scale_epsilon <= 0:

raise ValueError("scale_epsilon must be positive")

```
# Validate derived values

if not (0 &lt; self.alpha &lt;= 1):

raise ValueError(f"alpha={self.alpha} must be in (0, 1]")

if self.capacity &lt; 0:

raise ValueError(f"capacity={self.capacity} must be nonnegative")

Default AIEF-1.4 r2 configuration (matches frozen run 5a9f093a2fe0)

DEFAULT_MODEL_CONFIG = ModelConfig(

text

N=12,

dt=0.01,

tau=1.0,

omega=np.array([0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1], dtype=float),

a=1.0,

q=1.5,

delta=0.05,

eta=0.1,

mu=0.01,

gamma=0.5,
```

T_window=1.0,

B_c=1.0,

B_init_scale=0.0005,

scale_epsilon=1e-12,

)
==========
Protocol Configuration
==========

@dataclass(frozen=True)

class ProtocolConfig:

text

"""Protocol configuration for experiment execution."""

Simulation parameters

burn_fraction: float = 0.20

history_end_fraction: float = 0.60

future_end: float | None = None # Optional: end time for future simulation

Analysis windows (in steps, relative to split)

window_steps: int | None = None

recovery_end: int | None = None

Diagnostics

diagnostic_stride: int = 50

Branches to simulate

branches: tuple[str, ...] = ("continue",)

def validate(self, model: ModelConfig) -> None:

"""Validate protocol against model."""

if not (0 < self.burn_fraction < self.history_end_fraction < 1):

raise ValueError("Invalid fraction parameters")

Default protocol for AIEF-1.4 r2

DEFAULT_PROTOCOL_CONFIG = ProtocolConfig(

text

burn_fraction=0.20,

history_end_fraction=0.60,

future_end=None,

window_steps=None,

recovery_end=None,

diagnostic_stride=50,

branches=("continue", "reset_ema", "reset_B", "reset_B_ema", "B_carrier_only", "full_reset", "future_shuffle"),

)
==========
Dynamic State
==========

@dataclass

class DynamicState:

text

"""State container for AIEF-1.4 dynamics."""

theta: np.ndarray

B: np.ndarray

coherence_acc: np.ndarray

history: list[np.ndarray] # Using list for easier serialization than deque

```
def __post_init__(self):

"""Ensure arrays are copies."""

self.theta = np.asarray(self.theta, dtype=float).copy()

self.B = np.asarray(self.B, dtype=float).copy()

self.coherence_acc = np.asarray(self.coherence_acc, dtype=complex).copy()

self.history = [np.asarray(h, dtype=float).copy() for h in self.history]



def clone(self) -&gt; "DynamicState":

"""Create a deep copy of the state."""

return DynamicState(

theta=self.theta.copy(),

B=self.B.copy(),

coherence_acc=self.coherence_acc.copy(),

history=[h.copy() for h in self.history],

)
```

@property

def N(self) -> int:

return len(self.theta)

==========
Helper Functions
==========

def compute_edges(N: int) -> tuple[np.ndarray, np.ndarray]:

text

"""Get upper triangle indices for edges."""

return np.triu_indices(N, k=1)

def validate_B_matrix(B: np.ndarray, N: int, eps: float = 1e-12) -> None:

text

"""Validate B matrix: symmetric, nonnegative, zero diagonal."""

if B.shape != (N, N):

raise ValueError(f"B shape {B.shape} != ({N}, {N})")

if not np.allclose(B, B.T, atol=eps):

raise ValueError("B is not symmetric")

```
if not np.allclose(np.diag(B), 0, atol=eps):

raise ValueError("B diagonal is not zero")

if np.any(B &lt; -eps):

raise ValueError("B has negative entries")

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file."""

digest = hashlib.sha256()

with open(path, "rb") as handle:

for block in iter(lambda: [handle.read](http://handle.read)(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

def array_sha256(arr: np.ndarray) -> str:

text

"""Compute SHA-256 hash of a numpy array."""

digest = hashlib.sha256()

shape_bytes = np.asarray(arr.shape, dtype=np.int64).tobytes()

digest.update(shape_bytes)
```

```
digest.update(arr.dtype.str.encode())

contiguous = np.ascontiguousarray(arr)

if contiguous.size &gt; 0:

digest.update(contiguous.tobytes())

return digest.hexdigest()

def atomic_write_text(path: Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.", suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:
```

```
if os.path.exists(tmp_path):

os.remove(tmp_path)

def atomic_write_json(path: Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def atomic_write_npz(path: Path, **kwargs: Any) -> None:

text

"""Write NPZ atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp.npz")

try:

with os.fdopen(fd, "wb") as handle:

np.savez(handle, **kwargs)

handle.flush()

os.fsync(handle.fileno())
```

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

==
Simulation Core
==

def initialize_state(

text

seed: int,

config: ModelConfig,

) -> tuple[DynamicState, int]:

text

"""Initialize state from seed. Returns (state, steps_completed)."""

N = config.N

Initialize RNG

```
rng = np.random.default_rng(seed)



# Initial theta: uniform in [-pi, pi]

theta = rng.random(N) *2.0* np.pi - np.pi



# Initial B: small random symmetric matrix

B = config.B_init_scale * rng.random((N, N))

B = 0.5 * (B + B.T)

np.fill_diagonal(B, 0.0)



# Initial coherence accumulator

coherence_acc = np.zeros((N, N), dtype=complex)



# Initial history: DELAY_STEPS + 1 copies of theta

delay_steps = config.delay_steps

history = [theta.copy() for *in range(delay*steps + 1)]
```

```
state = DynamicState(

theta=theta,

B=B,

coherence_acc=coherence_acc,

history=history,

)



return state, 0

def compute_R(

text

state: DynamicState,

config: ModelConfig,

) -> np.ndarray:

text

"""Compute coherence matrix R from current state."""

N = state.N

delayed = state.history[0]
```

```
dphi = (

state.theta[:, None]

- delayed[None, :]

- (([config.omega](http://config.omega)[:, None] + [config.omega](http://config.omega)[None, :]) / 2.0)
* config.tau

)

z_inst = np.exp(1j * dphi)



# Update EMA

state.coherence_acc[:] = (

config.alpha *state.coherence_acc + (1.0 - config.alpha)* z_inst

)



# Compute magnitude and symmetrize

result = np.abs(state.coherence_acc)

result = 0.5 * (result + result.T)
```

```
np.fill_diagonal(result, 0.0)



return result

def advance_state(

text

state: DynamicState,

config: ModelConfig,

rng: np.random.Generator,

shuffle_driver: bool = False,

) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:

text

"""Advance state by one time step. Returns (R_physical, R_driver, info)."""

N = state.N

edges = compute_edges(N)



# Store previous B for projection

B_previous = state.B.copy()
```

Compute R matrices

R_physical = compute_R(state, config)

R_driver = R_physical

if shuffle_driver:

values = R_physical[edges].copy()

rng.shuffle(values)

R_driver = np.zeros_like(R_physical)

R_driver[edges] = values

R_driver[(edges[1], edges[0])] = values

Update B

B_unprojected = state.B + config.dt * (

(config.eta + config.mu) * R_driver

- config.mu

- config.gamma * state.B

```
)

B_unprojected = np.maximum(B_unprojected, 0.0)

B_unprojected = 0.5 * (B_unprojected + B_unprojected.T)

np.fill_diagonal(B_unprojected, 0.0)



# Check gap constraint

row_sum_pre = np.sum(B_unprojected, axis=1)

g_min_pre = float(np.min(config.a - config.q * row_sum_pre))

pre_violation = bool(g_min_pre &lt; [config.delta](http://config.delta))



# Project if needed

B_projected = B_unprojected

if pre_violation:

scale = np.minimum(1.0, config.capacity / (row_sum_pre + config.scale_epsilon))

B_projected = B_unprojected * np.minimum.outer(scale, scale)

np.fill_diagonal(B_projected, 0.0)
```

```
# Validate projection

state.B = B_projected

row_sum = np.sum(state.B, axis=1)

g_min = float(np.min(config.a - config.q * row_sum))

if g_min &lt; [config.delta](http://config.delta) - 1e-9:

raise RuntimeError(f"Gap projection failed: g_min={g_min}")



# Update theta

delayed = state.history[0]

coupling = np.sum(

state.B * np.sin(delayed[None, :] - state.theta[:, None]), axis=1

)

state.theta = np.mod(

state.theta + config.dt * ([config.omega](http://config.omega) + coupling),

2.0 * np.pi

)
```

```
# Update history

state.history.append(state.theta.copy())

if len(state.history) &gt; config.delay_steps + 1:

state.history.pop(0)



info = {

"pre_violation": pre_violation,

"g_min_pre": g_min_pre,

"g_min": g_min,

}



return R_physical, R_driver, info

def simulate_prefix(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,
```

) -> tuple[DynamicState, dict[str, Any]]:

text

"""Simulate prefix up to history_end. Returns (state, prefix_dict)."""

state, *= initialize*state(seed, config)

N = config.N

edges = compute_edges(N)

Compute total steps

total_steps = int(round(protocol.history_end_fraction * (protocol.future_end or 800.0) / config.dt))

burn_steps = int(round(protocol.burn_fraction * total_steps))

hist_end = total_steps

Accumulate R for past average

R_past_sum = np.zeros((N, N), dtype=float)

R_past_count = 0

```
for step in range(hist_end):

R_physical, *,* = advance_state(state, config, np.random.default_rng(0), shuffle_driver=False)


if burn_steps &lt;= step &lt; hist_end:

R_past_sum += R_physical

R_past_count += 1



prefix = {

"B_at_split": state.B.copy(),

"R_physical_past": R_past_sum / R_past_count if R_past_count &gt; 0 else np.zeros((N, N)),

"theta_at_split": state.theta.copy(),

"coherence_acc_at_split": state.coherence_acc.copy(),

"history_at_split": np.stack(state.history),

"selected_at_split": (state.B &gt; config.B_c).astype(np.uint8),

}



return state, prefix

def apply_intervention(
```

```
text

state: DynamicState,

branch: str,

config: ModelConfig,

rng: np.random.Generator,

) -> None:

text

"""Apply intervention to state. Mutates in place."""

N = state.N



if branch in {"reset_ema", "reset_B_ema", "B_carrier_only", "full_reset"}:

# Zero coherence accumulator

state.coherence_acc.fill(0.0)



if branch in {"reset_B", "reset_B_ema", "full_reset"}:

# Zero B

state.B.fill(0.0)
```

```
if branch in {"B_carrier_only", "full_reset"}:

# Randomize theta and history

new_theta = rng.random(N) *2.0* np.pi - np.pi

state.theta = new_theta

state.history = [new_theta.copy() for *in range(config.delay*steps + 1)]



# Ensure symmetry

state.B = 0.5 * (state.B + state.B.T)

np.fill_diagonal(state.B, 0.0)

def simulate_future(

text

split_state: DynamicState,

prefix: dict[str, Any],

branch: str,

config: ModelConfig,

protocol: ProtocolConfig,
```

```
rng: np.random.Generator,

) -> dict[str, Any]:

text

"""Simulate future trajectory from split state."""

state = split_state.clone()

apply_intervention(state, branch, config, rng)



N = state.N

edges = compute_edges(N)



# Determine future steps

if protocol.future_end is not None:

future_steps = int(round(protocol.future_end / config.dt))

else:

future_steps = 80000 # Default



# Initialize accumulators
```

```
sample_times = []

density_trajectory = []

similarity_trajectory = []



B_reference = prefix["B_at_split"]



for step in range(future_steps):

R_physical, *,* = advance_state(state, config, rng, shuffle_driver=(branch == "future_shuffle"))


if step % protocol.diagnostic_stride == 0:

t = (step + 1) * config.dt

sample_times.append(t)


# Density: fraction of edges above B_c

density = float(np.count_nonzero(state.B[edges] &gt; config.B_c) / len(edges[0]))

density_trajectory.append(density)


# Similarity: cosine between current and reference B
```

```
B_vec = state.B[edges]

B_ref_vec = B_reference[edges]

norm_B = np.linalg.norm(B_vec)

norm_ref = np.linalg.norm(B_ref_vec)

if norm_B &gt; 0 and norm_ref &gt; 0:

similarity = float([np.dot](http://np.dot)(B_vec, B_ref_vec) / (norm_B * norm_ref))

else:

similarity = float('nan')

similarity_trajectory.append(similarity)



result = {

"B_final": state.B.copy(),

"sample_times": np.array(sample_times, dtype=float),

"density_trajectory": np.array(density_trajectory, dtype=float),

"similarity_trajectory": np.array(similarity_trajectory, dtype=float),

}
```

return result

==========
Baseline Equivalence Test
==========

def baseline_equivalence_test(

text

config: ModelConfig,

protocol: ProtocolConfig,

frozen_manifest_path: str,

frozen_npz_path: str,

tolerance: float = 1e-15,

) -> dict[str, Any]:

text

"""Test that engine reproduces frozen baseline (rho=1, T_window=17.5)."""

import numpy as np

results = {

"status": "passed",

"config": {

"N": config.N,

"dt": config.dt,

"T_window": config.T_window,

"alpha": config.alpha,

},

"equality_checks": {},

"max_diffs": {},

}

Load frozen data

with np.load(frozen_npz_path, allow_pickle=False) as frozen_data:

frozen_files = set(frozen_data.files)

with open(frozen_manifest_path, "r") as f:

frozen_manifest = json.load(f)

```
# Get frozen model params

frozen_model = frozen_manifest["model"]

frozen_protocol = frozen_manifest["protocol"]



# Verify config matches frozen

if frozen_model["N"] != config.N:

results["status"] = "failed"

results["error"] = f"N mismatch: config={config.N}, frozen={frozen_model['N']}"

return results



if not np.allclose(frozen_model["omega"], [config.omega](http://config.omega), rtol=tolerance):

results["status"] = "failed"

results["error"] = "omega mismatch"

return results



# For now, just check that we can load and compare

# Full equivalence would require running the frozen seeds
```

results["note"] = "Full baseline equivalence requires running frozen seeds - implement in study driver"

return results

==
Main API
==

def run_seed(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

) -> dict[str, Any]:

text

"""Run a single seed through the full experiment."""

Simulate prefix

split_state, prefix = simulate_prefix(seed, config, protocol)

```
# Create branch RNGs

branch_rngs = {}

for branch in protocol.branches:

# Deterministic RNG from seed and branch

branch_rngs[branch] = np.random.default_rng(

hash((seed, branch)) % (2**64)

)



# Simulate each branch

branch_results = {}

for branch in protocol.branches:

branch_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,
```

```
branch_rngs[branch],

)



return {

"seed": seed,

"config": {

"N": config.N,

"dt": config.dt,

"T_window": config.T_window,

"alpha": config.alpha,

},

"protocol": {

"burn_fraction": protocol.burn_fraction,

"history_end_fraction": protocol.history_end_fraction,

},

"prefix": prefix,

"branches": branch_results,
```

}

if name == "main":

text

Quick validation

config = DEFAULT_MODEL_CONFIG

config.validate()

print(f"Engine v1 loaded successfully")

print(f" N={config.N}, dt={config.dt}, T_window={config.T_window}")

print(f" alpha={config.alpha:.6f}, capacity={config.capacity:.6f}")

print(f" delay_steps={config.delay_steps}")

Next:

"""AIEF-1.4 T_window Sensitivity Study Driver.

Study design: Regenerate full trajectories for each seed and rho value,

where rho = T_window / T_* and T_* = 25 / mean(omega) = 17.5.

Primary estimands:

A. Regime robustness: Does mature regime persist across rho?

B. Topology identity: Does each seed reconstruct same edge organization?

C. Recovery scaling: Does t_rec / T_window remain stable?

D. Weight reconstruction: Does reset_B reconstruct across EMA timescales?

E. Maintenance identity: Does future_shuffle remain destructive?

DO NOT reuse split states from other rho values. Full regeneration required.

"""

```
from future import annotations

import argparse

import hashlib

import json

import os

import sys

from dataclasses import dataclass, field

from pathlib import Path

from typing import Any

import numpy as np

import pandas as pd

from aief_experiment_engine_v1 import (

text
```

ModelConfig,

ProtocolConfig,

DynamicState,

compute_edges,

file_sha256,

atomic_write_text,

atomic_write_json,

atomic_write_npz,

initialize_state,

advance_state,

compute_R,

apply_intervention,

simulate_prefix,

simulate_future,

baseline_equivalence_test,

)
==========
Study Configuration
==========

T_* = 25 / mean(omega) = 25 / 0.55 = 45.4545...
But the spec says T_* = 25 / mean(omega) = 17.5
This implies mean(omega) = 25 / 17.5 = 1.42857...
However, the default omega has mean = 0.55
So T_* for default omega = 25 / 0.55 = 45.4545...
The spec says T_* = 17.5, so we need omega with mean = 25/17.5 = 1.42857...
But the frozen run uses omega = [0.0, 0.1, ..., 1.1] with mean = 0.55
Let me re-read: "Let T_* = 25/mean(omega) = 17.5"
This means mean(omega) = 25/17.5 = 1.42857...
But the default omega in the engine has mean = 0.55
I think the spec is using a different omega for the T_window study.
For now, let me use the frozen run's omega and compute T_* from it.
For the default omega [0.0, 0.1, ..., 1.1], mean = 0.55
So T_* = 25 / 0.55 = 45.4545...
And rho = T_window / T_* gives T_window = rho * 45.4545...
For rho=1: T_window = 45.4545...
For rho=0.5: T_window = 22.727...
etc.
But the spec says "T_* = 25/mean(omega) = 17.5"
This is inconsistent with the default omega.
I think the spec is defining T_* for a specific omega set.
Let me just use the formula as given and compute T_window from rho.
Actually, re-reading: "Let T_* = 25/mean(omega) = 17.5"
This seems to be stating that for the study, T_* = 17.5
So mean(omega) = 25/17.5 = 1.42857...
But we're using the frozen run's omega which has mean = 0.55
So for the frozen run, T_* = 25/0.55 = 45.4545...
I think the spec is using a placeholder. Let me just define:
T_* = 25 / mean(omega) where omega is from the config
Then rho = T_window / T_*
And T_window = rho T_
Predeclared rho grid

RHO_GRID = (0.5, 0.75, 1.0, 1.5, 2.0)
Primary outcomes for regime robustness

PRIMARY_REGIME_OUTCOMES = (

text

"D_B_final",

"P_mean_late",

"Total_edge_weight",

"Effective_edges",

"Nonzero_edge_fraction",

"Weight_gini",

"Weight_entropy",

"Mean_row_sum",

"Max_row_sum",

"Corr_Rpast_Plate",

)
Cross-T topology outcomes

CROSS_T_OUTCOMES = (

text

"cross_T_weight_cosine",

"cross_T_threshold_Jaccard",

"cross_T_normalized_weight_distance",

"cross_T_component_signature_match",

)

Recovery outcomes

RECOVERY_OUTCOMES = (

text

"recovery_probability",

"first_crossing",

"sustained_90_rest",

"held_90_for_1Twindow",

"restricted_mean_recovery_time",

"AUC_deficit_vs_continue",

)
Branch set for this study

STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
==========
Horizon Definitions
==========

@dataclass(frozen=True)

class HorizonSystem:

text

"""Horizon system definition."""

name: str

immediate_end: float | None

recovery_end: float | None

def get_horizon(self, t: float) -> str:

"""Classify time t into horizon."""

if self.immediate_end is not None and t <= self.immediate_end + 1e-12:

return "immediate"

if self.recovery_end is not None and t <= self.recovery_end + 1e-12:

return "recovery"

return "late"

Fixed physical horizons (from baseline AIEF-1.4)

FIXED_PHYSICAL_HORIZONS = HorizonSystem(

text

name="fixed_physical",

immediate_end=17.5,

recovery_end=87.5,

)
EMA-normalized horizons

def make_normalized_horizons(T_window: float) -> HorizonSystem:

text

"""Create EMA-normalized horizon system."""

return HorizonSystem(

name="normalized_EMA",

immediate_end=T_window,

recovery_end=5 * T_window,

)

==========
Study Driver
==========

def create_Twindow_config(base_config: ModelConfig, rho: float) -> ModelConfig:

text

"""Create model config for a specific rho value.

T_window = rho *T_* where T_* = 25 / mean(omega)

"""

T_star = 25.0 / float(np.mean(base_config.omega))

T_window = rho * T_star

return ModelConfig(

N=base_config.N,

dt=base_config.dt,

tau=base_config.tau,

omega=base_config.omega.copy(),

a=base_config.a,

q=base_config.q,

delta=base_config.delta,

eta=base_config.eta,

mu=base_config.mu,

gamma=base_config.gamma,

T_window=T_window,

B_c=base_config.B_c,

```
B_init_scale=base_config.B_init_scale,

scale_epsilon=base_config.scale_epsilon,

)

def create_protocol_config(

text

config: ModelConfig,

future_end: float = 320.0,

) -> ProtocolConfig:

text

"""Create protocol config for the study."""

return ProtocolConfig(

burn_fraction=0.20,

history_end_fraction=0.60,

future_end=future_end,

window_steps=None,

recovery_end=None,

diagnostic_stride=50,

branches=STUDY_BRANCHES,
```

```
)

def run_single_condition(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

) -> dict[str, Any]:

text

"""Run a single (seed, config) condition."""

from aief_experiment_engine_v1 import run_seed



result = run_seed(seed, config, protocol)



# Add metadata

result["rho"] = config.T_window / (25.0 / float(np.mean([config.omega](http://config.omega))))

result["T_window"] = config.T_window

result["alpha"] = config.alpha
```

```
return result

def run_Twindow_study(

text

seeds: list[int],

base_config: ModelConfig,

rho_grid: tuple[float, ...] = RHO_GRID,

output_dir: Path | None = None,

future_end: float = 320.0,

overwrite: bool = False,

) -> dict[str, Any]:

text

"""Run the full T_window sensitivity study."""



if output_dir is not None:

output_dir.mkdir(parents=True, exist_ok=True)
```

```
# First: Baseline equivalence test at rho=1

rho_1_config = create_Twindow_config(base_config, 1.0)

rho_1_protocol = create_protocol_config(rho_1_config, future_end)



# For now, skip full baseline test (would need frozen data)

# Just run the study



all_results = {}



for rho in rho_grid:

config = create_Twindow_config(base_config, rho)

protocol = create_protocol_config(config, future_end)



rho_results = {}



for seed in seeds:

try:
```

```
result = run_single_condition(seed, config, protocol)

result["status"] = "success"

rho_results[str(seed)] = result

except Exception as e:

rho_results[str(seed)] = {

"status": f"failed: {type(e).__name__}: {e}",

"seed": seed,

"rho": rho,

"T_window": config.T_window,

}


all_results[f"rho_{rho}"] = rho_results



# Save outputs

if output_dir is not None:

# Save combined results

timestamp = [pd.Timestamp.now](http://pd.Timestamp.now)().strftime("%Y%m%d_%H%M%S")
```

```
# Save raw results

raw_path = output_dir / f"twindow_raw_results_{timestamp}.json"

# For now, just save a summary

summary = {

"study": "AIEF-1.4 T_window sensitivity",

"rho_grid": list(rho_grid),

"n_seeds": len(seeds),

"base_config": {

"N": base_config.N,

"dt": base_config.dt,

"mean_omega": float(np.mean(base_[config.omega](http://config.omega))),

"T_star": 25.0 / float(np.mean(base_[config.omega](http://config.omega))),

},

"status": {

rho: {

seed: results[seed].get("status", "unknown")

for seed in results

}
```

```
for rho, results in all_results.items()

},

}

atomic_write_json(output_dir / "twindow_condition_manifest.json", summary)



return all_results

================================================================================
CLI
================================================================================

def parse_arguments():

text

parser = argparse.ArgumentParser(description=__doc__)

parser.add_argument(

"--seeds", nargs="+", type=int,

help="Seed IDs to process",

)

parser.add_argument(
```

```
"--output-dir", type=Path, default=Path("twindow_output"),

help="Output directory",

)

parser.add_argument(

"--future-end", type=float, default=320.0,

help="Future simulation end time",

)

parser.add_argument(

"--rho-grid", nargs="+", type=float, default=list(RHO_GRID),

help="Rho grid values",

)

parser.add_argument(

"--overwrite", action="store_true",

help="Overwrite existing outputs",

)

return parser.parse_args()

def main():

text
```

```
args = parse_arguments()



# Use default model config

base_config = ModelConfig(

N=12,

dt=0.01,

tau=1.0,

omega=np.array([0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1], dtype=float),

a=1.0,

q=1.5,

delta=0.05,

eta=0.1,

mu=0.01,

gamma=0.5,

T_window=1.0, # Will be overridden by rho

B_c=1.0,

B_init_scale=0.0005,
```

```
scale_epsilon=1e-12,

)



base_config.validate()



print(f"Running T_window sensitivity study")

print(f" Seeds: {len(args.seeds)}")

print(f" Rho grid: {args.rho_grid}")

print(f" Output: {args.output_dir}")



results = run_Twindow_study(

seeds=args.seeds,

base_config=base_config,

rho_grid=tuple(args.rho_grid),

output_dir=args.output_dir,

future_end=args.future_end,

overwrite=args.overwrite,
```

)

Print summary

for rho, rho_results in results.items():

n_success = sum(1 for r in rho_results.values() if r.get("status") == "success")

n_total = len(rho_results)

print(f" {rho}: {n_success}/{n_total} seeds succeeded")

print(f"\nDone. Output saved to {args.output_dir}")

if name == "main":

text

main()

Next:

"""AIEF-1.4 T_window Sensitivity Analysis Driver.

Computes:

Paired contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Branch DID: I_{s,ρ,b}(Y) = C_{s,ρ,b}(Y) - C_{s,1,b}(Y)

Recovery scaling: t_rec and t_rec/T_window

Bootstrap intervals

Tables and report

Statistical unit: 40 seed/graphs (not edges, components, branches, or conditions).

"""

from future import annotations

import argparse

import json

import os

from dataclasses import dataclass

from pathlib import Path

from typing import Any

import numpy as np

import pandas as pd
==
Constants
==

RHO_GRID = (0.5, 0.75, 1.0, 1.5, 2.0)

```
STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Primary regime outcomes

PRIMARY_REGIME_OUTCOMES = (

text

"D_B_final",

"P_mean_late",

"Total_edge_weight",

"Effective_edges",

"Nonzero_edge_fraction",

"Weight_gini",

"Weight_entropy",

"Mean_row_sum",

"Max_row_sum",

"Corr_Rpast_Plate",

)
Recovery outcomes

RECOVERY_OUTCOMES = (

text

"recovery_probability",
```

```
"first_crossing",

"sustained_90_rest",

"held_90_for_1Twindow",

"restricted_mean_recovery_time",

"AUC_deficit_vs_continue",

)
# ============================================================================
Bootstrap Functions
# ============================================================================

def bootstrap_ci(

text

values: np.ndarray,

repetitions: int = 1000,

seed: int = 42,

confidence: float = 0.95,

) -> tuple[float, float, float]:

text

"""Compute bootstrap confidence interval for the mean."""
```

values = np.asarray(values, dtype=float)

values = values[np.isfinite(values)]

if len(values) < 2:

return (float(np.nan), float(np.nan), float(np.nan))

rng = np.random.default_rng(seed)

bootstrap_means = np.zeros(repetitions)

for i in range(repetitions):

indices = rng.integers(0, len(values), size=len(values))

bootstrap_means[i] = np.mean(values[indices])

alpha = (1.0 - confidence) / 2.0

ci_lower = float(np.percentile(bootstrap_means, 100 * alpha))

ci_upper = float(np.percentile(bootstrap_means, 100 * (1 - alpha)))

mean_val = float(np.mean(values))

return (mean_val, ci_lower, ci_upper)

==========
Analysis Functions
==========

def compute_regime_contrasts(

text

results: dict[str, Any],

base_rho: float = 1.0,

) -> pd.DataFrame:

text

"""Compute regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}"""

rows = []

for rho in RHO_GRID:

if rho == base_rho:

continue

```
rho_key = f"rho_{rho}"

base_key = f"rho_{base_rho}"


if rho_key not in results or base_key not in results:

continue


for seed_str, seed_results in results[rho_key].items():

if seed_results.get("status") != "success":

continue


base_seed_results = results[base_key].get(seed_str, {})

if base_seed_results.get("status") != "success":

continue


# Get continue branch results

rho_continue = seed_results.get("branches", {}).get("continue", {})

base_continue = base_seed_results.get("branches", {}).get("continue", {})


for outcome in PRIMARY_REGIME_OUTCOMES:

rho_val = rho_continue.get(outcome)
```

```
base_val = base_continue.get(outcome)


if rho_val is None or base_val is None:

continue


if isinstance(rho_val, (int, float, np.number)) and isinstance(base_val, (int, float, np.number)):

contrast = float(rho_val) - float(base_val)

rows.append({

"seed": seed_str,

"rho": rho,

"outcome": outcome,

"contrast": contrast,

"type": "regime",

})


return pd.DataFrame(rows)

def compute_branch_did(

text
```

```
results: dict[str, Any],

base_rho: float = 1.0,

) -> pd.DataFrame:

text

"""Compute branch difference-in-differences: I_{s,ρ,b} = C_{s,ρ,b} - C_{s,1,b}



where C_{s,ρ,b} = Y_{s,ρ,b} - Y_{s,ρ,continue}

"""

rows = []



for rho in RHO_GRID:

if rho == base_rho:

continue



rho_key = f"rho_{rho}"

base_key = f"rho_{base_rho}"



if rho_key not in results or base_key not in results:
```

```
continue

for seed_str, seed_results in results[rho_key].items():
if seed_results.get("status") != "success":
continue

base_seed_results = results[base_key].get(seed_str, {})
if base_seed_results.get("status") != "success":
continue

for branch in STUDY_BRANCHES:
if branch == "continue":
continue

# C_{s,ρ,b} = Y_{s,ρ,b} - Y_{s,ρ,continue}
rho_branch = seed_results.get("branches", {}).get(branch, {})
rho_continue = seed_results.get("branches", {}).get("continue", {})

base_branch = base_seed_results.get("branches", {}).get(branch, {})
```

```
base_continue = base_seed_results.get("branches", {}).get("continue", {})


for outcome in PRIMARY_REGIME_OUTCOMES:

rho_branch_val = rho_branch.get(outcome)

rho_continue_val = rho_continue.get(outcome)

base_branch_val = base_branch.get(outcome)

base_continue_val = base_continue.get(outcome)


if any(v is None for v in [rho_branch_val, rho_continue_val, base_branch_val, base_continue_val]):

continue


if all(isinstance(v, (int, float, np.number)) for v in [rho_branch_val, rho_continue_val,
base_branch_val, base_continue_val]):

C_rho = float(rho_branch_val) - float(rho_continue_val)

C_base = float(base_branch_val) - float(base_continue_val)

did = C_rho - C_base


rows.append({

"seed": seed_str,

"rho": rho,
```

```
"branch": branch,

"outcome": outcome,

"C_rho": C_rho,

"C_base": C_base,

"DID": did,

"type": "branch_did",

})



return pd.DataFrame(rows)

def compute_recovery_scaling(

text

results: dict[str, Any],

) -> pd.DataFrame:

text

"""Compute recovery scaling: t_rec and t_rec/T_window"""

rows = []
```

```
for rho_key, rho_results in results.items():

rho = float(rho_key.replace("rho_", ""))


for seed_str, seed_results in rho_results.items():

if seed_results.get("status") != "success":

continue


config = seed_results.get("config", {})

T_window = config.get("T_window", 1.0)


for branch in STUDY_BRANCHES:

branch_results = seed_results.get("branches", {}).get(branch, {})


# Get recovery times

for outcome in RECOVERY_OUTCOMES:

val = branch_results.get(outcome)

if val is not None and isinstance(val, (int, float, np.number)):

rows.append({

"seed": seed_str,
```

```
"rho": rho,

"T_window": T_window,

"branch": branch,

"outcome": outcome,

"physical_time": float(val),

"normalized_time": float(val) / T_window if T_window &gt; 0 else float('nan'),

"type": "recovery",

})



return pd.DataFrame(rows)

def summarize_contrasts(

text

df: pd.DataFrame,

bootstraps: int = 1000,

seed: int = 42,

) -> pd.DataFrame:

text
```

"""Summarize contrasts with bootstrap CIs."""

summary_rows = []

for (rho, outcome), group in df.groupby(["rho", "outcome"]):

values = group["contrast"].dropna().to_numpy()

if len(values) >= 2:

mean_val, ci_low, ci_high = bootstrap_ci(values, bootstraps, seed)

n_pos = int(np.sum(values > 0))

n_zero = int(np.sum(values == 0))

n_neg = int(np.sum(values < 0))

summary_rows.append({

"rho": rho,

"outcome": outcome,

"type": group["type"].iloc[0],

"n": len(values),

"mean": mean_val,

```
"ci_low": ci_low,

"ci_high": ci_high,

"n_positive": n_pos,

"n_zero": n_zero,

"n_negative": n_neg,

})



return pd.DataFrame(summary_rows)

def make_report(

text

regime_summary: pd.DataFrame,

did_summary: pd.DataFrame,

recovery_summary: pd.DataFrame,

) -> str:

text

"""Generate analysis report."""

lines = [
```

"=" * 70,

"AIEF-1.4 T_window Sensitivity Analysis",

"=" * 70,

"",

"Scientific Questions:",

"",

"Q1. Regime robustness: Does the sparse mature regime persist over",

" 0.5T_* <= T_window <= 2T_*?",

"",

"Q2. Topology identity: Does each seed reconstruct approximately the",

" same edge organization as at rho=1?",

"",

"Q3. EMA recovery scaling: Does t_rec/T_window remain stable?",

"",

"Q4. Weight reconstruction: Does reset_B reconstruct across EMA timescales?",

"",

"Q5. Maintenance identity: Does future_shuffle remain destructive?",

```
"",
"-" * 70,
"Regime Contrasts Summary",
"-" * 70,
"",
]


if not regime_summary.empty:
for *, row in regime*summary.iterrows():
lines.append(
f" rho={row['rho']}, {row['outcome']}: "
f"mean={row['mean']:.6g} [CI: {row['ci_low']:.6g}, {row['ci_high']:.6g}], "
f"n={row['n']}, +={row['n_positive']}, 0={row['n_zero']}, -={row['n_negative']}"
)
else:
lines.append(" No regime contrasts computed")
```

```
lines.extend([

"",

"-" * 70,

"Branch Difference-in-Differences Summary",

"-" * 70,

"",

])



if not did_summary.empty:

for *, row in did*summary.iterrows():

lines.append(

f" rho={row['rho']}, {row['branch']}, {row['outcome']}: "

f"mean={row['mean']:.6g} [CI: {row['ci_low']:.6g}, {row['ci_high']:.6g}], "

f"n={row['n']}, +={row['n_positive']}, 0={row['n_zero']}, -={row['n_negative']}"

)

else:

lines.append(" No branch DID computed")
```

```
lines.extend([

"",

"-" * 70,

"Recovery Scaling Summary",

"-" * 70,

"",

])



if not recovery_summary.empty:

for *, row in recovery*summary.iterrows():

lines.append(

f" rho={row['rho']}, {row['branch']}, {row['outcome']}: "

f"physical={row['physical_time']:.4f}, normalized={row['normalized_time']:.4f}"

)

else:

lines.append(" No recovery scaling computed")
```

```
lines.extend([
    "",
    "-" * 70,
    "Interpretation Constraints",
    "-" * 70,
    "",
    "* First crossing and sustained recovery are different estimands.",
    "* Late convergence does not imply no transient loss.",
    "* Density recovery does not imply edge identity recovery.",
    "* The seed/graph is the inferential unit (40 seeds).",
    "* Parameter levels are repeated conditions within each seed.",
    "",
])

return "\n".join(lines)
================================================================================
```

Main Analysis Function
==========
====

def run_analysis(

text

results_path: Path,

output_dir: Path,

bootstraps: int = 1000,

analysis_seed: int = 42,

) -> dict[str, Any]:

text

"""Run full analysis on study results."""

output_dir.mkdir(parents=True, exist_ok=True)

Load results

with open(results_path, "r") as f:

results = json.load(f)

```
# Compute contrasts

regime_contrasts = compute_regime_contrasts(results)

branch_did = compute_branch_did(results)

recovery_scaling = compute_recovery_scaling(results)



# Summarize

regime_summary = summarize_contrasts(regime_contrasts, bootstraps, analysis_seed)

did_summary = summarize_contrasts(branch_did, bootstraps, analysis_seed)

recovery_summary = recovery_scaling.groupby(["rho", "branch", "outcome"]).agg({

"physical_time": "mean",

"normalized_time": "mean",

}).reset_index()



# Save outputs

regime_[contrasts.to](http://contrasts.to)_csv(output_dir / "twindow_paired_contrasts.csv",
index=False)

branch_[did.to](http://did.to)_csv(output_dir / "twindow_branch_interactions.csv", index=False)
```

```
recovery_[scaling.to](http://scaling.to)_csv(output_dir / "twindow_recovery_summary.csv",
index=False)

regime_[summary.to](http://summary.to)_csv(output_dir / "twindow_regime_summary.csv",
index=False)

did_[summary.to](http://summary.to)_csv(output_dir / "twindow_did_summary.csv", index=False)



# Generate report

report = make_report(regime_summary, did_summary, recovery_summary)

with open(output_dir / "twindow_sensitivity_report.txt", "w") as f:

f.write(report)



# Save manifest

manifest = {

"analysis": "AIEF-1.4 T_window sensitivity",

"results_path": str(results_path.resolve()),

"bootstrap_repetitions": bootstraps,

"analysis_seed": analysis_seed,

"rho_grid": list(RHO_GRID),
```

```
"branches": list(STUDY_BRANCHES),

"horizon_systems": ["fixed_physical", "normalized_EMA"],

"statistical_unit": "40 seed/graphs",

"outputs": [

"twindow_paired_contrasts.csv",

"twindow_branch_interactions.csv",

"twindow_recovery_summary.csv",

"twindow_regime_summary.csv",

"twindow_did_summary.csv",

"twindow_sensitivity_report.txt",

],

}

with open(output_dir / "analysis_manifest.json", "w") as f:

json.dump(manifest, f, indent=2, sort_keys=True)

f.write("\n")



return {
```

"regime_contrasts": regime_contrasts,

"branch_did": branch_did,

"recovery_scaling": recovery_scaling,

"regime_summary": regime_summary,

"did_summary": did_summary,

}

==========
CLI
==========

def parse_arguments():

text

parser = argparse.ArgumentParser(description=__doc__)

parser.add_argument(

"--results", type=Path, required=True,

help="Path to raw results JSON",

)

parser.add_argument(

"--output-dir", type=Path, default=Path("twindow_analysis"),

```
help="Output directory",

)

parser.add_argument(

"--bootstraps", type=int, default=1000,

help="Bootstrap repetitions",

)

parser.add_argument(

"--analysis-seed", type=int, default=42,

help="Analysis RNG seed",

)

return parser.parse_args()

def main():

text

args = parse_arguments()



print(f"Running T_window sensitivity analysis")

print(f" Results: {args.results}")
```

print(f" Output: {args.output_dir}")

results = run_analysis(

args.results,

args.output_dir,

bootstraps=args.bootstraps,

analysis_seed=args.analysis_seed,

)

print(f"\nDone. Analysis saved to {args.output_dir}")

if name == "main":

text

main()

These scripts are not usable for the TwindowTwindow study. They contain fundamental model-definition errors, skip the mandatory baseline-equivalence gate, do not save the raw results expected by the analysis driver, and implement incompatible output schemas.

I would discard this implementation rather than patch it incrementally.
1. The claimed baseline configuration is entirely wrong

The engine says:

Python

matches frozen run 5a9f093a2fe0

but defines:

Python

tau=1.0
omega=[0.0, 0.1, ..., 1.1]
eta=0.1
mu=0.01
gamma=0.5
T_window=1.0
B_c=1.0

The frozen production values are:

Python

tau = 0.5

omega = [
1.00, 1.02, 1.50, 2.00,
2.01, 1.01, 1.03, 1.51,
2.02, 2.00, 1.00, 1.04,
]

eta = 0.04
mu = 0.03
gamma = 0.015
T_window = 17.5
B_c = 0.12

Therefore the engine does not represent AIEF-1.4.

The T-window driver itself notices the contradiction in a long comment, but proceeds with the incorrect configuration anyway. That is a hard stop.

2. The baseline equivalence test is a stub that reports success

This function initializes:

Python

```
results = {
"status": "passed",
...
}
```

but never runs a seed or compares a trajectory. It ends with:

Python

```
"Full baseline equivalence requires running frozen seeds"
```

and still returns "passed".

The study driver then explicitly skips the gate:

Python

```
# For now, skip full baseline test
```

This violates the central requirement of the reusable-engine design.

A nonexecuted equivalence test must return:

text

```
status = not_run
```

or fail. It must never report “passed.”
3. Prefix timing is wrong

Production uses:
Tsim=800,
Tsim=800,
burn end=0.2Tsim=160,
burn end=0.2Tsim=160,
split time=0.6Tsim=480,
split time=0.6Tsim=480,
future duration=320.
future duration=320.

The engine computes:

Python

```
total_steps = (
history_end_fraction
* future_end
/ dt
)
```

The T-window driver passes:

Python

```
future_end = 320
```

so the split occurs at:
0.6×320=192,
0.6×320=192,

not 480480.

It also computes:

Python

```
burn_steps = burn_fraction * total_steps
```

which makes burn-in 20%20% of the already truncated prefix rather than 20%20% of the full 800-unit simulation.

The protocol needs distinct fields:

text

T_sim = 800
split_time = 480
future_duration = 320
burn_end = 160

Do not reuse one future_end variable for all three meanings.
4. Initialization does not reproduce production RNG

Production initialization uses a named child of:

Python

np.random.SeedSequence(seed).spawn(...)

The engine uses:

Python

np.random.default_rng(seed)

directly.

That changes every initial phase and BB value.

It also initializes theta in:

Python

[-pi, pi)

rather than the production:

Python

[0, 2*pi)

Although those ranges are phase-equivalent mathematically, they will not reproduce the frozen trajectory bitwise.
5. Branch RNG derivation is nondeterministic across Python processes

The engine uses:

Python

hash((seed, branch))

Python string/tuple hashes are process-randomized unless PYTHONHASHSEED is fixed. This is not a reproducible RNG namespace.

Use the frozen SeedSequence branch algorithm or a stable SHA-256-based namespace with an explicit manifest. For baseline equivalence, use the original algorithm exactly.
6. Future duration defaults to the wrong value

If future_end is absent:

Python

future_steps = 80000

which means 800 time units.

Production future duration is 320 time units:

Python

FUTURE_STEPS = 32000

Again, total simulation time and future duration are conflated.
7. The engine does not calculate the outcomes required by the study

simulate_future returns only:

text

B_final
sample_times
density_trajectory
similarity_trajectory

The driver and analysis expect:

text

D_B_final
P_mean_late
Total_edge_weight
Effective_edges
Nonzero_edge_fraction
Weight_gini
Weight_entropy
Mean_row_sum
Max_row_sum
Corr_Rpast_Plate
recovery outcomes

None of those are produced.

As a result:

regime contrast tables will be empty;
branch DID tables will be empty;
recovery scaling will be empty.

The engine and analysis schemas are not compatible.
8. The study driver saves no raw results

run_Twindow_study constructs a nested dictionary containing NumPy arrays, but never serializes it.

It defines:

Python

raw_path = ...

and does nothing with it.

The only output is a condition manifest containing status strings.

But the analysis driver requires:

Python

--results <raw results JSON>

No such JSON is created.

Even if attempted directly, NumPy arrays are not JSON serializable without conversion. Large scientific arrays should be saved as NPZ, with scalar tables in CSV.
9. Fixed and normalized horizons are defined but never used

The driver defines:

Python

FIXED_PHYSICAL_HORIZONS
make_normalized_horizons(...)

but no simulation or analysis code uses them.

Thus the promised distinction between:

fixed physical horizons;
EMA-normalized horizons

does not exist in the outputs.
10. The T∗T∗ calculation is wrong because the base omega is wrong

The driver spends many comments noticing that:
25/ω⁻
25/ω⁻

does not equal 17.517.5 under its fake omega vector.

With the actual frozen frequencies:
ω⁻=107,
ω⁻=710,

so:
T∗=2510/7=17.5.
T∗=10/725=17.5.

The contradiction disappears immediately when the correct frozen model is used.

Proceeding after noticing a baseline contradiction is especially concerning.
11. Analysis driver has an immediate DID bug

compute_branch_did stores the effect under:

Python

"DID": did

but calls:

Python

summarize_contrasts(branch_did)

which expects a column named:

Python

"contrast"

This raises a KeyError.

Either name the column contrast consistently or parameterize the summary function.
12. Branch DID summaries pool different branches together

summarize_contrasts groups by:

Python

["rho", "outcome"]

It does not group by branch.

Therefore, after fixing the missing column, it would pool:

text

reset_ema
reset_B
future_shuffle

into one meaningless average.

DID summaries must group by:

Python

["rho", "branch", "outcome"]

13. Recovery analysis is absent

The engine does not compute any member of:

Python

RECOVERY_OUTCOMES

The analysis driver merely looks for nonexistent keys.

There is no:

continue-relative threshold;
sustained recovery;
held-duration recovery;
censoring;
restricted time;
AUC deficit;
fixed versus normalized horizon.

14. Cross-TT topology identity is not implemented

The study declares:

text

cross_T_weight_cosine
cross_T_threshold_Jaccard
cross_T_normalized_weight_distance
cross_T_component_signature_match

but no code computes them.

Thus scientific question Q2 is not addressed.
15. Failure handling is not production-grade

Every condition catches exceptions and stores a failure row, then proceeds.

There is no requirement that:

baseline equivalence passed;
all 40 seeds completed;
every ρρ completed;
outputs are freezeable;
failed conditions are excluded visibly from inference.

A sensitivity study with missing conditions could still generate a report.
16. Output and manifest standards are missing

The driver lacks:

source hashes;
engine hash;
frozen input hashes;
command;
exact condition grid;
per-condition status CSV;
checkpoint/resume;
atomic CSV outputs;
branch diagnostics NPZ;
baseline equivalence certificate;
interpretation constraints;
freezeability flag.

The analysis driver writes files nonatomically and does not hash its input.
17. ModelConfig is not actually immutable

The dataclass is frozen, but:

Python

omega: np.ndarray

is mutable. A caller can change:

Python

config.omega[:] = ...

despite frozen=True.

Use an immutable tuple internally or copy and mark the NumPy array read-only in __post_init__.

This is secondary compared with the model errors.
18. What can be salvaged

The following design ideas are worth retaining:

immutable parameter object;
explicit derived αα, capacity, and delay steps;
shared core and thin drivers;
fixed versus normalized horizon abstraction;
condition-level metadata;
separate simulation and analysis drivers;
atomic helper concepts.

But the implementation must restart from the actual production source, not from this engine.
Correct redevelopment path
Step 1 — copy the production kernel exactly

Start from:

text

AIEF_1_4_memory_ablation_updated.py

Do not rewrite from memory.

Replace only hidden globals with reads from ModelConfig, one function at a time, while preserving:

arithmetic order;
RNG hierarchy;
initialization;
delay deque semantics;
EMA update;
positivity;
symmetrization;
gate test;
endpoint scaling;
phase update;
diagnostic timing;
branch interventions.

Step 2 — use the real frozen defaults

The baseline config must be built directly from the frozen manifest, not hardcoded independently.

Python

```
ModelConfig.from_manifest(
frozen_manifest["model"]
)
```

Then require exact equality with imported production constants.
Step 3 — define protocol times explicitly

Use:

Python

```
T_sim = 800
burn_end_time = 160
split_time = 480
future_duration = 320
```

or their exact step equivalents.

Do not derive split timing from future duration.

Step 4 — exact baseline equivalence

Before any ρ≠1ρ =1 condition, reproduce for all 40 seeds:

text

split theta
split B
split C
split history
continue B_final
continue P_late
continue sampled trajectories
reset_ema anchor
reset_B anchor
future_shuffle anchor

Require NaN-aware exact equality.

Any failure stops the study.
Step 5 — only then run the grid

Use the actual:
T∗=17.5
T∗=17.5

and:
Twindow=
Twindow=

\rho T_*,
\qquad
\rho\in
{0.5,0.75,1,1.5,2}.
]

All other config fields remain exact frozen values.
Step 6 — save compatible outputs

The simulation driver must produce the scalar and trajectory schema expected by the analysis driver.

Prefer:

text

twindow_condition_manifest.csv
twindow_seed_status.csv
twindow_scalar_summary.csv
twindow_branch_diagnostics.npz
baseline_equivalence.csv
analysis_manifest.json

Then let the analysis driver compute cross-condition outcomes and recovery.
Rating
Architecture concept: 7/10
Fidelity to frozen AIEF-1.4: 0.5/10
Baseline validation: 0/10
Driver/analysis compatibility: 1/10
Current executability as a scientific study: 1/10
Overall: 1.5/10

This is not a near-miss. It is a different model with an unexecuted equivalence test and incompatible drivers.

Do not run the parameter grid. Rebuild the engine from the frozen production module and make exact baseline reproduction the first executable deliverable.

Evaluate these re writes:

#!/usr/bin/env python3

"""AIEF Experiment Engine v1 - Parameterized simulation kernel.

This is a reusable, validated simulation engine that reproduces the frozen

AIEF-1.4 production run exactly and supports parameterized studies.

FROZEN PRODUCTION VALUES (from AIEF_1_4_memory_ablation_updated_REAL.py):

N = 12

DT = 0.01

TAU = 0.5

OMEGA = [1.00, 1.02, 1.50, 2.00, 2.01, 1.01, 1.03, 1.51, 2.02, 2.00, 1.00, 1.04]

A = 1.0

Q = 1.5

DELTA = 0.05

ETA = 0.04

GAMMA = 0.015

MU = 0.03

T_WINDOW = 25.0 / mean(OMEGA) = 17.5

B_C = 0.12

B_INIT_SCALE = 0.0005

PROTOCOL (from frozen run):

T_SIM = 800.0

N_STEPS = int(round(T_SIM / DT)) = 80000

BURN_END = int(0.20 * N_STEPS) = 16000 steps = 160.0 time units

HIST_END = int(0.60 * N_STEPS) = 48000 steps = 480.0 time units

FUTURE_STEPS = N_STEPS - HIST_END = 32000 steps = 320.0 time units

DELAY_STEPS = int(round(TAU / DT)) = 50

WINDOW_STEPS = int(round(T_WINDOW / DT)) = 1750

RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS) = 8750

Architecture:

ModelConfig: frozen dataclass for all model parameters

ProtocolConfig: frozen dataclass for protocol parameters

DynamicState: state container (immutable where possible)

Core functions: initialize, advance, simulate, interventions

Validation: baseline equivalence test

Serialization: checkpoint I/O

DO NOT MODIFY: This is the reference implementation.

"""

from future import annotations

import copy

```
import hashlib

import json

import math

import os

import tempfile

import time

from dataclasses import dataclass, field

from pathlib import Path

from typing import Any

import numpy as np
================================================================================
Frozen Production Constants (DO NOT CHANGE)
================================================================================

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,
```

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005

FROZEN_ZERO_TOL = 1e-12

FROZEN_PROJECTION_NORM_TOL = 1e-14

FROZEN_GAP_TOL = 1e-9
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA)) # 1.428333...

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.5

FROZEN_T_WINDOW = FROZEN_T_STAR # 17.5

FROZEN_ALPHA = math.exp(-FROZEN_DT / FROZEN_T_WINDOW)

FROZEN_CAPACITY = (FROZEN_A - FROZEN_DELTA) / FROZEN_Q
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_WINDOW / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
==========
Model Configuration
==========

@dataclass(frozen=True)

class ModelConfig:

text

"""Immutable model configuration.

All model-changing quantities must come from here. No hidden module globals.

"""

Core parameters

N: int

dt: float

tau: float

omega: tuple[float, ...] # Immutable tuple

Coupling parameters

a: float

q: float

delta: float

Dynamics parameters

eta: float

mu: float

gamma: float

```
# EMA/threshold parameters

T_window: float

B_c: float

B_init_scale: float

scale_epsilon: float = 1e-12



# Derived properties

@property

def mean_omega(self) -&gt; float:

"""Mean of omega frequencies."""

return float(np.mean([self.omega](http://self.omega)))



@property

def T_star(self) -&gt; float:

"""T_* = 25 / mean(omega)"""

return 25.0 / self.mean_omega
```

```
@property

def alpha(self) -&gt; float:

"""EMA smoothing factor: exp(-dt / T_window)"""

return math.exp(-self.dt / self.T_window)



@property

def capacity(self) -&gt; float:

"""Projection capacity: (a - delta) / q"""

return (self.a - [self.delta](http://self.delta)) / self.q



@property

def delay_steps(self) -&gt; int:

"""Delay steps: round(tau / dt)"""

return int(round(self.tau / self.dt))



def validate(self) -&gt; None:
```

"""Validate model configuration."""

if self.N <= 0:

raise ValueError("N must be positive")

if self.dt <= 0:

raise ValueError("dt must be positive")

if self.tau <= 0:

raise ValueError("tau must be positive")

if len(self.omega) != self.N:

raise ValueError(f"omega must have {self.N} elements")

if self.a <= 0 or self.q <= 0 or self.delta < 0:

raise ValueError("a, q must be positive; delta must be nonnegative")

if self.eta < 0 or self.mu < 0 or self.gamma < 0:

raise ValueError("eta, mu, gamma must be nonnegative")

if self.T_window <= 0:

raise ValueError("T_window must be positive")

if self.B_c < 0:

raise ValueError("B_c must be nonnegative")

if self.B_init_scale < 0:

raise ValueError("B_init_scale must be nonnegative")

if self.scale_epsilon <= 0:

raise ValueError("scale_epsilon must be positive")

Validate derived values

if not (0 < self.alpha <= 1):

raise ValueError(f"alpha={self.alpha} must be in (0, 1]")

if self.capacity < 0:

raise ValueError(f"capacity={self.capacity} must be nonnegative")

@dataclass(frozen=True)

class ProtocolConfig:

text

"""Protocol configuration for experiment execution."""

Simulation time parameters (in time units, not steps)

T_sim: float

burn_end_time: float

split_time: float

future_duration: float

Diagnostics

diagnostic_stride: int

Branches to simulate

branches: tuple[str, ...]

@property

def N(self) -> int:

"""Total steps."""

return int(round(self.T_sim / 0.01)) # Assuming dt=0.01

@property

def burn_end(self) -> int:

"""Burn-in end step."""

return int(round(self.burn_end_time / 0.01))

```
@property

def hist_end(self) -&gt; int:

"""History end step (split point)."""

return int(round(self.split_time / 0.01))



@property

def future_steps(self) -&gt; int:

"""Future simulation steps."""

return int(round(self.future_duration / 0.01))



def validate(self, model: ModelConfig) -&gt; None:

"""Validate protocol against model."""

if self.T_sim &lt;= 0:

raise ValueError("T_sim must be positive")

if not (0 &lt; self.burn_end_time &lt; self.split_time &lt; self.T_sim):

raise ValueError("Invalid time parameters: must have 0 &lt; burn_end &lt; split &lt; T_sim")
```

if self.future_duration <= 0:

raise ValueError("future_duration must be positive")

if self.diagnostic_stride <= 0:

raise ValueError("diagnostic_stride must be positive")

Default frozen production configuration

DEFAULT_MODEL_CONFIG = ModelConfig(

text

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=FROZEN_T_WINDOW,

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=FROZEN_ZERO_TOL,

)
Default protocol matching frozen run

DEFAULT_PROTOCOL_CONFIG = ProtocolConfig(

text

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=("continue", "reset_ema", "reset_B", "reset_B_ema", "B_carrier_only", "full_reset", "future_shuffle"),

)
==========
Utility Functions
==========

def compute_edges(N: int) -> tuple[np.ndarray, np.ndarray]:

```
text

"""Get upper triangle indices for edges."""

return np.triu_indices(N, k=1)

def validate_B_matrix(B: np.ndarray, N: int, eps: float = 1e-12) -> None:

text

"""Validate B matrix: symmetric, nonnegative, zero diagonal."""

if B.shape != (N, N):

raise ValueError(f"B shape {B.shape} != ({N}, {N})")

if not np.allclose(B, B.T, atol=eps):

raise ValueError("B is not symmetric")

if not np.allclose(np.diag(B), 0, atol=eps):

raise ValueError("B diagonal is not zero")

if np.any(B &lt; -eps):

raise ValueError("B has negative entries")

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file."""

digest = hashlib.sha256()
```

```
with open(path, "rb") as handle:
for block in iter(lambda: [handle.read](http://handle.read)(1024 * 1024), b""):
digest.update(block)
return digest.hexdigest()
def array_sha256(arr: np.ndarray) -> str:
text
"""Compute SHA-256 hash of a numpy array."""
digest = hashlib.sha256()
shape_bytes = np.asarray(arr.shape, dtype=np.int64).tobytes()
digest.update(shape_bytes)
digest.update(arr.dtype.str.encode())
contiguous = np.ascontiguousarray(arr)
if contiguous.size &gt; 0:
digest.update(contiguous.tobytes())
return digest.hexdigest()
def atomic_write_text(path: Path, text: str) -> None:
text
```

"""Write text atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{path.name}.", suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

def atomic_write_json(path: Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

```
def atomic_write_npz(path: Path, **kwargs: Any) -> None:

text

"""Write NPZ atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp.npz")

try:

with os.fdopen(fd, "wb") as handle:

np.savez(handle, **kwargs)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

================================================================================
RNG Utilities (matching frozen production exactly)
================================================================================

def make_seed_sequence(seed: int, child_index: int) -> int:
```

```
text

"""Generate a child seed from SeedSequence(seed).spawn(...)[child_index].



This matches the frozen production RNG hierarchy exactly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

children = root.spawn(child_index + 1)

return int(children[child_index].generate_state(1, dtype=np.uint64)[0])

def make_prefix_rng(seed: int) -> np.random.Generator:

text

"""Create prefix RNG matching frozen production.



Frozen: root = SeedSequence(seed); init_sequence, _ = root.spawn(2)

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)
```

```
init_sequence, _ = root.spawn(2)

return np.random.default_rng(int(init_sequence.generate_state(1, dtype=np.uint64)[0]))

def make_branch_rngs(seed: int, branch_order: tuple[str, ...]) -> dict[str, np.random.Generator]:

text

"""Create branch RNGs matching frozen production algorithm.



Frozen: *, branch*root = SeedSequence(seed).spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

branch_rng_seeds = {branch: int(sequence.generate_state(1, dtype=np.uint64)[0])

for branch, sequence in zip(branch_order, branch_sequences)}

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

*, branch*root = root.spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))



return {
```

branch: np.random.default_rng(int(sequence.generate_state(1, dtype=np.uint64)[0]))

for branch, sequence in zip(branch_order, branch_sequences)

}

==========
Dynamic State
==========

class DynamicState:

text

"""State container for AIEF-1.4 dynamics.

Matches the frozen production implementation exactly.

"""

def **init**(

self,

theta: np.ndarray,

B: np.ndarray,

```
coherence_acc: np.ndarray,

history: list[np.ndarray] | np.ndarray,

):

self.theta = np.asarray(theta, dtype=float).copy()

self.B = np.asarray(B, dtype=float).copy()

self.coherence_acc = np.asarray(coherence_acc, dtype=complex).copy()


# Handle history input - ensure it's a list of arrays

if isinstance(history, np.ndarray):

# If it's a 2D array, convert to list of 1D arrays

self.history = [np.asarray(h, dtype=float).copy() for h in history]

else:

self.history = [np.asarray(h, dtype=float).copy() for h in history]



def clone(self) -&gt; "DynamicState":

"""Create a deep copy of the state."""

return DynamicState(

theta=self.theta.copy(),
```

B=self.B.copy(),

coherence_acc=self.coherence_acc.copy(),

history=[h.copy() for h in self.history],

)

@property

def N(self) -> int:

return len(self.theta)

==========
Simulation Core (exact reproduction of frozen production)
==========

def initialize_state(

text

seed: int,

config: ModelConfig,

) -> DynamicState:

text

"""Initialize state from seed using frozen algorithm.

Matches production: simulate_prefix() initialization.

Uses SeedSequence(seed).spawn(2)[0] for init_sequence.

"""

N = config.N

rng = make_prefix_rng(seed)

Initialize theta: uniform in [0, 2*pi) - matching production

theta = rng.random(N) *2.0* np.pi

Initialize B: small random symmetric matrix

B = config.B_init_scale * rng.random((N, N))

B = 0.5 * (B + B.T)

np.fill_diagonal(B, 0.0)

Initialize coherence accumulator

coherence_acc = np.zeros((N, N), dtype=complex)

Initialize history: DELAY_STEPS + 1 copies of theta

history = [theta.copy() for *in range(config.delay*steps + 1)]

return DynamicState(

theta=theta,

B=B,

coherence_acc=coherence_acc,

history=history,

)

def compute_R(

text

state: DynamicState,

config: ModelConfig,

) -> np.ndarray:

text

"""Compute coherence matrix R from current state.

Exact reproduction of production compute_R().

"""

N = state.N

delayed = state.history[0]

Convert omega to numpy array for indexing

omega_arr = np.asarray(config.omega, dtype=float)

dphi = (

state.theta[:, None]

- delayed[None, :]

- ((omega_arr[:, None] + omega_arr[None, :]) / 2.0) * config.tau

)

z_inst = np.exp(1j * dphi)

```
# Update EMA

state.coherence_acc[:] = (

config.alpha *state.coherence_acc + (1.0 - config.alpha)* z_inst

)



# Compute magnitude and symmetrize

result = np.abs(state.coherence_acc)

result = 0.5 * (result + result.T)

np.fill_diagonal(result, 0.0)



return result

def advance_state(

text

state: DynamicState,

config: ModelConfig,

rng: np.random.Generator,

shuffle_driver: bool = False,
```

) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:

text

"""Advance state by one time step.

Exact reproduction of production advance_state().

Returns (R_physical, R_driver, info).

"""

N = state.N

edges = compute_edges(N)

Store previous B for projection

B_previous = state.B.copy()

Compute R matrices

R_physical = compute_R(state, config)

R_driver = R_physical

```
if shuffle_driver:

values = R_physical[edges].copy()

rng.shuffle(values)

R_driver = np.zeros_like(R_physical)

R_driver[edges] = values

R_driver[(edges[1], edges[0])] = values



# Update B

B_unprojected = state.B + config.dt * (

(config.eta + [config.mu](http://config.mu)) * R_driver

- [config.mu](http://config.mu)

- config.gamma * state.B

)

B_unprojected = np.maximum(B_unprojected, 0.0)

B_unprojected = 0.5 * (B_unprojected + B_unprojected.T)

np.fill_diagonal(B_unprojected, 0.0)
```

Check gap constraint

row_sum_pre = np.sum(B_unprojected, axis=1)

g_min_pre = float(np.min(config.a - config.q * row_sum_pre))

pre_violation = bool(g_min_pre < config.delta)

Project if needed

B_projected = B_unprojected

if pre_violation:

scale = np.minimum(1.0, config.capacity / (row_sum_pre + config.scale_epsilon))

B_projected = B_unprojected * np.minimum.outer(scale, scale)

np.fill_diagonal(B_projected, 0.0)

Validate projection

state.B = B_projected

row_sum = np.sum(state.B, axis=1)

g_min = float(np.min(config.a - config.q * row_sum))

if g_min < config.delta - 1e-9:

```
raise RuntimeError(f"Gap projection failed: g_min={g_min}")



# Update theta

delayed = state.history[0]

omega_arr = np.asarray([config.omega](http://config.omega), dtype=float)

coupling = np.sum(

state.B * np.sin(delayed[None, :] - state.theta[:, None]), axis=1

)

state.theta = np.mod(

state.theta + config.dt * (omega_arr + coupling),

2.0 * np.pi

)



# Update history

state.history.append(state.theta.copy())

if len(state.history) &gt; config.delay_steps + 1:

state.history.pop(0)
```

```
info = {

"pre_violation": pre_violation,

"g_min_pre": g_min_pre,

"g_min": g_min,

}



return R_physical, R_driver, info

def simulate_prefix(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

) -> tuple[DynamicState, dict[str, Any]]:

text

"""Simulate prefix up to split_time.
```

```
Exact reproduction of production simulate_prefix().

Returns (state, prefix_dict).

"""

N = config.N

edges = compute_edges(N)



# Initialize state

state = initialize_state(seed, config)



# Get RNG for prefix simulation

# Frozen: root = SeedSequence(seed); init_sequence, _ = root.spawn(2)

# rng = default_rng(init_sequence.generate_state(1)[0])

# But for the prefix, we use a fresh RNG that's not used for branches

from numpy.random import SeedSequence

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = np.random.default_rng(int(init_sequence.generate_state(1, dtype=np.uint64)[0]))
```

Simulate to split point

R_past_sum = np.zeros((N, N), dtype=float)

R_past_count = 0

burn_end = protocol.burn_end

hist_end = protocol.hist_end

for step in range(hist_end):

R_physical, *,* = advance_state(state, config, rng, shuffle_driver=False)

if burn_end <= step < hist_end:

R_past_sum += R_physical

R_past_count += 1

prefix = {

"B_at_split": state.B.copy(),

"R_physical_past": R_past_sum / R_past_count if R_past_count > 0 else np.zeros((N, N)),

"theta_at_split": state.theta.copy(),

"coherence_acc_at_split": state.coherence_acc.copy(),

"history_at_split": np.stack(state.history),

"selected_at_split": (state.B > config.B_c).astype(np.uint8),

}

return state, prefix

def apply_intervention(

text

state: DynamicState,

branch: str,

config: ModelConfig,

rng: np.random.Generator,

) -> None:

text

"""Apply intervention to state. Mutates in place.

Exact reproduction of production apply_intervention().

"""

N = state.N

if branch in {"reset_ema", "reset_B_ema", "B_carrier_only", "full_reset"}:

Zero coherence accumulator

state.coherence_acc.fill(0.0)

if branch in {"reset_B", "reset_B_ema", "full_reset"}:

Zero B

state.B.fill(0.0)

if branch in {"B_carrier_only", "full_reset"}:

Randomize theta and history

new_theta = rng.random(N) *2.0* np.pi

state.theta = new_theta

```
state.history = [new_theta.copy() for *in range(config.delay*steps + 1)]



# Ensure symmetry

state.B = 0.5 * (state.B + state.B.T)

np.fill_diagonal(state.B, 0.0)

def simulate_future(

text

split_state: DynamicState,

prefix: dict[str, Any],

branch: str,

config: ModelConfig,

protocol: ProtocolConfig,

rng: np.random.Generator,

) -> dict[str, Any]:

text

"""Simulate future trajectory from split state.
```

```
Exact reproduction of production simulate_future() with all required outcomes.
"""
state = split_state.clone()
apply_intervention(state, branch, config, rng)

N = state.N
edges = compute_edges(N)

future_steps = protocol.future_steps

# Initialize accumulators
sample_times = []
density_trajectory = []
similarity_trajectory = []

B_reference = prefix["B_at_split"]
```

```
for step in range(future_steps):

R_physical, *,* = advance_state(state, config, rng, shuffle_driver=(branch == "future_shuffle"))


if step % protocol.diagnostic_stride == 0:

t = (step + 1) * config.dt

sample_times.append(t)


# Density: fraction of edges above B_c

density = float(np.count_nonzero(state.B[edges] &gt; config.B_c) / len(edges[0]))

density_trajectory.append(density)


# Similarity: cosine between current and reference B

B_vec = state.B[edges]

B_ref_vec = B_reference[edges]

norm_B = np.linalg.norm(B_vec)

norm_ref = np.linalg.norm(B_ref_vec)

if norm_B &gt; 0 and norm_ref &gt; 0:

similarity = float([np.dot](http://np.dot)(B_vec, B_ref_vec) / (norm_B * norm_ref))
```

```
else:

similarity = float('nan')

similarity_trajectory.append(similarity)



# Compute final and summary outcomes

B_final = state.B.copy()



result = {

"B_final": B_final,

"sample_times": np.array(sample_times, dtype=float),

"density_trajectory": np.array(density_trajectory, dtype=float),

"similarity_trajectory": np.array(similarity_trajectory, dtype=float),



# Scalar outcomes

"D_B_final": float(np.count_nonzero(state.B[edges] &gt; config.B_c) / len(edges[0])),

"Total_edge_weight": float(np.sum(state.B[edges])),

"Effective_edges": effective_number(state.B[edges]),

"Nonzero_edge_fraction": float(np.mean(state.B[edges] &gt; config.scale_epsilon)),
```

"Mean_row_sum": float(np.mean(np.sum(state.B, axis=1))),

"Max_row_sum": float(np.max(np.sum(state.B, axis=1))),

Weight statistics

"Weight_gini": gini_coefficient(state.B[edges]),

"Weight_entropy": normalized_entropy(state.B[edges]),

Correlation with past

"Corr_Rpast_Plate": safe_corr(prefix["R_physical_past"][edges], state.B[edges]),

}

return result

==========
Metric Helpers
==========

def safe_corr(x: np.ndarray, y: np.ndarray) -> float:

text

"""Compute correlation coefficient safely."""

```
x = np.asarray(x, dtype=float)

y = np.asarray(y, dtype=float)

valid = np.isfinite(x) &amp;amp; np.isfinite(y)

x = x[valid]

y = y[valid]

if len(x) &amp;lt; 3 or np.std(x) &amp;lt; 1e-12 or np.std(y) &amp;lt; 1e-12:

return float('nan')

return float(np.corrcoef(x, y)[0, 1])

def effective_number(weights: np.ndarray) -> float:

text

"""Compute effective number of edges."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

sum_sq = float(np.sum(weights**2))

if total &amp;lt;= 1e-12 or sum_sq &amp;lt;= 1e-12:

return 0.0

return total**2 / sum_sq

def gini_coefficient(weights: np.ndarray) -> float:
```

text

"""Compute Gini coefficient."""

x = np.sort(np.asarray(weights, dtype=float))

total = float(np.sum(x))

if total <= 1e-12:

return 0.0

n = len(x)

ranks = np.arange(1, n + 1, dtype=float)

return float(2.0 *np.dot(ranks, x) / (n* total) - (n + 1.0) / n)

def normalized_entropy(weights: np.ndarray) -> float:

text

"""Compute normalized entropy."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

if total <= 1e-12:

return 0.0

probabilities = weights / total

positive = probabilities > 0.0

entropy = -float(np.sum(probabilities[positive] * np.log(probabilities[positive])))

return entropy / float(np.log(len(weights)))

```
==========
```

Baseline Equivalence Test

```
==========
```

def exact_array_equal_with_nan(a: np.ndarray, b: np.ndarray) -> bool:

text

"""Compare arrays with NaN-aware exact equality."""

a = np.asarray(a)

b = np.asarray(b)

if a.shape != b.shape or a.dtype != b.dtype:

return False

if np.issubdtype(a.dtype, np.floating) or np.issubdtype(a.dtype, np.complexfloating):

a_nan = np.isnan(a)

b_nan = np.isnan(b)

if not np.array_equal(a_nan, b_nan):

return False

```
finite = ~a_nan

return np.array_equal(a[finite], b[finite])

return np.array_equal(a, b)

def run_baseline_equivalence(

text

seeds: list[int],

config: ModelConfig,

protocol: ProtocolConfig,

frozen_manifest_path: str,

frozen_npz_path: str,

branch_order: tuple[str, ...],

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).



MANDATORY: Must reproduce frozen run exactly for all 40 seeds.

Requires NaN-aware exact equality for:
```

- split theta

- split B

- split coherence_acc

- split history

- continue B_final

- continue all trajectory keys

- all original branch anchors

Returns:

status: "passed" | "failed" | "not_run"

seed_results: per-seed validation details

"""

import numpy as np

results = {

"status": "not_run",

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"mean_omega": config.mean_omega,

"T_window": config.T_window,

"T_star": config.T_star,

"alpha": config.alpha,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

},

"n_seeds": len(seeds),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"seed_results": {},

```
}


# Load frozen manifest
with open(frozen_manifest_path, "r") as f:
frozen_manifest = json.load(f)


# Load frozen data
with np.load(frozen_npz_path, allow_pickle=False) as frozen_data:
frozen_files = dict(frozen_data)


# Validate config matches frozen manifest model
frozen_model = frozen_manifest["model"]


if frozen_model["N"] != config.N:
results["status"] = "failed"
results["error"] = f"N mismatch: config={config.N}, frozen={frozen_model['N']}"
```

```
return results



if not np.allclose(frozen_model["omega"], list([config.omega](http://config.omega)), rtol=1e-15):

results["status"] = "failed"

results["error"] = "omega mismatch"

return results



# Check other model params

model_checks = [

("dt", frozen_model["dt"], config.dt),

("tau", frozen_model.get("tau", frozen_model.get("T_window", 1.0)), config.tau),

("eta", frozen_model["eta"], config.eta),

("mu", frozen_model["mu"], [config.mu](http://config.mu)),

("gamma", frozen_model["gamma"], config.gamma),

("B_c", frozen_model["B_c"], config.B_c),

]
```

```
for name, frozen_val, config_val in model_checks:

if not np.isclose(frozen_val, config_val, rtol=1e-15):

results["status"] = "failed"

results["error"] = f"{name} mismatch: frozen={frozen_val}, config={config_val}"

return results



# Now run actual equivalence test for each seed

print("Running baseline equivalence test...")

all_passed = True



for seed in seeds:

seed_str = str(seed)

seed_results = {"status": "passed", "checks": {}, "max_diffs": {}}



try:

# Simulate with our engine

split_state, prefix = simulate_prefix(seed, config, protocol)

branch_rngs = make_branch_rngs(seed, branch_order)
```

```
# Simulate each branch

engine_results = {}

for branch in protocol.branches:

rng = branch_rngs[branch]

engine_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)


# Compare with frozen data

for branch in protocol.branches:

# Check split state (same for all branches)

if branch == protocol.branches[0]:
```

```
# Check split state keys

split_checks = [

("theta_at_split", prefix["theta_at_split"], frozen_files.get(f"prefix_seed_{seed_str}_theta_at_split")),

("B_at_split", prefix["B_at_split"], frozen_files.get(f"prefix_seed_{seed_str}_B_at_split")),

("coherence_acc_at_split", prefix["coherence_acc_at_split"], frozen_files.get(f"prefix_seed_{seed_str}
_coherence_acc_at_split")),

("history_at_split", prefix["history_at_split"], frozen_files.get(f"prefix_seed_{seed_str}
_history_at_split")),

]


for key, engine_val, frozen_val in split_checks:

if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"split_{key}_missing"] = True

all_passed = False

continue


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"split_{key}_exact"] = True
```

```
else:
seed_results["checks"][f"split_{key}_exact"] = False
seed_results["status"] = "failed"
all_passed = False
# Compute max diff for diagnostics
if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):
e = np.where(np.isnan(engine_val), 0.0, engine_val)
f = np.where(np.isnan(frozen_val), 0.0, frozen_val)
seed_results["max_diffs"][f"split_{key}"] = float(np.max(np.abs(e - f)))

# Check branch outputs
engine_branch = engine_results[branch]
frozen_prefix = f"{branch}_seed_{seed_str}"

for key, engine_val in engine_branch.items():
if isinstance(engine_val, np.ndarray):
frozen_key = f"{frozen_prefix}_{key}"
frozen_val = frozen_files.get(frozen_key)
```

```
if frozen_val is None:
seed_results["checks"][f"{branch}_{key}_missing"] = True
seed_results["status"] = "failed"
all_passed = False
continue

if exact_array_equal_with_nan(engine_val, frozen_val):
seed_results["checks"][f"{branch}_{key}_exact"] = True
else:
seed_results["checks"][f"{branch}_{key}_exact"] = False
seed_results["status"] = "failed"
all_passed = False
e = np.where(np.isnan(engine_val), 0.0, engine_val)
f = np.where(np.isnan(frozen_val), 0.0, frozen_val)
seed_results["max_diffs"][f"{branch}_{key}"] = float(np.max(np.abs(e - f)))

except Exception as e:
seed_results["status"] = "failed"
```

```
seed_results["error"] = str(e)

all_passed = False


results["seed_results"][seed_str] = seed_results

print(f" Seed {seed_str}: {seed_results['status']}")



if all_passed:

results["status"] = "passed"

print("Baseline equivalence: PASSED")

else:

results["status"] = "failed"

print("Baseline equivalence: FAILED")

raise RuntimeError("Baseline equivalence test FAILED. Study stopped.")



return results

================================================================================
Main API
```

```
================================================================================
====

def run_seed(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

branch_order: tuple[str, ...],

) -> dict[str, Any]:

text

"""Run a single seed through prefix + all branches.



Returns complete seed results with all branches.

"""

# Simulate prefix

split_state, prefix = simulate_prefix(seed, config, protocol)



# Create branch RNGs using frozen algorithm
```

```
branch_rngs = make_branch_rngs(seed, branch_order)



branch_results = {}

for branch in protocol.branches:

if branch in branch_rngs:

rng = branch_rngs[branch]

else:

# Fallback for branches not in branch_order

from numpy.random import SeedSequence

root = SeedSequence(seed)

*, branch*root = root.spawn(2)

sequences = branch_root.spawn(len(protocol.branches))

branch_idx = list(protocol.branches).index(branch)

rng = np.random.default_rng(int(sequences[branch_idx].generate_state(1, dtype=np.uint64)[0]))



branch_results[branch] = simulate_future(

split_state.clone(),
```

```
prefix,

branch,

config,

protocol,

rng,

)



return {

"seed": int(seed),

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,
```

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

if name == "main":

text

Quick validation

config = DEFAULT_MODEL_CONFIG

config.validate()

protocol = DEFAULT_PROTOCOL_CONFIG

protocol.validate(config)

print(f"Engine v1 loaded successfully")

print(f" N={config.N}, dt={config.dt}, tau={config.tau}")

print(f" T_window={config.T_window}, alpha={config.alpha:.6f}")

print(f" mean_omega={config.mean_omega:.6f}, T_*={config.T_star:.6f}")

print(f" delay_steps={config.delay_steps}")

print(f" Protocol: T_sim={protocol.T_sim}, burn={protocol.burn_end_time}, "

f"split={protocol.split_time}, future={protocol.future_duration}")

This rewrite is a major improvement in parameter fidelity and protocol timing, but it still will not reproduce the frozen baseline. The mandatory equivalence gate should fail because prefix RNG initialization differs from production. The engine also still omits several outputs required by the TwindowTwindow study.

Do not run the parameter grid yet.
1. Baseline RNG initialization is still wrong

Production does:

Python

root = np.random.SeedSequence(seed)
init_sequence, _ = root.spawn(2)
rng = np.random.default_rng(
init_sequence
)

The engine does:

Python

rng = np.random.default_rng(
int(
init_sequence.generate_state(
1,
dtype=np.uint64,
)[0]
)
)

These are not generally the same RNG stream.

The distinction is important:

Python

default_rng(SeedSequence_object)

is not equivalent to:

Python

default_rng(
SeedSequence_object.generate_state(...)
)

The branch RNGs use the generated uint64 in production, but prefix initialization does not.

Correct make_prefix_rng:

Python

def make_prefix_rng(
seed: int,
) -> np.random.Generator:
root = np.random.SeedSequence(seed)
init_sequence, _ = root.spawn(2)

return np.random.default_rng(
init_sequence
)

Then initialize_state should use this directly.

Inside simulate_prefix, the RNG passed to advance_state is unused when shuffle is false. For exact source correspondence, use:

Python

unused_rng = np.random.default_rng(0)

as production does.

This RNG correction is mandatory before baseline testing.
2. Protocol step properties hardcode dt=0.01dt=0.01

ProtocolConfig contains:

Python

return int(round(
self.T_sim / 0.01
))

and equivalent hardcoded calculations for burn, split, and future steps.

That contradicts the claim that model-changing quantities come from ModelConfig. It will also prevent later use for the timestep study.

Use methods taking the model:

Python

```
def n_steps(
self,
model: ModelConfig,
) -> int:
return int(round(
self.T_sim / model.dt
))


def burn_end(
self,
model: ModelConfig,
) -> int:
return int(round(
self.burn_end_time / model.dt
))
```

or store dt in ProtocolConfig and verify equality with the model.

For a TwindowTwindow-only v1, hardcoding dtdt does not change results, but it undermines the intended reusable architecture.
3. Baseline equivalence is incomplete

The function claims to require all original branch anchors, but it only compares keys that the engine happens to return:

Python

```
for key, engine_val in engine_branch.items():
```

Since simulate_future omits most production matrix outputs, the test cannot detect their absence.

It must iterate over the required frozen schema:

Python

```
for key in REQUIRED_BRANCH_MATRIX_KEYS:
if key not in engine_branch:
fail
```

if frozen_key not in frozen_data:
fail
compare(...)

The required baseline set should include at least:

text

B_final
B_mean_late
P_full
P_immediate
P_recovery
P_late
R_physical_future
R_driver_future
sample_times
density_trajectory
similarity_trajectory

Also compare production scalar outcomes.

As written, baseline equivalence could pass while the engine lacks most outputs needed by the study.
4. simulate_future still lacks required mature-window outcomes

The study expects:

text

P_mean_late
Corr_Rpast_Plate
B_mean_late
P_late
retention
emergence

but the engine does not accumulate any PP or BB horizon means.

This field is mislabeled:

Python

```
"Corr_Rpast_Plate":
safe_corr(
R_past,
B_final,
)
```

It is a correlation between past RR and final BB, not between past RR and PlatePlate.

Either rename it:

text

```
Corr_Rpast_Bfinal
```

or implement the actual PlatePlate accumulation.

The primary TwindowTwindow estimand requires fixed-physical and normalized-EMA horizons. The engine needs to accumulate both explicitly.

For example:

Python

```
horizon_systems = {
"fixed_physical": ...,
"normalized_EMA": ...,
}
```

For each system, accumulate:

text

P_immediate
P_recovery
P_late
B_mean_late
counts

Do not silently use one horizon system for both.
5. Recovery outcomes are still absent

The engine returns trajectories, which is useful, but the study driver must compute:

condition-specific continue thresholds;
first crossing;
sustained recovery;
held duration;
censoring;
restricted recovery time;
AUC deficit;
physical and TwindowTwindow-normalized time.

If the previously shown analysis driver is unchanged, it looks for keys that the engine never creates and will produce empty recovery tables.

Either:

calculate recovery outcomes in the study analysis from the saved trajectories; or
add them to the simulation driver.

Keeping them in the analysis layer is cleaner.
6. Baseline parameter checks use tolerance incorrectly

This uses:

Python

np.allclose(
frozen_omega,
config.omega,

rtol=1e-15,
)

but leaves the default:

Python

atol=1e-8

So differences below 10−810−8 can pass.

For the frozen baseline, use exact equality:

Python

np.array_equal(
np.asarray(
frozen_model["omega"],
dtype=float,
),
np.asarray(
config.omega,
dtype=float,
),
)

Likewise, np.isclose defaults to atol=1e-8. Use exact stored values or specify:

Python

rtol=0.0,
atol=0.0,

for baseline parameters.

The model checks also omit:

text

a
q
delta
T_window
B_init_scale
scale_epsilon
capacity
alpha
delay_steps

Add them.
Tau fallback is wrong

This code:

Python

frozen_model.get(
"tau",
frozen_model.get(
"T_window",
1.0,
),
)

would use T_window as tau if tau were missing. A missing required key should fail, not substitute an unrelated parameter.
7. run_seed should not invent RNG rules for unknown branches

This fallback:

Python

if branch not in branch_rngs:
invent branch seed from protocol order

can silently create a new randomization scheme.

For this engine version, require every branch to be in the frozen branch namespace:

Python

```
if branch not in branch_rngs:
raise ValueError(
f"unregistered branch: {branch}"
)
```

A future study-specific intervention should define an explicit named RNG namespace and record it in the manifest.

8. The engine is not yet a checkpointing engine

The header advertises:

text

```
Serialization: checkpoint I/O
```

but there is no checkpoint save/load or resume implementation.

Either implement it or remove the claim for v1.

This is not a baseline-equivalence blocker, but the API documentation should match reality.

9. Baseline comparison should include source/import authority

The engine duplicates production formulas. Before treating it as a reference, its equivalence artifact should record:

```
engine source hash;
frozen production source hash;
frozen manifest hash;
frozen NPZ hash;
all 40 seeds;
required-key schema;
```

exact equality results;
maximum differences;
environment versions.

A printed “PASSED” is not enough. Save:

text

baseline_equivalence_by_seed.csv
baseline_equivalence_manifest.json

10. Dynamic state list is acceptable but should be validated

Using a list plus:

Python

append(...)
pop(0)

can reproduce a deque with fixed maximum length, provided:
|H|=delay steps+1
|H|=delay steps+1

at every step.

Add assertions:

Python

if len(state.history) != (
config.delay_steps + 1
):
raise RuntimeError(...)

before and after each update in validation mode.

For performance, a deque is preferable, but a list is not scientifically wrong if exact equivalence passes.
11. Frozen constants are now substantially correct

The actual values are now appropriate:

N=12N=12;
dt=0.01dt=0.01;
τ=0.5τ=0.5;
correct ωω;
η=0.04η=0.04;
μ=0.03μ=0.03;
γ=0.015γ=0.015;
Bc=0.12Bc=0.12;
exact T∗=25/ω¯T∗=25/ω¯.

Note that:
T∗=
T∗=

17.5029171528588\ldots,
]

not exactly 17.517.5. The production alpha correctly uses the unrounded value:
α=0.9994288298464774…
α=0.9994288298464774…

Use exact FROZEN_T_WINDOW, while describing it as approximately 17.517.5.
12. Metric helpers now mostly match production

The implementations of:

effective number;
Gini;
normalized entropy;
total edge weight;
density;
cosine similarity

are broadly aligned with production.

But baseline equivalence must compare their actual returned scalar values, not merely assume formula similarity.

13. If the previous study/analysis drivers are unchanged, they remain broken

The earlier T-window driver:

did not save raw results;
skipped baseline equivalence;
used the wrong model config;
did not use horizon systems.

The earlier analysis driver:

expected a nonexistent JSON;
expected missing outcomes;
pooled different branches in DID summaries;
expected contrast while DID used DID;
did not implement cross-TT topology outcomes.

A corrected engine does not automatically repair those drivers. They need separate review.

Recommended next implementation gate

Do not revise the entire study stack again yet.

First make the engine prove exact baseline equivalence.

Minimal sequence

Fix prefix RNG.
Add the complete production branch output schema.
Make baseline validation iterate over required keys.
Run one seed.
Run all 40 seeds.
Save the equivalence certificate.

Only after the engine passes 40/40 should the T-window driver be written against its actual output schema.

Rating
Parameter fidelity: 8.5/10
Protocol fidelity: 7.5/10
RNG fidelity: 4/10 until prefix RNG is fixed
Baseline validation: 5/10 because it is real but incomplete
Study-output readiness: 4/10
Overall engine: 6/10

This is now a plausible reimplementation of the production kernel, not a different model. But it is not yet a validated reference engine. The prefix RNG error alone should cause the mandatory baseline test to fail, which is exactly why that test must be completed before further study code is trusted.

Now this one:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Driver.

Parameterized study varying T_window = rho T_ where T_* = 25 / mean(OMEGA) = 17.5

and rho ∈ {0.5, 0.75, 1, 1.5, 2}.

Study Design:

For each seed s ∈ seeds (40 seeds)

For each rho ∈ {0.5, 0.75, 1, 1.5, 2}

Set T_window = rho * 17.5

Set alpha = exp(-dt / T_window)

Full regeneration for each (s, rho) - NO split state reuse

Branches: continue, reset_ema, reset_B, future_shuffle

Fixed physical horizons: immediate=0-17.5, recovery=17.5-87.5, late=87.5-320

EMA-normalized horizons: immediate=0-1 T_window, recovery=1-5 T_window

Output:

Scalars in CSV: per (seed, rho, branch) scalar outcomes

Arrays in NPZ: per (seed, rho, branch) trajectory and matrix outcomes

MANDATORY: Baseline equivalence test at rho=1 must pass against frozen production.

"""

```
from future import annotations

import argparse

import csv

import hashlib

import json

import os

import platform

import secrets

import sys

import time

from pathlib import Path
```

```
from typing import Any

import numpy as np
Import the experiment engine

import aief_experiment_engine_v1 as engine
================================================================================
Study Configuration
================================================================================

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"

WORKFLOW_VERSION = "v1.0"
Frozen production values (from AIEF_1_4_memory_ablation_updated_REAL.py)

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0
```

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA))

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.5
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_STAR / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Rho values for T_window sensitivity

RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)
Branches for this study (subset of full branches)

STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Fixed physical horizon times (in time units, not steps)

PHYSICAL_IMMEDIATE_END = 17.5

PHYSICAL_RECOVERY_END = 87.5

PHYSICAL_LATE_START = 87.5

PHYSICAL_FUTURE_END = 320.0
Defaults

DEFAULT_MASTER_SEED = 20260714

DEFAULT_N_SEEDS = 40

DEFAULT_RUN_ROOT = "runs"
Outcome keys

PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)

BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

"Minimum_similarity",

"Time_to_similarity_below_0_9",

"Time_to_similarity_below_0_5",

"Late_persistence_ratio_to_continue",

"Late_similarity_ratio_to_continue",

Cross-T topology metrics

"T_window",

"rho",

)

BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
==========
Utility Functions
==========

def parse_master_seed(value: str) -> int:

text

"""Parse master seed from string."""

if value.strip().lower() == "random":

return secrets.randbits(64)

try:

```
result = int(value, 0)

except ValueError as exc:

raise argparse.ArgumentTypeError(

"master seed must be a nonnegative integer or 'random'"

) from exc

if result &lt; 0:

raise argparse.ArgumentTypeError("master seed must be nonnegative")

return result

def generate_seed_set(master_seed: int, count: int) -> list[int]:

text

"""Generate deterministic seed set from master seed."""

if count &lt;= 0:

raise ValueError("seed count must be positive")

children = np.random.SeedSequence(master_seed).spawn(count)

seeds = [

int(child.generate_state(1, dtype=np.uint64)[0])

for child in children

]
```

```
if len(set(seeds)) != len(seeds):

raise RuntimeError("duplicate generated seeds")

return seeds

def source_sha256() -> str | None:

text

"""Compute SHA-256 of this source file."""

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None

def atomic_write_text(path: str | Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temporary = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:
```

```
with open(temporary, "w", encoding="utf-8", newline="") as handle:
handle.write(text)
handle.flush()
os.fsync(handle.fileno())
os.replace(str(temporary), str(path))
finally:
if temporary.exists():
temporary.unlink()
def atomic_write_json(path: str | Path, data: dict[str, Any]) -> None:
text
"""Write JSON atomically with fsync."""
text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"
atomic_write_text(path, text)
def save_npz_atomic(path: str | Path, payload: dict[str, Any]) -> None:
text
"""Save NPZ atomically with fsync."""
path = Path(path)
path.parent.mkdir(parents=True, exist_ok=True)
```

```
temporary = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")
try:
np.savez_compressed(str(temporary), **payload)
os.replace(str(temporary), str(path))
finally:
if temporary.exists():
temporary.unlink()
def load_npz(path: str | Path) -> dict[str, Any]:
text
"""Load NPZ file."""
payload: dict[str, Any] = {}
with np.load(str(path), allow_pickle=False) as data:
for key in data.files:
value = data[key]
payload[key] = value.item() if value.ndim == 0 else value.copy()
return payload
================================================================================
```

Configuration and Validation
==========
====

def create_model_config(rho: float) -> engine.ModelConfig:

text

"""Create ModelConfig for a given rho value."""

T_window = rho * FROZEN_T_STAR

return engine.ModelConfig(

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=T_window,

```
B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=1e-12,

)

def create_protocol_config() -> engine.ProtocolConfig:

text

"""Create ProtocolConfig matching frozen production."""

return engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=STUDY_BRANCHES,

)

def compute_horizon_indices(

text
```

future_steps: int,

T_window: float,

dt: float,

) -> dict[str, tuple[int, int]]:

text

"""Compute horizon step indices for a given T_window.

Fixed physical horizons:

- immediate: 0-17.5 time units

- recovery: 17.5-87.5 time units

- late: 87.5-320 time units

EMA-normalized horizons:

- immediate: 0-1 T_window

- recovery: 1-5 T_window

- late: 5 T_window to end

```
Returns dict mapping horizon name to (start_step, end_step) inclusive start, exclusive end.

"""

window_steps = int(round(T_window / dt))



# Fixed physical horizons (in steps)

immediate_end_physical = int(round(PHYSICAL_IMMEDIATE_END / dt))

recovery_end_physical = int(round(PHYSICAL_RECOVERY_END / dt))



# EMA-normalized horizons (in steps)

immediate_end_normalized = window_steps # 0-1 T_window

recovery_end_normalized = 5 * window_steps # 1-5 T_window



# Use the more restrictive (shorter) horizon for each phase

immediate_end = min(immediate_end_physical, immediate_end_normalized)

recovery_end = min(recovery_end_physical, recovery_end_normalized)



# Clamp to future_steps
```

```
immediate_end = min(immediate_end, future_steps)

recovery_end = min(recovery_end, future_steps)



return {

"immediate": (0, immediate_end),

"recovery": (immediate_end, recovery_end),

"late": (recovery_end, future_steps),

}

# ============================================================================
# Simulation Functions
# ============================================================================

def simulate_seed_rho(

text

seed: int,

rho: float,

config: engine.ModelConfig,

protocol: engine.ProtocolConfig,

branch_order: tuple[str, ...],
```

) -> dict[str, Any]:

text

"""Simulate a single (seed, rho) combination.

Returns complete results for all branches at this (seed, rho).

"""

Run the seed through the engine

result = engine.run_seed(

seed=seed,

config=config,

protocol=protocol,

branch_order=branch_order,

)

Add rho and T_window to all branch results

T_window = config.T_window

for branch, branch_result in result["branches"].items():

```
branch_result["T_window"] = T_window

branch_result["rho"] = rho



return result

def run_baseline_equivalence(

text

seeds: list[int],

frozen_manifest_path: str,

frozen_npz_path: str,

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).



MANDATORY: Must reproduce frozen run exactly for all seeds.

This uses the engine's built-in baseline equivalence test.

"""

# Create config for rho=1
```

config = create_model_config(rho=1.0)

protocol = create_protocol_config()

Run the engine's baseline equivalence test

results = engine.run_baseline_equivalence(

seeds=seeds,

config=config,

protocol=protocol,

frozen_manifest_path=frozen_manifest_path,

frozen_npz_path=frozen_npz_path,

branch_order=STUDY_BRANCHES,

)

return results

==========
Manifest and Output
==========

```
def build_manifest(
text
master_seed: int,
seeds: list[int],
rho_values: tuple[float, ...],
) -> dict[str, Any]:
text
"""Build study manifest."""
# Compute config hash from model parameters
model_params = {
"N": FROZEN_N,
"dt": FROZEN_DT,
"tau": FROZEN_TAU,
"omega": FROZEN_OMEGA.tolist(),
"a": FROZEN_A,
"q": FROZEN_Q,
"delta": FROZEN_DELTA,
"eta": FROZEN_ETA,
```

"gamma": FROZEN_GAMMA,

"mu": FROZEN_MU,

"B_c": FROZEN_B_C,

"B_init_scale": FROZEN_B_INIT_SCALE,

"T_star": FROZEN_T_STAR,

}

protocol_params = {

"T_sim": FROZEN_T_SIM,

"burn_end_time": 160.0,

"split_time": 480.0,

"future_duration": 320.0,

"diagnostic_stride": 50,

"branches": list(STUDY_BRANCHES),

}

study_params = {

```
"rho_values": list(rho_values),

"T_window_values": [rho * FROZEN_T_STAR for rho in rho_values],

"n_seeds": len(seeds),

"statistical_unit": "40 seed/graphs",

}



manifest = {

"study_name": STUDY_NAME,

"workflow_version": WORKFLOW_VERSION,

"source_sha256": source_sha256(),

"model": model_params,

"protocol": protocol_params,

"study": study_params,

"randomization": {

"master_seed": int(master_seed),

"paired_seeds": [int(seed) for seed in seeds],

"seed_generation": "SeedSequence(master_seed).spawn(n_seeds)",
```

"branch_generation": "second per-seed child sequence spawns named uint64 branch seeds",

"bit_generator": np.random.default_rng(0).bit_generator.__class__.__name__,

},

"environment": {

"python": platform.python_version(),

"numpy": np.__version__,

"platform": platform.platform(),

},

"schema": {

"prefix_keys": list(PREFIX_KEYS),

"branch_scalar_keys": list(BRANCH_SCALAR_KEYS),

"branch_matrix_keys": list(BRANCH_MATRIX_KEYS),

},

"notes": {

"purpose": "T_window sensitivity study - varying T_window = rho *T_*",

"T_*": FROZEN_T_STAR,

"fixed_physical_horizons": {

"immediate": f"0-{PHYSICAL_IMMEDIATE_END} time units",

"recovery": f"{PHYSICAL_IMMEDIATE_END}-{PHYSICAL_RECOVERY_END} time units",

"late": f"{PHYSICAL_RECOVERY_END}-{PHYSICAL_FUTURE_END} time units",

},

"ema_normalized_horizons": {

"immediate": "0-1 T_window",

"recovery": "1-5 T_window",

"late": "5 T_window to end",

},

"full_regeneration": "Each (seed, rho) pair is independently initialized",

},

}

return manifest

def flatten_result(

text

seed: int,

```
rho: float,
result: dict[str, Any],
) -> dict[str, Any]:
text
"""Flatten a (seed, rho) result into a flat dict for NPZ storage."""
payload: dict[str, Any] = {
f"seed_{seed}_rho_{rho}_config_T_window": result["config"]["T_window"],
f"seed_{seed}_rho_{rho}_config_alpha": result["config"]["alpha"],
}


# Add prefix keys
for key, value in result["prefix"].items():
payload[f"prefix_seed_{seed}_rho_{rho}_{key}"] = value


# Add branch results
for branch, branch_result in result["branches"].items():
for key, value in branch_result.items():
```

```
if isinstance(value, np.ndarray):

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value

else:

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value



return payload

def extract_scalar_row(

text

seed: int,

seed_index: int,

rho: float,

branch: str,

branch_result: dict[str, Any],

) -> dict[str, Any]:

text

"""Extract a scalar row for CSV output."""

row = {
```

```
"seed_index": seed_index,

"seed": seed,

"rho": rho,

"T_window": branch_result.get("T_window", float('nan')),

"branch": branch,

}



for key in BRANCH_SCALAR_KEYS:

if key in branch_result:

row[key] = branch_result[key]

else:

row[key] = float('nan')



return row

================================================================================
Main Execution
================================================================================

def parse_arguments() -> argparse.Namespace:
```

```
text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

parser.add_argument(

"--master-seed",

type=parse_master_seed,

default=DEFAULT_MASTER_SEED,

help="fixed integer master seed or 'random' (default: 20260714)",

)

parser.add_argument(

"--n-seeds",

type=int,

default=DEFAULT_N_SEEDS,

help=f"number of seeds (default: {DEFAULT_N_SEEDS})",
```

)

parser.add_argument(

"--run-root",

default=DEFAULT_RUN_ROOT,

help=f"output root directory (default: {DEFAULT_RUN_ROOT})",

)

parser.add_argument(

"--frozen-manifest",

default=None,

help="path to frozen manifest for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-npz",

default=None,

help="path to frozen NPZ for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

```
)
parser.add_argument(
"--skip-baseline",
action="store_true",
help="skip baseline equivalence test (NOT RECOMMENDED)",
)
parser.add_argument(
"--dry-run",
action="store_true",
help="validate configuration and exit without running",
)
return parser.parse_args()
def main() -> None:
text
"""Main entry point."""
args = parse_arguments()
```

```
# Generate seeds

seeds = generate_seed_set(int(args.master_seed), args.n_seeds)



# Build manifest

manifest = build_manifest(

master_seed=int(args.master_seed),

seeds=seeds,

rho_values=RHO_VALUES,

)



# Compute config hash

manifest_text = json.dumps(

manifest, sort_keys=True, separators=(",", ":"), allow_nan=False

)

config_hash = hashlib.sha256(manifest_text.encode()).hexdigest()[:12]



# Create run directory
```

```
run_dir = Path([args.run](http://args.run)_root) / config_hash

run_dir.mkdir(parents=True, exist_ok=True)



# Save manifest

manifest["config_hash"] = config_hash

manifest_path = run_dir / "manifest.json"

atomic_write_json(manifest_path, manifest)



print(f"{STUDY_NAME}")

print(f" master seed: {args.master_seed}")

print(f" n_seeds: {args.n_seeds}")

print(f" rho values: {RHO_VALUES}")

print(f" configuration: {config_hash}")

print(f" output: {run_dir}")

print()



# Dry run check
```

```
if args.dry_run:

print("Dry run: configuration validated successfully.")

print(f"Manifest saved to {manifest_path}")

return



# Create protocol config (same for all rho)

protocol = create_protocol_config()



# MANDATORY: Run baseline equivalence test at rho=1

if not args.skip_baseline:

print("=" * 80)

print("Running MANDATORY baseline equivalence test at rho=1...")

print("=" * 80)



# Check if frozen data paths are provided

if args.frozen_manifest is None or args.frozen_npz is None:

print("\nFrozen data paths not provided!")
```

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then provide the paths with --frozen-manifest and --frozen-npz")

sys.exit(1)

Check if frozen data paths exist

if not os.path.exists(args.frozen_manifest):

print(f"\nFrozen manifest not found: {args.frozen_manifest}")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)

if not os.path.exists(args.frozen_npz):

print(f"\nFrozen NPZ not found: {args.frozen_npz}")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

```
print("Then use the generated run directory path.")

sys.exit(1)


try:

baseline_results = run_baseline_equivalence(

seeds=seeds[:10], # Test with first 10 seeds for speed

frozen_manifest_path=args.frozen_manifest,

frozen_npz_path=args.frozen_npz,

)


if baseline_results["status"] != "passed":

error_msg = baseline_results.get('error', 'unknown')

print(f"\nBaseline equivalence test FAILED: {error_msg}")


# Check if it's a configuration mismatch

if "mismatch" in error_msg.lower():

print("\nConfiguration mismatch detected!")

print("The frozen data does not match the expected production configuration.")
```

```
print("Expected configuration (from AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)):")

print(f" N={FROZEN_N}, DT={FROZEN_DT}, TAU={FROZEN_TAU}")

print(f" OMEGA={FROZEN_OMEGA.tolist()}")

print(f" T_WINDOW={FROZEN_T_STAR:.6f}")

print("\nTo generate correct frozen data:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path with --frozen-manifest and --frozen-npz")



sys.exit(1)

else:

print("\nBaseline equivalence test PASSED")

print("=" * 80)

except Exception as e:

print(f"\nBaseline equivalence test FAILED with exception: {e}")

import traceback

traceback.print_exc()

sys.exit(1)
```

```
else:

print("WARNING: Skipping baseline equivalence test (--skip-baseline)")

print("=" * 80)



# Run the study

print("\nRunning T_window sensitivity study...")

print(f" rho values: {RHO_VALUES}")

print(f" branches: {STUDY_BRANCHES}")

print(f" seeds: {len(seeds)}")



all_scalar_rows: list[dict[str, Any]] = []

all_matrix_payload: dict[str, Any] = {}



started = time.time()



for seed_index, seed in enumerate(seeds):

for rho in RHO_VALUES:
```

```
# Create config for this rho

config = create_model_config(rho)


# Run simulation

result = simulate_seed_rho(

seed=seed,

rho=rho,

config=config,

protocol=protocol,

branch_order=STUDY_BRANCHES,

)


# Extract scalar rows

for branch in STUDY_BRANCHES:

branch_result = result["branches"][branch]

row = extract_scalar_row(

seed=seed,

seed_index=seed_index,
```

```
rho=rho,

branch=branch,

branch_result=branch_result,

)

all_scalar_rows.append(row)


# Extract matrix payload

matrix_payload = flatten_result(seed, rho, result)

all_matrix_payload.update(matrix_payload)


# Status

elapsed = time.time() - started

total_combinations = len(seeds) * len(RHO_VALUES)

current_combination = seed_index * len(RHO_VALUES) + list(RHO_VALUES).index(rho) + 1

print(f" [{current_combination}/{total_combinations}] seed={seed}, rho={rho:.2f} "

f"({elapsed:.1f}s)")



# Save outputs
```

```
csv_path = run_dir / f"{STUDY_NAME}_{config_hash}_scalar_summary.csv"

npz_path = run_dir / f"{STUDY_NAME}_{config_hash}_branch_diagnostics.npz"



# Write CSV

if all_scalar_rows:

# Sort by seed_index, rho, branch for consistent ordering

all_scalar_rows.sort(key=lambda r: (r["seed_index"], r["rho"], r["branch"]))



# Get all fieldnames

fieldnames = list(all_scalar_rows[0].keys())



temporary_csv = csv_path.with_suffix(csv_path.suffix + ".tmp")

try:

with open(temporary_csv, "w", newline="", encoding="utf-8") as handle:

writer = csv.DictWriter(handle, fieldnames=fieldnames)

writer.writeheader()

writer.writerows(all_scalar_rows)

handle.flush()
```

```
os.fsync(handle.fileno())

os.replace(str(temporary_csv), str(csv_path))

finally:

if temporary_csv.exists():

temporary_csv.unlink()



# Write NPZ

if all_matrix_payload:

save_npz_atomic(npz_path, all_matrix_payload)



# Print summary

elapsed = time.time() - started

print(f"\nCompleted in {elapsed:.1f}s")

print(f"Saved {csv_path}")

print(f"Saved {npz_path}")

print(f"Manifest: {manifest_path}")
```

```
# Quick summary statistics

print("\n" + "=" * 80)

print("Summary Statistics by (rho, branch)")

print("=" * 80)



# Group by rho and branch

import itertools

for rho in RHO_VALUES:

for branch in STUDY_BRANCHES:

rows = [r for r in all_scalar_rows if r["rho"] == rho and r["branch"] == branch]

if rows:

keys = ["D_B_final", "P_mean_late", "Weighted_similarity_late", "Retention_contrast_late"]

stats = {}

for key in keys:

values = [r[key] for r in rows if key in r and isinstance(r[key], (int, float)) and not np.isnan(r[key])]

if values:

stats[key] = f"{np.mean(values):.4f}±{np.std(values):.4f}"
```

else:

stats[key] = "N/A"

print(f" rho={rho:.2f}, branch={branch:<15} | " +

" | ".join(f"{k}={stats.get(k, 'N/A')}" for k in keys))

if name == "main":

text

main()

This driver has the correct high-level study grid and full-regeneration intent, but it does not yet implement the stated scientific estimands. It also contains two execution-critical problems: incorrect branch RNG mapping for future_shuffle and a broken atomic NPZ writer.

Do not run the full grid.
1. Branch RNG mapping is wrong for the subset

The frozen branch order is:

Python

```
(
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
"full_reset",
"future_shuffle",
)
```

The study subset is:

Python

```
(
"continue",
"reset_ema",
"reset_B",
"future_shuffle",
)
```

The engine currently spawns RNG sequences using the subset order. The fourth generated sequence is therefore assigned to future_shuffle, but in production the fourth sequence belongs to reset_B_ema. The production future_shuffle RNG is the seventh sequence.

This will make the frozen future_shuffle baseline anchor fail.

Always reconstruct RNGs using the complete frozen branch order:

Python

```
FROZEN_BRANCH_ORDER = (
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
"full_reset",
"future_shuffle",
)
```

Then simulate only STUDY_BRANCHES:

Python

```
branch_rngs = make_branch_rngs(
seed,
FROZEN_BRANCH_ORDER,
)
```

for branch in STUDY_BRANCHES:
rng = branch_rngs[branch]

Separate:

text

RNG branch namespace

from:

text

branches selected for this study

2. Baseline equivalence tests only 10 seeds

The script states that baseline equivalence is mandatory for all 40 seeds but calls:

Python

seeds=seeds[:10]

This is not sufficient for a freezeable five-level study.

Run:

Python

seeds=seeds

and require 40/40 exact success.

The baseline test must also verify that the generated seed list equals:

Python

```
frozen_manifest[
"randomization"
]["paired_seeds"]
```

in the same order.

Do not infer the seed list independently without comparing it to the frozen manifest.
3. --skip-baseline is unsafe

The CLI permits:

text

```
--skip-baseline
```

and then runs the complete grid normally.

If retained for debugging:

output must go to a separate nonproduction namespace;
manifest must contain:

JSON

```
"baseline_equivalence": "skipped",
"freezeable": false
```

report must display a prominent warning.

For the default production path, baseline equivalence must be mandatory.
4. Atomic NPZ writing is broken

The temporary filename is created as:

text

output.npz.tmp-...

Then:

Python

np.savez_compressed(
str(temporary),
...
)

appends .npz because the temporary name does not end in .npz. The actual file becomes:

text

output.npz.tmp-....npz

but os.replace tries to move:

text

output.npz.tmp-...

which does not exist.

Use a temporary file handle:

Python

def save_npz_atomic(
path: Path,
payload: dict[str, Any],
) -> None:
path.parent.mkdir(

```
parents=True,
exist_ok=True,
)

fd, temporary = tempfile.mkstemp(
dir=path.parent,
prefix=f".{path.name}.",
suffix=".tmp.npz",
)

try:
with os.fdopen(fd, "wb") as handle:
np.savez_compressed(
handle,
**payload,
)
handle.flush()
os.fsync(handle.fileno())

os.replace(temporary, path)
except Exception:
try:
os.unlink(temporary)
except FileNotFoundError:
pass
raise
```

5. Horizon systems are not implemented

The study promises:

fixed physical horizons;
EMA-normalized horizons.

But compute_horizon_indices is never used.

Worse, it calculates:

Python

```
min(
physical_boundary,
normalized_boundary,
)
```

which merges the two definitions rather than reporting both.

The study requires two separate horizon maps:

Python

```
{
"fixed_physical": {
"immediate": ...,
"recovery": ...,
"late": ...,
},
"normalized_EMA": {
"immediate": ...,
"recovery": ...,
"late": ...,
},
}
```

Every horizon-dependent scalar row must include:

text

```
horizon_system
```

Do not take the minimum of the two boundaries.
6. The engine does not produce the declared scalar schema

The driver expects:

text

P_mean_late
Weighted_similarity_late
Jaccard_final
Retention_selected_late
Emergence_unselected_late
Corr_Rpast_Plate
r_mean_future
projection diagnostics
...

The current engine does not return most of these.

extract_scalar_row silently fills missing outcomes with NaN, so the study can complete while producing mostly empty tables.

This must be a validation error:

Python

```
missing = (
set(REQUIRED_SCALAR_KEYS)
- set(branch_result)
)

if missing:
raise RuntimeError(
f"missing scalar outputs: {missing}"
)
```

The engine and driver need one authoritative schema, not separate copied lists.
7. Recovery scaling is not implemented

The driver does not compute:

first crossing;
sustained recovery;
held recovery;

censoring;
restricted time;
AUC deficit;
trec/Twindowtrec/Twindow.

It merely saves production branch outputs.

The analysis layer can compute recovery from trajectories, but the output must contain:

condition-specific continue trajectory;
branch trajectory;
physical sample times;
ρ,Twindow,αρ,Twindow,α;
horizon system definitions.

The current engine provides some trajectories, but no analysis in this driver applies them.
8. Cross-TT topology identity is absent

The scientific question asks whether each seed reaches the same topology as at ρ=1ρ=1, but the driver never computes:

text

cross_T_weight_cosine
cross_T_threshold_Jaccard
cross_T_normalized_weight_distance
cross_T_component_signature_match

These can be calculated in the analysis script from saved B_final, but they must actually be implemented there. Merely listing the desired outcomes is insufficient.
9. The manifest is written before validation and never finalized

The driver writes manifest.json before:

baseline equivalence;
condition execution;
seed completion;
output creation.

It never updates the manifest with:

text

baseline equivalence status
successful/failed conditions
output hashes
elapsed time
freezeability
actual output filenames

Use an initial run specification plus a final analysis manifest, or atomically rewrite the manifest at completion.

A failed baseline should mark the run failed/nonfreezeable.
10. No condition-status table or checkpointing

The grid contains:
40×5=200
40×5=200

fully regenerated prefixes and:
200×4=800
200×4=800

future branch trajectories.

The script stores everything in memory and writes only after completion. One late failure loses the entire run.

Implement per-(seed,ρ)(seed,ρ) atomic checkpoints containing:

text

seed
rho

config hash
run status
prefix arrays
branch results
source/input hashes

At minimum save:

text

twindow_seed_condition_status.csv

with every requested combination.

Resume must verify config hashes before accepting a checkpoint.
11. Condition failures are not handled

The study loop has no per-condition status handling. Any exception aborts the run without:

a failure manifest;
seed/rho status;
preserved completed conditions.

This is unacceptable for a large regenerated study.

The production rule should be:

checkpoint completed conditions;
record failure;
stop the freezeable run;
do not generate scientific summaries from an incomplete grid.

12. Memory use may become excessive

all_matrix_payload retains all arrays for all 200 conditions and 800 branches in memory until the end.

Once full production matrices such as PlatePlate, Bmean,lateBmean,late, and trajectories are added, this becomes unnecessary risk.

Write per-condition checkpoints or per-condition NPZ files, then consolidate only after successful completion.
13. The summary uses outcomes likely to be NaN

This section prints:

Python

P_mean_late
Weighted_similarity_late
Retention_contrast_late

but the engine currently does not generate them.

The likely output is:

text

N/A

for most columns. Add schema checks before simulation rather than discover this after 160 million steps.
14. Seed-generation comparison is missing

Generating the same master-seed cohort is appropriate, but the driver must compare:

Python

seeds

against the frozen manifest’s paired seed list.

If they differ, baseline equivalence is not testing the intended cohort.

15. The fixed physical boundary should be described exactly

The true baseline memory time is:
$T^*=$
$T^*=$

17.50291715\ldots
]

while production:

Python

WINDOW_STEPS = 1750

corresponds to 17.517.5 sampled physical units.

Using:

text

17.5 and 87.5

for fixed physical horizons is defensible because it matches production step boundaries. Document that these are step-derived baseline boundaries, not the exact unrounded T^*T^*.
16. Output naming is incomplete

The required study outputs include:

text

twindow_condition_manifest.csv
twindow_seed_status.csv
twindow_scalar_summary.csv
twindow_cross_condition_topology.csv
twindow_recovery_summary.csv
twindow_paired_contrasts.csv

twindow_branch_interactions.csv
twindow_branch_diagnostics.npz
twindow_sensitivity_report.txt
analysis_manifest.json

The current driver creates only:

scalar CSV;
branch NPZ;
initial manifest.

It does not produce the condition/status and analysis artifacts.

That is acceptable only if a separate analysis script reliably produces them. The manifest should explicitly distinguish simulation outputs from analysis outputs.
What is correct

correct frozen parameters;
exact five-level ρρ grid;
full prefix regeneration per (s,ρ)(s,ρ);
same initial seeds across conditions;
sensible branch subset;
correct physical study duration;
explicit seed-level pairing;
deterministic manifest/config hash;
atomic CSV writing;
branch arrays preserved in NPZ.

Required sequence before running the grid

Fix the engine prefix RNG.
Make the engine return the required output schema.
Fix branch RNG reconstruction using the full frozen branch order.
Fix atomic NPZ output.
Run baseline equivalence on all 40 seeds.
Save the 40-seed equivalence certificate.
Run one nonbaseline (seed,ρ)(seed,ρ) smoke condition.
Validate both horizon systems and recovery analysis.
Add condition-level checkpoint/resume.

Only then run 200 regenerated conditions.

Rating
Study design: 8.5/10
Pairing/regeneration logic: 8/10
Baseline-gate enforcement: 5/10
Output-schema compatibility: 3/10
Recovery/horizon implementation: 2/10
Large-run robustness: 3/10
Current overall driver: 4.5/10

The driver is a good study specification embedded in incomplete production code. Its main risk is that it can run an expensive grid while silently producing NaN for the primary outcomes. The correct next deliverable is not the full sensitivity run; it is a verified 40-seed baseline-equivalence certificate from the engine.

And this one:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Analysis Driver.

Analysis driver for T_window sensitivity study results.

Input:

CSV file with scalar outcomes from AIEF_1_4_Twindow_sensitivity.py

NPZ file with matrix/trajectory outcomes

Output:

Analysis reports (text and JSON)

Aggregated statistics

Cross-T topology metrics

Visualization-ready data

Features:

Per-rho aggregation across seeds

Per-branch aggregation across seeds and rho

Cross-T topology metric computation

Statistical significance testing

Report generation

"""

from future import annotations

import argparse

import csv

import json

import os

import sys

from pathlib import Path

from typing import Any

import numpy as np

```
================================================================================
Configuration
================================================================================

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"

ANALYSIS_VERSION = "v1.0"
Expected values from study design

RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)

T_STAR = 17.5

T_WINDOW_VALUES = tuple(rho * T_STAR for rho in RHO_VALUES)

STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Primary outcomes

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late",

"Retention_contrast_late",

"P_mean_late",

)
Key metrics for cross-T analysis

CROSS_T_METRICS = (

text
```

"D_B_final",

"P_mean_late",

"Weighted_similarity_late",

"Retention_contrast_late",

"Retention_selected_late",

"Emergence_unselected_late",

"Total_edge_weight",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"Projection_active_fraction",

"Minimum_similarity",

)
Branch families for aggregation

BRANCH_FAMILIES = {

text

"reference": ("continue",),

"memory_ablation": ("reset_ema", "reset_B"),

"driver_control": ("future_shuffle",),

}
==========
Utility Functions
==========

def load_csv(path: str | Path) -> list[dict[str, Any]]:

text

"""Load CSV file into list of dicts."""

rows = []

with open(path, "r", encoding="utf-8", newline="") as handle:

reader = csv.DictReader(handle)

for row in reader:

Convert numeric fields

for key, value in row.items():

try:

if "." in value or "e" in value.lower():

row[key] = float(value)

elif value.lstrip("-").isdigit():

```
row[key] = int(value)

except (ValueError, AttributeError):

pass

rows.append(row)

return rows

def load_npz(path: str | Path) -> dict[str, Any]:

text

"""Load NPZ file."""

payload: dict[str, Any] = {}

with np.load(str(path), allow_pickle=False) as data:

for key in data.files:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

return payload

def load_manifest(path: str | Path) -> dict[str, Any]:

text

"""Load manifest JSON."""

with open(path, "r", encoding="utf-8") as handle:
```

```
return json.load(handle)

def save_json(path: str | Path, data: dict[str, Any]) -> None:

text

"""Save JSON file."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

with open(path, "w", encoding="utf-8") as handle:

json.dump(data, handle, indent=2, sort_keys=True, allow_nan=False)

handle.write("\n")

def save_csv(path: str | Path, rows: list[dict[str, Any]]) -> None:

text

"""Save CSV file."""

if not rows:

return

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

fieldnames = list(rows[0].keys())
```

```
with open(path, "w", newline="", encoding="utf-8") as handle:

writer = csv.DictWriter(handle, fieldnames=fieldnames)

writer.writeheader()

writer.writerows(rows)

def parse_run_dir(run_dir: str | Path) -> tuple[Path, Path, Path, Path]:

text

"""Parse run directory to find input files."""

run_dir = Path(run_dir)



# Find manifest

manifest_path = run_dir / "manifest.json"

if not manifest_path.exists():

raise FileNotFoundError(f"manifest.json not found in {run_dir}")



# Find CSV

csv_files = list(run_dir.glob(f"{STUDY_NAME}_*_scalar_summary.csv"))

if not csv_files:
```

```
raise FileNotFoundError(f"CSV file not found in {run_dir}")

csv_path = csv_files[0]



# Find NPZ

npz_files = list(run_dir.glob(f"{STUDY_NAME}_*_branch_diagnostics.npz"))

if not npz_files:

raise FileNotFoundError(f"NPZ file not found in {run_dir}")

npz_path = npz_files[0]



return run_dir, manifest_path, csv_path, npz_path

================================================================================
Data Aggregation
================================================================================

def aggregate_by_group(

text

data: list[dict[str, Any]],

group_keys: tuple[str, ...],

value_keys: tuple[str, ...],
```

) -> dict[tuple, dict[str, dict[str, float]]]:

text

"""Aggregate data by grouping keys, computing statistics for each value key.

Returns nested dict: {(group_values): {value_key: {stat: value}}}

"""

from collections import defaultdict

groups = defaultdict(lambda: defaultdict(list))

for row in data:

Create group key

group_key = tuple(row[k] for k in group_keys)

for vk in value_keys:

if vk in row and isinstance(row[vk], (int, float)):

groups[group_key][vk].append(float(row[vk]))

```
# Compute statistics

result = {}

for group_key, values in groups.items():

result[group_key] = {}

for vk, vals in values.items():

vals = np.array(vals)

finite_vals = vals[np.isfinite(vals)]

if len(finite_vals) &gt; 0:

result[group_key][vk] = {

"n": int(len(finite_vals)),

"mean": float(np.mean(finite_vals)),

"std": float(np.std(finite_vals)),

"min": float(np.min(finite_vals)),

"max": float(np.max(finite_vals)),

"median": float(np.median(finite_vals)),

"q25": float(np.percentile(finite_vals, 25)),
```

"q75": float(np.percentile(finite_vals, 75)),

}

else:

result[group_key][vk] = {

"n": 0,

"mean": float('nan'),

"std": float('nan'),

"min": float('nan'),

"max": float('nan'),

"median": float('nan'),

"q25": float('nan'),

"q75": float('nan'),

}

return result

def aggregate_per_rho(

text

```
data: list[dict[str, Any]],

value_keys: tuple[str, ...] = CROSS_T_METRICS,

) -> dict[float, dict[str, dict[str, float]]]:

text

"""Aggregate data per rho value, across all seeds and branches."""

return aggregate_by_group(data, ("rho",), value_keys)

def aggregate_per_branch(

text

data: list[dict[str, Any]],

value_keys: tuple[str, ...] = CROSS_T_METRICS,

) -> dict[str, dict[str, dict[str, float]]]:

text

"""Aggregate data per branch, across all seeds and rho."""

return aggregate_by_group(data, ("branch",), value_keys)

def aggregate_per_rho_branch(

text

data: list[dict[str, Any]],

value_keys: tuple[str, ...] = CROSS_T_METRICS,
```

) -> dict[tuple, dict[str, dict[str, float]]]:

text

"""Aggregate data per (rho, branch) combination."""

return aggregate_by_group(data, ("rho", "branch"), value_keys)

==========
Cross-T Topology Metrics
==========

def compute_cross_t_metrics(

text

data: list[dict[str, Any]],

) -> dict[str, Any]:

text

"""Compute cross-T topology metrics comparing different T_window values.

Metrics:

- Consistency: How similar are topologies across different T_window values?

- Stability: How stable is topology within a T_window across seeds?

- Sensitivity: How much do metrics change with T_window?

"""

results = {}

Group by seed and rho

seed_rho_data = {}

for row in data:

key = (row["seed"], row["rho"])

if key not in seed_rho_data:

seed_rho_data[key] = {}

seed_rho_data[key][row["branch"]] = row

Compute pairwise similarities between rho values (for continue branch)

rho_pairs = []

for i, rho1 in enumerate(RHO_VALUES):

for rho2 in RHO_VALUES[i+1:]:

rho_pairs.append((rho1, rho2))

```
pairwise_similarities = {}

for rho1, rho2 in rho_pairs:

similarities = []

for seed in set(r["seed"] for r in data):

key1 = (seed, rho1)

key2 = (seed, rho2)

if key1 in seed_rho_data and key2 in seed_rho_data:

if "continue" in seed_rho_data[key1] and "continue" in seed_rho_data[key2]:

sim1 = seed_rho_data[key1]["continue"].get("Weighted_similarity_late", np.nan)

sim2 = seed_rho_data[key2]["continue"].get("Weighted_similarity_late", np.nan)

# We want similarity between B matrices at different T_window

# For now, use the similarity metric itself as proxy

if np.isfinite(sim1) and np.isfinite(sim2):

similarities.append(abs(sim1 - sim2))


if similarities:

pairwise_similarities[(rho1, rho2)] = {
```

```
"mean_diff": float(np.mean(similarities)),

"std_diff": float(np.std(similarities)),

"n": len(similarities),

}



results["pairwise_rho_similarity_differences"] = pairwise_similarities



# Compute sensitivity metrics (slope of metrics vs rho)

sensitivity = {}

for metric in CROSS_T_METRICS:

# Collect (rho, value) pairs for continue branch

rho_values = []

metric_values = []

for row in data:

if row["branch"] == "continue" and metric in row:

rho_values.append(row["rho"])

metric_values.append(row[metric])
```

if len(rho_values) >= 2:

Fit linear regression

x = np.array(rho_values)

y = np.array(metric_values)

finite = np.isfinite(y)

x = x[finite]

y = y[finite]

if len(x) >= 2:

A = np.vstack([x, np.ones(len(x))]).T

m, c = np.linalg.lstsq(A, y, rcond=None)[0]

sensitivity[metric] = {

"slope": float(m),

"intercept": float(c),

"r_squared": float(1 - np.sum((y - (m*x + c))**2) / np.sum((y - np.mean(y))**2)) if np.std(y) > 0 else float('nan'),

}

```
else:
sensitivity[metric] = {
"slope": float('nan'),
"intercept": float('nan'),
"r_squared": float('nan'),
}
else:
sensitivity[metric] = {
"slope": float('nan'),
"intercept": float('nan'),
"r_squared": float('nan'),
}

results["sensitivity_slopes"] = sensitivity

# Compute topology consistency across T_window
# For each seed, compute correlation of metrics across rho values
```

```
seed_consistency = {}

for seed in set(r["seed"] for r in data):

seed_data = [r for r in data if r["seed"] == seed and r["branch"] == "continue"]

if len(seed_data) &gt;= 2:

# Compute correlation matrix of metrics across rho

metrics = list(CROSS_T_METRICS)

n_metrics = len(metrics)

corr_matrix = np.zeros((n_metrics, n_metrics))


for i, m1 in enumerate(metrics):

for j, m2 in enumerate(metrics):

vals1 = np.array([r.get(m1, np.nan) for r in seed_data])

vals2 = np.array([r.get(m2, np.nan) for r in seed_data])

finite = np.isfinite(vals1) &amp;amp; np.isfinite(vals2)

if np.sum(finite) &gt;= 2:

corr_matrix[i, j] = float(np.corrcoef(vals1[finite], vals2[finite])[0, 1])

else:

corr_matrix[i, j] = float('nan')
```

```
seed_consistency[seed] = {

"correlation_matrix": corr_matrix.tolist(),

"metrics": list(metrics),

}



results["seed_consistency"] = seed_consistency



return results

================================================================================
Statistical Analysis
================================================================================

def compute_statistical_tests(

text

data: list[dict[str, Any]],

comparisons: list[tuple[str, str, str]], # (metric, branch1, branch2)

) -> dict[str, Any]:

text
```

"""Compute statistical tests for specified comparisons.

Uses paired t-tests (since seeds are paired across branches).

"""

results = {}

for metric, branch1, branch2 in comparisons:

Collect paired values

values1 = []

values2 = []

for seed in set(r["seed"] for r in data):

seed_data = [r for r in data if r["seed"] == seed]

r1 = next((r for r in seed_data if r["branch"] == branch1), None)

r2 = next((r for r in seed_data if r["branch"] == branch2), None)

if r1 and r2 and metric in r1 and metric in r2:

v1 = r1[metric]

```
v2 = r2[metric]

if isinstance(v1, (int, float)) and isinstance(v2, (int, float)):

if np.isfinite(v1) and np.isfinite(v2):

values1.append(float(v1))

values2.append(float(v2))


if len(values1) &gt;= 2:

# Paired t-test

diffs = np.array(values1) - np.array(values2)

n = len(diffs)

mean_diff = float(np.mean(diffs))

std_diff = float(np.std(diffs, ddof=1))

se = std_diff / np.sqrt(n)

t_stat = mean_diff / se if se &gt; 0 else float('nan')


# Two-tailed p-value

from scipy import stats

p_value = float(stats.t.sf(abs(t_stat), n-1) * 2) if not np.isnan(t_stat) else float('nan')
```

```
results[f"{metric}_{branch1}_vs_{branch2}"] = {
"n": n,
"mean_diff": mean_diff,
"std_diff": std_diff,
"t_stat": t_stat,
"p_value": p_value,
"mean_branch1": float(np.mean(values1)),
"mean_branch2": float(np.mean(values2)),
}
else:
results[f"{metric}_{branch1}_vs_{branch2}"] = {
"n": len(values1),
"mean_diff": float('nan'),
"std_diff": float('nan'),
"t_stat": float('nan'),
"p_value": float('nan'),
"mean_branch1": float('nan'),
```

```
"mean_branch2": float('nan'),

}



return results

# ============================================================================
# Report Generation
# ============================================================================

def generate_text_report(

text

data: list[dict[str, Any]],

manifest: dict[str, Any],

cross_t_metrics: dict[str, Any],

statistical_tests: dict[str, Any],

) -> str:

text

"""Generate human-readable text report."""

lines = []

lines.append("=" * 80)
```

```
lines.append(f"AIEF-1.4 T_window Sensitivity Study Analysis Report")

lines.append("=" * 80)

lines.append("")



# Study info

lines.append("STUDY INFORMATION")

lines.append("-" * 40)

lines.append(f" Study name: {manifest.get('study_name', 'N/A')}")

lines.append(f" Workflow: {manifest.get('workflow_version', 'N/A')}")

lines.append(f" Config hash: {manifest.get('config_hash', 'N/A')}")

lines.append(f" N seeds: {manifest.get('study', {}).get('n_seeds', 'N/A')}")

lines.append(f" Rho values: {manifest.get('study', {}).get('rho_values', 'N/A')}")

lines.append(f" T_*: {manifest.get('notes', {}).get('T_*', 'N/A')}")

lines.append("")



# Model parameters

lines.append("MODEL PARAMETERS")
```

```
lines.append("-" * 40)

model = manifest.get("model", {})

for key, value in model.items():

if isinstance(value, list):

lines.append(f" {key}: {value[:5]}..." if len(value) &gt; 5 else f" {key}: {value}")

else:

lines.append(f" {key}: {value}")

lines.append("")



# Primary outcomes summary

lines.append("PRIMARY OUTCOMES BY (rho, branch)")

lines.append("-" * 40)



agg = aggregate_per_rho_branch(data, PRIMARY_OUTCOMES)

for (rho, branch) in sorted(agg.keys()):

stats = agg[(rho, branch)]

cells = []
```

```
for metric in PRIMARY_OUTCOMES:

if metric in stats:

s = stats[metric]

cells.append(f"{s['mean']:.4f}±{s['std']:.4f}")

else:

cells.append("N/A")

lines.append(f" rho={rho:.2f}, branch={branch:&lt;15} | " + " | ".join(cells))

lines.append("")



# Sensitivity analysis

lines.append("SENSITIVITY ANALYSIS (Slope vs rho)")

lines.append("-" * 40)

sensitivity = cross_t_metrics.get("sensitivity_slopes", {})

for metric, stats in sorted(sensitivity.items()):

if metric in PRIMARY_OUTCOMES:

lines.append(f" {metric}: slope={stats.get('slope', float('nan')):.4f}, "

f"r²={stats.get('r_squared', float('nan')):.4f}")
```

```
lines.append("")



# Pairwise rho differences

lines.append("PAIRWISE RHO SIMILARITY DIFFERENCES (continue branch)")

lines.append("-" * 40)

pairwise = cross_t_metrics.get("pairwise_rho_similarity_differences", {})

for (rho1, rho2), stats in sorted(pairwise.items()):

lines.append(f" rho={rho1:.2f} vs {rho2:.2f}: mean_diff={stats.get('mean_diff', float('nan')):.4f}, "

f"std={stats.get('std_diff', float('nan')):.4f}, n={stats.get('n', 0)}")

lines.append("")



# Statistical tests

lines.append("STATISTICAL TESTS (Paired t-tests)")

lines.append("-" * 40)

for test_name, stats in sorted(statistical_tests.items()):

if stats.get("p_value", float('nan')) &lt; 0.05:

sig = "*"
```

```
else:

sig = ""

lines.append(f" {test_name}{sig}: t={stats.get('t_stat', float('nan')):.4f}, "

f"p={stats.get('p_value', float('nan')):.4f}, "

f"n={stats.get('n', 0)}")

lines.append("")



# Branch comparisons

lines.append("BRANCH COMPARISONS (mean ± std across all rho)")

lines.append("-" * 40)



agg_branch = aggregate_per_branch(data, PRIMARY_OUTCOMES)

for branch in STUDY_BRANCHES:

if branch in agg_branch:

stats = agg_branch[branch]

cells = []

for metric in PRIMARY_OUTCOMES:
```

```
if metric in stats:

s = stats[metric]

cells.append(f"{s['mean']:.4f}±{s['std']:.4f}")

else:

cells.append("N/A")

lines.append(f" {branch:&lt;15} | " + " | ".join(cells))

lines.append("")



lines.append("=" * 80)

lines.append("END OF REPORT")

lines.append("=" * 80)



return "\n".join(lines)

def generate_json_report(

text

data: list[dict[str, Any]],

manifest: dict[str, Any],
```

```
cross_t_metrics: dict[str, Any],

statistical_tests: dict[str, Any],

) -> dict[str, Any]:

text

"""Generate machine-readable JSON report."""

report = {

"study_name": STUDY_NAME,

"analysis_version": ANALYSIS_VERSION,

"timestamp": int(time.time()),

"manifest": manifest,

"summary_statistics": {

"per_rho": aggregate_per_rho(data, CROSS_T_METRICS),

"per_branch": aggregate_per_branch(data, CROSS_T_METRICS),

"per_rho_branch": aggregate_per_rho_branch(data, CROSS_T_METRICS),

},

"cross_t_metrics": cross_t_metrics,

"statistical_tests": statistical_tests,
```

"primary_outcomes": list(PRIMARY_OUTCOMES),

"rho_values": list(RHO_VALUES),

"t_window_values": list(T_WINDOW_VALUES),

"branches": list(STUDY_BRANCHES),

}

return report

==
Visualization Data
==

def prepare_visualization_data(

text

data: list[dict[str, Any]],

) -> dict[str, Any]:

text

"""Prepare data for visualization in a structured format."""

viz_data = {

"metadata": {

```
"study_name": STUDY_NAME,

"rho_values": list(RHO_VALUES),

"t_window_values": list(T_WINDOW_VALUES),

"branches": list(STUDY_BRANCHES),

"n_seeds": len(set(r["seed"] for r in data)),

},

"line_plots": {},

"bar_plots": {},

"scatter_plots": {},

}



# Line plots: metric vs rho for each branch

for metric in CROSS_T_METRICS:

plot_data = {}

for branch in STUDY_BRANCHES:

rho_vals = []

mean_vals = []
```

```
std_vals = []


for rho in RHO_VALUES:

rho_data = [r for r in data if r["rho"] == rho and r["branch"] == branch]

vals = [r.get(metric, np.nan) for r in rho_data if metric in r]

vals = np.array(vals)

finite = vals[np.isfinite(vals)]


if len(finite) &gt; 0:

rho_vals.append(rho)

mean_vals.append(float(np.mean(finite)))

std_vals.append(float(np.std(finite)))


if rho_vals:

plot_data[branch] = {

"rho": rho_vals,

"mean": mean_vals,

"std": std_vals,
```

```
}

if plot_data:
viz_data["line_plots"][metric] = plot_data


# Bar plots: metric by branch for each rho
for metric in PRIMARY_OUTCOMES:
plot_data = {}
for rho in RHO_VALUES:
branch_means = []
branch_stds = []
branch_names = []

for branch in STUDY_BRANCHES:
rho_branch_data = [r for r in data if r["rho"] == rho and r["branch"] == branch]
vals = [r.get(metric, np.nan) for r in rho_branch_data if metric in r]
vals = np.array(vals)
finite = vals[np.isfinite(vals)]
```

```
if len(finite) &gt; 0:

branch_names.append(branch)

branch_means.append(float(np.mean(finite)))

branch_stds.append(float(np.std(finite)))


if branch_names:

plot_data[f"rho_{rho}"] = {

"branch": branch_names,

"mean": branch_means,

"std": branch_stds,

}


if plot_data:

viz_data["bar_plots"][metric] = plot_data



# Scatter plots: pairwise metric comparisons

for i, metric1 in enumerate(CROSS_T_METRICS[:5]): # Limit to first 5 for demo

for metric2 in CROSS_T_METRICS[i+1:]:
```

```
x_vals = []

y_vals = []

labels = []


for rho in RHO_VALUES:

for branch in STUDY_BRANCHES:

rho_branch_data = [r for r in data if r["rho"] == rho and r["branch"] == branch]

if rho_branch_data:

x = np.mean([r.get(metric1, np.nan) for r in rho_branch_data if metric1 in r])

y = np.mean([r.get(metric2, np.nan) for r in rho_branch_data if metric2 in r])

if np.isfinite(x) and np.isfinite(y):

x_vals.append(float(x))

y_vals.append(float(y))

labels.append(f"rho={rho}, {branch}")


if x_vals:

viz_data["scatter_plots"][f"{metric1}_vs_{metric2}"] = {

"x": x_vals,
```

```
"y": y_vals,

"labels": labels,

"x_label": metric1,

"y_label": metric2,

}



return viz_data

# ============================================================================
# Main Execution
# ============================================================================

def parse_arguments() -> argparse.Namespace:

text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)
```

```
parser.add_argument(
"run_dir",
type=str,
help="run directory containing study outputs",
)
parser.add_argument(
"--output",
type=str,
default=None,
help="output directory for analysis (default: &lt;run_dir&gt;/analysis)",
)
parser.add_argument(
"--csv-only",
action="store_true",
help="only output CSV files (no JSON/text reports)",
)
parser.add_argument(
"--json-only",
```

```
action="store_true",

help="only output JSON report (no text/CSV)",

)

parser.add_argument(

"--viz-data",

action="store_true",

help="generate visualization data JSON",

)

return parser.parse_args()

def main() -> None:

text

"""Main entry point."""

args = parse_arguments()



# Parse run directory

run_dir, manifest_path, csv_path, npz_path = parse_run_dir([args.run](http://args.run)_dir)
```

```
# Load data

print(f"Loading data from {run_dir}...")

data = load_csv(csv_path)

manifest = load_manifest(manifest_path)

npz_data = load_npz(npz_path)



print(f" Loaded {len(data)} rows from CSV")

print(f" Loaded {len(npz_data)} arrays from NPZ")



# Set up output directory

if args.output is None:

output_dir = run_dir / "analysis"

else:

output_dir = Path(args.output)

output_dir.mkdir(parents=True, exist_ok=True)



# Compute aggregations
```

```
print("\nComputing aggregations...")

per_rho = aggregate_per_rho(data)

per_branch = aggregate_per_branch(data)

per_rho_branch = aggregate_per_rho_branch(data)



# Compute cross-T metrics

print("Computing cross-T topology metrics...")

cross_t_metrics = compute_cross_t_metrics(data)



# Compute statistical tests

print("Computing statistical tests...")



# Define comparisons: (metric, branch1, branch2)

comparisons = []

for metric in PRIMARY_OUTCOMES:

for i, b1 in enumerate(STUDY_BRANCHES):

for b2 in STUDY_BRANCHES[i+1:]:
```

```
comparisons.append((metric, b1, b2))



try:

statistical_tests = compute_statistical_tests(data, comparisons)

except ImportError:

print(" scipy not available, skipping statistical tests")

statistical_tests = {}



# Generate reports

print("\nGenerating reports...")



if not args.json_only:

# Text report

text_report = generate_text_report(data, manifest, cross_t_metrics, statistical_tests)

text_path = output_dir / "report.txt"

with open(text_path, "w", encoding="utf-8") as f:

f.write(text_report)
```

```
print(f" Saved text report to {text_path}")


# Print to console

print("\n" + "=" * 80)

print("ANALYSIS REPORT")

print("=" * 80)

print(text_report)



if not args.csv_only:

# JSON report

json_report = generate_json_report(data, manifest, cross_t_metrics, statistical_tests)

json_path = output_dir / "report.json"

save_json(json_path, json_report)

print(f" Saved JSON report to {json_path}")



# Aggregated CSV files

print("\nSaving aggregated data...")
```

```
# Per (rho, branch) aggregation

agg_rows = []

for (rho, branch), metrics in per_rho_branch.items():

row = {"rho": rho, "branch": branch}

for metric, stats in metrics.items():

row[f"{metric}_mean"] = stats["mean"]

row[f"{metric}_std"] = stats["std"]

row[f"{metric}_n"] = stats["n"]

agg_rows.append(row)



agg_csv_path = output_dir / "aggregated_by_rho_branch.csv"

save_csv(agg_csv_path, agg_rows)

print(f" Saved aggregated data to {agg_csv_path}")



# Per rho aggregation

rho_agg_rows = []
```

```
for rho, metrics in per_rho.items():

row = {"rho": rho}

for metric, stats in metrics.items():

row[f"{metric}_mean"] = stats["mean"]

row[f"{metric}_std"] = stats["std"]

rho_agg_rows.append(row)



rho_csv_path = output_dir / "aggregated_by_rho.csv"

save_csv(rho_csv_path, rho_agg_rows)

print(f" Saved per-rho aggregation to {rho_csv_path}")



# Per branch aggregation

branch_agg_rows = []

for branch, metrics in per_branch.items():

row = {"branch": branch}

for metric, stats in metrics.items():

row[f"{metric}_mean"] = stats["mean"]
```

```
row[f"{metric}_std"] = stats["std"]

branch_agg_rows.append(row)



branch_csv_path = output_dir / "aggregated_by_branch.csv"

save_csv(branch_csv_path, branch_agg_rows)

print(f" Saved per-branch aggregation to {branch_csv_path}")



# Cross-T metrics

cross_t_path = output_dir / "cross_t_metrics.json"

save_json(cross_t_path, cross_t_metrics)

print(f" Saved cross-T metrics to {cross_t_path}")



# Statistical tests

if statistical_tests:

stats_path = output_dir / "statistical_tests.json"

save_json(stats_path, statistical_tests)

print(f" Saved statistical tests to {stats_path}")
```

```
# Visualization data

if args.viz_data:

print("\nGenerating visualization data...")

viz_data = prepare_visualization_data(data)

viz_path = output_dir / "visualization_data.json"

save_json(viz_path, viz_data)

print(f" Saved visualization data to {viz_path}")



print("\n" + "=" * 80)

print("ANALYSIS COMPLETE")

print("=" * 80)

if name == "main":

text

main()
```

This analysis script does not implement the predeclared TwindowTwindow estimands. It replaces the required paired seed-level contrasts and branch difference-in-differences with marginal aggregation, proxy “topology” metrics, and incorrectly paired t-tests. It also contains JSON/runtime errors.

I would reject and rewrite it around the prespecified estimands.
1. Cross-TT topology is not computed

The script loads the NPZ:

Python

npz_data = load_npz(npz_path)

but never uses it.

Instead, it defines “cross-T similarity” as:

Python

abs(
Weighted_similarity_late_rho1
- Weighted_similarity_late_rho2
)

This is not similarity between the two regenerated topologies. Each Weighted_similarity_late is measured relative to that condition’s own split-state reference.

The required comparison is directly between final weight matrices:
Ss,ρ=
Ss,ρ=

\cos\left(
B^{\rm final}{s,\rho,\rm continue},
B^{\rm final}{s,1,\rm continue}
\right).
]

Also compute:
Js,ρ=
Js,ρ=

\operatorname{Jaccard}\left(
B^{\rm final}{s,\rho,\rm continue},
B^{\rm final}{s,1,\rm continue}
\right),
]

and:
Ds,ρ=
Ds,ρ=
\frac{
|B^{\rm final}_{s,\rho,\rm continue}

B^{\rm final}_{s,1,\rm continue}|2
}{
|B^{\rm final}{s,1,\rm continue}|_2
}.
]

These require the NPZ arrays. The current script's "pairwise rho similarity differences" should be removed.
2. The primary paired regime estimand is absent

The predeclared estimand is:
Δs,ρregime(Y)=
Δs,ρregime(Y)=
Y_{s,\rho,\rm continue}

Y_{s,1,\rm continue}.
]

The script never computes this.

Instead, it aggregates raw outcomes per rho across all seeds and branches:

Python

aggregate_per_rho(data)

That mixes:

continue;
reset EMA;
reset BB;
future shuffle.

A per-rho mean across branches has no scientific interpretation.

The analysis needs a tidy paired-contrast table:

text

seed
rho
outcome
Y_rho_continue
Y_baseline_continue
paired_contrast

Then bootstrap the 40 seed-level contrasts.
3. Branch difference-in-differences is absent

The required branch estimand is:
Is,ρ,b(Y)=
Is,ρ,b(Y)=
\left[
Y_{s,\rho,b}
Y_{s,\rho,\rm continue}
\right]
\left[
Y_{s,1,b}

Y_{s,1,\rm continue}
\right].
]

The current script never calculates it.

This must be the primary branch sensitivity analysis for:

text

reset_ema
reset_B
future_shuffle

A tidy output should contain:

text

seed
rho
branch
outcome
branch_effect_rho
branch_effect_baseline
difference_in_differences

4. Statistical tests are paired incorrectly

For each seed, this code selects:

Python

next(
row
for row in seed_data
if row["branch"] == branch
)

There are five rows per branch—one for each rho. next simply selects the first row encountered, usually one arbitrary rho.

Thus the “paired t-test” does not identify which TwindowTwindow condition is being compared.

The test must pair by:

text

seed
rho
branch

Moreover, all pairwise branch t-tests are not the predeclared analysis. The primary inference should be deterministic seed-level bootstrap intervals for:

Δs,ρregimeΔs,ρregime;
Is,ρ,bIs,ρ,b.

If t-tests are retained as secondary diagnostics, they must be grouped by rho and multiplicity-labeled.
5. Sensitivity slopes treat repeated conditions as independent

The code pools all 200 seed/rho observations into:

Python

metric ~ rho

without seed effects or paired centering.

The slope is confounded by between-seed heterogeneity. The reported R2R2 is not a within-seed sensitivity measure.

If a trend estimate is desired, fit seed-centered contrasts:
Ys,ρ−Ys,1
Ys,ρ−Ys,1

against a prespecified function of rho, or fit a repeated-measures/mixed model. Levelwise paired contrasts should remain primary.
6. “Seed consistency” is not topology consistency

For each seed, the script computes correlations among different scalar metrics across five rho values.

That measures whether, for example, density and entropy co-vary across five points. It does not measure whether topology is stable across TwindowTwindow.

Remove this section or label it as a five-point scalar co-variation diagnostic. It is not a topology consistency result.
7. Recovery scaling is absent

The script does not calculate:

condition-specific continue targets;
first crossing;
sustained recovery;
held-duration recovery;
censoring;
restricted recovery time;
AUC deficit;
trec/Twindowtrec/Twindow;
fixed versus normalized horizons.

The raw trajectories are loaded into memory and then ignored.

For each seed/rho/branch, compute recovery relative to the same seed/rho continue trajectory.

Report both:
trec
trec

and:
trec/Twindow.
trec/Twindow.

Use all seeds in censoring denominators.
8. Fixed and normalized horizons are absent

The report mentions no actual horizon-specific computations.

The analysis must separately produce:

text

horizon_system = fixed_physical
horizon_system = normalized_EMA

Never merge these systems.
9. JSON output will fail

Several dictionaries use tuple keys:

Python

(rho1, rho2)

and:

Python

(rho, branch)

JSON object keys must be strings.

Also, allow_nan=False will reject the numerous NaN values explicitly inserted into the aggregation dictionaries.

The generated JSON report will therefore fail unless keys are converted to rows/strings and nonfinite values are sanitized.
Missing import

generate_json_report calls:

Python

time.time()

but time is not imported. This is another immediate runtime error.
10. The NPZ is loaded unnecessarily into memory

The analysis loads every array:

Python

npz_data = load_npz(npz_path)

but does not use any of them.

For a complete sensitivity run this could consume substantial memory for no reason. Once cross-TT and recovery analyses are implemented, load arrays selectively by key.
11. Hardcoded design values ignore the run manifest

The script hardcodes:

Python

RHO_VALUES
T_STAR = 17.5
STUDY_BRANCHES

It should read the actual grid, exact T∗T∗, and branches from the run manifest, then validate them against expected values.

The exact production value is approximately:
T∗=17.50291715,
T∗=17.50291715,

not exactly 17.517.5.
12. Completeness and baseline status are not validated

Before analysis, require:

baseline-equivalence status is passed;
run is freezeable;
all 40 seeds are present;
all five rho values are present for every seed;
all four branches are present for every seed/rho;
no duplicate rows;
no failed condition statuses.

Expected scalar row count:
40×5×4=800.
40×5×4=800.

Do not analyze an incomplete grid silently.
13. Marginal aggregation across rho is scientifically misleading

This report section:

text

BRANCH COMPARISONS
(mean ± std across all rho)

treats five repeated parameter conditions per seed as extra observations and averages different dynamical regimes together.

Remove it. Branch effects should be reported by rho and through the prespecified DID.

Similarly:

Python

aggregate_per_rho(data)

across all branches should not be a primary result.
14. CSV parsing is fragile

The manual parser may leave:

text

nan
NaN
inf
-inf
empty strings

as strings.

Use pandas with explicit dtypes, especially keeping seed IDs as strings:

Python

```
frame = pd.read_csv(
csv_path,
dtype={"seed": str},
)
```

Then validate numeric columns with:

Python

```
pd.to_numeric(..., errors="raise")
```

15. CSV and JSON writes are not atomic

This analysis belongs to the frozen evidence lineage and should use the same atomic-output standards as the simulation driver.

It should also write an analysis_manifest.json containing:

analysis source hash;
simulation manifest hash;
scalar CSV hash;
NPZ hash;

command;
bootstrap settings;
output hashes;
row-completeness checks;
interpretation constraints.

The current report.json is not an adequate analysis manifest.
16. Output mode flags do not do what they say

--csv-only still writes the text report.

--json-only still writes aggregated CSV files.

The output branches should be corrected or the flags removed.
17. Visualization scatter plots are aggregate-of-aggregate diagnostics

The scatter plots compare branch/rho means rather than seed-level paired observations. They can be descriptive but should not be used inferentially.

They are not a priority before the primary contrast tables are correct.
Required rewrite structure

The analysis should have five explicit stages.
Stage 1 — validation

Validate:

text

800 scalar rows
40 unique seeds
5 rho values per seed
4 branches per seed/rho
baseline equivalence passed
run freezeable
no duplicate condition keys

Stage 2 — regime contrasts

For every nonbaseline rho:
Ys,ρ,continue−
Ys,ρ,continue−

Y_{s,1,\rm continue}.
]

Save:

text

twindow_paired_regime_contrasts.csv

Stage 3 — branch DID

For every nonbaseline rho and noncontinue branch:
(Ys,ρ,b−Ys,ρ,c)−
(Ys,ρ,b−Ys,ρ,c)−

(Y_{s,1,b}-Y_{s,1,c}).
]

Save:

text

twindow_branch_interactions.csv

Stage 4 — topology and recovery

Use NPZ matrices and trajectories to compute:

text

cross-T weight cosine

threshold Jaccard
normalized distance
recovery probability
sustained recovery
restricted time
AUC deficit
t_rec / T_window

for both horizon systems.
Stage 5 — seed-level summaries

Bootstrap only seed-level paired contrasts across 40 seeds.

Save:

text

twindow_regime_summary.csv
twindow_branch_DID_summary.csv
twindow_cross_condition_topology.csv
twindow_recovery_summary.csv
twindow_sensitivity_report.txt
analysis_manifest.json

Rating
Data aggregation implementation: 4/10
Statistical estimand fidelity: 1/10
Cross-T topology analysis: 1/10
Recovery/horizon analysis: 0.5/10
Runtime/output correctness: 2/10
Overall: 2/10

This is a generic marginal-summary script, not the specified paired TwindowTwindow sensitivity analysis. It should be rewritten around the seed-level regime contrast and branch difference-in-differences before any simulation grid is analyzed.

Now these rewrites:

"""AIEF Experiment Engine v1 - Parameterized simulation kernel.

This is a reusable, validated simulation engine that reproduces the frozen

AIEF-1.4 production run exactly and supports parameterized studies.

FROZEN PRODUCTION VALUES (from AIEF_1_4_memory_ablation_updated_REAL.py):

N = 12

DT = 0.01

TAU = 0.5

OMEGA = [1.00, 1.02, 1.50, 2.00, 2.01, 1.01, 1.03, 1.51, 2.02, 2.00, 1.00, 1.04]

A = 1.0

Q = 1.5

DELTA = 0.05

ETA = 0.04

GAMMA = 0.015

MU = 0.03

T_WINDOW = 25.0 / mean(OMEGA) = 17.5

B_C = 0.12

B_INIT_SCALE = 0.0005

PROTOCOL (from frozen run):

T_SIM = 800.0

N_STEPS = int(round(T_SIM / DT)) = 80000

BURN_END = int(0.20 * N_STEPS) = 16000 steps = 160.0 time units

HIST_END = int(0.60 * N_STEPS) = 48000 steps = 480.0 time units

FUTURE_STEPS = N_STEPS - HIST_END = 32000 steps = 320.0 time units

DELAY_STEPS = int(round(TAU / DT)) = 50

WINDOW_STEPS = int(round(T_WINDOW / DT)) = 1750

RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS) = 8750

Architecture:

ModelConfig: frozen dataclass for all model parameters

ProtocolConfig: frozen dataclass for protocol parameters

DynamicState: state container (immutable where possible)

Core functions: initialize, advance, simulate, interventions

Validation: baseline equivalence test

DO NOT MODIFY: This is the reference implementation.

"""

from future import annotations

```
import copy

import hashlib

import json

import math

import os

import tempfile

import time

from dataclasses import dataclass, field

from pathlib import Path

from typing import Any

import numpy as np
================================================================================
Frozen Production Constants (DO NOT CHANGE)
================================================================================

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text
```

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005

FROZEN_ZERO_TOL = 1e-12

FROZEN_PROJECTION_NORM_TOL = 1e-14

FROZEN_GAP_TOL = 1e-9
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA)) # 1.428333...

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.5

FROZEN_T_WINDOW = FROZEN_T_STAR # 17.5

FROZEN_ALPHA = math.exp(-FROZEN_DT / FROZEN_T_WINDOW)

FROZEN_CAPACITY = (FROZEN_A - FROZEN_DELTA) / FROZEN_Q
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_WINDOW / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
==========
Model Configuration
==========

@dataclass(frozen=True)

class ModelConfig:

text

"""Immutable model configuration.

All model-changing quantities must come from here. No hidden module globals.

"""

Core parameters

N: int

dt: float

tau: float

omega: tuple[float, ...] # Immutable tuple

Coupling parameters

a: float

q: float

delta: float

Dynamics parameters

eta: float

mu: float

gamma: float

EMA/threshold parameters

T_window: float

B_c: float

B_init_scale: float

scale_epsilon: float = 1e-12

Derived properties

@property

def mean_omega(self) -> float:

"""Mean of omega frequencies."""

return float(np.mean(self.omega))

@property

def T_star(self) -> float:

"""T_* = 25 / mean(omega)"""

```
return 25.0 / self.mean_omega



@property

def alpha(self) -&gt; float:

"""EMA smoothing factor: exp(-dt / T_window)"""

return math.exp(-self.dt / self.T_window)



@property

def capacity(self) -&gt; float:

"""Projection capacity: (a - delta) / q"""

return (self.a - [self.delta](http://self.delta)) / self.q



@property

def delay_steps(self) -&gt; int:

"""Delay steps: round(tau / dt)"""

return int(round(self.tau / self.dt))
```

```
def validate(self) -&gt; None:

"""Validate model configuration."""

if self.N &lt;= 0:

raise ValueError("N must be positive")

if self.dt &lt;= 0:

raise ValueError("dt must be positive")

if self.tau &lt;= 0:

raise ValueError("tau must be positive")

if len([self.omega](http://self.omega)) != self.N:

raise ValueError(f"omega must have {self.N} elements")

if self.a &lt;= 0 or self.q &lt;= 0 or [self.delta](http://self.delta) &lt; 0:

raise ValueError("a, q must be positive; delta must be nonnegative")

if self.eta &lt; 0 or [self.mu](http://self.mu) &lt; 0 or self.gamma &lt; 0:

raise ValueError("eta, mu, gamma must be nonnegative")

if self.T_window &lt;= 0:

raise ValueError("T_window must be positive")

if self.B_c &lt; 0:

raise ValueError("B_c must be nonnegative")
```

if self.B_init_scale < 0:

raise ValueError("B_init_scale must be nonnegative")

if self.scale_epsilon <= 0:

raise ValueError("scale_epsilon must be positive")

Validate derived values

if not (0 < self.alpha <= 1):

raise ValueError(f"alpha={self.alpha} must be in (0, 1]")

if self.capacity < 0:

raise ValueError(f"capacity={self.capacity} must be nonnegative")

@dataclass(frozen=True)

class ProtocolConfig:

text

"""Protocol configuration for experiment execution."""

Simulation time parameters (in time units, not steps)

T_sim: float

burn_end_time: float

split_time: float

future_duration: float

Diagnostics

diagnostic_stride: int

Branches to simulate

branches: tuple[str, ...]

def n_steps(self, model: ModelConfig) -> int:

"""Total steps."""

return int(round(self.T_sim / model.dt))

def burn_end(self, model: ModelConfig) -> int:

"""Burn-in end step."""

return int(round(self.burn_end_time / model.dt))

def hist_end(self, model: ModelConfig) -> int:

"""History end step (split point)."""

return int(round(self.split_time / model.dt))

def future_steps(self, model: ModelConfig) -> int:

"""Future simulation steps."""

return int(round(self.future_duration / model.dt))

def validate(self, model: ModelConfig) -> None:

"""Validate protocol against model."""

if self.T_sim <= 0:

raise ValueError("T_sim must be positive")

if not (0 < self.burn_end_time < self.split_time < self.T_sim):

raise ValueError("Invalid time parameters: must have 0 < burn_end < split < T_sim")

if self.future_duration <= 0:

raise ValueError("future_duration must be positive")

if self.diagnostic_stride <= 0:

raise ValueError("diagnostic_stride must be positive")

Validate that all time parameters are compatible with dt

if not np.isclose(self.T_sim / model.dt, round(self.T_sim / model.dt), rtol=1e-10):

raise ValueError(f"T_sim ({self.T_sim}) must be divisible by dt ({model.dt})")

if not np.isclose(self.burn_end_time / model.dt, round(self.burn_end_time / model.dt), rtol=1e-10):

raise ValueError(f"burn_end_time ({self.burn_end_time}) must be divisible by dt ({model.dt})")

if not np.isclose(self.split_time / model.dt, round(self.split_time / model.dt), rtol=1e-10):

raise ValueError(f"split_time ({self.split_time}) must be divisible by dt ({model.dt})")

if not np.isclose(self.future_duration / model.dt, round(self.future_duration / model.dt), rtol=1e-10):

raise ValueError(f"future_duration ({self.future_duration}) must be divisible by dt ({model.dt})")

Default frozen production configuration

DEFAULT_MODEL_CONFIG = ModelConfig(

text

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=FROZEN_T_WINDOW,

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=FROZEN_ZERO_TOL,

)
Default protocol matching frozen run

DEFAULT_PROTOCOL_CONFIG = ProtocolConfig(

text

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=("continue", "reset_ema", "reset_B", "reset_B_ema", "B_carrier_only", "full_reset", "future_shuffle"),

)
==========
Utility Functions
==========

def compute_edges(N: int) -> tuple[np.ndarray, np.ndarray]:

text

"""Get upper triangle indices for edges."""

return np.triu_indices(N, k=1)

def validate_B_matrix(B: np.ndarray, N: int, eps: float = 1e-12) -> None:

text

"""Validate B matrix: symmetric, nonnegative, zero diagonal."""

if B.shape != (N, N):

raise ValueError(f"B shape {B.shape} != ({N}, {N})")

if not np.allclose(B, B.T, atol=eps):

raise ValueError("B is not symmetric")

if not np.allclose(np.diag(B), 0, atol=eps):

raise ValueError("B diagonal is not zero")

if np.any(B < -eps):

raise ValueError("B has negative entries")

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file."""

digest = hashlib.sha256()

with open(path, "rb") as handle:

for block in iter(lambda: handle.read(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

def array_sha256(arr: np.ndarray) -> str:

text

"""Compute SHA-256 hash of a numpy array."""

digest = hashlib.sha256()

shape_bytes = np.asarray(arr.shape, dtype=np.int64).tobytes()

digest.update(shape_bytes)

digest.update(arr.dtype.str.encode())

```
contiguous = np.ascontiguousarray(arr)

if contiguous.size &gt; 0:

digest.update(contiguous.tobytes())

return digest.hexdigest()

def atomic_write_text(path: Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):
```

```
os.remove(tmp_path)

def atomic_write_json(path: Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def atomic_write_npz(path: Path, **kwargs: Any) -> None:

text

"""Write NPZ atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp.npz")

try:

with os.fdopen(fd, "wb") as handle:

np.savez(handle, **kwargs)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))
```

```
finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

================================================================================
RNG Utilities (matching frozen production exactly)
================================================================================

def make_seed_sequence(seed: int, child_index: int) -> int:

text

"""Generate a child seed from SeedSequence(seed).spawn(...)[child_index].



This matches the frozen production RNG hierarchy exactly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

children = root.spawn(child_index + 1)

return int(children[child_index].generate_state(1, dtype=np.uint64)[0])

def make_prefix_rng(seed: int) -> np.random.Generator:
```

text

"""Create prefix RNG matching frozen production exactly.

Frozen production does:

root = np.random.SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = np.random.default_rng(init_sequence)

NOTE: default_rng(SeedSequence_object) is NOT the same as

default_rng(SeedSequence_object.generate_state(...))

The production uses the SeedSequence object directly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

return np.random.default_rng(init_sequence)

def make_branch_rngs(seed: int, branch_order: tuple[str, ...]) -> dict[str, np.random.Generator]:

```
text

"""Create branch RNGs matching frozen production algorithm.



Frozen: *, branch*root = SeedSequence(seed).spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

branch_rng_seeds = {branch: int(sequence.generate_state(1, dtype=np.uint64)[0])

for branch, sequence in zip(branch_order, branch_sequences)}

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

*, branch*root = root.spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))



return {

branch: np.random.default_rng(int(sequence.generate_state(1, dtype=np.uint64)[0]))

for branch, sequence in zip(branch_order, branch_sequences)

}
```

```
================================================================================
Dynamic State
================================================================================

class DynamicState:

text

"""State container for AIEF-1.4 dynamics.



Matches the frozen production implementation exactly.

"""



def **init**(

self,

theta: np.ndarray,

B: np.ndarray,

coherence_acc: np.ndarray,

history: list[np.ndarray] | np.ndarray,

):

self.theta = np.asarray(theta, dtype=float).copy()
```

```
self.B = np.asarray(B, dtype=float).copy()

self.coherence_acc = np.asarray(coherence_acc, dtype=complex).copy()


# Handle history input - ensure it's a list of arrays

if isinstance(history, np.ndarray):

# If it's a 2D array, convert to list of 1D arrays

self.history = [np.asarray(h, dtype=float).copy() for h in history]

else:

self.history = [np.asarray(h, dtype=float).copy() for h in history]



def clone(self) -&gt; "DynamicState":

"""Create a deep copy of the state."""

return DynamicState(

theta=self.theta.copy(),

B=self.B.copy(),

coherence_acc=self.coherence_acc.copy(),

history=[h.copy() for h in self.history],
```

)

@property

def N(self) -> int:

return len(self.theta)

def validate(self, config: ModelConfig, step: int | None = None) -> None:

"""Validate state consistency."""

Check history length

expected_history_len = config.delay_steps + 1

if len(self.history) != expected_history_len:

raise RuntimeError(

f"History length {len(self.history)} != expected {expected_history_len} "

f"(delay_steps={config.delay_steps}) at step {step}"

)

Check theta shape

if self.theta.shape != (self.N,):

```
raise RuntimeError(f"theta shape {self.theta.shape} != ({self.N},)")

# Check B shape

if self.B.shape != (self.N, self.N):

raise RuntimeError(f"B shape {self.B.shape} != ({self.N}, {self.N})")

# Check coherence_acc shape

if self.coherence_acc.shape != (self.N, self.N):

raise RuntimeError(f"coherence_acc shape {self.coherence_acc.shape} != ({self.N}, {self.N})")

# Check B symmetry

if not np.allclose(self.B, self.B.T, atol=1e-12):

raise RuntimeError("B matrix is not symmetric")

# Check B diagonal

if not np.allclose(np.diag(self.B), 0, atol=1e-12):

raise RuntimeError("B matrix diagonal is not zero")

# Check B non-negativity

if np.any(self.B &lt; -1e-12):

raise RuntimeError("B matrix has negative entries")

================================================================================
Simulation Core (exact reproduction of frozen production)
```

```
================================================================================
====

def initialize_state(

text

seed: int,

config: ModelConfig,

) -> DynamicState:

text

"""Initialize state from seed using frozen algorithm exactly.



Matches production: simulate_prefix() initialization.

Uses: root = SeedSequence(seed); init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence)

"""

N = config.N

rng = make_prefix_rng(seed)



# Initialize theta: uniform in [0, 2*pi) - matching production
```

```
theta = rng.random(N) *2.0* np.pi


# Initialize B: small random symmetric matrix

B = config.B_init_scale * rng.random((N, N))

B = 0.5 * (B + B.T)

np.fill_diagonal(B, 0.0)


# Initialize coherence accumulator

coherence_acc = np.zeros((N, N), dtype=complex)


# Initialize history: DELAY_STEPS + 1 copies of theta

history = [theta.copy() for *in range(config.delay*steps + 1)]


return DynamicState(

theta=theta,

B=B,
```

```
coherence_acc=coherence_acc,

history=history,

)

def compute_R(

text

state: DynamicState,

config: ModelConfig,

) -> np.ndarray:

text

"""Compute coherence matrix R from current state.



Exact reproduction of production compute_R().

"""

N = state.N

delayed = state.history[0]



# Convert omega to numpy array for indexing
```

```
omega_arr = np.asarray([config.omega](http://config.omega), dtype=float)



dphi = (

state.theta[:, None]

- delayed[None, :]

- ((omega_arr[:, None] + omega_arr[None, :]) / 2.0) * config.tau

)

z_inst = np.exp(1j * dphi)



# Update EMA

state.coherence_acc[:] = (

config.alpha *state.coherence_acc + (1.0 - config.alpha)* z_inst

)



# Compute magnitude and symmetrize

result = np.abs(state.coherence_acc)

result = 0.5 * (result + result.T)
```

```
np.fill_diagonal(result, 0.0)



return result

def advance_state(

text

state: DynamicState,

config: ModelConfig,

rng: np.random.Generator,

shuffle_driver: bool = False,

) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:

text

"""Advance state by one time step.



Exact reproduction of production advance_state().

Returns (R_physical, R_driver, info).

"""

N = state.N
```

```
edges = compute_edges(N)



# Store previous B for projection

B_previous = state.B.copy()



# Compute R matrices

R_physical = compute_R(state, config)

R_driver = R_physical



if shuffle_driver:

values = R_physical[edges].copy()

rng.shuffle(values)

R_driver = np.zeros_like(R_physical)

R_driver[edges] = values

R_driver[(edges[1], edges[0])] = values



# Update B
```

```
B_unprojected = state.B + config.dt * (
(config.eta + [config.mu](http://config.mu)) * R_driver
- [config.mu](http://config.mu)
- config.gamma * state.B
)
B_unprojected = np.maximum(B_unprojected, 0.0)
B_unprojected = 0.5 * (B_unprojected + B_unprojected.T)
np.fill_diagonal(B_unprojected, 0.0)


# Check gap constraint
row_sum_pre = np.sum(B_unprojected, axis=1)
g_min_pre = float(np.min(config.a - config.q * row_sum_pre))
pre_violation = bool(g_min_pre &lt; [config.delta](http://config.delta))


# Project if needed
B_projected = B_unprojected
if pre_violation:
```

```
scale = np.minimum(1.0, config.capacity / (row_sum_pre + config.scale_epsilon))

B_projected = B_unprojected * np.minimum.outer(scale, scale)

np.fill_diagonal(B_projected, 0.0)



# Validate projection

state.B = B_projected

row_sum = np.sum(state.B, axis=1)

g_min = float(np.min(config.a - config.q * row_sum))

if g_min &lt; [config.delta](http://config.delta) - 1e-9:

raise RuntimeError(f"Gap projection failed: g_min={g_min}")



# Update theta

delayed = state.history[0]

omega_arr = np.asarray([config.omega](http://config.omega), dtype=float)

coupling = np.sum(

state.B * np.sin(delayed[None, :] - state.theta[:, None]), axis=1

)
```

```
state.theta = np.mod(

state.theta + config.dt * (omega_arr + coupling),

2.0 * np.pi

)



# Update history

state.history.append(state.theta.copy())

if len(state.history) &gt; config.delay_steps + 1:

state.history.pop(0)



info = {

"pre_violation": pre_violation,

"g_min_pre": g_min_pre,

"g_min": g_min,

}



return R_physical, R_driver, info
```

def simulate_prefix(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

) -> tuple[DynamicState, dict[str, Any]]:

text

"""Simulate prefix up to split_time.

Exact reproduction of production simulate_prefix().

Returns (state, prefix_dict).

Production uses:

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence) # for initialization

Then in the loop, uses:

```
unused_rng = default_rng(0) # for advance_state when shuffle_driver=False

"""

N = config.N

edges = compute_edges(N)



# Initialize state (uses init_sequence via make_prefix_rng)

state = initialize_state(seed, config)



# Get RNG for prefix simulation loop

# Production uses unused_rng = default_rng(0) for the advance_state calls

# when shuffle_driver=False

unused_rng = np.random.default_rng(0)



# Simulate to split point

R_past_sum = np.zeros((N, N), dtype=float)

R_past_count = 0
```

```
burn_end = protocol.burn_end(config)

hist_end = protocol.hist_end(config)



for step in range(hist_end):

R_physical, *,* = advance_state(state, config, unused_rng, shuffle_driver=False)


if burn_end &lt;= step &lt; hist_end:

R_past_sum += R_physical

R_past_count += 1



prefix = {

"B_at_split": state.B.copy(),

"R_physical_past": R_past_sum / R_past_count if R_past_count &gt; 0 else np.zeros((N, N)),

"theta_at_split": state.theta.copy(),

"coherence_acc_at_split": state.coherence_acc.copy(),

"history_at_split": np.stack(state.history),

"selected_at_split": (state.B &gt; config.B_c).astype(np.uint8),
```

```
}



return state, prefix

def apply_intervention(

text

state: DynamicState,

branch: str,

config: ModelConfig,

rng: np.random.Generator,

) -> None:

text

"""Apply intervention to state. Mutates in place.



Exact reproduction of production apply_intervention().

"""

N = state.N
```

```
if branch in {"reset_ema", "reset_B_ema", "B_carrier_only", "full_reset"}:

# Zero coherence accumulator

state.coherence_acc.fill(0.0)



if branch in {"reset_B", "reset_B_ema", "full_reset"}:

# Zero B

state.B.fill(0.0)



if branch in {"B_carrier_only", "full_reset"}:

# Randomize theta and history

new_theta = rng.random(N) *2.0* np.pi

state.theta = new_theta

state.history = [new_theta.copy() for *in range(config.delay*steps + 1)]



# Ensure symmetry

state.B = 0.5 * (state.B + state.B.T)

np.fill_diagonal(state.B, 0.0)
```

def simulate_future(

text

split_state: DynamicState,

prefix: dict[str, Any],

branch: str,

config: ModelConfig,

protocol: ProtocolConfig,

rng: np.random.Generator,

) -> dict[str, Any]:

text

"""Simulate future trajectory from split state.

Reproduces production simulate_future() with ALL required outputs.

Production accumulates:

- P_full, P_immediate, P_recovery, P_late (incidence matrices)

- B_mean_full, B_mean_immediate, B_mean_recovery, B_mean_late

- R_physical_future, R_driver_future

- Various scalar metrics

Uses horizon system from production:

- WINDOW_STEPS = round(T_window / dt)

- RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS)

- immediate: 0 to WINDOW_STEPS

- recovery: WINDOW_STEPS to RECOVERY_END

- late: RECOVERY_END to FUTURE_STEPS

"""

state = split_state.clone()

apply_intervention(state, branch, config, rng)

N = state.N

edges = compute_edges(N)

future_steps = protocol.future_steps(config)

```
# Compute horizon step boundaries (matching production exactly)

window_steps = int(round(config.T_window / config.dt))

recovery_end = min(5 * window_steps, future_steps)



# Initialize accumulators for horizon-based metrics

# P matrices (incidence: B &gt; B_c as 0/1)

P_sums = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}



# B matrices (mean B values)

B_sums = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),
```

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}

Counts for each horizon

counts = {

"full": 0,

"immediate": 0,

"recovery": 0,

"late": 0,

}

R matrices

R_physical_sum = np.zeros((N, N), dtype=float)

R_driver_sum = np.zeros((N, N), dtype=float)

Trajectory accumulators

```
sample_times = []

density_trajectory = []

similarity_trajectory = []



# For retention/emergence calculations

r_sum = 0.0

r_count = 0

r_late_sum = 0.0

r_late_count = 0



B_reference = prefix["B_at_split"]

shuffle_driver = (branch == "future_shuffle")



for step in range(future_steps):

R_physical, R_driver, *= advance*state(

state, config, rng, shuffle_driver=shuffle_driver

)
```

```
# Accumulate R matrices

R_physical_sum += R_physical

R_driver_sum += R_driver


# Determine which horizons this step belongs to

selected = (state.B &gt; config.B_c).astype(float)


# Full horizon: all steps

P_sums["full"] += selected

B_sums["full"] += state.B

counts["full"] += 1


# Time-based horizons (production uses step-based)

if step &lt; window_steps:

# immediate: 0 to WINDOW_STEPS (exclusive)

P_sums["immediate"] += selected

B_sums["immediate"] += state.B

counts["immediate"] += 1
```

if window_steps <= step < recovery_end:

recovery: WINDOW_STEPS to RECOVERY_END (exclusive)

P_sums["recovery"] += selected

B_sums["recovery"] += state.B

counts["recovery"] += 1

if recovery_end <= step:

late: RECOVERY_END to end

P_sums["late"] += selected

B_sums["late"] += state.B

counts["late"] += 1

Track late-phase metrics

r_value = float(np.abs(np.mean(np.exp(1j * state.theta))))

r_late_sum += r_value

r_late_count += 1

Trajectory sampling

```
if step % protocol.diagnostic_stride == 0:

t = (step + 1) * config.dt

sample_times.append(t)


# Density: fraction of edges above B_c

density = float(np.count_nonzero(state.B[edges] &gt; config.B_c) / len(edges[0]))

density_trajectory.append(density)


# Similarity: cosine between current and reference B

B_vec = state.B[edges]

B_ref_vec = B_reference[edges]

norm_B = np.linalg.norm(B_vec)

norm_ref = np.linalg.norm(B_ref_vec)

if norm_B &gt; 0 and norm_ref &gt; 0:

similarity = float([np.dot](http://np.dot)(B_vec, B_ref_vec) / (norm_B * norm_ref))

else:

similarity = float('nan')

similarity_trajectory.append(similarity)
```

```
# Track overall r

r_value = float(np.abs(np.mean(np.exp(1j * state.theta))))

r_sum += r_value

r_count += 1



# Compute mean matrices

P = {name: P_sums[name] / counts[name] if counts[name] &gt; 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean = {name: B_sums[name] / counts[name] if counts[name] &gt; 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}



# Compute R means

R_physical_mean = R_physical_sum / future_steps if future_steps &gt; 0 else np.zeros((N, N))

R_driver_mean = R_driver_sum / future_steps if future_steps &gt; 0 else np.zeros((N, N))



# Final state

B_final = state.B.copy()
```

```
final_weights = state.B[edges]

final_rows = np.sum(state.B, axis=1)



# Retention and emergence (from production)

split_selected = prefix["selected_at_split"][edges].astype(bool)

p_late_edges = P["late"][edges]



retention = (

float(np.mean(p_late_edges[split_selected]))

if np.any(split_selected)

else float('nan')

)

emergence = (

float(np.mean(p_late_edges[~split_selected]))

if np.any(~split_selected)

else float('nan')

)
```

```
# Minimum similarity from trajectory

similarity_array = np.array(similarity_trajectory, dtype=float)

finite_similarity = similarity_array[np.isfinite(similarity_array)]

minimum_similarity = (

float(np.min(finite_similarity)) if len(finite_similarity) &gt; 0

else float('nan')

)



# Build result with ALL production outputs

result = {

# Matrix outputs (required for baseline equivalence)

"B_final": B_final,

"B_mean_late": B_mean["late"],

"P_full": P["full"],

"P_immediate": P["immediate"],

"P_recovery": P["recovery"],
```

"P_late": P["late"],

"R_physical_future": R_physical_mean,

"R_driver_future": R_driver_mean,

Trajectory outputs

"sample_times": np.array(sample_times, dtype=float),

"density_trajectory": np.array(density_trajectory, dtype=float),

"similarity_trajectory": np.array(similarity_trajectory, dtype=float),

Scalar outcomes

"D_B_final": float(np.count_nonzero(final_weights > config.B_c) / len(edges[0])),

"P_mean_full": float(np.mean(P["full"][edges])),

"P_mean_immediate": float(np.mean(P["immediate"][edges])),

"P_mean_recovery": float(np.mean(P["recovery"][edges])),

"P_mean_late": float(np.mean(P["late"][edges])),

Similarity metrics

"Weighted_similarity_final": cosine_edges(B_reference, B_final, edges),

"Weighted_similarity_late": cosine_edges(B_reference, B_mean["late"], edges),

"Jaccard_final": jaccard_edges(B_reference, B_final, edges, config.B_c),

Retention/emergence

"Retention_selected_late": retention,

"Emergence_unselected_late": emergence,

"Retention_contrast_late": retention - emergence,

Correlations with past R

"Corr_Rpast_Pfull": safe_corr(prefix["R_physical_past"][edges], P["full"][edges]),

"Corr_Rpast_Plate": safe_corr(prefix["R_physical_past"][edges], P["late"][edges]),

Weight statistics

"Total_edge_weight": float(np.sum(final_weights)),

"Mean_row_sum": float(np.mean(final_rows)),

"Max_row_sum": float(np.max(final_rows)),

"Nonzero_edge_fraction": float(np.mean(final_weights > config.scale_epsilon)),

"Effective_edges": effective_number(final_weights),

"Weight_gini": gini_coefficient(final_weights),

"Weight_entropy": normalized_entropy(final_weights),

Order parameter

"r_mean_future": float(r_sum / r_count) if r_count > 0 else float('nan'),

"r_mean_late": float(r_late_sum / r_late_count) if r_late_count > 0 else float('nan'),

Minimum similarity

"Minimum_similarity": minimum_similarity,

}

return result

==========
Metric Helpers
==========

def cosine_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None) -> float:

text

"""Compute cosine similarity between reference and current B matrices over edges.

If edges is None, computes over all upper triangle edges.

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = np.asarray(reference[edges], dtype=float)

y = np.asarray(current[edges], dtype=float)

denominator = float(np.linalg.norm(x) * np.linalg.norm(y))

if denominator <= 1e-12:

return float('nan')

return float(np.dot(x, y) / denominator)

def jaccard_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None, threshold: float | None = None) -> float:

text

"""Compute Jaccard similarity between reference and current B matrices over edges.

Uses B > B_c as the threshold for edge presence.

If edges is None, computes over all upper triangle edges.

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

if threshold is None:

Use default B_c from calling context - this will be passed explicitly

threshold = 1e-12 # Should not be used, will be overridden

x = reference[edges] > threshold

y = current[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x & y) / union)

```
def safe_corr(x: np.ndarray, y: np.ndarray) -> float:

text

"""Compute correlation coefficient safely."""

x = np.asarray(x, dtype=float)

y = np.asarray(y, dtype=float)

valid = np.isfinite(x) &amp;amp; np.isfinite(y)

x = x[valid]

y = y[valid]

if len(x) &amp;lt; 3 or np.std(x) &amp;lt; 1e-12 or np.std(y) &amp;lt; 1e-12:

return float('nan')

return float(np.corrcoef(x, y)[0, 1])

def effective_number(weights: np.ndarray) -> float:

text

"""Compute effective number of edges."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

sum_sq = float(np.sum(weights**2))

if total &amp;lt;= 1e-12 or sum_sq &amp;lt;= 1e-12:
```

```
return 0.0

return total**2 / sum_sq

def gini_coefficient(weights: np.ndarray) -> float:

text

"""Compute Gini coefficient."""

x = np.sort(np.asarray(weights, dtype=float))

total = float(np.sum(x))

if total &lt;= 1e-12:

return 0.0

n = len(x)

ranks = np.arange(1, n + 1, dtype=float)

return float(2.0 *[np.dot](http://np.dot)(ranks, x) / (n* total) - (n + 1.0) / n)

def normalized_entropy(weights: np.ndarray) -> float:

text

"""Compute normalized entropy."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))
```

if total <= 1e-12:

return 0.0

probabilities = weights / total

positive = probabilities > 0.0

entropy = -float(np.sum(probabilities[positive] * np.log(probabilities[positive])))

return entropy / float(np.log(len(weights)))

==========
Required Output Schema for Baseline Equivalence
==========
These are the keys that MUST be present in both engine output and frozen data
for baseline equivalence to pass

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)

REQUIRED_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)

REQUIRED_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

)
==========
Baseline Equivalence Test
==========

def exact_array_equal_with_nan(a: np.ndarray, b: np.ndarray) -> bool:

text

"""Compare arrays with NaN-aware exact equality."""

a = np.asarray(a)

b = np.asarray(b)

if a.shape != b.shape or a.dtype != b.dtype:

return False

if np.issubdtype(a.dtype, np.floating) or np.issubdtype(a.dtype, np.complexfloating):

a_nan = np.isnan(a)

b_nan = np.isnan(b)

```
if not np.array_equal(a_nan, b_nan):

return False

finite = ~a_nan

return np.array_equal(a[finite], b[finite])

return np.array_equal(a, b)

def run_baseline_equivalence(

text

seeds: list[int],

config: ModelConfig,

protocol: ProtocolConfig,

frozen_manifest_path: str,

frozen_npz_path: str,

branch_order: tuple[str, ...],

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).
```

MANDATORY: Must reproduce frozen run exactly for all seeds.

This function validates that the engine produces EXACTLY the same outputs

as the frozen production run for all required keys.

Requires NaN-aware exact equality for ALL required schema keys:

- All prefix keys

- All branch matrix keys

- All branch scalar keys

Returns:

status: "passed" | "failed" | "not_run"

seed_results: per-seed validation details

engine_source_sha256: hash of this engine source

frozen_manifest_hash: hash of frozen manifest

frozen_npz_hash: hash of frozen NPZ

"""

```
import numpy as np

from pathlib import Path

import hashlib



# Helper function to compute source hash

def *source*sha256() -&gt; str | None:

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None



# Record engine source hash

engine_source_hash = *source*sha256()



results = {

"status": "not_run",

"config": {
```

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"mean_omega": config.mean_omega,

"T_window": config.T_window,

"T_star": config.T_star,

"alpha": config.alpha,

"a": config.a,

"q": config.q,

"delta": config.delta,

"eta": config.eta,

"mu": config.mu,

"gamma": config.gamma,

"B_c": config.B_c,

"B_init_scale": config.B_init_scale,

"scale_epsilon": config.scale_epsilon,

"capacity": config.capacity,

```
"delay_steps": config.delay_steps,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

"diagnostic_stride": protocol.diagnostic_stride,

"branches": list(protocol.branches),

},

"n_seeds": len(seeds),

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),

"engine_source_sha256": engine_source_hash,

"seed_results": {},

}
```

```
# Load frozen manifest

with open(frozen_manifest_path, "r") as f:

frozen_manifest = json.load(f)



# Load frozen data

with np.load(frozen_npz_path, allow_pickle=False) as frozen_data:

frozen_files = dict(frozen_data)



# Record frozen hashes

results["frozen_manifest_hash"] = hashlib.sha256(

json.dumps(frozen_manifest, sort_keys=True).encode()

).hexdigest()

results["frozen_npz_hash"] = file_sha256(frozen_npz_path)



# Validate config matches frozen manifest model - use EXACT equality

frozen_model = frozen_manifest["model"]
```

```
# Check N

if frozen_model["N"] != config.N:

results["status"] = "failed"

results["error"] = f"N mismatch: config={config.N}, frozen={frozen_model['N']}"

return results



# Check omega - use exact equality

frozen_omega_arr = np.array(frozen_model["omega"], dtype=float)

config_omega_arr = np.array(list([config.omega](http://config.omega)), dtype=float)

if not np.array_equal(frozen_omega_arr, config_omega_arr):

results["status"] = "failed"

results["error"] = f"omega mismatch: frozen={frozen_model['omega']}, config={list([config.omega]
(http://config.omega))}"

return results



# Check all other model params with EXACT equality (rtol=0, atol=0)

model_checks = [

("dt", frozen_model["dt"], config.dt),
```

("tau", frozen_model["tau"], config.tau),

("a", frozen_model["a"], config.a),

("q", frozen_model["q"], config.q),

("delta", frozen_model["delta"], config.delta),

("eta", frozen_model["eta"], config.eta),

("mu", frozen_model["mu"], config.mu),

("gamma", frozen_model["gamma"], config.gamma),

("B_c", frozen_model["B_c"], config.B_c),

("B_init_scale", frozen_model.get("B_init_scale", 0.0), config.B_init_scale),

("T_window", frozen_model["T_window"], config.T_window),

]

for name, frozen_val, config_val in model_checks:

if not np.isclose(frozen_val, config_val, rtol=0.0, atol=0.0):

results["status"] = "failed"

results["error"] = f"{name} mismatch: frozen={frozen_val}, config={config_val}"

return results

```
# Check derived values

if not np.isclose(frozen_model.get("alpha", 0.0), config.alpha, rtol=0.0, atol=1e-15):

results["status"] = "failed"

results["error"] = f"alpha mismatch: frozen={frozen_model.get('alpha')}, config={config.alpha}"

return results



# Now run actual equivalence test for each seed

print("Running baseline equivalence test...")

all_passed = True



for seed in seeds:

seed_str = str(seed)

seed_results = {"status": "passed", "checks": {}, "max_diffs": {}}



try:

# Simulate with our engine
```

```
split_state, prefix = simulate_prefix(seed, config, protocol)

branch_rngs = make_branch_rngs(seed, branch_order)


# Simulate each branch

engine_results = {}

for branch in protocol.branches:

rng = branch_rngs[branch]

engine_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)


# Check prefix state - iterate over REQUIRED_PREFIX_KEYS

for key in REQUIRED_PREFIX_KEYS:
```

```
engine_val = prefix.get(key)

frozen_key = f"prefix_seed_{seed_str}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"prefix_{key}_missing"] = True

all_passed = False

continue


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"prefix_{key}_exact"] = True

else:

seed_results["checks"][f"prefix_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)
```

```
f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"prefix_{key}"] = float(np.max(np.abs(e - f)))


# Check branch outputs - iterate over REQUIRED keys

for branch in protocol.branches:

engine_branch = engine_results[branch]

frozen_prefix = f"{branch}_seed_{seed_str}"


# Check matrix keys

for key in REQUIRED_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing"] = True

all_passed = False

continue
```

```
if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"{branch}_{key}"] = float(np.max(np.abs(e - f)))
```

```
# Check scalar keys

for key in REQUIRED_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

# This scalar might not be in frozen data - that's OK for now

# but we should track it

seed_results["checks"][f"{branch}_{key}_missing_in_frozen"] = True

continue


if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue


# For scalars, compare directly
```

```
if isinstance(engine_val, (int, float, np.number)):

if np.isclose(engine_val, frozen_val, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}"] = float(abs(engine_val - frozen_val))


except Exception as e:

seed_results["status"] = "failed"

seed_results["error"] = str(e)

import traceback

seed_results["traceback"] = traceback.format_exc()

all_passed = False


results["seed_results"][seed_str] = seed_results

print(f" Seed {seed_str}: {seed_results['status']}")
```

```
if all_passed:

results["status"] = "passed"

print("Baseline equivalence: PASSED")

else:

results["status"] = "failed"

print("Baseline equivalence: FAILED")

# Save equivalence certificate even on failure

raise RuntimeError("Baseline equivalence test FAILED. Study stopped.")



return results

================================================================================
Main API
================================================================================

def run_seed(

text

seed: int,
```

```
config: ModelConfig,

protocol: ProtocolConfig,

branch_order: tuple[str, ...],

) -> dict[str, Any]:

text

"""Run a single seed through prefix + all branches.



Returns complete seed results with all branches.

"""

# Simulate prefix

split_state, prefix = simulate_prefix(seed, config, protocol)



# Create branch RNGs using frozen algorithm

branch_rngs = make_branch_rngs(seed, branch_order)



# Use branch_order to determine which branches to run

# All branches in branch_order must have RNGs
```

```
branch_results = {}

for branch in branch_order:

if branch not in branch_rngs:

raise ValueError(

f"Branch '{branch}' not found in branch_rngs. "

f"All branches must be registered in branch_order for explicit RNG namespace. "

f"Registered branches: {list(branch_rngs.keys())}"

)

rng = branch_rngs[branch]


branch_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)
```

```
return {
"seed": int(seed),
"config": {
"N": config.N,
"dt": config.dt,
"tau": config.tau,
"T_window": config.T_window,
"alpha": config.alpha,
"mean_omega": config.mean_omega,
"T_star": config.T_star,
},
"protocol": {
"T_sim": protocol.T_sim,
"burn_end_time": protocol.burn_end_time,
"split_time": protocol.split_time,
"future_duration": protocol.future_duration,
```

```
},

"prefix": prefix,

"branches": branch_results,

}

if name == "main":

text

# Quick validation

config = DEFAULT_MODEL_CONFIG

config.validate()

protocol = DEFAULT_PROTOCOL_CONFIG

protocol.validate(config)



print(f"Engine v1 loaded successfully")

print(f" N={config.N}, dt={config.dt}, tau={config.tau}")

print(f" T_window={config.T_window}, alpha={config.alpha:.6f}")

print(f" mean_omega={config.mean_omega:.6f}, T_*={config.T_star:.6f}")

print(f" delay_steps={config.delay_steps}")
```

print(f" Protocol: T_sim={protocol.T_sim}, burn={protocol.burn_end_time}, "

f"split={protocol.split_time}, future={protocol.future_duration}")

This rewrite fixes the frozen parameters, timing, and prefix RNG. The core step map is now plausibly close to bitwise production equivalence. However, the engine’s branch API still conflates RNG namespace with selected branches, the scalar-equivalence test does not actually validate scalars, and the engine does not yet support both required horizon systems.

The next action should be an actual one-seed baseline run, but only after fixing the branch RNG interface.
1. Prefix RNG is now correct

This is the key correction:

Python

rng = np.random.default_rng(
init_sequence
)

rather than seeding from init_sequence.generate_state(...).

The initialization now matches production:

same SeedSequence hierarchy;
theta in [0,2π)[0,2π);
same BB draw;
same repeated initial history;
same prefix timing;
same unused_rng=default_rng(0) in the nonshuffle prefix.

This removes the primary expected source of split-state mismatch.
2. Branch RNG namespace is still conflated with branches-to-run

run_seed currently does:

Python

```
branch_rngs = make_branch_rngs(
seed,
branch_order,
)

for branch in branch_order:
simulate branch
```

But the TwindowTwindow study needs:

RNG seeds generated from the complete frozen seven-branch order;
only four selected branches simulated.

These are different objects.

Use:

Python

```
def run_seed(
seed: int,
config: ModelConfig,
protocol: ProtocolConfig,
rng_branch_order: tuple[str, ...],
) -> dict[str, Any]:
split_state, prefix = simulate_prefix(
seed,
config,
protocol,
)

branch_rngs = make_branch_rngs(
seed,
rng_branch_order,
)
```

branch_results = {}

for branch in protocol.branches:
if branch not in branch_rngs:
raise ValueError(
f"branch {branch} absent from "
"RNG namespace"
)

branch_results[branch] = (
simulate_future(
split_state,
prefix,
branch,
config,
protocol,
branch_rngs[branch],
)
)

Then:

Python

FROZEN_BRANCH_ORDER = (
"continue",
"reset_ema",
"reset_B",
"reset_B_ema",
"B_carrier_only",
"full_reset",
"future_shuffle",
)

STUDY_BRANCHES = (
"continue",
"reset_ema",
"reset_B",
"future_shuffle",
)

Without this correction, future_shuffle uses the wrong RNG stream whenever only the four-study branch list is passed.
3. r_mean_late does not match production sampling

Production calculates both r_mean_future and r_mean_late only at diagnostic sample times.

The engine currently accumulates:

Python

r_late_sum

at every late step.

Move the late accumulation into the diagnostic-stride block:

Python

if step % protocol.diagnostic_stride == 0:
r_value = ...

r_sum += r_value
r_count += 1

if step >= recovery_end:
r_late_sum += r_value
r_late_count += 1

Do not accumulate r_late at unsampled steps.

This difference will not be detected by matrix-anchor checks because scalar outcomes are not stored in the frozen NPZ.
4. Scalar equivalence is not actually tested

The frozen branch NPZ contains matrix/trajectory keys, not the scalar summary columns.

The baseline code does:

Python

```
frozen_val = frozen_files.get(
frozen_scalar_key
)

if frozen_val is None:
mark missing
continue
```

and does not fail.

Therefore none of the scalar outcomes are validated despite the function claiming exact scalar equivalence.

Pass the frozen scalar CSV into the equivalence test, load it with seed IDs as strings, and compare every required scalar by (seed, branch).

For NaN, use explicit NaN-aware equality.

Alternatively, narrow the claim:

Baseline matrix/trajectory equivalence

and do not claim scalar equivalence until the CSV is incorporated.
5. Required scalar schema is still incomplete

Production also returns:

text

```
G_min_future
G_min_pre_future
```

Pre_gap_violation_fraction
Projection_active_fraction
Projection_active_given_violation
Projection_norm_mean_active
Projection_norm_max
Projection_relative_state_mean_active
Projection_relative_state_max
Projection_relative_update_mean_active
Projection_relative_update_max

The engine omits them.

If these are required by the TwindowTwindow driver—and the earlier driver lists them—the engine must port the exact ProjectionTotals logic from production.

Otherwise remove them from the study schema. Do not silently fill them with NaN.
6. Recovery-reference fields are intentionally absent but must be assigned later

Fields such as:

text

Late_persistence_ratio_to_continue
Late_similarity_ratio_to_continue
Time_to_90pct...

are populated in production only after all branches have been simulated and the continue reference is known.

It is reasonable for simulate_future not to calculate them. But the study driver or analysis layer must explicitly calculate them before declaring schema completion.

Document which fields belong to:

branch simulation;
post-branch within-condition analysis.

7. Fixed physical horizons are still not represented

simulate_future uses:

Python

```
window_steps = round(
T_window / dt
)
recovery_end = 5 * window_steps
```

This gives the EMA-normalized horizon system.

The primary TwindowTwindow study also requires fixed physical horizons:

text

```
immediate: 0–17.5
recovery: 17.5–87.5
late: 87.5–320
```

The engine should either:

accumulate both systems; or
expose a horizon specification argument.

For example:

Python

```
@dataclass(frozen=True)
class HorizonDefinition:
name: str
immediate_end_steps: int
recovery_end_steps: int
```

Then simulate_future can accumulate:

text

P_fixed_physical_late
P_normalized_EMA_late
B_fixed_physical_mean_late
B_normalized_EMA_mean_late

Do not overwrite one horizon system with the other.
8. Baseline equivalence certificate is not saved

run_baseline_equivalence constructs detailed seed results but either:

returns on parameter mismatch; or
raises on trajectory mismatch.

It does not save a certificate itself.

The study driver should always save:

text

baseline_equivalence_by_seed.csv
baseline_equivalence_manifest.json

before deciding whether to continue.

On failure, save the detailed results and stop. On success, hash the certificate and include it in the T-window run manifest.
9. Frozen manifest hash is not the file hash

This calculates:

Python

sha256(
json.dumps(
frozen_manifest,

sort_keys=True,
)
)

That is a canonicalized-content hash, not the SHA-256 of the frozen manifest file.

Record both if useful, but label them correctly:

text

frozen_manifest_file_sha256
frozen_manifest_canonical_json_sha256

The historical artifact authority is the file hash.
10. Baseline model validation still has small gaps

Add exact checks for:

text

scale_epsilon
capacity
delay_steps
T_sim
future_steps
window_steps
recovery_end
diagnostic_stride
branch order

The tau fallback has been removed, which is good.

For exact frozen values, np.isclose(..., atol=0, rtol=0) is acceptable, although direct equality is clearer for values loaded from the same manifest representation.
11. History list behavior is probably equivalent

The list implementation:

Python

append(new)
if len > max:
pop(0)

should produce the same history sequence as the production fixed-length deque.

The baseline split-history comparison will verify it.

For defensive validation, optionally call:

Python

state.validate(config, step)

at initialization, split, and final state—not every step unless debugging, because that would add substantial cost.
12. The engine now returns most important production outcomes

The following are now correctly present:

all required branch matrix outputs;
threshold density;
horizon persistence matrices;
late mean BB;
final and late cosine similarity;
Jaccard;
retention/emergence;
past-RR/persistence correlations;
total weight;
effective edges;
Gini and entropy;
sampled trajectories.

That is a major improvement.

The remaining schema gap is mainly the projection diagnostics and continue-relative postprocessing.

13. Baseline test must be run with the full frozen branch order

For the baseline certificate, use all seven branches:

Python

```
protocol = DEFAULT_PROTOCOL_CONFIG
rng_branch_order = FROZEN_BRANCH_ORDER
```

For the T-window study:

Python

```
protocol.branches = STUDY_BRANCHES
rng_branch_order = FROZEN_BRANCH_ORDER
```

This is the cleanest separation.
14. Do not call it the reference implementation yet

The header says:

text

```
DO NOT MODIFY: This is the reference implementation.
```

That status should be granted only after 40/40 exact baseline equivalence.

Until then:

Candidate parameterized engine v1.

After passing, freeze its source hash and equivalence certificate. Future modifications should create v2.
Recommended immediate test sequence
Step 1 — one seed, all seven baseline branches

Require exact equality for:

text

6 prefix arrays
11 matrix/trajectory arrays × 7 branches

and scalar comparison against the frozen scalar CSV.
Step 2 — all 40 seeds, all seven branches

Save the full equivalence certificate.
Step 3 — one nonbaseline condition

Use one seed and ρ=0.5ρ=0.5, simulating only:

text

continue
reset_ema
reset_B
future_shuffle

while retaining the seven-branch RNG namespace.

Verify output schemas and both horizon systems.

Only then run the full grid.
Rating
Area Rating
Frozen parameter fidelity 9/10
Prefix RNG fidelity 9/10
Core step-map fidelity 8.5/10
Protocol timing 9/10
Branch RNG API 5/10
Output-schema completeness 7/10
Baseline validation 7/10
T-window study readiness 6/10
Overall engine: 7.5/10

This is now a serious candidate engine. The next meaningful evidence is not another rewrite—it is an actual one-seed exact equivalence result.

now this:

"""AIEF-1.4 T_window Sensitivity Study Analysis Driver v2.

Correct implementation of the prespecified estimands:

Regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

Cross-T topology: S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Recovery scaling: t_rec / T_window for both horizon systems

Output:

twindow_paired_regime_contrasts.csv

twindow_branch_interactions.csv

twindow_cross_condition_topology.csv

twindow_recovery_summary.csv

twindow_sensitivity_report.txt

analysis_manifest.json

This script implements the 5-stage analysis structure as specified.

"""

```
from future import annotations

import argparse

import csv

import hashlib

import json

import os

import sys

import time

from pathlib import Path

from typing import Any

import numpy as np

import pandas as pd
================================================================================
Configuration
================================================================================

ANALYSIS_VERSION = "v2.0"

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"
Expected values (for validation)

EXPECTED_N_SEEDS = 40
```

```
EXPECTED_RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)

EXPECTED_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")

EXPECTED_N_CONDITIONS = EXPECTED_N_SEEDS * len(EXPECTED_RHO_VALUES)

EXPECTED_N_ROWS = EXPECTED_N_CONDITIONS * len(EXPECTED_BRANCHES)
Primary outcomes

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late",

"Retention_contrast_late",

"P_mean_late",

)
All scalar outcomes for validation

ALL_SCALAR_OUTCOMES = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",
```

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

)
Matrix keys needed for cross-T topology

MATRIX_KEYS = (

text

"B_final",

)
==========
Utility Functions
==========

def load_csv_atomic(path: str | Path) -> pd.DataFrame:

text

"""Load CSV file atomically with validation."""

path = Path(path)

Use pandas with explicit dtypes

df = pd.read_csv(

path,

dtype={"seed": str, "branch": str}, # Keep seeds as strings

)

Convert numeric columns, coercing errors to NaN

for col in df.columns:

if col not in ("seed", "seed_index", "branch", "rho"):

try:

df[col] = pd.to_numeric(df[col], errors="coerce")

except Exception:

pass

return df

def load_npz_selective(path: str | Path, keys: list[str] | None = None) -> dict[str, Any]:

text

"""Load only specified keys from NPZ file to save memory."""

path = Path(path)

payload: dict[str, Any] = {}

```
try:

with np.load(str(path), allow_pickle=False) as data:

if keys is None:

keys = list(data.files)

for key in keys:

if key in data:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

except Exception as e:

print(f"Warning: Could not load NPZ {path}: {e}")



return payload

def save_csv_atomic(path: str | Path, df: pd.DataFrame) -> None:

text

"""Save CSV atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)
```

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

df.to_csv(temp_path, index=False)

os.replace(str(temp_path), str(path))

finally:

if temp_path.exists():

temp_path.unlink()

def save_json_atomic(path: str | Path, data: dict[str, Any]) -> None:

text

"""Save JSON atomically with fsync, handling NaN values."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

Convert NaN to None for JSON serialization

def sanitize(obj: Any) -> Any:

if isinstance(obj, dict):

```
return {k: sanitize(v) for k, v in obj.items()}

elif isinstance(obj, list):

return [sanitize(v) for v in obj]

elif isinstance(obj, float) and np.isnan(obj):

return None

elif isinstance(obj, (np.integer, np.floating)):

return obj.item()

elif isinstance(obj, np.ndarray):

return obj.tolist()

else:

return obj



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:

json.dump(sanitize(data), f, indent=2, sort_keys=True, allow_nan=False)

f.write("\n")
```

```
f.flush()

os.fsync(f.fileno())

os.replace(str(temp_path), str(path))

finally:

if temp_path.exists():

temp_path.unlink()

def file_sha256(path: str | Path) -> str:

text

"""Compute SHA-256 hash of a file."""

import hashlib

digest = hashlib.sha256()

with open(path, "rb") as f:

for block in iter(lambda: [f.read](http://f.read)(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

================================================================================
Metric Functions
================================================================================
```

def cosine_similarity(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute cosine similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

norm_x = np.linalg.norm(x)

norm_y = np.linalg.norm(y)

if norm_x <= 1e-12 or norm_y <= 1e-12:

return float('nan')

return float(np.dot(x, y) / (norm_x * norm_y))

def jaccard_similarity(B1: np.ndarray, B2: np.ndarray, threshold: float = 0.12) -> float:

text

"""Compute Jaccard similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges] > threshold

y = B2[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x & y) / union)

def normalized_distance(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute normalized L2 distance between two B matrices."""

diff = B1 - B2

norm_diff = np.linalg.norm(diff)

norm_B2 = np.linalg.norm(B2)

if norm_B2 <= 1e-12:

return float('nan')

return float(norm_diff / norm_B2)

==========
Stage 1: Validation
==========

def validate_run(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

) -> dict[str, Any]:

text

"""Validate the run before analysis.

Checks:

- 800 scalar rows (40 seeds × 5 rho × 4 branches)

- 40 unique seeds

- 5 rho values per seed

- 4 branches per seed/rho

- baseline equivalence passed

- run is freezeable

- no duplicate condition keys

"""

results = {

"status": "passed",

"errors": [],

"warnings": [],

}

Load manifest

try:

with open(manifest_path, "r") as f:

manifest = json.load(f)

except Exception as e:

```
results["status"] = "failed"

results["errors"].append(f"Cannot load manifest: {e}")

return results



# Check baseline equivalence

baseline_eq = manifest.get("baseline_equivalence", {})

if baseline_eq.get("status") != "passed":

results["status"] = "failed"

results["errors"].append(

f"Baseline equivalence not passed: {baseline_eq.get('status', 'unknown')}"

)

return results



# Load CSV

try:

df = load_csv_atomic(csv_path)

except Exception as e:
```

```
results["status"] = "failed"

results["errors"].append(f"Cannot load CSV: {e}")

return results



# Validate row count

expected_rows = EXPECTED_N_ROWS

actual_rows = len(df)

if actual_rows != expected_rows:

results["status"] = "failed"

results["errors"].append(

f"Row count mismatch: expected {expected_rows}, got {actual_rows}"

)



# Validate unique seeds

unique_seeds = df["seed"].unique()

if len(unique_seeds) != EXPECTED_N_SEEDS:

results["status"] = "failed"
```

```
results["errors"].append(
f"Seed count mismatch: expected {EXPECTED_N_SEEDS}, got {len(unique_seeds)}"
)

# Validate rho values
unique_rho = sorted(df["rho"].unique())
expected_rho = sorted(EXPECTED_RHO_VALUES)
if unique_rho != expected_rho:
results["status"] = "failed"
results["errors"].append(
f"Rho values mismatch: expected {expected_rho}, got {unique_rho}"
)

# Validate branches
unique_branches = sorted(df["branch"].unique())
expected_branches = sorted(EXPECTED_BRANCHES)
if unique_branches != expected_branches:
```

```
results["status"] = "failed"

results["errors"].append(

f"Branches mismatch: expected {expected_branches}, got {unique_branches}"

)



# Validate completeness: each seed/rho/branch combination should exist

for seed in unique_seeds:

seed_data = df[df["seed"] == seed]

for rho in unique_rho:

rho_data = seed_data[seed_data["rho"] == rho]

for branch in unique_branches:

if not ((rho_data["branch"] == branch).any()):

results["status"] = "failed"

results["errors"].append(

f"Missing: seed={seed}, rho={rho}, branch={branch}"

)
```

```
# Check for duplicates

duplicate_rows = df.duplicated(subset=["seed", "rho", "branch"])

if duplicate_rows.any():

results["status"] = "failed"

results["errors"].append(

f"Duplicate rows found: {duplicate_rows.sum()} duplicates"

)



# Check freezeable status

if not manifest.get("freezeable", True):

results["warnings"].append("Run marked as non-freezeable")



# Record validation info

results["validation"] = {

"n_rows": actual_rows,

"n_seeds": len(unique_seeds),

"n_rho": len(unique_rho),
```

```
"n_branches": len(unique_branches),

"n_duplicates": int(duplicate_rows.sum()),

"baseline_equivalence": baseline_eq.get("status"),

"freezeable": manifest.get("freezeable", True),

}



if results["status"] == "passed":

print("✅ Stage 1: Validation PASSED")

print(f" {actual_rows} rows, {len(unique_seeds)} seeds, {len(unique_rho)} rho,
{len(unique_branches)} branches")

else:

print("❌ Stage 1: Validation FAILED")

for error in results["errors"]:

print(f" {error}")



return results

================================================================================
====
```

Stage 2: Regime Contrasts

==========

====

def compute_regime_contrasts(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

For each seed and non-baseline rho, compute the difference between

the outcome at that rho and the outcome at rho=1 (baseline).

"""

Filter to continue branch only

continue_df = df[df["branch"] == "continue"].copy()

Pivot to have seed × rho as index, outcomes as columns

```
pivot_df = continue_df.pivot_table(

index=["seed", "rho"],

columns=["branch"],

values=list(outcomes),

)



# Get baseline (rho=1) values

baseline_df = pivot_df.xs(1.0, level="rho")

baseline_df.columns = [f"{col}_baseline" for col in baseline_df.columns]



# Merge with all rho values

merged = pivot_df.merge(baseline_df, left_index=True, right_index=True)



# Compute contrasts

contrast_rows = []

for (seed, rho), row in merged.iterrows():

if rho == 1.0:
```

```
continue # Skip baseline

for outcome in outcomes:
baseline_key = f"{outcome}_baseline"
contrast_key = f"{outcome}_regime_contrast"

if outcome in row.index and baseline_key in row.index:
contrast = row[outcome] - row[baseline_key]
contrast_rows.append({
"seed": seed,
"rho": rho,
"outcome": outcome,
f"Y_rho_continue": row[outcome],
f"Y_baseline_continue": row[baseline_key],
"paired_regime_contrast": contrast,
})

if not contrast_rows:
```

raise RuntimeError("No regime contrasts computed - check input data")

contrasts_df = pd.DataFrame(contrast_rows)

print(f"✅ Stage 2: Regime Contrasts - {len(contrast_rows)} contrasts computed")

print(f" Seeds: {contrasts_df['seed'].nunique()}, Rho values: {contrasts_df['rho'].nunique()}, Outcomes: {contrasts_df['outcome'].nunique()}")

return contrasts_df

==========
Stage 3: Branch Difference-in-Differences
==========

def compute_branch_did(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences relative to continue branch.

"""

did_rows = []

for seed in df["seed"].unique():

seed_data = df[df["seed"] == seed]

for rho in sorted(df["rho"].unique()):

rho_data = seed_data[seed_data["rho"] == rho]

for branch in df["branch"].unique():

if branch == "continue":

continue # Skip continue branch

for outcome in outcomes:

```
# Get Y_{s,ρ,b}

y_rho_b = rho_data[rho_data["branch"] == branch][outcome].values

if len(y_rho_b) == 0:

continue

y_rho_b = y_rho_b[0]


# Get Y_{s,ρ,continue}

y_rho_c = rho_data[rho_data["branch"] == "continue"][outcome].values[0]


# Get Y_{s,1,b}

baseline_data = seed_data[seed_data["rho"] == 1.0]

y_base_b = baseline_data[baseline_data["branch"] == branch][outcome].values

if len(y_base_b) == 0:

continue

y_base_b = y_base_b[0]


# Get Y_{s,1,continue}

y_base_c = baseline_data[baseline_data["branch"] == "continue"][outcome].values[0]
```

```
# Compute DID

branch_effect_rho = y_rho_b - y_rho_c

branch_effect_baseline = y_base_b - y_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"outcome": outcome,

"Y_rho_branch": y_rho_b,

"Y_rho_continue": y_rho_c,

"branch_effect_rho": branch_effect_rho,

"Y_baseline_branch": y_base_b,

"Y_baseline_continue": y_base_c,

"branch_effect_baseline": branch_effect_baseline,

"difference_in_differences": did,

})
```

```
if not did_rows:

raise RuntimeError("No branch DID computed - check input data")



did_df = pd.DataFrame(did_rows)



print(f"✅ Stage 3: Branch DID - {len(did_rows)} DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, Branches:
{did_df['branch'].nunique()}")



return did_df

================================================================================
Stage 4: Cross-T Topology and Recovery
================================================================================

def compute_cross_t_topology(

text

npz_path: Path,

df: pd.DataFrame,
```

```
seeds: list,

rho_values: list,

) -> pd.DataFrame:

text

"""Compute cross-T topology metrics.



For each seed and rho, compute:

- S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- J_{s,ρ} = Jaccard(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- D_{s,ρ} = normalized_L2_distance(B_final_{s,ρ,continue}, B_final_{s,1,continue})

"""

topology_rows = []



# Load only B_final matrices from NPZ

npz_data = load_npz_selective(npz_path)



# Get baseline (rho=1) B_final for continue branch
```

```
baseline_keys = {}

for seed in seeds:

key_pattern = f"continue_seed_{seed}_rho_1.0_B_final"

# Try different key formats

for possible_key in npz_data.keys():

if f"continue_seed_{seed}_rho_1" in possible_key and "B_final" in possible_key:

baseline_keys[seed] = possible_key

break



for seed in seeds:

for rho in rho_values:

if rho == 1.0:

continue # Skip baseline


# Get B_final for this (seed, rho, continue)

rho_key_pattern = f"continue_seed_{seed}_rho_{rho}_B_final"

rho_key = None
```

```
for possible_key in npz_data.keys():

if f"continue_seed_{seed}_rho_{rho}" in possible_key and "B_final" in possible_key:

rho_key = possible_key

break


if rho_key is None or seed not in baseline_keys:

continue


baseline_key = baseline_keys[seed]


if rho_key not in npz_data or baseline_key not in npz_data:

continue


B_rho = npz_data[rho_key]

B_baseline = npz_data[baseline_key]


# Compute metrics

cos_sim = cosine_similarity(B_rho, B_baseline)

jaccard = jaccard_similarity(B_rho, B_baseline)

norm_dist = normalized_distance(B_rho, B_baseline)
```

```
topology_rows.append({

"seed": seed,

"rho": rho,

"cross_T_cosine": cos_sim,

"cross_T_Jaccard": jaccard,

"cross_T_normalized_distance": norm_dist,

})



if not topology_rows:

print("⚠️ Stage 4: Cross-T Topology - No matrices found in NPZ")

print(" This is expected if NPZ uses different key format")

# Return empty DataFrame with correct columns

return pd.DataFrame(columns=["seed", "rho", "cross_T_cosine", "cross_T_Jaccard",
"cross_T_normalized_distance"])



topology_df = pd.DataFrame(topology_rows)
```

print(f"✅ Stage 4: Cross-T Topology - {len(topology_rows)} topology comparisons computed")

print(f" Seeds: {topology_df['seed'].nunique()}, Rho: {topology_df['rho'].nunique()}")

return topology_df

def compute_recovery_scaling(

text

df: pd.DataFrame,

npz_path: Path,

horizon_system: str = "fixed_physical",

) -> pd.DataFrame:

text

"""Compute recovery scaling metrics.

For each seed/rho/branch, compute:

- First crossing of continue branch threshold

- Sustained recovery duration

- Held recovery time

- Censoring indicator

- Restricted recovery time

- AUC deficit

- t_rec / T_window

Uses trajectories from NPZ to compute recovery relative to continue branch.

"""

This is a placeholder - full implementation requires loading trajectories

and computing recovery metrics relative to continue branch

For now, return empty DataFrame with correct columns

print(f"⚠️ Stage 4: Recovery Scaling - Not yet implemented for {horizon_system}")

print(" Will be added in future version")

return pd.DataFrame(columns=[

"seed", "rho", "branch", "horizon_system",

"first_crossing", "sustained_recovery", "held_duration",

"censored", "restricted_time", "auc_deficit", "t_rec_over_Twindow"

])

==========
Stage 5: Seed-Level Summaries
==========

def compute_seed_level_summaries(

text

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

n_bootstrap: int = 1000,

) -> dict[str, pd.DataFrame]:

text

"""Compute seed-level summaries with bootstrap confidence intervals.

For each outcome and rho, compute:

- Mean contrast

- Std error

- 95% bootstrap CI

- p-value (if applicable)

"""

summaries = {}

Regime contrast summaries

if not regime_contrasts.empty:

regime_summary = regime_contrasts.groupby(["rho", "outcome"])["paired_regime_contrast"].agg([

("mean", "mean"),

("std", "std"),

("n", "count"),

]).reset_index()

Add bootstrap CI

for (rho, outcome), group in regime_contrasts.groupby(["rho", "outcome"]):

values = group["paired_regime_contrast"].dropna().values

if len(values) > 0:

```
# Simple bootstrap

bootstrap_means = []

for *in range(n*bootstrap):

sample = np.random.choice(values, size=len(values), replace=True)

bootstrap_means.append(np.mean(sample))


ci_lower = np.percentile(bootstrap_means, 2.5)

ci_upper = np.percentile(bootstrap_means, 97.5)


idx = regime_summary[(regime_summary["rho"] == rho) &amp;amp; (regime_summary["outcome"] ==
outcome)].index

regime_summary.loc[idx, "ci_lower"] = ci_lower

regime_summary.loc[idx, "ci_upper"] = ci_upper


summaries["regime_contrasts"] = regime_summary

print(f"✅ Stage 5: Regime Contrast Summaries - {len(regime_summary)} groups")



# Branch DID summaries

if not branch_did.empty:
```

```
did_summary = branch_did.groupby(["rho", "branch", "outcome"])["difference_in_differences"].agg([

("mean", "mean"),

("std", "std"),

("n", "count"),

]).reset_index()


# Add bootstrap CI

for (rho, branch, outcome), group in branch_did.groupby(["rho", "branch", "outcome"]):

values = group["difference_in_differences"].dropna().values

if len(values) &gt; 0:

bootstrap_means = []

for *in range(n*bootstrap):

sample = np.random.choice(values, size=len(values), replace=True)

bootstrap_means.append(np.mean(sample))


ci_lower = np.percentile(bootstrap_means, 2.5)

ci_upper = np.percentile(bootstrap_means, 97.5)
```

```
idx = did_summary[(did_summary["rho"] == rho) &amp;amp; (did_summary["branch"] == branch) &amp;amp; (did_summary["outcome"] == outcome)].index

did_summary.loc[idx, "ci_lower"] = ci_lower

did_summary.loc[idx, "ci_upper"] = ci_upper


summaries["branch_did"] = did_summary

print(f"✅ Stage 5: Branch DID Summaries - {len(did_summary)} groups")



# Topology summaries

if not topology.empty:

topology_summary = topology.groupby("rho").agg([

("mean", "mean"),

("std", "std"),

("n", "count"),

]).reset_index()

summaries["topology"] = topology_summary

print(f"✅ Stage 5: Topology Summaries - {len(topology_summary)} rho values")
```

```
return summaries

# ============================================================================
# Report Generation
# ============================================================================

def generate_text_report(

text

validation: dict[str, Any],

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

regime_summaries: pd.DataFrame,

did_summaries: pd.DataFrame,

topology_summaries: pd.DataFrame,

) -> str:

text

"""Generate human-readable text report."""

lines = []

lines.append("=" * 80)
```

```
lines.append("AIEF-1.4 T_window Sensitivity Study - Analysis Report")

lines.append("=" * 80)

lines.append("")



# Validation summary

lines.append("VALIDATION SUMMARY")

lines.append("-" * 40)

val = validation.get("validation", {})

lines.append(f" Status: {validation.get('status', 'unknown')}")

lines.append(f" Rows: {val.get('n_rows', 0)} (expected: {EXPECTED_N_ROWS})")

lines.append(f" Seeds: {val.get('n_seeds', 0)} (expected: {EXPECTED_N_SEEDS})")

lines.append(f" Rho values: {val.get('n_rho', 0)} (expected: {len(EXPECTED_RHO_VALUES)})")

lines.append(f" Branches: {val.get('n_branches', 0)} (expected: {len(EXPECTED_BRANCHES)})")

lines.append(f" Duplicates: {val.get('n_duplicates', 0)}")

lines.append(f" Baseline equivalence: {val.get('baseline_equivalence', 'unknown')}")

lines.append(f" Freezeable: {val.get('freezeable', False)}")

lines.append("")
```

```
# Regime contrasts

lines.append("REGIME CONTRASTS (Δ = Y_{s,ρ,continue} - Y_{s,1,continue})")

lines.append("-" * 40)

if not regime_summaries.empty:

for rho in sorted(regime_summaries["rho"].unique()):

rho_data = regime_summaries[regime_summaries["rho"] == rho]

for outcome in PRIMARY_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = out_data[("paired_regime_contrast", "mean")].values[0]

std_val = out_data[("paired_regime_contrast", "std")].values[0]

n_val = out_data[("paired_regime_contrast", "n")].values[0]

ci_lower = out_data.get("ci_lower", [float('nan')])[0]

ci_upper = out_data.get("ci_upper", [float('nan')])[0]


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")
```

```
else:

lines.append(" No regime contrast data")

lines.append("")



# Branch DID

lines.append("BRANCH DIFFERENCE-IN-DIFFERENCES")

lines.append("-" * 40)

lines.append("I_{s,ρ,b} = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]")

lines.append("")

if not did_summaries.empty:

for rho in sorted(did_summaries["rho"].unique()):

rho_data = did_summaries[did_summaries["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in PRIMARY_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:
```

```
mean_val = out_data[("difference_in_differences", "mean")].values[0]

std_val = out_data[("difference_in_differences", "std")].values[0]

n_val = out_data[("difference_in_differences", "n")].values[0]

ci_lower = out_data.get("ci_lower", [float('nan')])[0]

ci_upper = out_data.get("ci_upper", [float('nan')])[0]


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No branch DID data")

lines.append("")



# Cross-T topology

lines.append("CROSS-T TOPOLOGY METRICS")

lines.append("-" * 40)

if not topology_summaries.empty:

for rho in sorted(topology_summaries.index):
```

```
row = topology_summaries.loc[rho]

lines.append(f" rho={rho:.2f}:")

lines.append(f" Cosine similarity: {row[('cross_T_cosine', 'mean')]:.4f} ± {row[('cross_T_cosine', 'std')]:.4f}")

lines.append(f" Jaccard similarity: {row[('cross_T_Jaccard', 'mean')]:.4f} ± {row[('cross_T_Jaccard', 'std')]:.4f}")

lines.append(f" Normalized distance: {row[('cross_T_normalized_distance', 'mean')]:.4f} ± {row[('cross_T_normalized_distance', 'std')]:.4f}")

else:

lines.append(" No cross-T topology data")

lines.append("")



lines.append("=" * 80)

lines.append("END OF REPORT")

lines.append("=" * 80)



return "\n".join(lines)

def generate_analysis_manifest(
```

```
text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

validation: dict[str, Any],

output_files: dict[str, Path],

) -> dict[str, Any]:

text

"""Generate analysis manifest with complete provenance."""

# Compute hashes

csv_hash = file_sha256(csv_path) if csv_path.exists() else "N/A"

npz_hash = file_sha256(npz_path) if npz_path.exists() else "N/A"

manifest_hash = file_sha256(manifest_path) if manifest_path.exists() else "N/A"



output_hashes = {}

for name, path in output_files.items():

if path.exists():
```

```
output_hashes[name] = file_sha256(path)

else:

output_hashes[name] = "N/A"



manifest = {

"analysis_version": ANALYSIS_VERSION,

"timestamp": int(time.time()),

"study_name": STUDY_NAME,


"input_hashes": {

"scalar_csv": csv_hash,

"branch_npz": npz_hash,

"simulation_manifest": manifest_hash,

},


"validation": validation,


"output_hashes": output_hashes,
```

```
"environment": {
"python": sys.version,
"numpy": np.__version__,
"pandas": pd.__version__,
},

"notes": {
"primary_outcomes": list(PRIMARY_OUTCOMES),
"expected_n_seeds": EXPECTED_N_SEEDS,
"expected_rho_values": list(EXPECTED_RHO_VALUES),
"expected_branches": list(EXPECTED_BRANCHES),
},
}

return manifest

================================================================================
Main Execution
================================================================================
```

```
def parse_arguments() -> argparse.Namespace:

text

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

parser.add_argument(

"run_dir",

type=str,

help="run directory containing study outputs",

)

parser.add_argument(

"--output",

type=str,

default=None,

help="output directory for analysis (default: &lt;run_dir&gt;/analysis)",

)

parser.add_argument(
```

"--n-bootstrap",

type=int,

default=1000,

help="number of bootstrap iterations (default: 1000)",

)

return parser.parse_args()

def main() -> None:

text

args = parse_arguments()

Parse run directory

run_dir = Path(args.run_dir)

csv_files = list(run_dir.glob(f"{STUDY_NAME}_*_scalar_summary.csv"))

npz_files = list(run_dir.glob(f"{STUDY_NAME}_*_branch_diagnostics.npz"))

manifest_files = list(run_dir.glob("manifest.json"))

if not csv_files:

```
print(f"No CSV file found in {run_dir}")

sys.exit(1)

if not npz_files:

print(f"No NPZ file found in {run_dir}")

sys.exit(1)

if not manifest_files:

print(f"No manifest found in {run_dir}")

sys.exit(1)



csv_path = csv_files[0]

npz_path = npz_files[0]

manifest_path = manifest_files[0]



# Set up output directory

if args.output is None:

output_dir = run_dir / "analysis"

else:
```

```
output_dir = Path(args.output)

output_dir.mkdir(parents=True, exist_ok=True)



print(f"Analyzing {run_dir}")

print(f" CSV: {csv_[path.name](http://path.name)}")

print(f" NPZ: {npz_[path.name](http://path.name)}")

print(f" Manifest: {manifest_[path.name](http://path.name)}")

print(f" Output: {output_dir}")

print()



# Stage 1: Validation

print("Stage 1: Validating run...")

validation = validate_run(csv_path, npz_path, manifest_path)



if validation["status"] == "failed":

print("\n❌ VALIDATION FAILED - Cannot proceed with analysis")

for error in validation["errors"]:
```

```
print(f" {error}")

for warning in validation["warnings"]:

print(f" WARNING: {warning}")

sys.exit(1)



# Load CSV data

print("\nLoading scalar data...")

df = load_csv_atomic(csv_path)



# Get study parameters from manifest

with open(manifest_path, "r") as f:

manifest = json.load(f)



seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]

rho_values = manifest["study"]["rho_values"]

branches = manifest["protocol"]["branches"]
```

```
print(f" Loaded {len(df)} rows")

print(f" Seeds: {len(seeds)}, Rho: {len(rho_values)}, Branches: {len(branches)}")



# Stage 2: Regime Contrasts

print("\nStage 2: Computing regime contrasts...")

regime_contrasts = compute_regime_contrasts(df, PRIMARY_OUTCOMES)



# Stage 3: Branch DID

print("\nStage 3: Computing branch difference-in-differences...")

branch_did = compute_branch_did(df, PRIMARY_OUTCOMES)



# Stage 4: Cross-T Topology

print("\nStage 4: Computing cross-T topology metrics...")

topology = compute_cross_t_topology(npz_path, df, seeds, rho_values)



# Stage 4b: Recovery Scaling (placeholder)
```

```
print("\nStage 4b: Recovery scaling...")

recovery = compute_recovery_scaling(df, npz_path)



# Stage 5: Seed-Level Summaries

print("\nStage 5: Computing seed-level summaries...")

summaries = compute_seed_level_summaries(

regime_contrasts, branch_did, topology, n_bootstrap=args.n_bootstrap

)



regime_summaries = summaries.get("regime_contrasts", pd.DataFrame())

did_summaries = summaries.get("branch_did", pd.DataFrame())

topology_summaries = summaries.get("topology", pd.DataFrame())



# Save outputs

print("\nSaving outputs...")



# Paired regime contrasts
```

```
regime_contrast_path = output_dir / "twindow_paired_regime_contrasts.csv"

save_csv_atomic(regime_contrast_path, regime_contrasts)

print(f" Saved: {regime_contrast_[path.name](http://path.name)}")



# Branch DID

branch_did_path = output_dir / "twindow_branch_interactions.csv"

save_csv_atomic(branch_did_path, branch_did)

print(f" Saved: {branch_did_[path.name](http://path.name)}")



# Cross-T topology

if not topology.empty:

topology_path = output_dir / "twindow_cross_condition_topology.csv"

save_csv_atomic(topology_path, topology)

print(f" Saved: {topology_[path.name](http://path.name)}")



# Recovery scaling (if implemented)

if not recovery.empty:
```

```
recovery_path = output_dir / "twindow_recovery_summary.csv"

save_csv_atomic(recovery_path, recovery)

print(f" Saved: {recovery_[path.name](http://path.name)}")



# Summaries

if not regime_summaries.empty:

regime_summary_path = output_dir / "twindow_regime_summary.csv"

save_csv_atomic(regime_summary_path, regime_summaries)

print(f" Saved: {regime_summary_[path.name](http://path.name)}")



if not did_summaries.empty:

did_summary_path = output_dir / "twindow_branch_DID_summary.csv"

save_csv_atomic(did_summary_path, did_summaries)

print(f" Saved: {did_summary_[path.name](http://path.name)}")



if not topology_summaries.empty:

topology_summary_path = output_dir / "twindow_topology_summary.csv"
```

```
save_csv_atomic(topology_summary_path, topology_summaries)

print(f" Saved: {topology_summary_[path.name](http://path.name)}")



# Text report

report = generate_text_report(

validation, regime_contrasts, branch_did, topology,

regime_summaries, did_summaries, topology_summaries

)

report_path = output_dir / "twindow_sensitivity_report.txt"

with open(report_path, "w", encoding="utf-8") as f:

f.write(report)

print(f" Saved: {report_[path.name](http://path.name)}")



# Analysis manifest

output_files = {

"regime_contrasts": regime_contrast_path,

"branch_did": branch_did_path,
```

```
"topology": topology_path,

"recovery": recovery_path,

"regime_summary": regime_summary_path,

"did_summary": did_summary_path,

"topology_summary": topology_summary_path,

"report": report_path,

}



analysis_manifest = generate_analysis_manifest(

csv_path, npz_path, manifest_path, validation, output_files

)

manifest_path_out = output_dir / "analysis_manifest.json"

save_json_atomic(manifest_path_out, analysis_manifest)

print(f" Saved: {manifest_path_[out.name](http://out.name)}")



# Print report to console

print("\n" + "=" * 80)
```

print("ANALYSIS REPORT")

print("=" * 80)

print(report)

print("\n" + "=" * 80)

print("ANALYSIS COMPLETE")

print("=" * 80)

if name == "main":

text

main()

This rewrite moves toward the correct five-stage analysis, but it still has multiple runtime failures and the recovery/horizon stage remains a placeholder. It is not ready to analyze a production sensitivity run.
Immediate runtime failures
1. Regime contrast pivot is incorrectly constructed

The code first filters to continue:

Python

continue_df = df[
df["branch"] == "continue"
]

but then pivots with:

Python

columns=["branch"]

This creates MultiIndex columns such as:

Python

("P_mean_late", "continue")

Later it tests:

Python

if outcome in row.index

where outcome is a plain string. That condition will be false, so no contrasts are created, followed by:

text

RuntimeError: No regime contrasts computed

Use a direct merge instead:

Python

```
def compute_regime_contrasts(
frame: pd.DataFrame,
outcomes: tuple[str, ...],
) -> pd.DataFrame:
cont = frame.loc[
frame["branch"] == "continue",
["seed", "rho", *outcomes],
].copy()

baseline = (
```

```
cont.loc[cont["rho"] == 1.0]
.drop(columns="rho")
.rename(
columns={
outcome: f"{outcome}_baseline"
for outcome in outcomes
}
)
)

merged = cont.merge(
baseline,
on="seed",
how="inner",
validate="many_to_one",
)

merged = merged.loc[
merged["rho"] != 1.0
]

rows = []

for _, row in merged.iterrows():
for outcome in outcomes:
value = row[outcome]
reference = row[
f"{outcome}_baseline"
]

rows.append({
"seed": row["seed"],
"rho": row["rho"],
"outcome": outcome,
"Y_rho_continue": value,
"Y_baseline_continue":
reference,
"paired_regime_contrast":
value - reference,
})
```

return pd.DataFrame(rows)

2. Summary tables and report expect incompatible column structures

This creates flat columns:

Python

```
regime_contrasts.groupby(...)[
"paired_regime_contrast"
].agg([
("mean", "mean"),
("std", "std"),
("n", "count"),
]).reset_index()
```

The resulting columns are approximately:

text

```
rho
outcome
mean
std
n
```

But the report accesses:

Python

```
out_data[
("paired_regime_contrast", "mean")
]
```

That column does not exist.

Use named aggregation:

Python

```
regime_summary = (
regime_contrasts
.groupby(
["rho", "outcome"],
as_index=False,
)
.agg(
mean=(
"paired_regime_contrast",
"mean",
),
std=(
"paired_regime_contrast",
"std",
),
n=(
"paired_regime_contrast",
"count",
),
)
)
```

Then report:

Python

```
mean_val = out_data["mean"].iloc[0]
```

Apply the same correction to DID summaries.
3. Topology summary indexing is wrong

The code applies:

Python

.reset_index()

but later iterates:

Python

for rho in topology_summaries.index:

That iterates row numbers 0,1,2,30,1,2,3, not rho values.

Use:

Python

```
for _, row in (
topology_summaries.iterrows()
):
rho = row["rho"]
```

Also flatten the MultiIndex metric columns before writing/reporting.
4. Conditional output paths are referenced when undefined

Recovery is always empty because it is a placeholder:

Python

recovery = empty DataFrame

Therefore this block never defines:

Python

recovery_path

but later:

Python

```
output_files = {
...
"recovery": recovery_path,
}
```

raises UnboundLocalError.

The same can happen for topology or summary paths.

Construct output_files incrementally:

Python

```
output_files = {}

if not recovery.empty:
recovery_path = ...
save(...)
output_files[
"recovery"
] = recovery_path
```

Scientific omissions
5. Recovery scaling remains explicitly unimplemented

This is still a placeholder:

Python

```
print(
"Recovery Scaling - Not yet implemented"
)
return empty DataFrame
```

Therefore the script does not answer:

trec/Twindow
trec/Twindow

and does not calculate:

condition-specific continue targets;
first crossing;
sustained recovery;
held recovery;
censoring;
restricted time;
AUC deficit;
fixed versus normalized horizons.

The script should not claim to implement the five-stage analysis while Stage 4b is absent.

Either implement recovery fully or fail with:

Python

raise NotImplementedError(
"recovery scaling is required "
"before production analysis"
)

Silently emitting no recovery output is worse.
6. Fixed and normalized horizon systems are still absent

No calculation uses:

text

fixed_physical
normalized_EMA

The scalar P_mean_late and Weighted_similarity_late produced by the current engine use condition-dependent, EMA-normalized windows. Comparing them across rho does not provide the fixed-physical primary estimand.

This cannot be repaired entirely in analysis if the simulation did not save fixed-physical PP and BB-mean accumulators.

The simulation engine/driver must save both horizon systems.
Cross-T topology
7. Cross-T topology is now conceptually correct, but key matching is unsafe

The correct matrices are finally compared:
Bs,ρ,continueversusBs,1,continue.
Bs,ρ,continueversusBs,1,continue.

However, baseline matching searches for substrings:

Python

"rho_1" in key

which also matches:

text

rho_1.5

Use exact keys:

Python

```
baseline_key = (
f"continue_seed_{seed}_"
"rho_1.0_B_final"
)

rho_key = (
f"continue_seed_{seed}_"
f"rho_{rho}_B_final"
)
```

Better still, define a canonical rho ID in the simulation driver:

text

rho_0p5
rho_0p75
rho_1
rho_1p5
rho_2

and use the same helper in simulation and analysis.
8. Missing topology matrices must be fatal

The expected number of nonbaseline comparisons is:
40×4=160.
40×4=160.

The current code prints a warning and returns an empty DataFrame when matrices are missing.

Require exactly 160 rows. Any missing matrix is a schema failure.
9. NPZ loading is not selective

Despite the function name, it is called without keys:

Python

load_npz_selective(npz_path)

which loads the entire NPZ.

Construct the exact list of 200 continue B_final keys and load only those.
Difference-in-differences
10. Baseline rho should be skipped

The DID loop includes ρ=1ρ=1, producing rows that should be exactly zero.

Add:

Python

```
if rho == 1.0:
continue
```

Expected DID count for three noncontinue branches and three outcomes is:
40×4×3×3=
40×4×3×3=

]

Validate this count.
11. Missing/NaN values are not validated

The code assumes continue rows exist and accesses:

Python

```
.values[0]
```

Validation checks combination presence but not that every primary outcome is numeric and finite.

Before contrasts, require all primary outcome columns to exist and define a missing-value policy. For primary freezeable analyses, unexpected NaN should be fatal unless the outcome is documented as undefined.
Bootstrap reproducibility
12. Bootstrap uses global random state

This uses:

Python

```
np.random.choice(...)
```

without a seeded generator. Results are not reproducible and the CLI does not offer a bootstrap seed.

Add:

Python

--bootstrap-seed

and use:

Python

rng = np.random.default_rng(
bootstrap_seed
)

For each summary group:

Python

indices = rng.integers(
0,
len(values),
size=(
n_bootstrap,
len(values),
),
)
bootstrap_means = np.mean(
values[indices],
axis=1,
)

Record seed and repetitions in the manifest.
Validation
13. Nonfreezeable runs should fail, not warn

This currently adds only a warning:

Python

if not manifest["freezeable"]:
warnings.append(...)

The production analysis should stop unless an explicit debugging flag is provided.

Likewise, require:

text

baseline equivalence certificate path/hash
full cohort completed
all condition statuses successful

not merely a nested status string.
14. Required outcome columns are not checked

Validation confirms rows, seeds, rho values, and branches, but not the presence/completeness of:

text

Weighted_similarity_late
Retention_contrast_late
P_mean_late

Add exact schema validation and numeric conversion.
JSON and output handling
15. Analysis manifest lacks its own source hash and command

Add:

text

analysis source path/hash

command line
bootstrap seed/repetitions
simulation input hashes
baseline certificate hash
output hashes
interpretation constraints

The current manifest hashes inputs and outputs but not the analysis source itself.
16. Text report is written nonatomically

Use the same atomic writer as other project analyses.
17. save_csv_atomic does not fsync

It writes to a temporary path and replaces it, which is partially atomic, but does not flush/fsync explicitly. Align it with the established atomic-output helper.
Smaller points

time is now imported, which fixes the earlier JSON-report NameError.
Pandas loading is an improvement.
Keep seed IDs as strings, as done.
The Jaccard threshold should come from the simulation manifest rather than hardcoded 0.12.
Normalized distance should use upper-triangle edge vectors for consistency, though using full symmetric matrices gives the same ratio when diagonals are zero.
df.pivot_table silently averages duplicate rows; validation already rejects duplicates, so direct indexing is safer.
The report function receives raw contrast tables but does not use them.
compute_seed_level_summaries docstring mentions p-values but computes none; remove that claim.

Required rewrite gate

Before using this analysis:

fix regime contrast construction;
fix summary column/report access;
implement recovery scaling;
implement both horizon systems in simulation outputs;
enforce exact topology key schema and 160 comparisons;
seed the bootstrap;
fail on nonfreezeable/incomplete runs;

make outputs and manifest production-grade.

Rating
Area Rating
Prespecified estimand structure 7.5/10
Regime contrast implementation 2/10
Branch DID implementation 6/10
Cross-T topology implementation 6/10
Recovery/horizon implementation 0.5/10
Reproducibility 4/10
Current runtime correctness 3/10
Overall: 4/10

This is conceptually much closer to the intended analysis, but it will fail in Stage 2 or report generation and cannot answer the recovery-scaling question until Stage 4b and the two horizon systems are implemented.

```
#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Driver.

Parameterized study varying T_window = rho T_ where T_* = 25 / mean(OMEGA) = 17.5

and rho ∈ {0.5, 0.75, 1, 1.5, 2}.

Study Design:

For each seed s ∈ seeds (40 seeds)

For each rho ∈ {0.5, 0.75, 1, 1.5, 2}

Set T_window = rho * 17.5

Set alpha = exp(-dt / T_window)

Full regeneration for each (s, rho) - NO split state reuse
```

Branches: continue, reset_ema, reset_B, future_shuffle

Fixed physical horizons: immediate=0-17.5, recovery=17.5-87.5, late=87.5-320

EMA-normalized horizons: immediate=0-1 T_window, recovery=1-5 T_window

Output:

Scalars in CSV: per (seed, rho, branch) scalar outcomes

Arrays in NPZ: per (seed, rho, branch) trajectory and matrix outcomes

MANDATORY: Baseline equivalence test at rho=1 must pass against frozen production.

"""

```
from future import annotations

import argparse

import csv

import hashlib

import json

import os

import platform

import secrets

import sys

import time
```

```
from pathlib import Path

from typing import Any

import numpy as np
Import the experiment engine

import aief_experiment_engine_v1 as engine
================================================================================
Study Configuration
================================================================================

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"

WORKFLOW_VERSION = "v1.0"
Frozen production values (from AIEF_1_4_memory_ablation_updated_REAL.py)

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)
```

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA))

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.5
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_STAR / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Rho values for T_window sensitivity

RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)
Frozen branch order from AIEF_1_4_memory_ablation_updated_REAL.py

FROZEN_BRANCH_ORDER = (

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",

)
Study subset of branches

STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Fixed physical horizon times (in time units, not steps)

PHYSICAL_IMMEDIATE_END = 17.5

PHYSICAL_RECOVERY_END = 87.5

PHYSICAL_LATE_START = 87.5

PHYSICAL_FUTURE_END = 320.0
Defaults

DEFAULT_MASTER_SEED = 20260714

DEFAULT_N_SEEDS = 40

DEFAULT_RUN_ROOT = "runs"
==========
REQUIRED OUTPUT SCHEMA
==========
These are the MINIMUM required outputs that must be present in engine results
If any are missing, the study should fail with a clear error

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)

REQUIRED_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
Primary outcomes (must be present)

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late",

"Retention_contrast_late",

"P_mean_late",

)
Required scalar keys that the engine currently produces
These are validated in validate_branch_outputs()

REQUIRED_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Metadata

"T_window",

"rho",

)

Desired scalar keys (not yet produced by engine, but included in CSV for future)

DESIRED_BRANCH_SCALAR_KEYS = (

text

Projection diagnostics (not yet in engine)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

Time-based metrics (not yet in engine)

"Time_to_similarity_below_0_9",

"Time_to_similarity_below_0_5",

Ratio metrics (not yet in engine)

"Late_persistence_ratio_to_continue",

"Late_similarity_ratio_to_continue",

)
All scalar keys (required + desired) for CSV output

BRANCH_SCALAR_KEYS = REQUIRED_BRANCH_SCALAR_KEYS +
DESIRED_BRANCH_SCALAR_KEYS
All matrix keys

BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS

def validate_branch_outputs(branch: str, branch_result: dict[str, Any]) -> None:

text

"""Validate that a branch result contains all required outputs.

Raises RuntimeError if required keys are missing.

"""

missing_scalars = set(REQUIRED_BRANCH_SCALAR_KEYS) - set(branch_result.keys())

missing_matrices = set(REQUIRED_BRANCH_MATRIX_KEYS) - set(branch_result.keys())

if missing_scalars or missing_matrices:

missing = list(missing_scalars) + list(missing_matrices)

raise RuntimeError(

```
f"Branch '{branch}' missing required outputs: {missing[:10]}... "

f"({len(missing)} total missing)"

)

# ============================================================================
# Utility Functions
# ============================================================================

def parse_master_seed(value: str) -> int:

text

"""Parse master seed from string."""

if value.strip().lower() == "random":

return secrets.randbits(64)

try:

result = int(value, 0)

except ValueError as exc:

raise argparse.ArgumentTypeError(

"master seed must be a nonnegative integer or 'random'"

) from exc

if result &lt; 0:
```

raise argparse.ArgumentTypeError("master seed must be nonnegative")

return result

def generate_seed_set(master_seed: int, count: int) -> list[int]:

text

"""Generate deterministic seed set from master seed."""

if count <= 0:

raise ValueError("seed count must be positive")

children = np.random.SeedSequence(master_seed).spawn(count)

seeds = [

int(child.generate_state(1, dtype=np.uint64)[0])

for child in children

]

if len(set(seeds)) != len(seeds):

raise RuntimeError("duplicate generated seeds")

return seeds

def source_sha256() -> str | None:

text

"""Compute SHA-256 of this source file."""

```
try:
return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
except (NameError, OSError):
return None
def atomic_write_text(path: str | Path, text: str) -> None:
text
"""Write text atomically with fsync."""
path = Path(path)
path.parent.mkdir(parents=True, exist_ok=True)
temporary = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")
try:
with open(temporary, "w", encoding="utf-8", newline="") as handle:
handle.write(text)
handle.flush()
os.fsync(handle.fileno())
os.replace(str(temporary), str(path))
finally:
```

```
if temporary.exists():

temporary.unlink()

def atomic_write_json(path: str | Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def save_npz_atomic(path: str | Path, payload: dict[str, Any]) -> None:

text

"""Save NPZ atomically with fsync.



Uses a temporary file with .npz suffix so np.savez_compressed doesn't add extra suffix.

"""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

# Use tempfile.mkstemp to create a temporary file in the same directory

fd, temporary = tempfile.mkstemp(
```

```
dir=str(path.parent),

prefix=f".{[path.name](http://path.name)}.",

suffix=".tmp.npz",

)

try:

with os.fdopen(fd, "wb") as handle:

np.savez_compressed(handle, **payload)

handle.flush()

os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)

def load_npz(path: str | Path) -> dict[str, Any]:

text

"""Load NPZ file."""

payload: dict[str, Any] = {}

with np.load(str(path), allow_pickle=False) as data:
```

```
for key in data.files:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

return payload

================================================================================
Configuration and Validation
================================================================================

def create_model_config(rho: float) -> engine.ModelConfig:

text

"""Create ModelConfig for a given rho value."""

T_window = rho * FROZEN_T_STAR

return engine.ModelConfig(

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,
```

```
q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=T_window,

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=1e-12,

)

def create_protocol_config() -> engine.ProtocolConfig:

text

"""Create ProtocolConfig matching frozen production."""

return engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480
```

diagnostic_stride=50,

branches=STUDY_BRANCHES,

)

def compute_horizon_indices(

text

future_steps: int,

T_window: float,

dt: float,

) -> dict[str, tuple[int, int]]:

text

"""Compute horizon step indices for a given T_window.

Fixed physical horizons:

- immediate: 0-17.5 time units

- recovery: 17.5-87.5 time units

- late: 87.5-320 time units

EMA-normalized horizons:

- immediate: 0-1 T_window

- recovery: 1-5 T_window

- late: 5 T_window to end

Returns dict mapping horizon name to (start_step, end_step) inclusive start, exclusive end.

"""

window_steps = int(round(T_window / dt))

Fixed physical horizons (in steps)

immediate_end_physical = int(round(PHYSICAL_IMMEDIATE_END / dt))

recovery_end_physical = int(round(PHYSICAL_RECOVERY_END / dt))

EMA-normalized horizons (in steps)

immediate_end_normalized = window_steps # 0-1 T_window

recovery_end_normalized = 5 * window_steps # 1-5 T_window

```
# Use the more restrictive (shorter) horizon for each phase

immediate_end = min(immediate_end_physical, immediate_end_normalized)

recovery_end = min(recovery_end_physical, recovery_end_normalized)



# Clamp to future_steps

immediate_end = min(immediate_end, future_steps)

recovery_end = min(recovery_end, future_steps)



return {

"immediate": (0, immediate_end),

"recovery": (immediate_end, recovery_end),

"late": (recovery_end, future_steps),

}

============================================================================
====
Simulation Functions
============================================================================
====

def simulate_seed_rho(
```

```
text

seed: int,

rho: float,

config: engine.ModelConfig,

protocol: engine.ProtocolConfig,

study_branches: tuple[str, ...],

) -> dict[str, Any]:

text

"""Simulate a single (seed, rho) combination.



CRITICAL: Uses FROZEN_BRANCH_ORDER for RNG namespace, but only simulates

study_branches subset. This ensures future_shuffle gets the 7th RNG sequence

(not the 4th) as in production.



Returns complete results for all branches at this (seed, rho).

"""

# Create branch RNGs using FULL frozen branch order for correct namespace
```

```
branch_rngs = engine.make_branch_rngs(seed, FROZEN_BRANCH_ORDER)



# Create a protocol with just the study branches

study_protocol = engine.ProtocolConfig(

T_sim=protocol.T_sim,

burn_end_time=protocol.burn_end_time,

split_time=protocol.split_time,

future_duration=protocol.future_duration,

diagnostic_stride=protocol.diagnostic_stride,

branches=study_branches,

)



# Simulate prefix

split_state, prefix = engine.simulate_prefix(seed, config, study_protocol)



# Simulate only study branches, using RNGs from frozen namespace

branch_results = {}
```

```
for branch in study_branches:

if branch not in branch_rngs:

raise RuntimeError(

f"Branch '{branch}' not in FROZEN_BRANCH_ORDER. "

f"Cannot map to production RNG namespace."

)

rng = branch_rngs[branch]

branch_results[branch] = engine.simulate_future(

split_state.clone(),

prefix,

branch,

config,

study_protocol,

rng,

)

# Add metadata

branch_results[branch]["T_window"] = config.T_window

branch_results[branch]["rho"] = rho
```

```
return {

"seed": int(seed),

"rho": rho,

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},

"protocol": {

"T_sim": study_protocol.T_sim,

"burn_end_time": study_protocol.burn_end_time,

"split_time": study_protocol.split_time,
```

"future_duration": study_protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

def run_baseline_equivalence(

text

seeds: list[int],

frozen_manifest_path: str,

frozen_npz_path: str,

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for ALL seeds.

Verifies:

1. Seed list matches frozen manifest exactly (same seeds, same order)

2. All 40 seeds pass equivalence (if 40 seeds provided)

3. Engine produces all required outputs

"""

import json

Load frozen manifest to compare seeds

with open(frozen_manifest_path, 'r') as f:

frozen_manifest = json.load(f)

frozen_seeds = frozen_manifest["randomization"]["paired_seeds"]

Verify seed list matches frozen manifest EXACTLY

This is critical: baseline equivalence tests a specific cohort

if len(seeds) != len(frozen_seeds):

return {

"status": "failed",

```
"error": f"Seed count mismatch: generated={len(seeds)}, frozen={len(frozen_seeds)}",

"frozen_seeds": frozen_seeds,

"generated_seeds": [int(s) for s in seeds],

"freezeable": False,

}



for i, (gen_seed, frozen_seed) in enumerate(zip(seeds, frozen_seeds)):

if int(gen_seed) != int(frozen_seed):

return {

"status": "failed",

"error": f"Seed mismatch at index {i}: generated={gen_seed}, frozen={frozen_seed}",

"frozen_seeds": frozen_seeds,

"generated_seeds": [int(s) for s in seeds],

"freezeable": False,

}



# Verify master seed matches
```

```
frozen_master_seed = frozen_manifest["randomization"]["master_seed"]

# Note: We can't check the generated master seed here as it's not passed,

# but the seed generation method should match



# Create config for rho=1

config = create_model_config(rho=1.0)

protocol = create_protocol_config()



# Run the engine's baseline equivalence test with ALL seeds

results = [engine.run](http://engine.run)_baseline_equivalence(

seeds=seeds,

config=config,

protocol=protocol,

frozen_manifest_path=frozen_manifest_path,

frozen_npz_path=frozen_npz_path,

branch_order=FROZEN_BRANCH_ORDER, # Use FULL branch order for RNG namespace

)
```

```
# Add seed comparison info

results["seed_comparison"] = {

"generated_seeds": [int(s) for s in seeds],

"frozen_seeds": [int(s) for s in frozen_seeds],

"seeds_match": True,

}



return results

================================================================================
Manifest and Output
================================================================================

def build_manifest(

text

master_seed: int,

seeds: list[int],

rho_values: tuple[float, ...],
```

) -> dict[str, Any]:

text

"""Build study manifest."""

Compute config hash from model parameters

model_params = {

"N": FROZEN_N,

"dt": FROZEN_DT,

"tau": FROZEN_TAU,

"omega": FROZEN_OMEGA.tolist(),

"a": FROZEN_A,

"q": FROZEN_Q,

"delta": FROZEN_DELTA,

"eta": FROZEN_ETA,

"gamma": FROZEN_GAMMA,

"mu": FROZEN_MU,

"B_c": FROZEN_B_C,

"B_init_scale": FROZEN_B_INIT_SCALE,

"T_star": FROZEN_T_STAR,

```
}


protocol_params = {
"T_sim": FROZEN_T_SIM,
"burn_end_time": 160.0,
"split_time": 480.0,
"future_duration": 320.0,
"diagnostic_stride": 50,
"branches": list(STUDY_BRANCHES),
}


study_params = {
"rho_values": list(rho_values),
"T_window_values": [rho * FROZEN_T_STAR for rho in rho_values],
"n_seeds": len(seeds),
"statistical_unit": "40 seed/graphs",
}
```

```
manifest = {
"study_name": STUDY_NAME,
"workflow_version": WORKFLOW_VERSION,
"source_sha256": source_sha256(),
"model": model_params,
"protocol": protocol_params,
"study": study_params,
"randomization": {
"master_seed": int(master_seed),
"paired_seeds": [int(seed) for seed in seeds],
"seed_generation": "SeedSequence(master_seed).spawn(n_seeds)",
"branch_generation": "second per-seed child sequence spawns named uint64 branch seeds",
"bit_generator": np.random.default_rng(0).bit_generator.__class__.__name__,
},
"environment": {
"python": platform.python_version(),
```

"numpy": np.__version__,

"platform": platform.platform(),

},

"schema": {

"prefix_keys": list(REQUIRED_PREFIX_KEYS),

"branch_scalar_keys": list(BRANCH_SCALAR_KEYS),

"branch_matrix_keys": list(BRANCH_MATRIX_KEYS),

"required_branch_scalar_keys": list(REQUIRED_BRANCH_SCALAR_KEYS),

"required_branch_matrix_keys": list(REQUIRED_BRANCH_MATRIX_KEYS),

},

"notes": {

"purpose": "T_window sensitivity study - varying T_window = rho *T_*",

"T_*": FROZEN_T_STAR,

"fixed_physical_horizons": {

"immediate": f"0-{PHYSICAL_IMMEDIATE_END} time units",

"recovery": f"{PHYSICAL_IMMEDIATE_END}-{PHYSICAL_RECOVERY_END} time units",

"late": f"{PHYSICAL_RECOVERY_END}-{PHYSICAL_FUTURE_END} time units",

},

"ema_normalized_horizons": {

"immediate": "0-1 T_window",

"recovery": "1-5 T_window",

"late": "5 T_window to end",

},

"full_regeneration": "Each (seed, rho) pair is independently initialized",

},

}

return manifest

def flatten_result(

text

seed: int,

rho: float,

result: dict[str, Any],

) -> dict[str, Any]:

```
text

"""Flatten a (seed, rho) result into a flat dict for NPZ storage."""

payload: dict[str, Any] = {

f"seed_{seed}_rho_{rho}_config_T_window": result["config"]["T_window"],

f"seed_{seed}_rho_{rho}_config_alpha": result["config"]["alpha"],

}



# Add prefix keys

for key, value in result["prefix"].items():

payload[f"prefix_seed_{seed}_rho_{rho}_{key}"] = value



# Add branch results

for branch, branch_result in result["branches"].items():

for key, value in branch_result.items():

if isinstance(value, np.ndarray):

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value

else:
```

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value

return payload

def extract_scalar_row(

text

seed: int,

seed_index: int,

rho: float,

branch: str,

branch_result: dict[str, Any],

) -> dict[str, Any]:

text

"""Extract a scalar row for CSV output.

Validates that all required outputs are present before extraction.

"""

Validate required outputs

```
validate_branch_outputs(branch, branch_result)



row = {

"seed_index": seed_index,

"seed": seed,

"rho": rho,

"T_window": branch_result.get("T_window", float('nan')),

"branch": branch,

}



for key in BRANCH_SCALAR_KEYS:

if key in branch_result:

row[key] = branch_result[key]

else:

row[key] = float('nan')



return row
```

```
================================================================================
Main Execution
================================================================================

def parse_arguments() -> argparse.Namespace:

text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

parser.add_argument(

"--master-seed",

type=parse_master_seed,

default=DEFAULT_MASTER_SEED,

help="fixed integer master seed or 'random' (default: 20260714)",

)

parser.add_argument(

"--n-seeds",
```

```
type=int,

default=DEFAULT_N_SEEDS,

help=f"number of seeds (default: {DEFAULT_N_SEEDS})",

)

parser.add_argument(

"--run-root",

default=DEFAULT_RUN_ROOT,

help=f"output root directory (default: {DEFAULT_RUN_ROOT})",

)

parser.add_argument(

"--frozen-manifest",

default=None,

help="path to frozen manifest for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-npz",
```

default=None,

help="path to frozen NPZ for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

NOTE: --skip-baseline removed for production safety

Baseline equivalence is MANDATORY and cannot be skipped

For debugging, use a separate nonproduction namespace

parser.add_argument(

"--dry-run",

action="store_true",

help="validate configuration and exit without running",

)

return parser.parse_args()

def main() -> None:

text

"""Main entry point."""

args = parse_arguments()

```
# Generate seeds

seeds = generate_seed_set(int(args.master_seed), args.n_seeds)



# Build manifest

manifest = build_manifest(

master_seed=int(args.master_seed),

seeds=seeds,

rho_values=RHO_VALUES,

)



# Compute config hash

manifest_text = json.dumps(

manifest, sort_keys=True, separators=(",", ":"), allow_nan=False

)

config_hash = hashlib.sha256(manifest_text.encode()).hexdigest()[:12]
```

```
# Create run directory

run_dir = Path([args.run](http://args.run)_root) / config_hash

run_dir.mkdir(parents=True, exist_ok=True)



# Save manifest

manifest["config_hash"] = config_hash

manifest_path = run_dir / "manifest.json"

atomic_write_json(manifest_path, manifest)



print(f"{STUDY_NAME}")

print(f" master seed: {args.master_seed}")

print(f" n_seeds: {args.n_seeds}")

print(f" rho values: {RHO_VALUES}")

print(f" configuration: {config_hash}")

print(f" output: {run_dir}")

print()
```

```
# Dry run check

if args.dry_run:

print("Dry run: configuration validated successfully.")

print(f"Manifest saved to {manifest_path}")

return



# Create protocol config (same for all rho)

protocol = create_protocol_config()



# MANDATORY: Run baseline equivalence test at rho=1

# This cannot be skipped - baseline equivalence is required for freezeability

print("=" * 80)

print("Running MANDATORY baseline equivalence test at rho=1...")

print("=" * 80)



# Check if frozen data paths are provided

if args.frozen_manifest is None or args.frozen_npz is None:
```

```
print("\nFrozen data paths not provided!")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then provide the paths with --frozen-manifest and --frozen-npz")

sys.exit(1)



# Check if frozen data paths exist

if not os.path.exists(args.frozen_manifest):

print(f"\nFrozen manifest not found: {args.frozen_manifest}")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)



if not os.path.exists(args.frozen_npz):

print(f"\nFrozen NPZ not found: {args.frozen_npz}")
```

```
print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)



# Run baseline equivalence - MANDATORY for all seeds

baseline_results = run_baseline_equivalence(

seeds=seeds,

frozen_manifest_path=args.frozen_manifest,

frozen_npz_path=args.frozen_npz,

)



if baseline_results["status"] != "passed":

error_msg = baseline_results.get('error', 'unknown')

print(f"\nBaseline equivalence test FAILED: {error_msg}")



# Check if it's a configuration mismatch
```

```
if "mismatch" in error_msg.lower():

print("\nConfiguration mismatch detected!")

print("The frozen data does not match the expected production configuration.")

print("Expected configuration (from AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)):")

print(f" N={FROZEN_N}, DT={FROZEN_DT}, TAU={FROZEN_TAU}")

print(f" OMEGA={FROZEN_OMEGA.tolist()}")

print(f" T_WINDOW={FROZEN_T_STAR:.6f}")

print("\nTo generate correct frozen data:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path with --frozen-manifest and --frozen-npz")



# Save failure manifest

failure_manifest = {

**manifest,

"config_hash": config_hash,

"status": "FAILED",

"failure_reason": "baseline_equivalence_failed",
```

```
"failure_details": baseline_results,

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)

print(f"\nFailure manifest saved to {failure_manifest_path}")


sys.exit(1)

else:

print("\nBaseline equivalence test PASSED")

print("=" * 80)


# Update manifest with baseline equivalence certificate

manifest["baseline_equivalence"] = {

"status": "passed",

"n_seeds_tested": baseline_results["n_seeds"],

"n_seeds_passed": baseline_results["n_seeds"],

"engine_source_sha256": baseline_results.get("engine_source_sha256"),
```

```
"frozen_manifest_hash": baseline_results.get("frozen_manifest_hash"),

"frozen_npz_hash": baseline_results.get("frozen_npz_hash"),

}


# Re-save manifest with baseline certificate

atomic_write_json(manifest_path, manifest)



# Run the study

print("\nRunning T_window sensitivity study...")

print(f" rho values: {RHO_VALUES}")

print(f" branches: {STUDY_BRANCHES}")

print(f" seeds: {len(seeds)}")



all_scalar_rows: list[dict[str, Any]] = []

all_matrix_payload: dict[str, Any] = {}



started = time.time()
```

```
for seed_index, seed in enumerate(seeds):

for rho in RHO_VALUES:

# Create config for this rho

config = create_model_config(rho)


# Run simulation

result = simulate_seed_rho(

seed=seed,

rho=rho,

config=config,

protocol=protocol,

study_branches=STUDY_BRANCHES,

)


# Extract scalar rows

for branch in STUDY_BRANCHES:

branch_result = result["branches"][branch]

row = extract_scalar_row(
```

```
seed=seed,

seed_index=seed_index,

rho=rho,

branch=branch,

branch_result=branch_result,

)

all_scalar_rows.append(row)


# Extract matrix payload

matrix_payload = flatten_result(seed, rho, result)

all_matrix_payload.update(matrix_payload)


# Status

elapsed = time.time() - started

total_combinations = len(seeds) * len(RHO_VALUES)

current_combination = seed_index * len(RHO_VALUES) + list(RHO_VALUES).index(rho) + 1

print(f" [{current_combination}/{total_combinations}] seed={seed}, rho={rho:.2f} "

f"({elapsed:.1f}s)")
```

```
# Save outputs

csv_path = run_dir / f"{STUDY_NAME}_{config_hash}_scalar_summary.csv"

npz_path = run_dir / f"{STUDY_NAME}_{config_hash}_branch_diagnostics.npz"



# Write CSV

if all_scalar_rows:

# Sort by seed_index, rho, branch for consistent ordering

all_scalar_rows.sort(key=lambda r: (r["seed_index"], r["rho"], r["branch"]))


# Get all fieldnames

fieldnames = list(all_scalar_rows[0].keys())


temporary_csv = csv_path.with_suffix(csv_path.suffix + ".tmp")

try:

with open(temporary_csv, "w", newline="", encoding="utf-8") as handle:

writer = csv.DictWriter(handle, fieldnames=fieldnames)

writer.writeheader()
```

```
writer.writerows(all_scalar_rows)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary_csv), str(csv_path))

finally:

if temporary_csv.exists():

temporary_csv.unlink()



# Write NPZ

if all_matrix_payload:

save_npz_atomic(npz_path, all_matrix_payload)



# Print summary

elapsed = time.time() - started

print(f"\nCompleted in {elapsed:.1f}s")

print(f"Saved {csv_path}")

print(f"Saved {npz_path}")
```

```
print(f"Manifest: {manifest_path}")



# Quick summary statistics

print("\n" + "=" * 80)

print("Summary Statistics by (rho, branch)")

print("=" * 80)



# Validate we have primary outcomes before summarizing

missing_primary = []

for row in all_scalar_rows:

for key in PRIMARY_OUTCOMES:

if key not in row or (isinstance(row[key], float) and np.isnan(row[key])):

missing_primary.append(f"{row['seed']}_rho_{row['rho']}_branch_{row['branch']}_{key}")



if missing_primary:

print(f"WARNING: {len(missing_primary)} primary outcomes are NaN or missing!")

print(f" First 5 missing: {missing_primary[:5]}")
```

```
print(" This indicates engine is not producing required outputs.")



# Group by rho and branch

for rho in RHO_VALUES:

for branch in STUDY_BRANCHES:

rows = [r for r in all_scalar_rows if r["rho"] == rho and r["branch"] == branch]

if rows:

# Only show keys that are actually present and not all NaN

available_keys = [k for k in PRIMARY_OUTCOMES if any(

k in r and not (isinstance(r[k], float) and np.isnan(r[k]))

for r in rows

)]


if not available_keys:

print(f" rho={rho:.2f}, branch={branch:&lt;15} | ALL PRIMARY OUTCOMES MISSING/NaN!")

continue


stats = {}
```

for key in available_keys:

values = [r[key] for r in rows if key in r and isinstance(r[key], (int, float)) and not np.isnan(r[key])]

if values:

stats[key] = f"{np.mean(values):.4f}±{np.std(values):.4f}"

else:

stats[key] = "N/A"

print(f" rho={rho:.2f}, branch={branch:<15} | " +

" | ".join(f"{k}={stats.get(k, 'N/A')}" for k in available_keys))

if name == "main":

text

main()

hello

try again

try again

This driver is now much closer to a runnable regenerated study. The full frozen RNG namespace is correctly separated from the four selected study branches, baseline equivalence is mandatory for all requested seeds, NPZ writing is fixed, and required output keys are validated.

The main remaining scientific blocker is that the two horizon systems are still not implemented separately. The driver claims fixed-physical primary outcomes, but the engine currently returns only TwindowTwindow-normalized horizon outcomes.
1. Branch RNG handling is now correct

This is the right structure:

Python

branch_rngs = engine.make_branch_rngs(
seed,
FROZEN_BRANCH_ORDER,
)

for branch in study_branches:
rng = branch_rngs[branch]

Thus future_shuffle receives the seventh frozen branch RNG stream rather than the fourth subset stream.

This resolves one of the previous driver’s most important reproducibility errors.
2. Baseline equivalence still validates only four branches

run_baseline_equivalence creates:

Python

protocol = create_protocol_config()

whose branches are:

Python

STUDY_BRANCHES

Therefore the engine validates only:

text

continue
reset_ema
reset_B
future_shuffle

despite comments saying “all original branch anchors.”

For the TwindowTwindow study, exact equivalence of these four branches is sufficient, but the report and certificate must say:

All four study branches passed baseline equivalence.

If the engine itself is to be frozen as a general AIEF replacement, validate all seven production branches with:

Python

baseline_protocol = engine.ProtocolConfig(
...,
branches=FROZEN_BRANCH_ORDER,
)

I recommend all seven once, because the engine is intended as reusable infrastructure.
3. Baseline failure diagnostics may be lost

The latest engine’s equivalence function raises on failure:

Python

raise RuntimeError(...)

The driver calls it without wrapping the call. Therefore the subsequent code intended to create FAILED_manifest.json may never execute.

Use:

Python

```
try:
baseline_results = (
run_baseline_equivalence(...)
)
except Exception as exc:
failure_manifest = {
**manifest,
"status": "FAILED",
"failure_reason":
"baseline_equivalence_exception",
"exception_type":
type(exc).__name__,
"exception_message": str(exc),
"freezeable": False,
}
atomic_write_json(
run_dir / "FAILED_manifest.json",
failure_manifest,
)
raise
```

Better still, make the engine return the failed certificate rather than raising before it can be saved.
4. Save the detailed baseline-equivalence certificate

The driver stores only a small summary in the manifest:

text

```
status
n_seeds
hashes
```

The engine calculated detailed per-seed/per-key results. Save them as:

text

baseline_equivalence_by_seed.csv
baseline_equivalence_manifest.json

and hash both in the final run manifest.

Without this, “40/40 passed” cannot be independently audited.
5. Fixed and normalized horizon systems remain conflated

compute_horizon_indices still calculates:

Python

```
min(
physical_boundary,
normalized_boundary,
)
```

This creates a hybrid horizon that is neither fixed-physical nor EMA-normalized.

The function is currently unused, but its logic is wrong and should not be activated.

Return two systems:

Python

```
def compute_horizon_systems(
future_steps,
T_window,
dt,
):
fixed_immediate = int(round(
17.5 / dt
))
fixed_recovery = int(round(
87.5 / dt
))

normalized_immediate = int(round(
```

```
T_window / dt
))
normalized_recovery = int(round(
5.0 * T_window / dt
))

return {
"fixed_physical": {
"immediate": (
0,
fixed_immediate,
),
"recovery": (
fixed_immediate,
fixed_recovery,
),
"late": (
fixed_recovery,
future_steps,
),
},
"normalized_EMA": {
"immediate": (
0,
normalized_immediate,
),
"recovery": (
normalized_immediate,
normalized_recovery,
),
"late": (
normalized_recovery,
future_steps,
),
},
}
```

Clamp each boundary independently to future_steps.
6. The engine currently returns only normalized-EMA P/BP/B windows

The engine uses:

Python

```
window_steps = round(
T_window / dt
)

recovery_end = 5 * window_steps
```

Therefore:

text

```
P_mean_late
Weighted_similarity_late
Retention_contrast_late
```

change both because the dynamics changed and because the averaging window changed.

Those outcomes cannot serve as the fixed-physical primary estimands.

The engine needs to accumulate both systems and expose explicit names, for example:

text

```
P_mean_late_fixed_physical
P_mean_late_normalized_EMA

Weighted_similarity_late_fixed_physical
Weighted_similarity_late_normalized_EMA

Retention_selected_late_fixed_physical
Retention_selected_late_normalized_EMA

Emergence_unselected_late_fixed_physical
Emergence_unselected_late_normalized_EMA
```

At ρ=1ρ=1, the two horizon systems should coincide to the production step boundaries and should match exactly.

The driver’s primary schema should use fixed-physical fields. Normalized fields should be secondary.

This is the main scientific correction still required.
7. Required-schema validation is good

The new:

Python

validate_branch_outputs(...)

prevents a long grid from silently producing NaN for required outcomes.

That is a major improvement.

However, once dual horizons are added, the required schema should explicitly contain both sets rather than generic names.

Desired-but-absent projection diagnostics may remain optional, provided they are clearly marked NaN and excluded from primary analyses.
8. Run completion and freezeability are not finalized

The manifest is written before baseline and simulation. After the grid completes, it is not rewritten with:

text

status = completed
freezeable = true/false
condition count
successful seed/rho count
failed conditions
scalar output hash

NPZ output hash
elapsed time
baseline certificate hashes

Add a final manifest rewrite.

At completion require:
40×5=200
40×5=200

successful seed/rho conditions and:
200×4=800
200×4=800

scalar rows.

Record exact counts.
9. No condition-status table or checkpoints

A 200-prefix regenerated study is too expensive to keep only in memory until completion.

Add:

text

twindow_condition_status.csv

with one row per seed/rho:

text

seed
rho
T_window
alpha
status
elapsed_seconds

checkpoint_path
config_hash

Write an atomic checkpoint after every successful seed/rho condition.

On resume:

validate the global config hash;
validate the condition config hash;
reject incompatible checkpoints.

A failed condition should make the production run nonfreezeable.
10. Full results are accumulated in memory

all_matrix_payload retains all branch arrays for 800 branch trajectories.

This may be manageable, but it is unnecessary and increases the cost of late failure.

Prefer:

text

conditions/seed_<seed>/rho_<id>.npz

then consolidate after 200/200 successful conditions.
11. One-seed smoke testing is awkward

The baseline wrapper requires the generated seed list to exactly equal the frozen manifest list.
Therefore:

text

--n-seeds 1

cannot perform a one-seed baseline smoke test.

Add a dedicated debug option:

text

--smoke-seed <frozen-seed>

that:

selects one seed from the frozen cohort;
compares it to the corresponding frozen arrays;
marks output run_scope=smoke;
sets freezeable=false.

The production path must still require all 40.
12. Random master seeds are incompatible with frozen baseline

The CLI allows:

text

--master-seed random

but mandatory baseline equivalence requires the exact frozen seed cohort.

Remove random for the production T-window study, or mark random runs as new nonbaseline experiments without frozen equivalence.

The default production path should require:

Python

master_seed == 20260714
n_seeds == 40

13. Final output manifest needs stronger provenance

Add:

driver source hash;
engine source hash;
frozen production source hash;
frozen manifest and NPZ file hashes;
baseline equivalence certificate hashes;
exact rho-to-TwindowTwindow-to-alpha table;
horizon definitions in steps and physical time;
branch RNG namespace order;
selected study branches;
command;
output hashes;
environment versions;
interpretation constraints.

14. Minor points

PHYSICAL_LATE_START duplicates PHYSICAL_RECOVERY_END.
FROZEN_T_STAR is approximately 17.50291717.502917, while fixed physical boundaries intentionally use the step-derived 17.517.5 and 87.587.5. Document this.
The output summary should use sample standard deviation if interpreted inferentially.
DESIRED_BRANCH_SCALAR_KEYS filled with NaN should not appear in a freezeable primary table without an explicit availability flag.
The references to a REAL.py file should use the actual frozen production filename.

What is now solid

correct model values;
exact regenerated prefix per seed/rho;
correct full branch RNG namespace;
four predeclared study branches;
five predeclared rho levels;
required schema checks;
all-40 baseline seed matching;
atomic CSV and fixed NPZ writing;
deterministic run/config hash;
exact seed pairing across rho.

Current rating

Area Rating
Study design 9/10
Full regeneration/pairing 9/10
RNG correctness 9/10
Baseline gate 7.5/10
Output schema enforcement 8/10
Horizon estimand implementation 3/10
Large-run robustness 4/10
Final provenance 5/10
Overall driver: 7/10

The remaining issue is no longer basic model fidelity. It is ensuring that the outcomes actually correspond to the prespecified fixed-physical and EMA-normalized estimands. Implement dual horizon accumulation and condition checkpointing before launching the complete grid.

Evaluate this:L

#!/usr/bin/env python3

"""AIEF Experiment Engine v1 - Parameterized simulation kernel.

This is a reusable, validated simulation engine that reproduces the frozen

AIEF-1.4 production run exactly and supports parameterized studies.

FROZEN PRODUCTION VALUES (from AIEF_1_4_memory_ablation_updated_REAL.py):

N = 12

DT = 0.01

TAU = 0.5

OMEGA = [1.00, 1.02, 1.50, 2.00, 2.01, 1.01, 1.03, 1.51, 2.02, 2.00, 1.00, 1.04]

A = 1.0

Q = 1.5

DELTA = 0.05

ETA = 0.04

GAMMA = 0.015

MU = 0.03

T_WINDOW = 25.0 / mean(OMEGA) = 17.5

B_C = 0.12

B_INIT_SCALE = 0.0005

PROTOCOL (from frozen run):

T_SIM = 800.0

N_STEPS = int(round(T_SIM / DT)) = 80000

BURN_END = int(0.20 * N_STEPS) = 16000 steps = 160.0 time units

HIST_END = int(0.60 * N_STEPS) = 48000 steps = 480.0 time units

FUTURE_STEPS = N_STEPS - HIST_END = 32000 steps = 320.0 time units

DELAY_STEPS = int(round(TAU / DT)) = 50

WINDOW_STEPS = int(round(T_WINDOW / DT)) = 1750

RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS) = 8750

Architecture:

ModelConfig: frozen dataclass for all model parameters

ProtocolConfig: frozen dataclass for protocol parameters

DynamicState: state container (immutable where possible)

Core functions: initialize, advance, simulate, interventions

Validation: baseline equivalence test

NOTE: This is a candidate reference implementation. After passing 40/40 baseline

text

equivalence tests, it will be frozen as the reference implementation.

"""

from future import annotations

import copy

import hashlib

import json

import math

import os

import tempfile

import time

```
from dataclasses import dataclass, field

from pathlib import Path

from typing import Any

import numpy as np
================================================================================
Frozen Production Constants (DO NOT CHANGE)
================================================================================

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04
```

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005

FROZEN_ZERO_TOL = 1e-12

FROZEN_PROJECTION_NORM_TOL = 1e-14

FROZEN_GAP_TOL = 1e-9
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA)) # 1.428333...

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.502917152858814

FROZEN_T_WINDOW = FROZEN_T_STAR # 17.502917152858814 (exact, NOT rounded)

FROZEN_ALPHA = math.exp(-FROZEN_DT / FROZEN_T_WINDOW)

FROZEN_CAPACITY = (FROZEN_A - FROZEN_DELTA) / FROZEN_Q
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_WINDOW / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Fixed physical horizon boundaries (in time units, NOT T_window dependent)
These are the PRIMARY estimand horizons

FIXED_PHYSICAL_IMMEDIATE_END = 17.5

FIXED_PHYSICAL_RECOVERY_END = 87.5

FIXED_PHYSICAL_FUTURE_END = 320.0
Derived fixed physical horizon steps

FIXED_PHYSICAL_IMMEDIATE_STEPS = int(round(FIXED_PHYSICAL_IMMEDIATE_END /
FROZEN_DT)) # 1750

FIXED_PHYSICAL_RECOVERY_STEPS = int(round(FIXED_PHYSICAL_RECOVERY_END /
FROZEN_DT)) # 8750
==========
Model Configuration
==========

@dataclass(frozen=True)

class ModelConfig:

text

"""Immutable model configuration.

All model-changing quantities must come from here. No hidden module globals.

"""

Core parameters

N: int

dt: float

tau: float

omega: tuple[float, ...] # Immutable tuple

Coupling parameters

a: float

q: float

delta: float

Dynamics parameters

eta: float

mu: float

gamma: float

EMA/threshold parameters

T_window: float

B_c: float

B_init_scale: float

scale_epsilon: float = 1e-12

Derived properties

@property

def mean_omega(self) -> float:

"""Mean of omega frequencies."""

return float(np.mean(self.omega))

@property

def T_star(self) -> float:

"""T_* = 25 / mean(omega)"""

```
return 25.0 / self.mean_omega



@property

def alpha(self) -&gt; float:

"""EMA smoothing factor: exp(-dt / T_window)"""

return math.exp(-self.dt / self.T_window)



@property

def capacity(self) -&gt; float:

"""Projection capacity: (a - delta) / q"""

return (self.a - [self.delta](http://self.delta)) / self.q



@property

def delay_steps(self) -&gt; int:

"""Delay steps: round(tau / dt)"""

return int(round(self.tau / self.dt))
```

```
def validate(self) -&gt; None:

"""Validate model configuration."""

if self.N &lt;= 0:

raise ValueError("N must be positive")

if self.dt &lt;= 0:

raise ValueError("dt must be positive")

if self.tau &lt;= 0:

raise ValueError("tau must be positive")

if len([self.omega](http://self.omega)) != self.N:

raise ValueError(f"omega must have {self.N} elements")

if self.a &lt;= 0 or self.q &lt;= 0 or [self.delta](http://self.delta) &lt; 0:

raise ValueError("a, q must be positive; delta must be nonnegative")

if self.eta &lt; 0 or [self.mu](http://self.mu) &lt; 0 or self.gamma &lt; 0:

raise ValueError("eta, mu, gamma must be nonnegative")

if self.T_window &lt;= 0:

raise ValueError("T_window must be positive")

if self.B_c &lt; 0:

raise ValueError("B_c must be nonnegative")
```

if self.B_init_scale < 0:

raise ValueError("B_init_scale must be nonnegative")

if self.scale_epsilon <= 0:

raise ValueError("scale_epsilon must be positive")

Validate derived values

if not (0 < self.alpha <= 1):

raise ValueError(f"alpha={self.alpha} must be in (0, 1]")

if self.capacity < 0:

raise ValueError(f"capacity={self.capacity} must be nonnegative")

@dataclass(frozen=True)

class ProtocolConfig:

text

"""Protocol configuration for experiment execution."""

Simulation time parameters (in time units, not steps)

T_sim: float

burn_end_time: float

split_time: float

future_duration: float

Diagnostics

diagnostic_stride: int

Branches to simulate

branches: tuple[str, ...]

def n_steps(self, model: ModelConfig) -> int:

"""Total steps."""

return int(round(self.T_sim / model.dt))

def burn_end(self, model: ModelConfig) -> int:

"""Burn-in end step."""

return int(round(self.burn_end_time / model.dt))

def hist_end(self, model: ModelConfig) -> int:

"""History end step (split point)."""

return int(round(self.split_time / model.dt))

def future_steps(self, model: ModelConfig) -> int:

"""Future simulation steps."""

return int(round(self.future_duration / model.dt))

def validate(self, model: ModelConfig) -> None:

"""Validate protocol against model."""

if self.T_sim <= 0:

raise ValueError("T_sim must be positive")

if not (0 < self.burn_end_time < self.split_time < self.T_sim):

raise ValueError("Invalid time parameters: must have 0 < burn_end < split < T_sim")

if self.future_duration <= 0:

raise ValueError("future_duration must be positive")

if self.diagnostic_stride <= 0:

raise ValueError("diagnostic_stride must be positive")

Validate that all time parameters are compatible with dt

if not np.isclose(self.T_sim / model.dt, round(self.T_sim / model.dt), rtol=1e-10):

raise ValueError(f"T_sim ({self.T_sim}) must be divisible by dt ({model.dt})")

if not np.isclose(self.burn_end_time / model.dt, round(self.burn_end_time / model.dt), rtol=1e-10):

raise ValueError(f"burn_end_time ({self.burn_end_time}) must be divisible by dt ({model.dt})")

if not np.isclose(self.split_time / model.dt, round(self.split_time / model.dt), rtol=1e-10):

raise ValueError(f"split_time ({self.split_time}) must be divisible by dt ({model.dt})")

if not np.isclose(self.future_duration / model.dt, round(self.future_duration / model.dt), rtol=1e-10):

raise ValueError(f"future_duration ({self.future_duration}) must be divisible by dt ({model.dt})")

Default frozen production configuration
NOTE: T_window must be the EXACT value 25.0/mean(OMEGA), not rounded

DEFAULT_MODEL_CONFIG = ModelConfig(

text

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=FROZEN_T_WINDOW, # 17.502917152858814 (exact)

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=FROZEN_ZERO_TOL,

)
Default protocol matching frozen run

DEFAULT_PROTOCOL_CONFIG = ProtocolConfig(

text

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=("continue", "reset_ema", "reset_B", "reset_B_ema", "B_carrier_only", "full_reset", "future_shuffle"),

)
==========
Utility Functions
==========

def compute_edges(N: int) -> tuple[np.ndarray, np.ndarray]:

text

"""Get upper triangle indices for edges."""

return np.triu_indices(N, k=1)

def validate_B_matrix(B: np.ndarray, N: int, eps: float = 1e-12) -> None:

text

"""Validate B matrix: symmetric, nonnegative, zero diagonal."""

if B.shape != (N, N):

raise ValueError(f"B shape {B.shape} != ({N}, {N})")

if not np.allclose(B, B.T, atol=eps):

raise ValueError("B is not symmetric")

if not np.allclose(np.diag(B), 0, atol=eps):

```
raise ValueError("B diagonal is not zero")

if np.any(B &lt; -eps):

raise ValueError("B has negative entries")

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file."""

digest = hashlib.sha256()

with open(path, "rb") as handle:

for block in iter(lambda: [handle.read](http://handle.read)(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

def array_sha256(arr: np.ndarray) -> str:

text

"""Compute SHA-256 hash of a numpy array."""

digest = hashlib.sha256()

shape_bytes = np.asarray(arr.shape, dtype=np.int64).tobytes()

digest.update(shape_bytes)
```

```
digest.update(arr.dtype.str.encode())

contiguous = np.ascontiguousarray(arr)

if contiguous.size &gt; 0:

digest.update(contiguous.tobytes())

return digest.hexdigest()

def atomic_write_text(path: Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.", suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:
```

```
if os.path.exists(tmp_path):

os.remove(tmp_path)

def atomic_write_json(path: Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def atomic_write_npz(path: Path, **kwargs: Any) -> None:

text

"""Write NPZ atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp.npz")

try:

with os.fdopen(fd, "wb") as handle:

np.savez(handle, **kwargs)

handle.flush()

os.fsync(handle.fileno())
```

```
os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

================================================================================
RNG Utilities (matching frozen production exactly)
================================================================================

def make_seed_sequence(seed: int, child_index: int) -> int:

text

"""Generate a child seed from SeedSequence(seed).spawn(...)[child_index].



This matches the frozen production RNG hierarchy exactly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

children = root.spawn(child_index + 1)

return int(children[child_index].generate_state(1, dtype=np.uint64)[0])

def make_prefix_rng(seed: int) -> np.random.Generator:
```

text

"""Create prefix RNG matching frozen production exactly.

Frozen production does:

root = np.random.SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = np.random.default_rng(init_sequence)

NOTE: default_rng(SeedSequence_object) is NOT the same as

default_rng(SeedSequence_object.generate_state(...))

The production uses the SeedSequence object directly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

return np.random.default_rng(init_sequence)

def make_branch_rngs(seed: int, branch_order: tuple[str, ...]) -> dict[str, np.random.Generator]:

```
text

"""Create branch RNGs matching frozen production algorithm.



Frozen: *, branch*root = SeedSequence(seed).spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

branch_rng_seeds = {branch: int(sequence.generate_state(1, dtype=np.uint64)[0])

for branch, sequence in zip(branch_order, branch_sequences)}

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

*, branch*root = root.spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))



return {

branch: np.random.default_rng(int(sequence.generate_state(1, dtype=np.uint64)[0]))

for branch, sequence in zip(branch_order, branch_sequences)

}
```

```
================================================================================
Projection Accumulator (matching production)
================================================================================

@dataclass

class ProjectionTotals:

text

"""Accumulator for projection diagnostics - exact match to production."""

steps: int = 0

violations: int = 0

active: int = 0

norm_sum: float = 0.0

norm_max: float = 0.0

relative_state_sum: float = 0.0

relative_state_max: float = 0.0

relative_update_sum: float = 0.0

relative_update_max: float = 0.0

g_min: float = np.inf
```

```
g_min_pre: float = np.inf

def update(self, info: dict[str, Any]) -&gt; None:

"""Update accumulator with step info."""

self.steps += 1

self.violations += int(info["pre_violation"])

self.g_min = min(self.g_min, float(info["g_min"]))

self.g_min_pre = min(self.g_min_pre, float(info["g_min_pre"]))

if info["projection_active"]:

self.active += 1

norm = float(info["projection_norm"])

rel_state = float(info["projection_relative_state"])

rel_update = float(info["projection_relative_update"])

self.norm_sum += norm

self.norm_max = max(self.norm_max, norm)

self.relative_state_sum += rel_state

self.relative_state_max = max(self.relative_state_max, rel_state)

self.relative_update_sum += rel_update

self.relative_update_max = max(self.relative_update_max, rel_update)
```

```
def metrics(self) -&gt; dict[str, float]:

"""Compute final metrics from accumulated values."""

active_denominator = max(self.active, 1)

return {

"G_min_future": float(self.g_min),

"G_min_pre_future": float(self.g_min_pre),

"Pre_gap_violation_fraction": float(self.violations / self.steps),

"Projection_active_fraction": float(self.active / self.steps),

"Projection_active_given_violation": float(self.active / self.violations)

if self.violations &gt; 0

else float('nan'),

"Projection_norm_mean_active": float(

self.norm_sum / active_denominator

),

"Projection_norm_max": float(self.norm_max),

"Projection_relative_state_mean_active": float(

self.relative_state_sum / active_denominator
```

),

"Projection_relative_state_max": float(self.relative_state_max),

"Projection_relative_update_mean_active": float(

self.relative_update_sum / active_denominator

),

"Projection_relative_update_max": float(self.relative_update_max),

}

==========
Dynamic State
==========

class DynamicState:

text

"""State container for AIEF-1.4 dynamics.

Matches the frozen production implementation exactly.

"""

def **init**(

```
self,

theta: np.ndarray,

B: np.ndarray,

coherence_acc: np.ndarray,

history: list[np.ndarray] | np.ndarray,

):

self.theta = np.asarray(theta, dtype=float).copy()

self.B = np.asarray(B, dtype=float).copy()

self.coherence_acc = np.asarray(coherence_acc, dtype=complex).copy()


# Handle history input - ensure it's a list of arrays

if isinstance(history, np.ndarray):

# If it's a 2D array, convert to list of 1D arrays

self.history = [np.asarray(h, dtype=float).copy() for h in history]

else:

self.history = [np.asarray(h, dtype=float).copy() for h in history]
```

```
def clone(self) -&gt; "DynamicState":

"""Create a deep copy of the state."""

return DynamicState(

theta=self.theta.copy(),

B=self.B.copy(),

coherence_acc=self.coherence_acc.copy(),

history=[h.copy() for h in self.history],

)



@property

def N(self) -&gt; int:

return len(self.theta)



def validate(self, config: ModelConfig, step: int | None = None) -&gt; None:

"""Validate state consistency."""

# Check history length

expected_history_len = config.delay_steps + 1
```

```
if len(self.history) != expected_history_len:

raise RuntimeError(

f"History length {len(self.history)} != expected {expected_history_len} "

f"(delay_steps={config.delay_steps}) at step {step}"

)

# Check theta shape

if self.theta.shape != (self.N,):

raise RuntimeError(f"theta shape {self.theta.shape} != ({self.N},)")

# Check B shape

if self.B.shape != (self.N, self.N):

raise RuntimeError(f"B shape {self.B.shape} != ({self.N}, {self.N})")

# Check coherence_acc shape

if self.coherence_acc.shape != (self.N, self.N):

raise RuntimeError(f"coherence_acc shape {self.coherence_acc.shape} != ({self.N}, {self.N})")

# Check B symmetry

if not np.allclose(self.B, self.B.T, atol=1e-12):

raise RuntimeError("B matrix is not symmetric")

# Check B diagonal
```

if not np.allclose(np.diag(self.B), 0, atol=1e-12):

raise RuntimeError("B matrix diagonal is not zero")

Check B non-negativity

if np.any(self.B < -1e-12):

raise RuntimeError("B matrix has negative entries")

==========
Simulation Core (exact reproduction of frozen production)
==========

def initialize_state(

text

seed: int,

config: ModelConfig,

) -> DynamicState:

text

"""Initialize state from seed using frozen algorithm exactly.

Matches production: simulate_prefix() initialization.

Uses: root = SeedSequence(seed); init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence)

"""

N = config.N

rng = make_prefix_rng(seed)

Initialize theta: uniform in [0, 2*pi) - matching production

theta = rng.random(N) *2.0* np.pi

Initialize B: small random symmetric matrix

B = config.B_init_scale * rng.random((N, N))

B = 0.5 * (B + B.T)

np.fill_diagonal(B, 0.0)

Initialize coherence accumulator

coherence_acc = np.zeros((N, N), dtype=complex)

```
# Initialize history: DELAY_STEPS + 1 copies of theta

history = [theta.copy() for *in range(config.delay*steps + 1)]



return DynamicState(

theta=theta,

B=B,

coherence_acc=coherence_acc,

history=history,

)

def compute_R(

text

state: DynamicState,

config: ModelConfig,

) -> np.ndarray:

text

"""Compute coherence matrix R from current state.
```

```
Exact reproduction of production compute_R().

"""

N = state.N

delayed = state.history[0]



# Convert omega to numpy array for indexing

omega_arr = np.asarray([config.omega](http://config.omega), dtype=float)



dphi = (

state.theta[:, None]

- delayed[None, :]

- ((omega_arr[:, None] + omega_arr[None, :]) / 2.0) * config.tau

)

z_inst = np.exp(1j * dphi)



# Update EMA

state.coherence_acc[:] = (
```

```
config.alpha *state.coherence_acc + (1.0 - config.alpha)* z_inst

)



# Compute magnitude and symmetrize

result = np.abs(state.coherence_acc)

result = 0.5 * (result + result.T)

np.fill_diagonal(result, 0.0)



return result

def advance_state(

text

state: DynamicState,

config: ModelConfig,

rng: np.random.Generator,

shuffle_driver: bool = False,

) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:

text
```

"""Advance state by one time step.

Exact reproduction of production advance_state().

Returns (R_physical, R_driver, info).

Production also returns projection diagnostics:

- projection_active: whether correction was applied

- projection_norm: norm of correction

- projection_relative_state: correction_norm / ||B_unprojected||

- projection_relative_update: correction_norm / ||B_unprojected - B_previous||

"""

N = state.N

edges = compute_edges(N)

Store previous B for projection

B_previous = state.B.copy()

```
# Compute R matrices

R_physical = compute_R(state, config)

R_driver = R_physical



if shuffle_driver:

values = R_physical[edges].copy()

rng.shuffle(values)

R_driver = np.zeros_like(R_physical)

R_driver[edges] = values

R_driver[(edges[1], edges[0])] = values



# Update B

B_unprojected = state.B + config.dt * (

(config.eta + [config.mu](http://config.mu)) * R_driver

- [config.mu](http://config.mu)

- config.gamma * state.B

)
```

```
B_unprojected = np.maximum(B_unprojected, 0.0)

B_unprojected = 0.5 * (B_unprojected + B_unprojected.T)

np.fill_diagonal(B_unprojected, 0.0)



# Projection diagnostics (matching production exactly)

proposed_update_norm = float(np.linalg.norm(B_unprojected - B_previous))



# Check gap constraint

row_sum_pre = np.sum(B_unprojected, axis=1)

g_min_pre = float(np.min(config.a - config.q * row_sum_pre))

pre_violation = bool(g_min_pre &lt; [config.delta](http://config.delta))



# Project if needed

B_projected = B_unprojected

correction_norm = 0.0

if pre_violation:

scale = np.minimum(1.0, config.capacity / (row_sum_pre + config.scale_epsilon))
```

```
B_projected = B_unprojected * np.minimum.outer(scale, scale)

np.fill_diagonal(B_projected, 0.0)

correction_norm = float(np.linalg.norm(B_projected - B_unprojected))



projection_active = bool(correction_norm &gt; FROZEN_PROJECTION_NORM_TOL)

relative_state = correction_norm / max(

float(np.linalg.norm(B_unprojected)), FROZEN_ZERO_TOL

)

relative_update = correction_norm / max(proposed_update_norm, FROZEN_ZERO_TOL)



# Validate projection

state.B = B_projected

row_sum = np.sum(state.B, axis=1)

g_min = float(np.min(config.a - config.q * row_sum))

if g_min &lt; [config.delta](http://config.delta) - 1e-9:

raise RuntimeError(f"Gap projection failed: g_min={g_min}")
```

```
# Update theta

delayed = state.history[0]

omega_arr = np.asarray([config.omega](http://config.omega), dtype=float)

coupling = np.sum(

state.B * np.sin(delayed[None, :] - state.theta[:, None]), axis=1

)

state.theta = np.mod(

state.theta + config.dt * (omega_arr + coupling),

2.0 * np.pi

)



# Update history

state.history.append(state.theta.copy())

if len(state.history) &gt; config.delay_steps + 1:

state.history.pop(0)



info = {
```

"pre_violation": pre_violation,

"projection_active": projection_active,

"projection_norm": correction_norm,

"projection_relative_state": relative_state,

"projection_relative_update": relative_update,

"g_min_pre": g_min_pre,

"g_min": g_min,

}

return R_physical, R_driver, info

def simulate_prefix(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

) -> tuple[DynamicState, dict[str, Any]]:

text

"""Simulate prefix up to split_time.

Exact reproduction of production simulate_prefix().

Returns (state, prefix_dict).

Production uses:

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence) # for initialization

Then in the loop, uses:

unused_rng = default_rng(0) # for advance_state when shuffle_driver=False

"""

N = config.N

edges = compute_edges(N)

Initialize state (uses init_sequence via make_prefix_rng)

state = initialize_state(seed, config)

```
# Get RNG for prefix simulation loop

# Production uses unused_rng = default_rng(0) for the advance_state calls

# when shuffle_driver=False

unused_rng = np.random.default_rng(0)



# Simulate to split point

R_past_sum = np.zeros((N, N), dtype=float)

R_past_count = 0



burn_end = protocol.burn_end(config)

hist_end = protocol.hist_end(config)



for step in range(hist_end):

R_physical, *,* = advance_state(state, config, unused_rng, shuffle_driver=False)



if burn_end &lt;= step &lt; hist_end:
```

```
R_past_sum += R_physical

R_past_count += 1



prefix = {

"B_at_split": state.B.copy(),

"R_physical_past": R_past_sum / R_past_count if R_past_count &gt; 0 else np.zeros((N, N)),

"theta_at_split": state.theta.copy(),

"coherence_acc_at_split": state.coherence_acc.copy(),

"history_at_split": np.stack(state.history),

"selected_at_split": (state.B &gt; config.B_c).astype(np.uint8),

}



return state, prefix

def apply_intervention(

text

state: DynamicState,

branch: str,
```

```
config: ModelConfig,

rng: np.random.Generator,

) -> None:

text

"""Apply intervention to state. Mutates in place.



Exact reproduction of production apply_intervention().

"""

N = state.N



if branch in {"reset_ema", "reset_B_ema", "B_carrier_only", "full_reset"}:

# Zero coherence accumulator

state.coherence_acc.fill(0.0)



if branch in {"reset_B", "reset_B_ema", "full_reset"}:

# Zero B

state.B.fill(0.0)
```

```
if branch in {"B_carrier_only", "full_reset"}:

# Randomize theta and history

new_theta = rng.random(N) *2.0* np.pi

state.theta = new_theta

state.history = [new_theta.copy() for *in range(config.delay*steps + 1)]



# Ensure symmetry

state.B = 0.5 * (state.B + state.B.T)

np.fill_diagonal(state.B, 0.0)

def simulate_future(

text

split_state: DynamicState,

prefix: dict[str, Any],

branch: str,

config: ModelConfig,

protocol: ProtocolConfig,
```

rng: np.random.Generator,

) -> dict[str, Any]:

text

"""Simulate future trajectory from split state.

Reproduces production simulate_future() with ALL required outputs.

CRITICAL: Now accumulates BOTH horizon systems:

Fixed Physical Horizons (time-based, NOT T_window dependent):

- immediate: 0 to 17.5 time units

- recovery: 17.5 to 87.5 time units

- late: 87.5 to 320 time units

EMA-Normalized Horizons (T_window dependent):

- immediate: 0 to 1*T_window

- recovery: 1*T_window to 5*T_window

- late: 5*T_window to end

At rho=1 (T_window=17.502917...), the fixed physical and normalized horizons

are approximately aligned but use exact step boundaries.

Production accumulates:

- P_full, P_immediate, P_recovery, P_late (incidence matrices)

- B_mean_full, B_mean_immediate, B_mean_recovery, B_mean_late

- R_physical_future, R_driver_future

- Projection diagnostics via ProjectionTotals

- Various scalar metrics

Uses horizon system from production for normalized:

- WINDOW_STEPS = round(T_window / dt)

- RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS)

For fixed physical:

- FIXED_IMMEDIATE_END = round(17.5 / dt)

- FIXED_RECOVERY_END = round(87.5 / dt)

"""

state = split_state.clone()

apply_intervention(state, branch, config, rng)

N = state.N

edges = compute_edges(N)

future_steps = protocol.future_steps(config)

Compute horizon step boundaries

Normalized EMA horizons (T_window dependent)

window_steps = int(round(config.T_window / config.dt))

recovery_end_ema = min(5 * window_steps, future_steps)

Fixed physical horizons (NOT T_window dependent)

```
# These use the production's physical time boundaries: 17.5, 87.5, 320

fixed_immediate_end = int(round(17.5 / config.dt)) # 17.5 time units

fixed_recovery_end = int(round(87.5 / config.dt)) # 87.5 time units

# Late goes from 87.5 to end (320 time units)



# Clamp all boundaries to future_steps

recovery_end_ema = min(recovery_end_ema, future_steps)

fixed_immediate_end = min(fixed_immediate_end, future_steps)

fixed_recovery_end = min(fixed_recovery_end, future_steps)



# Initialize accumulators for BOTH horizon systems

# P matrices (incidence: B &gt; B_c as 0/1)

P_sums_ema = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),
```

```
}

P_sums_fixed = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}



# B matrices (mean B values)

B_sums_ema = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}

B_sums_fixed = {

"full": np.zeros((N, N), dtype=float),
```

```
"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}



# Counts for each horizon in both systems

counts_ema = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}

counts_fixed = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}



# R matrices

R_physical_sum = np.zeros((N, N), dtype=float)

R_driver_sum = np.zeros((N, N), dtype=float)



# Projection accumulator (matching production)

projection = ProjectionTotals()



# Trajectory accumulators
```

```
sample_times = []

density_trajectory = []

similarity_trajectory = []



# For retention/emergence calculations

r_sum = 0.0

r_count = 0

r_late_ema_sum = 0.0

r_late_ema_count = 0

r_late_fixed_sum = 0.0

r_late_fixed_count = 0



B_reference = prefix["B_at_split"]

shuffle_driver = (branch == "future_shuffle")



for step in range(future_steps):

R_physical, R_driver, info = advance_state(
```

state, config, rng, shuffle_driver=shuffle_driver

)

Update projection accumulator

projection.update(info)

Accumulate R matrices

R_physical_sum += R_physical

R_driver_sum += R_driver

Determine which horizons this step belongs to for BOTH systems

selected = (state.B > config.B_c).astype(float)

=== EMA-Normalized Horizons ===

Full: all steps

P_sums_ema["full"] += selected

B_sums_ema["full"] += state.B

counts_ema["full"] += 1

immediate: 0 to window_steps (exclusive)

if step < window_steps:

P_sums_ema["immediate"] += selected

B_sums_ema["immediate"] += state.B

counts_ema["immediate"] += 1

recovery: window_steps to recovery_end_ema (exclusive)

if window_steps <= step < recovery_end_ema:

P_sums_ema["recovery"] += selected

B_sums_ema["recovery"] += state.B

counts_ema["recovery"] += 1

late: recovery_end_ema to end

if recovery_end_ema <= step:

P_sums_ema["late"] += selected

B_sums_ema["late"] += state.B

counts_ema["late"] += 1

=== Fixed Physical Horizons ===

Full: all steps (same as EMA)

P_sums_fixed["full"] += selected

B_sums_fixed["full"] += state.B

counts_fixed["full"] += 1

immediate: 0 to fixed_immediate_end (exclusive)

if step < fixed_immediate_end:

P_sums_fixed["immediate"] += selected

B_sums_fixed["immediate"] += state.B

counts_fixed["immediate"] += 1

recovery: fixed_immediate_end to fixed_recovery_end (exclusive)

if fixed_immediate_end <= step < fixed_recovery_end:

P_sums_fixed["recovery"] += selected

B_sums_fixed["recovery"] += state.B

counts_fixed["recovery"] += 1

late: fixed_recovery_end to end

if fixed_recovery_end <= step:

P_sums_fixed["late"] += selected

B_sums_fixed["late"] += state.B

counts_fixed["late"] += 1

Trajectory sampling - CRITICAL: r_mean_late is ONLY accumulated here

if step % protocol.diagnostic_stride == 0:

t = (step + 1) * config.dt

sample_times.append(t)

Density: fraction of edges above B_c

density = float(np.count_nonzero(state.B[edges] > config.B_c) / len(edges[0]))

density_trajectory.append(density)

Similarity: cosine between current and reference B

B_vec = state.B[edges]

B_ref_vec = B_reference[edges]

norm_B = np.linalg.norm(B_vec)

norm_ref = np.linalg.norm(B_ref_vec)

if norm_B > 0 and norm_ref > 0:

similarity = float(np.dot(B_vec, B_ref_vec) / (norm_B * norm_ref))

else:

similarity = float('nan')

similarity_trajectory.append(similarity)

Track overall r

r_value = float(np.abs(np.mean(np.exp(1j * state.theta))))

r_sum += r_value

r_count += 1

Track late-phase r for BOTH horizon systems

if recovery_end_ema <= step:

r_late_ema_sum += r_value

r_late_ema_count += 1

if fixed_recovery_end <= step:

r_late_fixed_sum += r_value

r_late_fixed_count += 1

Compute mean matrices for BOTH horizon systems

P_ema = {name: P_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

P_fixed = {name: P_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_ema = {name: B_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_fixed = {name: B_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

Compute R means

R_physical_mean = R_physical_sum / future_steps if future_steps > 0 else np.zeros((N, N))

R_driver_mean = R_driver_sum / future_steps if future_steps > 0 else np.zeros((N, N))

```
# Final state

B_final = state.B.copy()

final_weights = state.B[edges]

final_rows = np.sum(state.B, axis=1)



# Retention and emergence for BOTH horizon systems

split_selected = prefix["selected_at_split"][edges].astype(bool)



# EMA-normalized

p_late_edges_ema = P_ema["late"][edges]

retention_ema = (

float(np.mean(p_late_edges_ema[split_selected]))

if np.any(split_selected)

else float('nan')

)

emergence_ema = (
```

```
float(np.mean(p_late_edges_ema[~split_selected]))

if np.any(~split_selected)

else float('nan')

)



# Fixed physical

p_late_edges_fixed = P_fixed[''late''][edges]

retention_fixed = (

float(np.mean(p_late_edges_fixed[split_selected]))

if np.any(split_selected)

else float('nan')

)

emergence_fixed = (

float(np.mean(p_late_edges_fixed[~split_selected]))

if np.any(~split_selected)

else float('nan')

)
```

```
# Minimum similarity from trajectory

similarity_array = np.array(similarity_trajectory, dtype=float)

finite_similarity = similarity_array[np.isfinite(similarity_array)]

minimum_similarity = (

float(np.min(finite_similarity)) if len(finite_similarity) &gt; 0

else float('nan')

)



# Build result with BOTH horizon systems

result = {

# === Matrix outputs for EMA-normalized horizons (backward compatible) ===

"B_final": B_final,

"B_mean_late": B_mean_ema["late"],

"P_full": P_ema["full"],

"P_immediate": P_ema["immediate"],

"P_recovery": P_ema["recovery"],
```

"P_late": P_ema["late"],

"R_physical_future": R_physical_mean,

"R_driver_future": R_driver_mean,

=== Matrix outputs for FIXED-PHYSICAL horizons (NEW) ===

"B_mean_late_fixed_physical": B_mean_fixed["late"],

"P_full_fixed_physical": P_fixed["full"],

"P_immediate_fixed_physical": P_fixed["immediate"],

"P_recovery_fixed_physical": P_fixed["recovery"],

"P_late_fixed_physical": P_fixed["late"],

=== Matrix outputs for NORMALIZED-EMA horizons (NEW - explicit) ===

"B_mean_late_normalized_EMA": B_mean_ema["late"],

"P_full_normalized_EMA": P_ema["full"],

"P_immediate_normalized_EMA": P_ema["immediate"],

"P_recovery_normalized_EMA": P_ema["recovery"],

"P_late_normalized_EMA": P_ema["late"],

Trajectory outputs

"sample_times": np.array(sample_times, dtype=float),

"density_trajectory": np.array(density_trajectory, dtype=float),

"similarity_trajectory": np.array(similarity_trajectory, dtype=float),

Scalar outcomes - EMA-normalized (backward compatible names)

"D_B_final": float(np.count_nonzero(final_weights > config.B_c) / len(edges[0])),

"P_mean_full": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late": float(np.mean(P_ema["late"][edges])),

Scalar outcomes - FIXED-PHYSICAL (NEW - primary estimands)

"P_mean_full_fixed_physical": float(np.mean(P_fixed["full"][edges])),

"P_mean_immediate_fixed_physical": float(np.mean(P_fixed["immediate"][edges])),

"P_mean_recovery_fixed_physical": float(np.mean(P_fixed["recovery"][edges])),

"P_mean_late_fixed_physical": float(np.mean(P_fixed["late"][edges])),

Scalar outcomes - NORMALIZED-EMA (NEW - explicit)

"P_mean_full_normalized_EMA": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate_normalized_EMA": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery_normalized_EMA": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late_normalized_EMA": float(np.mean(P_ema["late"][edges])),

Similarity metrics - BOTH systems

"Weighted_similarity_final": cosine_edges(B_reference, B_final, edges),

"Weighted_similarity_late": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Weighted_similarity_late_fixed_physical": cosine_edges(B_reference, B_mean_fixed["late"], edges),

"Weighted_similarity_late_normalized_EMA": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Jaccard_final": jaccard_edges(B_reference, B_final, edges, config.B_c),

"Jaccard_final_fixed_physical": jaccard_edges(B_reference, B_final, edges, config.B_c),

Retention/emergence - BOTH systems

"Retention_selected_late": retention_ema,

"Emergence_unselected_late": emergence_ema,

"Retention_contrast_late": retention_ema - emergence_ema,

"Retention_selected_late_fixed_physical": retention_fixed,

"Emergence_unselected_late_fixed_physical": emergence_fixed,

"Retention_contrast_late_fixed_physical": retention_fixed - emergence_fixed,

"Retention_selected_late_normalized_EMA": retention_ema,

"Emergence_unselected_late_normalized_EMA": emergence_ema,

"Retention_contrast_late_normalized_EMA": retention_ema - emergence_ema,

Correlations with past R - BOTH systems

"Corr_Rpast_Pfull": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

"Corr_Rpast_Pfull_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["full"][edges]),

"Corr_Rpast_Plate_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["late"][edges]),

"Corr_Rpast_Pfull_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

Weight statistics (same for both - based on final state)

"Total_edge_weight": float(np.sum(final_weights)),

"Mean_row_sum": float(np.mean(final_rows)),

"Max_row_sum": float(np.max(final_rows)),

"Nonzero_edge_fraction": float(np.mean(final_weights > config.scale_epsilon)),

"Effective_edges": effective_number(final_weights),

"Weight_gini": gini_coefficient(final_weights),

"Weight_entropy": normalized_entropy(final_weights),

Order parameter - BOTH systems

"r_mean_future": float(r_sum / r_count) if r_count > 0 else float('nan'),

"r_mean_late": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

"r_mean_late_fixed_physical": float(r_late_fixed_sum / r_late_fixed_count) if r_late_fixed_count > 0 else float('nan'),

"r_mean_late_normalized_EMA": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

Minimum similarity

"Minimum_similarity": minimum_similarity,

Projection diagnostics (from ProjectionTotals)

```
**projection.metrics(),

# Horizon metadata
"window_steps": window_steps,
"recovery_end_ema": recovery_end_ema,
"fixed_immediate_end": fixed_immediate_end,
"fixed_recovery_end": fixed_recovery_end,

# Time-based metrics (placeholders - will be NaN for now)
"Time_to_50pct_continue_final_density": float('nan'),
"Time_to_90pct_continue_final_density": float('nan'),
"Time_to_50pct_continue_late_similarity": float('nan'),
"Time_to_90pct_continue_late_similarity": float('nan'),
"Time_to_similarity_below_0_9": float('nan'),
"Time_to_similarity_below_0_5": float('nan'),
"Late_persistence_ratio_to_continue": float('nan'),
"Late_similarity_ratio_to_continue": float('nan'),
}
```

return result

==========
Metric Helpers
==========

def cosine_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None) -> float:

text

"""Compute cosine similarity between reference and current B matrices over edges.

If edges is None, computes over all upper triangle edges.

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = np.asarray(reference[edges], dtype=float)

y = np.asarray(current[edges], dtype=float)

denominator = float(np.linalg.norm(x) * np.linalg.norm(y))

if denominator <= 1e-12:

return float('nan')

return float(np.dot(x, y) / denominator)

def jaccard_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None, threshold: float = FROZEN_ZERO_TOL) -> float:

text

"""Compute Jaccard similarity between reference and current B matrices over edges.

Uses B > threshold as the threshold for edge presence.

If edges is None, computes over all upper triangle edges.

Matches production: jaccard_edges(B_reference, state.B) uses B > B_C

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

```
x = reference[edges] &gt; threshold

y = current[edges] &gt; threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x &amp;amp; y) / union)

def safe_corr(x: np.ndarray, y: np.ndarray) -> float:

text

"""Compute correlation coefficient safely."""

x = np.asarray(x, dtype=float)

y = np.asarray(y, dtype=float)

valid = np.isfinite(x) &amp;amp; np.isfinite(y)

x = x[valid]

y = y[valid]

if len(x) &lt; 3 or np.std(x) &lt; 1e-12 or np.std(y) &lt; 1e-12:

return float('nan')

return float(np.corrcoef(x, y)[0, 1])
```

```
def effective_number(weights: np.ndarray) -> float:

text

"""Compute effective number of edges."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

sum_sq = float(np.sum(weights**2))

if total &lt;= 1e-12 or sum_sq &lt;= 1e-12:

return 0.0

return total**2 / sum_sq

def gini_coefficient(weights: np.ndarray) -> float:

text

"""Compute Gini coefficient."""

x = np.sort(np.asarray(weights, dtype=float))

total = float(np.sum(x))

if total &lt;= 1e-12:

return 0.0

n = len(x)

ranks = np.arange(1, n + 1, dtype=float)
```

return float(2.0 *np.dot(ranks, x) / (n* total) - (n + 1.0) / n)

def normalized_entropy(weights: np.ndarray) -> float:

text

"""Compute normalized entropy."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

if total <= 1e-12:

return 0.0

probabilities = weights / total

positive = probabilities > 0.0

entropy = -float(np.sum(probabilities[positive] * np.log(probabilities[positive])))

return entropy / float(np.log(len(weights)))

==========
====
Required Output Schema for Baseline Equivalence
==========
====
These are the keys that MUST be present in both engine output and frozen data
for baseline equivalence to pass

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)

REQUIRED_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

Fixed physical horizon matrices (NEW - primary estimands)

"B_mean_late_fixed_physical",

"P_full_fixed_physical",

"P_immediate_fixed_physical",

"P_recovery_fixed_physical",

"P_late_fixed_physical",

Normalized EMA horizon matrices (NEW - explicit)

"B_mean_late_normalized_EMA",

"P_full_normalized_EMA",

"P_immediate_normalized_EMA",

"P_recovery_normalized_EMA",

"P_late_normalized_EMA",

)

REQUIRED_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Fixed physical horizon scalars (NEW - PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (NEW - explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

Projection diagnostics (from ProjectionTotals)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

)
==========
====
Baseline Equivalence Test
==========
====

def exact_array_equal_with_nan(a: np.ndarray, b: np.ndarray) -> bool:

text

```
"""Compare arrays with NaN-aware exact equality."""

a = np.asarray(a)

b = np.asarray(b)

if a.shape != b.shape or a.dtype != b.dtype:

return False

if np.issubdtype(a.dtype, np.floating) or np.issubdtype(a.dtype, np.complexfloating):

a_nan = np.isnan(a)

b_nan = np.isnan(b)

if not np.array_equal(a_nan, b_nan):

return False

finite = ~a_nan

return np.array_equal(a[finite], b[finite])

return np.array_equal(a, b)

def run_baseline_equivalence(

text

seeds: list[int],

config: ModelConfig,

protocol: ProtocolConfig,
```

```
frozen_manifest_path: str,
frozen_npz_path: str,
branch_order: tuple[str, ...],
frozen_csv_path: str | None = None,
) -> dict[str, Any]:
text
"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for all seeds.

This function validates that the engine produces EXACTLY the same outputs
as the frozen production run for all required keys.

Requires NaN-aware exact equality for ALL required schema keys:
- All prefix keys
- All branch matrix keys
- All branch scalar keys
```

Returns:

status: "passed" | "failed" | "not_run"

seed_results: per-seed validation details

engine_source_sha256: hash of this engine source

frozen_manifest_hash: hash of frozen manifest

frozen_npz_hash: hash of frozen NPZ

"""

import numpy as np

from pathlib import Path

import hashlib

Helper function to compute source hash

def *source*sha256() -> str | None:

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None

Record engine source hash

engine_source_hash = *source*sha256()

results = {

"status": "not_run",

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"mean_omega": config.mean_omega,

"T_window": config.T_window,

"T_star": config.T_star,

"alpha": config.alpha,

"a": config.a,

"q": config.q,

"delta": config.delta,

"eta": config.eta,

"mu": config.mu,

"gamma": config.gamma,

"B_c": config.B_c,

"B_init_scale": config.B_init_scale,

"scale_epsilon": config.scale_epsilon,

"capacity": config.capacity,

"delay_steps": config.delay_steps,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

"diagnostic_stride": protocol.diagnostic_stride,

"branches": list(protocol.branches),

```
},
"n_seeds": len(seeds),
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"engine_source_sha256": engine_source_hash,
"seed_results": {},
}


# Load frozen manifest
with open(frozen_manifest_path, "r") as f:
frozen_manifest = json.load(f)


# Load frozen data
with np.load(frozen_npz_path, allow_pickle=False) as frozen_data:
frozen_files = dict(frozen_data)


# Load frozen scalar CSV if provided
```

```
frozen_scalar_csv: dict[str, dict[str, Any]] = {}

if frozen_csv_path is not None and os.path.exists(frozen_csv_path):

try:

import csv

with open(frozen_csv_path, "r", newline="", encoding="utf-8") as f:

reader = csv.DictReader(f)

for row in reader:

seed_id = row.get("seed", "")

branch = row.get("branch", "")

if seed_id and branch:

key = f"{branch}_seed_{seed_id}"

if key not in frozen_scalar_csv:

frozen_scalar_csv[key] = {}

for k, v in row.items():

if k not in ("seed", "seed_index", "branch", "rho", "T_window"):

try:

frozen_scalar_csv[key][k] = float(v)

except (ValueError, TypeError):
```

```
frozen_scalar_csv[key][k] = v

except Exception as e:

print(f"Warning: Could not load frozen CSV {frozen_csv_path}: {e}")



# Record frozen hashes

results["frozen_manifest_hash"] = hashlib.sha256(

json.dumps(frozen_manifest, sort_keys=True).encode()

).hexdigest()

results["frozen_npz_hash"] = file_sha256(frozen_npz_path)

if frozen_csv_path is not None and os.path.exists(frozen_csv_path):

results["frozen_csv_hash"] = file_sha256(frozen_csv_path)



# Validate config matches frozen manifest model - use EXACT equality

frozen_model = frozen_manifest["model"]



# Check N

if frozen_model["N"] != config.N:
```

```
results["status"] = "failed"

results["error"] = f"N mismatch: config={config.N}, frozen={frozen_model['N']}"

return results



# Check omega - use exact equality

frozen_omega_arr = np.array(frozen_model["omega"], dtype=float)

config_omega_arr = np.array(list([config.omega](http://config.omega)), dtype=float)

if not np.array_equal(frozen_omega_arr, config_omega_arr):

results["status"] = "failed"

results["error"] = f"omega mismatch: frozen={frozen_model['omega']}, config={list([config.omega]
(http://config.omega))}"

return results



# Check all other model params with EXACT equality (rtol=0, atol=0)

model_checks = [

("dt", frozen_model["dt"], config.dt),

("tau", frozen_model["tau"], config.tau),
```

("a", frozen_model["a"], config.a),

("q", frozen_model["q"], config.q),

("delta", frozen_model["delta"], config.delta),

("eta", frozen_model["eta"], config.eta),

("mu", frozen_model["mu"], config.mu),

("gamma", frozen_model["gamma"], config.gamma),

("B_c", frozen_model["B_c"], config.B_c),

("B_init_scale", frozen_model.get("B_init_scale", 0.0), config.B_init_scale),

("T_window", frozen_model["T_window"], config.T_window),

]

for name, frozen_val, config_val in model_checks:

if not np.isclose(frozen_val, config_val, rtol=0.0, atol=0.0):

results["status"] = "failed"

results["error"] = f"{name} mismatch: frozen={frozen_val}, config={config_val}"

return results

```
# Check derived values

if not np.isclose(frozen_model.get("alpha", 0.0), config.alpha, rtol=0.0, atol=1e-15):

results["status"] = "failed"

results["error"] = f"alpha mismatch: frozen={frozen_model.get('alpha')}, config={config.alpha}"

return results



# Now run actual equivalence test for each seed

print("Running baseline equivalence test...")

all_passed = True



for seed in seeds:

seed_str = str(seed)

seed_results = {"status": "passed", "checks": {}, "max_diffs": {}}



try:

# Simulate with our engine

split_state, prefix = simulate_prefix(seed, config, protocol)

branch_rngs = make_branch_rngs(seed, branch_order)
```

```
# Simulate each branch

engine_results = {}

for branch in protocol.branches:

rng = branch_rngs[branch]

engine_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)


# Check prefix state - iterate over REQUIRED_PREFIX_KEYS

for key in REQUIRED_PREFIX_KEYS:

engine_val = prefix.get(key)

frozen_key = f"prefix_seed_{seed_str}_{key}"
```

```
frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"prefix_{key}_missing"] = True

all_passed = False

continue


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"prefix_{key}_exact"] = True

else:

seed_results["checks"][f"prefix_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"prefix_{key}"] = float(np.max(np.abs(e - f)))
```

```
# Check branch outputs - iterate over REQUIRED keys

for branch in protocol.branches:

engine_branch = engine_results[branch]

frozen_prefix = f"{branch}_seed_{seed_str}"


# Check matrix keys

for key in REQUIRED_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing"] = True

all_passed = False

continue


if engine_val is None:
```

```
seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"{branch}_{key}"] = float(np.max(np.abs(e - f)))


# Check scalar keys

for key in REQUIRED_BRANCH_SCALAR_KEYS:
```

```
engine_val = engine_branch.get(key)

frozen_key = f"{frozen_prefix}_{key}"


# Try to get from CSV first (production stores scalars in CSV)

frozen_val = None

csv_key = f"{branch}_seed_{seed_str}"

if csv_key in frozen_scalar_csv and key in frozen_scalar_csv[csv_key]:

frozen_val = frozen_scalar_csv[csv_key][key]

else:

# Fallback to NPZ (for backwards compatibility)

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

# This scalar might not be in frozen data - that's OK for now

# but we should track it

seed_results["checks"][f"{branch}_{key}_missing_in_frozen"] = True

continue
```

```
if engine_val is None:
seed_results["status"] = "failed"
seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True
all_passed = False
continue

# For scalars, compare directly
if isinstance(engine_val, (int, float, np.number)):
# Handle NaN: both NaN means equal
engine_is_nan = np.isnan(engine_val) if isinstance(engine_val, (int, float, np.number)) else False
frozen_is_nan = np.isnan(frozen_val) if isinstance(frozen_val, (int, float, np.number)) else False

if engine_is_nan and frozen_is_nan:
seed_results["checks"][f"{branch}_{key}_exact"] = True
elif np.isclose(engine_val, frozen_val, rtol=0.0, atol=0.0):
seed_results["checks"][f"{branch}_{key}_exact"] = True
else:
seed_results["checks"][f"{branch}_{key}_exact"] = False
```

```
seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}"] = float(abs(engine_val - frozen_val)) if not
(engine_is_nan or frozen_is_nan) else float('nan')



except Exception as e:

seed_results["status"] = "failed"

seed_results["error"] = str(e)

import traceback

seed_results["traceback"] = traceback.format_exc()

all_passed = False



results["seed_results"][seed_str] = seed_results

print(f" Seed {seed_str}: {seed_results['status']}")



if all_passed:

results["status"] = "passed"

print("Baseline equivalence: PASSED")

else:
```

```
results["status"] = "failed"

print("Baseline equivalence: FAILED")

# Print details of failures

for seed_str, seed_results in results["seed_results"].items():

if seed_results.get("status") != "passed":

print(f" Seed {seed_str} failures:")

for check, val in seed_results.get("checks", {}).items():

if not val:

print(f" CHECK FAILED: {check}")

for key, val in seed_results.get("max_diffs", {}).items():

print(f" MAX DIFF: {key} = {val}")

# Save equivalence certificate even on failure

raise RuntimeError("Baseline equivalence test FAILED. Study stopped.")



return results

================================================================================
Main API
```

```
================================================================================
====

def run_seed(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

branch_order: tuple[str, ...],

) -> dict[str, Any]:

text

"""Run a single seed through prefix + all branches.



Returns complete seed results with all branches.

"""

# Simulate prefix

split_state, prefix = simulate_prefix(seed, config, protocol)



# Create branch RNGs using frozen algorithm
```

```
branch_rngs = make_branch_rngs(seed, branch_order)



# Use branch_order to determine which branches to run

# All branches in branch_order must have RNGs

branch_results = {}

for branch in branch_order:

if branch not in branch_rngs:

raise ValueError(

f"Branch '{branch}' not found in branch_rngs. "

f"All branches must be registered in branch_order for explicit RNG namespace. "

f"Registered branches: {list(branch_rngs.keys())}"

)

rng = branch_rngs[branch]


branch_results[branch] = simulate_future(

split_state.clone(),

prefix,
```

```
branch,

config,

protocol,

rng,

)



return {

"seed": int(seed),

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},
```

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

if name == "main":

text

Quick validation

config = DEFAULT_MODEL_CONFIG

config.validate()

protocol = DEFAULT_PROTOCOL_CONFIG

protocol.validate(config)

print(f"Engine v1 loaded successfully")

print(f" N={config.N}, dt={config.dt}, tau={config.tau}")

print(f" T_window={config.T_window}, alpha={config.alpha:.6f}")

print(f" mean_omega={config.mean_omega:.6f}, T_*={config.T_star:.6f}")

print(f" delay_steps={config.delay_steps}")

print(f" Protocol: T_sim={protocol.T_sim}, burn={protocol.burn_end_time}, "

f"split={protocol.split_time}, future={protocol.future_duration}")

This engine is now close to a valid parameterized production kernel. The dual-horizon accumulation and projection diagnostics are strong additions, and the core step/order/RNG logic appears faithful.

However, the current baseline equivalence test is guaranteed to fail because it requires newly introduced dual-horizon arrays to exist in the old frozen NPZ. The frozen run cannot contain fields that did not exist when it was generated.
1. Baseline equivalence currently requires impossible frozen keys

REQUIRED_BRANCH_MATRIX_KEYS now includes:

text

B_mean_late_fixed_physical
P_full_fixed_physical
P_immediate_fixed_physical
P_recovery_fixed_physical
P_late_fixed_physical

B_mean_late_normalized_EMA
P_full_normalized_EMA
P_immediate_normalized_EMA
P_recovery_normalized_EMA
P_late_normalized_EMA

These keys are new. They do not exist in the frozen AIEF-1.4 NPZ.

The equivalence code treats missing matrix keys as fatal:

Python

```
if frozen_val is None:
status = failed
```

Therefore every seed will fail baseline validation even if the legacy dynamics reproduce perfectly.

Separate the schemas.

Python

```
FROZEN_BRANCH_MATRIX_KEYS = (
"B_final",
"B_mean_late",
"P_full",
"P_immediate",
"P_recovery",
"P_late",
"R_physical_future",
"R_driver_future",
"sample_times",
"density_trajectory",
"similarity_trajectory",
)

EXTENDED_BRANCH_MATRIX_KEYS = (
"B_mean_late_fixed_physical",
"P_full_fixed_physical",
"P_immediate_fixed_physical",
"P_recovery_fixed_physical",
"P_late_fixed_physical",
"B_mean_late_normalized_EMA",
"P_full_normalized_EMA",
```

"P_immediate_normalized_EMA",
"P_recovery_normalized_EMA",
"P_late_normalized_EMA",
)

Compare only FROZEN_BRANCH_MATRIX_KEYS against the historical NPZ.
2. Validate new dual-horizon fields internally at ρ=1ρ=1

At the baseline:
round(T∗/dt)=1750,
round(T∗/dt)=1750,
5×1750=8750,
5×1750=8750,

which exactly match the fixed physical step boundaries.

Therefore, for every branch and seed, require internally:

Python

```
np.array_equal(
result["P_late_fixed_physical"],
result["P_late"],
)

np.array_equal(
result["P_late_normalized_EMA"],
result["P_late"],
)

np.array_equal(
result["B_mean_late_fixed_physical"],
result["B_mean_late"],
)

np.array_equal(
result["B_mean_late_normalized_EMA"],
result["B_mean_late"],
```

)

and equivalent checks for immediate/recovery/full matrices and scalar summaries.

This gives the new fields an exact validation gate without demanding impossible historical keys.
3. Separate frozen and extended scalar schemas

The same issue occurs for new scalar fields. Missing scalar keys currently do not fail, so this is less visible, but the logic is ambiguous.

Define:

Python

```
FROZEN_BRANCH_SCALAR_KEYS = (
# Exact historical production schema
)
```

and:

Python

```
EXTENDED_BRANCH_SCALAR_KEYS = (
"P_mean_late_fixed_physical",
"Weighted_similarity_late_fixed_physical",
"Retention_contrast_late_fixed_physical",
"Corr_Rpast_Plate_fixed_physical",
"r_mean_late_fixed_physical",
"P_mean_late_normalized_EMA",
"Weighted_similarity_late_normalized_EMA",
"Retention_contrast_late_normalized_EMA",
"Corr_Rpast_Plate_normalized_EMA",
"r_mean_late_normalized_EMA",
)
```

Historical scalar keys should be mandatory when the frozen scalar CSV is supplied. Extended keys should be checked against the legacy values internally at ρ=1ρ=1.
4. The driver must supply the frozen scalar CSV

run_baseline_equivalence can now load:

Python

frozen_csv_path

but the previously shown T-window driver passes only:

text

frozen manifest
frozen NPZ

Thus scalar equivalence is still not tested. Every historical scalar is marked missing and skipped.

Add:

text

--frozen-scalar

to the driver and pass it to:

Python

engine.run_baseline_equivalence(
...,
frozen_csv_path=args.frozen_scalar,
)

If the engine claims scalar equivalence, this input must be mandatory.
5. Missing historical scalar keys should be fatal

For fields in FROZEN_BRANCH_SCALAR_KEYS, this:

Python

if frozen_val is None:
mark missing
continue

should fail the seed.

Only new extended keys may be absent historically.
6. The dual-horizon implementation is conceptually correct

The engine now separately accumulates:

fixed physical windows;
normalized EMA windows.

It no longer takes the minimum of their boundaries.

The fixed-primary fields are explicit, and the old production names remain backward-compatible normalized-EMA outputs.

This is the correct architecture for the sensitivity study.
Baseline equality of horizons

At ρ=1ρ=1, both systems use:

text

immediate end = 1750
recovery end = 8750

so exact equality should hold.

At other rho values, they correctly diverge.
7. Projection diagnostics now match production structure

ProjectionTotals and the expanded advance_state information are substantially aligned with production:

proposal violation;
correction norm;
relative state/update correction;
minimum gap values;
active projection counts.

These fields should now be included in historical scalar comparison using the frozen scalar CSV.
8. run_seed still conflates RNG namespace and branches to run

The generic API remains:

Python

for branch in branch_order:

so it simulates every branch used to establish the RNG namespace.

The study driver correctly bypasses this function and performs its own subset loop, but the engine API itself remains misleading.

Change to:

Python

def run_seed(
seed,
config,
protocol,
rng_branch_order,
):
branch_rngs = make_branch_rngs(
seed,
rng_branch_order,
)

for branch in protocol.branches:

...

This makes the reusable API safe.
9. Equivalence certificate still needs explicit persistence

The function computes detailed per-seed checks but raises on failure and does not save them itself.

The driver should save the result—successful or failed—to:

text

baseline_equivalence_by_seed.csv
baseline_equivalence_manifest.json

before deciding whether to proceed.

For a failure, do not lose the detailed check table.
10. Manifest file hash remains mislabeled

This:

Python

sha256(
json.dumps(
frozen_manifest,
sort_keys=True,
)
)

is a canonical JSON hash, not the file SHA-256.

Use:

Python

file_sha256(
frozen_manifest_path
)

and optionally save the canonical hash under a separate name.
11. Metric details
Jaccard default

The helper default is FROZEN_ZERO_TOL, while the production topology Jaccard uses BcBc. Calls currently pass config.B_c, so results are correct. Remove the misleading default or require threshold explicitly.
Fixed-physical Jaccard

Jaccard_final_fixed_physical is identical to Jaccard_final because final state is horizon-independent. It is redundant and may be removed.
Full horizon duplication

P_full_fixed_physical and P_full_normalized_EMA are necessarily identical because both include every future step. Keeping both is harmless but redundant.
12. History and RNG fidelity now look strong

The corrected prefix RNG, branch RNG, initialization, history shift, intervention, and diagnostic sampling are now close to production.

The list-based history implementation should be accepted only after exact split-history and branch trajectory equivalence pass.
13. Remaining downstream synchronization

The T-window driver and analysis must update their required schemas to use:
Primary fixed-physical fields

text

P_mean_late_fixed_physical
Weighted_similarity_late_fixed_physical
Retention_contrast_late_fixed_physical
Corr_Rpast_Plate_fixed_physical
r_mean_late_fixed_physical

Secondary normalized fields

text

P_mean_late_normalized_EMA
Weighted_similarity_late_normalized_EMA
Retention_contrast_late_normalized_EMA
Corr_Rpast_Plate_normalized_EMA
r_mean_late_normalized_EMA

If they continue using generic legacy fields, they will analyze only normalized-EMA horizons.
Required next gate

Before any nonbaseline run:

split frozen versus extended schemas;
require the frozen scalar CSV;
compare frozen historical keys exactly;
validate extended fields internally at ρ=1ρ=1;
save the full equivalence certificate;
run one seed;
then all 40 seeds/all seven branches.

Rating
Area Rating
Core model fidelity 9/10
Prefix/branch RNG fidelity 9/10
Projection diagnostics 9/10
Dual-horizon implementation 9/10
Historical schema handling 4/10
Baseline certificate persistence 6/10
Reusable branch API 6/10
Study readiness after schema fix 8/10
Overall: 8/10 architecture, 6.5/10 current baseline execution

The engine is now scientifically well designed. The remaining blocker is schema logic: it must not require new dual-horizon fields to exist in an old frozen artifact. Once that is corrected, the actual equivalence test—not another static rewrite—should decide whether the engine is ready.

Evaluate this:

#!/usr/bin/env python3

"""AIEF Experiment Engine v1 - Parameterized simulation kernel.

This is a reusable, validated simulation engine that reproduces the frozen

AIEF-1.4 production run exactly and supports parameterized studies.

FROZEN PRODUCTION VALUES (from AIEF_1_4_memory_ablation_updated_REAL.py):

N = 12

DT = 0.01

TAU = 0.5

OMEGA = [1.00, 1.02, 1.50, 2.00, 2.01, 1.01, 1.03, 1.51, 2.02, 2.00, 1.00, 1.04]

A = 1.0

Q = 1.5

DELTA = 0.05

ETA = 0.04

GAMMA = 0.015

MU = 0.03

T_WINDOW = 25.0 / mean(OMEGA) = 17.5

B_C = 0.12

B_INIT_SCALE = 0.0005

PROTOCOL (from frozen run):

T_SIM = 800.0

N_STEPS = int(round(T_SIM / DT)) = 80000

BURN_END = int(0.20 * N_STEPS) = 16000 steps = 160.0 time units

HIST_END = int(0.60 * N_STEPS) = 48000 steps = 480.0 time units

FUTURE_STEPS = N_STEPS - HIST_END = 32000 steps = 320.0 time units

DELAY_STEPS = int(round(TAU / DT)) = 50

WINDOW_STEPS = int(round(T_WINDOW / DT)) = 1750

RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS) = 8750

Architecture:

ModelConfig: frozen dataclass for all model parameters

ProtocolConfig: frozen dataclass for protocol parameters

DynamicState: state container (immutable where possible)

Core functions: initialize, advance, simulate, interventions

Validation: baseline equivalence test

NOTE: This is a candidate reference implementation. After passing 40/40 baseline

text

equivalence tests, it will be frozen as the reference implementation.

"""

from future import annotations

import copy

import hashlib

import json

import math

import os

import tempfile

import time

from dataclasses import dataclass, field

from pathlib import Path

from typing import Any

import numpy as np
==
====
Frozen Production Constants (DO NOT CHANGE)
==
====

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005

FROZEN_ZERO_TOL = 1e-12

FROZEN_PROJECTION_NORM_TOL = 1e-14

FROZEN_GAP_TOL = 1e-9
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA)) # 1.428333...

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.502917152858814

FROZEN_T_WINDOW = FROZEN_T_STAR # 17.502917152858814 (exact, NOT rounded)

FROZEN_ALPHA = math.exp(-FROZEN_DT / FROZEN_T_WINDOW)

FROZEN_CAPACITY = (FROZEN_A - FROZEN_DELTA) / FROZEN_Q
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_WINDOW / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Fixed physical horizon boundaries (in time units, NOT T_window dependent)
These are the PRIMARY estimand horizons

FIXED_PHYSICAL_IMMEDIATE_END = 17.5

FIXED_PHYSICAL_RECOVERY_END = 87.5

FIXED_PHYSICAL_FUTURE_END = 320.0
Derived fixed physical horizon steps

FIXED_PHYSICAL_IMMEDIATE_STEPS = int(round(FIXED_PHYSICAL_IMMEDIATE_END / FROZEN_DT)) # 1750

FIXED_PHYSICAL_RECOVERY_STEPS = int(round(FIXED_PHYSICAL_RECOVERY_END / FROZEN_DT)) # 8750
==========
Model Configuration
==========

@dataclass(frozen=True)

class ModelConfig:

text

"""Immutable model configuration.

All model-changing quantities must come from here. No hidden module globals.

"""

Core parameters

N: int

dt: float

tau: float

omega: tuple[float, ...] # Immutable tuple

Coupling parameters

a: float

q: float

delta: float

Dynamics parameters

eta: float

mu: float

gamma: float

EMA/threshold parameters

T_window: float

B_c: float

B_init_scale: float

```
scale_epsilon: float = 1e-12



# Derived properties

@property

def mean_omega(self) -&gt; float:

"""Mean of omega frequencies."""

return float(np.mean([self.omega](http://self.omega)))



@property

def T_star(self) -&gt; float:

"""T_* = 25 / mean(omega)"""

return 25.0 / self.mean_omega



@property

def alpha(self) -&gt; float:

"""EMA smoothing factor: exp(-dt / T_window)"""

return math.exp(-self.dt / self.T_window)
```

@property

def capacity(self) -> float:

"""Projection capacity: (a - delta) / q"""

return (self.a - self.delta) / self.q

@property

def delay_steps(self) -> int:

"""Delay steps: round(tau / dt)"""

return int(round(self.tau / self.dt))

def validate(self) -> None:

"""Validate model configuration."""

if self.N <= 0:

raise ValueError("N must be positive")

if self.dt <= 0:

raise ValueError("dt must be positive")

if self.tau <= 0:

raise ValueError("tau must be positive")

if len(self.omega) != self.N:

raise ValueError(f"omega must have {self.N} elements")

if self.a <= 0 or self.q <= 0 or self.delta < 0:

raise ValueError("a, q must be positive; delta must be nonnegative")

if self.eta < 0 or self.mu < 0 or self.gamma < 0:

raise ValueError("eta, mu, gamma must be nonnegative")

if self.T_window <= 0:

raise ValueError("T_window must be positive")

if self.B_c < 0:

raise ValueError("B_c must be nonnegative")

if self.B_init_scale < 0:

raise ValueError("B_init_scale must be nonnegative")

if self.scale_epsilon <= 0:

raise ValueError("scale_epsilon must be positive")

Validate derived values

if not (0 < self.alpha <= 1):

raise ValueError(f"alpha={self.alpha} must be in (0, 1]")

if self.capacity < 0:

raise ValueError(f"capacity={self.capacity} must be nonnegative")

@dataclass(frozen=True)

class ProtocolConfig:

text

"""Protocol configuration for experiment execution."""

Simulation time parameters (in time units, not steps)

T_sim: float

burn_end_time: float

split_time: float

future_duration: float

Diagnostics

diagnostic_stride: int

```
# Branches to simulate

branches: tuple[str, ...]



def n_steps(self, model: ModelConfig) -&gt; int:

"""Total steps."""

return int(round(self.T_sim / model.dt))



def burn_end(self, model: ModelConfig) -&gt; int:

"""Burn-in end step."""

return int(round(self.burn_end_time / model.dt))



def hist_end(self, model: ModelConfig) -&gt; int:

"""History end step (split point)."""

return int(round(self.split_time / model.dt))



def future_steps(self, model: ModelConfig) -&gt; int:

"""Future simulation steps."""
```

```
return int(round(self.future_duration / model.dt))



def validate(self, model: ModelConfig) -&gt; None:

"""Validate protocol against model."""

if self.T_sim &lt;= 0:

raise ValueError("T_sim must be positive")

if not (0 &lt; self.burn_end_time &lt; self.split_time &lt; self.T_sim):

raise ValueError("Invalid time parameters: must have 0 &lt; burn_end &lt; split &lt; T_sim")

if self.future_duration &lt;= 0:

raise ValueError("future_duration must be positive")

if self.diagnostic_stride &lt;= 0:

raise ValueError("diagnostic_stride must be positive")

# Validate that all time parameters are compatible with dt

if not np.isclose(self.T_sim / model.dt, round(self.T_sim / model.dt), rtol=1e-10):

raise ValueError(f"T_sim ({self.T_sim}) must be divisible by dt ({model.dt})")

if not np.isclose(self.burn_end_time / model.dt, round(self.burn_end_time / model.dt), rtol=1e-10):

raise ValueError(f"burn_end_time ({self.burn_end_time}) must be divisible by dt ({model.dt})")
```

if not np.isclose(self.split_time / model.dt, round(self.split_time / model.dt), rtol=1e-10):

raise ValueError(f"split_time ({self.split_time}) must be divisible by dt ({model.dt})")

if not np.isclose(self.future_duration / model.dt, round(self.future_duration / model.dt), rtol=1e-10):

raise ValueError(f"future_duration ({self.future_duration}) must be divisible by dt ({model.dt})")

Default frozen production configuration
NOTE: T_window must be the EXACT value 25.0/mean(OMEGA), not rounded

DEFAULT_MODEL_CONFIG = ModelConfig(

text

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=FROZEN_T_WINDOW, # 17.502917152858814 (exact)

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=FROZEN_ZERO_TOL,

)
Default protocol matching frozen run

DEFAULT_PROTOCOL_CONFIG = ProtocolConfig(

text

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=("continue", "reset_ema", "reset_B", "reset_B_ema", "B_carrier_only", "full_reset", "future_shuffle"),

)
==========
Utility Functions
==========

def compute_edges(N: int) -> tuple[np.ndarray, np.ndarray]:

```
text

"""Get upper triangle indices for edges."""

return np.triu_indices(N, k=1)

def validate_B_matrix(B: np.ndarray, N: int, eps: float = 1e-12) -> None:

text

"""Validate B matrix: symmetric, nonnegative, zero diagonal."""

if B.shape != (N, N):

raise ValueError(f"B shape {B.shape} != ({N}, {N})")

if not np.allclose(B, B.T, atol=eps):

raise ValueError("B is not symmetric")

if not np.allclose(np.diag(B), 0, atol=eps):

raise ValueError("B diagonal is not zero")

if np.any(B &lt; -eps):

raise ValueError("B has negative entries")

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file."""
```

```
digest = hashlib.sha256()

with open(path, "rb") as handle:

for block in iter(lambda: [handle.read](http://handle.read)(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

def array_sha256(arr: np.ndarray) -> str:

text

"""Compute SHA-256 hash of a numpy array."""

digest = hashlib.sha256()

shape_bytes = np.asarray(arr.shape, dtype=np.int64).tobytes()

digest.update(shape_bytes)

digest.update(arr.dtype.str.encode())

contiguous = np.ascontiguousarray(arr)

if contiguous.size &gt; 0:

digest.update(contiguous.tobytes())

return digest.hexdigest()

def atomic_write_text(path: Path, text: str) -> None:

text
```

```
"""Write text atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.", suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

def atomic_write_json(path: Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)
```

def atomic_write_npz(path: Path, **kwargs: Any) -> None:

text

"""Write NPZ atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{path.name}.", suffix=".tmp.npz")

try:

with os.fdopen(fd, "wb") as handle:

np.savez(handle, **kwargs)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

==========
RNG Utilities (matching frozen production exactly)
==========

def make_seed_sequence(seed: int, child_index: int) -> int:

text

"""Generate a child seed from SeedSequence(seed).spawn(...)[child_index].

This matches the frozen production RNG hierarchy exactly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

children = root.spawn(child_index + 1)

return int(children[child_index].generate_state(1, dtype=np.uint64)[0])

def make_prefix_rng(seed: int) -> np.random.Generator:

text

"""Create prefix RNG matching frozen production exactly.

Frozen production does:

root = np.random.SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = np.random.default_rng(init_sequence)

NOTE: default_rng(SeedSequence_object) is NOT the same as

default_rng(SeedSequence_object.generate_state(...))

The production uses the SeedSequence object directly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

return np.random.default_rng(init_sequence)

def make_branch_rngs(seed: int, branch_order: tuple[str, ...]) -> dict[str, np.random.Generator]:

text

"""Create branch RNGs matching frozen production algorithm.

Frozen: *, branch*root = SeedSequence(seed).spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

branch_rng_seeds = {branch: int(sequence.generate_state(1, dtype=np.uint64)[0])

```
for branch, sequence in zip(branch_order, branch_sequences)}

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

*, branch*root = root.spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))



return {

branch: np.random.default_rng(int(sequence.generate_state(1, dtype=np.uint64)[0]))

for branch, sequence in zip(branch_order, branch_sequences)

}

================================================================================
Projection Accumulator (matching production)
================================================================================

@dataclass

class ProjectionTotals:

text

"""Accumulator for projection diagnostics - exact match to production."""
```

steps: int = 0

violations: int = 0

active: int = 0

norm_sum: float = 0.0

norm_max: float = 0.0

relative_state_sum: float = 0.0

relative_state_max: float = 0.0

relative_update_sum: float = 0.0

relative_update_max: float = 0.0

g_min: float = np.inf

g_min_pre: float = np.inf

def update(self, info: dict[str, Any]) -> None:

"""Update accumulator with step info."""

self.steps += 1

self.violations += int(info["pre_violation"])

self.g_min = min(self.g_min, float(info["g_min"]))

self.g_min_pre = min(self.g_min_pre, float(info["g_min_pre"]))

```
if info["projection_active"]:

self.active += 1

norm = float(info["projection_norm"])

rel_state = float(info["projection_relative_state"])

rel_update = float(info["projection_relative_update"])

self.norm_sum += norm

self.norm_max = max(self.norm_max, norm)

self.relative_state_sum += rel_state

self.relative_state_max = max(self.relative_state_max, rel_state)

self.relative_update_sum += rel_update

self.relative_update_max = max(self.relative_update_max, rel_update)

def metrics(self) -&gt; dict[str, float]:

"""Compute final metrics from accumulated values."""

active_denominator = max(self.active, 1)

return {

"G_min_future": float(self.g_min),

"G_min_pre_future": float(self.g_min_pre),

"Pre_gap_violation_fraction": float(self.violations / self.steps),
```

"Projection_active_fraction": float(self.active / self.steps),

"Projection_active_given_violation": float(self.active / self.violations)

if self.violations > 0

else float('nan'),

"Projection_norm_mean_active": float(

self.norm_sum / active_denominator

),

"Projection_norm_max": float(self.norm_max),

"Projection_relative_state_mean_active": float(

self.relative_state_sum / active_denominator

),

"Projection_relative_state_max": float(self.relative_state_max),

"Projection_relative_update_mean_active": float(

self.relative_update_sum / active_denominator

),

"Projection_relative_update_max": float(self.relative_update_max),

}

```
================================================================================
Dynamic State
================================================================================

class DynamicState:

text

"""State container for AIEF-1.4 dynamics.



Matches the frozen production implementation exactly.

"""



def **init**(

self,

theta: np.ndarray,

B: np.ndarray,

coherence_acc: np.ndarray,

history: list[np.ndarray] | np.ndarray,

):

self.theta = np.asarray(theta, dtype=float).copy()
```

```
self.B = np.asarray(B, dtype=float).copy()

self.coherence_acc = np.asarray(coherence_acc, dtype=complex).copy()


# Handle history input - ensure it's a list of arrays

if isinstance(history, np.ndarray):

# If it's a 2D array, convert to list of 1D arrays

self.history = [np.asarray(h, dtype=float).copy() for h in history]

else:

self.history = [np.asarray(h, dtype=float).copy() for h in history]



def clone(self) -&gt; "DynamicState":

"""Create a deep copy of the state."""

return DynamicState(

theta=self.theta.copy(),

B=self.B.copy(),

coherence_acc=self.coherence_acc.copy(),

history=[h.copy() for h in self.history],
```

)

@property

def N(self) -> int:

return len(self.theta)

def validate(self, config: ModelConfig, step: int | None = None) -> None:

"""Validate state consistency."""

Check history length

expected_history_len = config.delay_steps + 1

if len(self.history) != expected_history_len:

raise RuntimeError(

f"History length {len(self.history)} != expected {expected_history_len} "

f"(delay_steps={config.delay_steps}) at step {step}"

)

Check theta shape

if self.theta.shape != (self.N,):

```
raise RuntimeError(f"theta shape {self.theta.shape} != ({self.N},)")

# Check B shape

if self.B.shape != (self.N, self.N):

raise RuntimeError(f"B shape {self.B.shape} != ({self.N}, {self.N})")

# Check coherence_acc shape

if self.coherence_acc.shape != (self.N, self.N):

raise RuntimeError(f"coherence_acc shape {self.coherence_acc.shape} != ({self.N}, {self.N})")

# Check B symmetry

if not np.allclose(self.B, self.B.T, atol=1e-12):

raise RuntimeError("B matrix is not symmetric")

# Check B diagonal

if not np.allclose(np.diag(self.B), 0, atol=1e-12):

raise RuntimeError("B matrix diagonal is not zero")

# Check B non-negativity

if np.any(self.B &lt; -1e-12):

raise RuntimeError("B matrix has negative entries")

================================================================================
Simulation Core (exact reproduction of frozen production)
```

```
================================================================================
====

def initialize_state(

text

seed: int,

config: ModelConfig,

) -> DynamicState:

text

"""Initialize state from seed using frozen algorithm exactly.



Matches production: simulate_prefix() initialization.

Uses: root = SeedSequence(seed); init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence)

"""

N = config.N

rng = make_prefix_rng(seed)



# Initialize theta: uniform in [0, 2*pi) - matching production
```

```
theta = rng.random(N) *2.0* np.pi


# Initialize B: small random symmetric matrix

B = config.B_init_scale * rng.random((N, N))

B = 0.5 * (B + B.T)

np.fill_diagonal(B, 0.0)


# Initialize coherence accumulator

coherence_acc = np.zeros((N, N), dtype=complex)


# Initialize history: DELAY_STEPS + 1 copies of theta

history = [theta.copy() for *in range(config.delay*steps + 1)]


return DynamicState(

theta=theta,

B=B,
```

```
coherence_acc=coherence_acc,

history=history,

)

def compute_R(

text

state: DynamicState,

config: ModelConfig,

) -> np.ndarray:

text

"""Compute coherence matrix R from current state.



Exact reproduction of production compute_R().

"""

N = state.N

delayed = state.history[0]



# Convert omega to numpy array for indexing
```

```
omega_arr = np.asarray([config.omega](http://config.omega), dtype=float)



dphi = (

state.theta[:, None]

- delayed[None, :]

- ((omega_arr[:, None] + omega_arr[None, :]) / 2.0) * config.tau

)

z_inst = np.exp(1j * dphi)



# Update EMA

state.coherence_acc[:] = (

config.alpha *state.coherence_acc + (1.0 - config.alpha)* z_inst

)



# Compute magnitude and symmetrize

result = np.abs(state.coherence_acc)

result = 0.5 * (result + result.T)
```

np.fill_diagonal(result, 0.0)

return result

def advance_state(

text

state: DynamicState,

config: ModelConfig,

rng: np.random.Generator,

shuffle_driver: bool = False,

) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:

text

"""Advance state by one time step.

Exact reproduction of production advance_state().

Returns (R_physical, R_driver, info).

Production also returns projection diagnostics:

- projection_active: whether correction was applied

- projection_norm: norm of correction

- projection_relative_state: correction_norm / ||B_unprojected||

- projection_relative_update: correction_norm / ||B_unprojected - B_previous||

"""

N = state.N

edges = compute_edges(N)

Store previous B for projection

B_previous = state.B.copy()

Compute R matrices

R_physical = compute_R(state, config)

R_driver = R_physical

if shuffle_driver:

values = R_physical[edges].copy()

```
rng.shuffle(values)

R_driver = np.zeros_like(R_physical)

R_driver[edges] = values

R_driver[(edges[1], edges[0])] = values



# Update B

B_unprojected = state.B + config.dt * (

(config.eta + [config.mu](http://config.mu)) * R_driver

- [config.mu](http://config.mu)

- config.gamma * state.B

)

B_unprojected = np.maximum(B_unprojected, 0.0)

B_unprojected = 0.5 * (B_unprojected + B_unprojected.T)

np.fill_diagonal(B_unprojected, 0.0)



# Projection diagnostics (matching production exactly)

proposed_update_norm = float(np.linalg.norm(B_unprojected - B_previous))
```

```
# Check gap constraint

row_sum_pre = np.sum(B_unprojected, axis=1)

g_min_pre = float(np.min(config.a - config.q * row_sum_pre))

pre_violation = bool(g_min_pre &lt; [config.delta](http://config.delta))



# Project if needed

B_projected = B_unprojected

correction_norm = 0.0

if pre_violation:

scale = np.minimum(1.0, config.capacity / (row_sum_pre + config.scale_epsilon))

B_projected = B_unprojected * np.minimum.outer(scale, scale)

np.fill_diagonal(B_projected, 0.0)

correction_norm = float(np.linalg.norm(B_projected - B_unprojected))



projection_active = bool(correction_norm &gt; FROZEN_PROJECTION_NORM_TOL)

relative_state = correction_norm / max(
```

```
float(np.linalg.norm(B_unprojected)), FROZEN_ZERO_TOL

)

relative_update = correction_norm / max(proposed_update_norm, FROZEN_ZERO_TOL)



# Validate projection

state.B = B_projected

row_sum = np.sum(state.B, axis=1)

g_min = float(np.min(config.a - config.q * row_sum))

if g_min &lt; [config.delta](http://config.delta) - 1e-9:

raise RuntimeError(f"Gap projection failed: g_min={g_min}")



# Update theta

delayed = state.history[0]

omega_arr = np.asarray([config.omega](http://config.omega), dtype=float)

coupling = np.sum(

state.B * np.sin(delayed[None, :] - state.theta[:, None]), axis=1

)
```

```
state.theta = np.mod(

state.theta + config.dt * (omega_arr + coupling),

2.0 * np.pi

)



# Update history

state.history.append(state.theta.copy())

if len(state.history) &gt; config.delay_steps + 1:

state.history.pop(0)



info = {

"pre_violation": pre_violation,

"projection_active": projection_active,

"projection_norm": correction_norm,

"projection_relative_state": relative_state,

"projection_relative_update": relative_update,

"g_min_pre": g_min_pre,
```

"g_min": g_min,

}

return R_physical, R_driver, info

def simulate_prefix(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

) -> tuple[DynamicState, dict[str, Any]]:

text

"""Simulate prefix up to split_time.

Exact reproduction of production simulate_prefix().

Returns (state, prefix_dict).

Production uses:

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence) # for initialization

Then in the loop, uses:

unused_rng = default_rng(0) # for advance_state when shuffle_driver=False

"""

N = config.N

edges = compute_edges(N)

Initialize state (uses init_sequence via make_prefix_rng)

state = initialize_state(seed, config)

Get RNG for prefix simulation loop

Production uses unused_rng = default_rng(0) for the advance_state calls

when shuffle_driver=False

unused_rng = np.random.default_rng(0)

```
# Simulate to split point

R_past_sum = np.zeros((N, N), dtype=float)

R_past_count = 0



burn_end = protocol.burn_end(config)

hist_end = protocol.hist_end(config)



for step in range(hist_end):

R_physical, *,* = advance_state(state, config, unused_rng, shuffle_driver=False)


if burn_end &lt;= step &lt; hist_end:

R_past_sum += R_physical

R_past_count += 1



prefix = {

"B_at_split": state.B.copy(),

"R_physical_past": R_past_sum / R_past_count if R_past_count &gt; 0 else np.zeros((N, N)),
```

"theta_at_split": state.theta.copy(),

"coherence_acc_at_split": state.coherence_acc.copy(),

"history_at_split": np.stack(state.history),

"selected_at_split": (state.B > config.B_c).astype(np.uint8),

}

return state, prefix

def apply_intervention(

text

state: DynamicState,

branch: str,

config: ModelConfig,

rng: np.random.Generator,

) -> None:

text

"""Apply intervention to state. Mutates in place.

```
Exact reproduction of production apply_intervention().

"""

N = state.N



if branch in {"reset_ema", "reset_B_ema", "B_carrier_only", "full_reset"}:

# Zero coherence accumulator

state.coherence_acc.fill(0.0)



if branch in {"reset_B", "reset_B_ema", "full_reset"}:

# Zero B

state.B.fill(0.0)



if branch in {"B_carrier_only", "full_reset"}:

# Randomize theta and history

new_theta = rng.random(N) *2.0* np.pi

state.theta = new_theta

state.history = [new_theta.copy() for *in range(config.delay*steps + 1)]
```

```
# Ensure symmetry

state.B = 0.5 * (state.B + state.B.T)

np.fill_diagonal(state.B, 0.0)

def simulate_future(

text

split_state: DynamicState,

prefix: dict[str, Any],

branch: str,

config: ModelConfig,

protocol: ProtocolConfig,

rng: np.random.Generator,

) -> dict[str, Any]:

text

"""Simulate future trajectory from split state.



Reproduces production simulate_future() with ALL required outputs.
```

CRITICAL: Now accumulates BOTH horizon systems:

Fixed Physical Horizons (time-based, NOT T_window dependent):

- immediate: 0 to 17.5 time units

- recovery: 17.5 to 87.5 time units

- late: 87.5 to 320 time units

EMA-Normalized Horizons (T_window dependent):

- immediate: 0 to 1*T_window

- recovery: 1*T_window to 5*T_window

- late: 5*T_window to end

At rho=1 (T_window=17.502917...), the fixed physical and normalized horizons

are approximately aligned but use exact step boundaries.

Production accumulates:

- P_full, P_immediate, P_recovery, P_late (incidence matrices)

- B_mean_full, B_mean_immediate, B_mean_recovery, B_mean_late

- R_physical_future, R_driver_future

- Projection diagnostics via ProjectionTotals

- Various scalar metrics

Uses horizon system from production for normalized:

- WINDOW_STEPS = round(T_window / dt)

- RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS)

For fixed physical:

- FIXED_IMMEDIATE_END = round(17.5 / dt)

- FIXED_RECOVERY_END = round(87.5 / dt)

"""

state = split_state.clone()

apply_intervention(state, branch, config, rng)

```
N = state.N

edges = compute_edges(N)

future_steps = protocol.future_steps(config)



# Compute horizon step boundaries

# Normalized EMA horizons (T_window dependent)

window_steps = int(round(config.T_window / config.dt))

recovery_end_ema = min(5 * window_steps, future_steps)



# Fixed physical horizons (NOT T_window dependent)

# These use the production's physical time boundaries: 17.5, 87.5, 320

fixed_immediate_end = int(round(17.5 / config.dt)) # 17.5 time units

fixed_recovery_end = int(round(87.5 / config.dt)) # 87.5 time units

# Late goes from 87.5 to end (320 time units)



# Clamp all boundaries to future_steps

recovery_end_ema = min(recovery_end_ema, future_steps)
```

```
fixed_immediate_end = min(fixed_immediate_end, future_steps)

fixed_recovery_end = min(fixed_recovery_end, future_steps)



# Initialize accumulators for BOTH horizon systems

# P matrices (incidence: B &gt; B_c as 0/1)

P_sums_ema = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}

P_sums_fixed = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}
```

```
# B matrices (mean B values)
B_sums_ema = {
"full": np.zeros((N, N), dtype=float),
"immediate": np.zeros((N, N), dtype=float),
"recovery": np.zeros((N, N), dtype=float),
"late": np.zeros((N, N), dtype=float),
}
B_sums_fixed = {
"full": np.zeros((N, N), dtype=float),
"immediate": np.zeros((N, N), dtype=float),
"recovery": np.zeros((N, N), dtype=float),
"late": np.zeros((N, N), dtype=float),
}

# Counts for each horizon in both systems
counts_ema = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}
```

```
counts_fixed = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}



# R matrices

R_physical_sum = np.zeros((N, N), dtype=float)

R_driver_sum = np.zeros((N, N), dtype=float)



# Projection accumulator (matching production)

projection = ProjectionTotals()



# Trajectory accumulators

sample_times = []

density_trajectory = []

similarity_trajectory = []



# For retention/emergence calculations

r_sum = 0.0
```

```
r_count = 0

r_late_ema_sum = 0.0

r_late_ema_count = 0

r_late_fixed_sum = 0.0

r_late_fixed_count = 0



B_reference = prefix["B_at_split"]

shuffle_driver = (branch == "future_shuffle")



for step in range(future_steps):

R_physical, R_driver, info = advance_state(

state, config, rng, shuffle_driver=shuffle_driver

)



# Update projection accumulator

projection.update(info)



# Accumulate R matrices
```

R_physical_sum += R_physical

R_driver_sum += R_driver

Determine which horizons this step belongs to for BOTH systems

selected = (state.B > config.B_c).astype(float)

=== EMA-Normalized Horizons ===

Full: all steps

P_sums_ema["full"] += selected

B_sums_ema["full"] += state.B

counts_ema["full"] += 1

immediate: 0 to window_steps (exclusive)

if step < window_steps:

P_sums_ema["immediate"] += selected

B_sums_ema["immediate"] += state.B

counts_ema["immediate"] += 1

recovery: window_steps to recovery_end_ema (exclusive)

if window_steps <= step < recovery_end_ema:

P_sums_ema["recovery"] += selected

B_sums_ema["recovery"] += state.B

counts_ema["recovery"] += 1

late: recovery_end_ema to end

if recovery_end_ema <= step:

P_sums_ema["late"] += selected

B_sums_ema["late"] += state.B

counts_ema["late"] += 1

=== Fixed Physical Horizons ===

Full: all steps (same as EMA)

P_sums_fixed["full"] += selected

B_sums_fixed["full"] += state.B

counts_fixed["full"] += 1

immediate: 0 to fixed_immediate_end (exclusive)

if step < fixed_immediate_end:

```
P_sums_fixed["immediate"] += selected

B_sums_fixed["immediate"] += state.B

counts_fixed["immediate"] += 1


# recovery: fixed_immediate_end to fixed_recovery_end (exclusive)

if fixed_immediate_end &lt;= step &lt; fixed_recovery_end:

P_sums_fixed["recovery"] += selected

B_sums_fixed["recovery"] += state.B

counts_fixed["recovery"] += 1


# late: fixed_recovery_end to end

if fixed_recovery_end &lt;= step:

P_sums_fixed["late"] += selected

B_sums_fixed["late"] += state.B

counts_fixed["late"] += 1


# Trajectory sampling - CRITICAL: r_mean_late is ONLY accumulated here

if step % protocol.diagnostic_stride == 0:
```

```
t = (step + 1) * config.dt

sample_times.append(t)


# Density: fraction of edges above B_c

density = float(np.count_nonzero(state.B[edges] &gt; config.B_c) / len(edges[0]))

density_trajectory.append(density)


# Similarity: cosine between current and reference B

B_vec = state.B[edges]

B_ref_vec = B_reference[edges]

norm_B = np.linalg.norm(B_vec)

norm_ref = np.linalg.norm(B_ref_vec)

if norm_B &gt; 0 and norm_ref &gt; 0:

similarity = float([np.dot](http://np.dot)(B_vec, B_ref_vec) / (norm_B * norm_ref))

else:

similarity = float('nan')

similarity_trajectory.append(similarity)


# Track overall r
```

```
r_value = float(np.abs(np.mean(np.exp(1j * state.theta))))

r_sum += r_value

r_count += 1


# Track late-phase r for BOTH horizon systems

if recovery_end_ema &lt;= step:

r_late_ema_sum += r_value

r_late_ema_count += 1


if fixed_recovery_end &lt;= step:

r_late_fixed_sum += r_value

r_late_fixed_count += 1



# Compute mean matrices for BOTH horizon systems

P_ema = {name: P_sums_ema[name] / counts_ema[name] if counts_ema[name] &gt; 0 else
np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

P_fixed = {name: P_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] &gt; 0 else
np.zeros((N, N))
```

```
for name in ["full", "immediate", "recovery", "late"]}



B_mean_ema = {name: B_sums_ema[name] / counts_ema[name] if counts_ema[name] &gt; 0 else
np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_fixed = {name: B_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] &gt; 0 else
np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}



# Compute R means

R_physical_mean = R_physical_sum / future_steps if future_steps &gt; 0 else np.zeros((N, N))

R_driver_mean = R_driver_sum / future_steps if future_steps &gt; 0 else np.zeros((N, N))



# Final state

B_final = state.B.copy()

final_weights = state.B[edges]

final_rows = np.sum(state.B, axis=1)
```

```
# Retention and emergence for BOTH horizon systems

split_selected = prefix["selected_at_split"][edges].astype(bool)



# EMA-normalized

p_late_edges_ema = P_ema["late"][edges]

retention_ema = (

float(np.mean(p_late_edges_ema[split_selected]))

if np.any(split_selected)

else float('nan')

)

emergence_ema = (

float(np.mean(p_late_edges_ema[~split_selected]))

if np.any(~split_selected)

else float('nan')

)



# Fixed physical
```

```
p_late_edges_fixed = P_fixed["late"][edges]

retention_fixed = (

float(np.mean(p_late_edges_fixed[split_selected]))

if np.any(split_selected)

else float('nan')

)

emergence_fixed = (

float(np.mean(p_late_edges_fixed[~split_selected]))

if np.any(~split_selected)

else float('nan')

)



# Minimum similarity from trajectory

similarity_array = np.array(similarity_trajectory, dtype=float)

finite_similarity = similarity_array[np.isfinite(similarity_array)]

minimum_similarity = (

float(np.min(finite_similarity)) if len(finite_similarity) &gt; 0
```

```
else float('nan')

)



# Build result with BOTH horizon systems

result = {

# === Matrix outputs for EMA-normalized horizons (backward compatible) ===

"B_final": B_final,

"B_mean_late": B_mean_ema["late"],

"P_full": P_ema["full"],

"P_immediate": P_ema["immediate"],

"P_recovery": P_ema["recovery"],

"P_late": P_ema["late"],

"R_physical_future": R_physical_mean,

"R_driver_future": R_driver_mean,



# === Matrix outputs for FIXED-PHYSICAL horizons (NEW) ===

"B_mean_late_fixed_physical": B_mean_fixed["late"],
```

"P_full_fixed_physical": P_fixed["full"],

"P_immediate_fixed_physical": P_fixed["immediate"],

"P_recovery_fixed_physical": P_fixed["recovery"],

"P_late_fixed_physical": P_fixed["late"],

=== Matrix outputs for NORMALIZED-EMA horizons (NEW - explicit) ===

"B_mean_late_normalized_EMA": B_mean_ema["late"],

"P_full_normalized_EMA": P_ema["full"],

"P_immediate_normalized_EMA": P_ema["immediate"],

"P_recovery_normalized_EMA": P_ema["recovery"],

"P_late_normalized_EMA": P_ema["late"],

Trajectory outputs

"sample_times": np.array(sample_times, dtype=float),

"density_trajectory": np.array(density_trajectory, dtype=float),

"similarity_trajectory": np.array(similarity_trajectory, dtype=float),

Scalar outcomes - EMA-normalized (backward compatible names)

"D_B_final": float(np.count_nonzero(final_weights > config.B_c) / len(edges[0])),

```
"P_mean_full": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late": float(np.mean(P_ema["late"][edges])),


# Scalar outcomes - FIXED-PHYSICAL (NEW - primary estimands)

"P_mean_full_fixed_physical": float(np.mean(P_fixed["full"][edges])),

"P_mean_immediate_fixed_physical": float(np.mean(P_fixed["immediate"][edges])),

"P_mean_recovery_fixed_physical": float(np.mean(P_fixed["recovery"][edges])),

"P_mean_late_fixed_physical": float(np.mean(P_fixed["late"][edges])),


# Scalar outcomes - NORMALIZED-EMA (NEW - explicit)

"P_mean_full_normalized_EMA": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate_normalized_EMA": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery_normalized_EMA": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late_normalized_EMA": float(np.mean(P_ema["late"][edges])),


# Similarity metrics - BOTH systems
```

"Weighted_similarity_final": cosine_edges(B_reference, B_final, edges),

"Weighted_similarity_late": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Weighted_similarity_late_fixed_physical": cosine_edges(B_reference, B_mean_fixed["late"], edges),

"Weighted_similarity_late_normalized_EMA": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Jaccard_final": jaccard_edges(B_reference, B_final, edges, config.B_c),

"Jaccard_final_fixed_physical": jaccard_edges(B_reference, B_final, edges, config.B_c),

Retention/emergence - BOTH systems

"Retention_selected_late": retention_ema,

"Emergence_unselected_late": emergence_ema,

"Retention_contrast_late": retention_ema - emergence_ema,

"Retention_selected_late_fixed_physical": retention_fixed,

"Emergence_unselected_late_fixed_physical": emergence_fixed,

"Retention_contrast_late_fixed_physical": retention_fixed - emergence_fixed,

"Retention_selected_late_normalized_EMA": retention_ema,

"Emergence_unselected_late_normalized_EMA": emergence_ema,

"Retention_contrast_late_normalized_EMA": retention_ema - emergence_ema,

Correlations with past R - BOTH systems

"Corr_Rpast_Pfull": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

"Corr_Rpast_Pfull_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["full"][edges]),

"Corr_Rpast_Plate_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["late"][edges]),

"Corr_Rpast_Pfull_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

Weight statistics (same for both - based on final state)

"Total_edge_weight": float(np.sum(final_weights)),

"Mean_row_sum": float(np.mean(final_rows)),

"Max_row_sum": float(np.max(final_rows)),

"Nonzero_edge_fraction": float(np.mean(final_weights > config.scale_epsilon)),

"Effective_edges": effective_number(final_weights),

"Weight_gini": gini_coefficient(final_weights),

"Weight_entropy": normalized_entropy(final_weights),

Order parameter - BOTH systems

"r_mean_future": float(r_sum / r_count) if r_count > 0 else float('nan'),

"r_mean_late": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

"r_mean_late_fixed_physical": float(r_late_fixed_sum / r_late_fixed_count) if r_late_fixed_count > 0 else float('nan'),

"r_mean_late_normalized_EMA": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

Minimum similarity

"Minimum_similarity": minimum_similarity,

Projection diagnostics (from ProjectionTotals)

**projection.metrics(),

Horizon metadata

"window_steps": window_steps,

"recovery_end_ema": recovery_end_ema,

"fixed_immediate_end": fixed_immediate_end,

"fixed_recovery_end": fixed_recovery_end,

Time-based metrics (placeholders - will be NaN for now)

"Time_to_50pct_continue_final_density": float('nan'),

"Time_to_90pct_continue_final_density": float('nan'),

"Time_to_50pct_continue_late_similarity": float('nan'),

"Time_to_90pct_continue_late_similarity": float('nan'),

"Time_to_similarity_below_0_9": float('nan'),

"Time_to_similarity_below_0_5": float('nan'),

"Late_persistence_ratio_to_continue": float('nan'),

"Late_similarity_ratio_to_continue": float('nan'),

}

return result

==========
Metric Helpers
==========

def cosine_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None) -> float:

text

"""Compute cosine similarity between reference and current B matrices over edges.

If edges is None, computes over all upper triangle edges.

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = np.asarray(reference[edges], dtype=float)

y = np.asarray(current[edges], dtype=float)

denominator = float(np.linalg.norm(x) * np.linalg.norm(y))

if denominator <= 1e-12:

return float('nan')

return float(np.dot(x, y) / denominator)

def jaccard_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None, threshold: float = FROZEN_ZERO_TOL) -> float:

text

"""Compute Jaccard similarity between reference and current B matrices over edges.

Uses B > threshold as the threshold for edge presence.

If edges is None, computes over all upper triangle edges.

Matches production: jaccard_edges(B_reference, state.B) uses B > B_C

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = reference[edges] > threshold

y = current[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x & y) / union)

```
def safe_corr(x: np.ndarray, y: np.ndarray) -> float:

text

"""Compute correlation coefficient safely."""

x = np.asarray(x, dtype=float)

y = np.asarray(y, dtype=float)

valid = np.isfinite(x) &amp;amp; np.isfinite(y)

x = x[valid]

y = y[valid]

if len(x) &amp;lt; 3 or np.std(x) &amp;lt; 1e-12 or np.std(y) &amp;lt; 1e-12:

return float('nan')

return float(np.corrcoef(x, y)[0, 1])

def effective_number(weights: np.ndarray) -> float:

text

"""Compute effective number of edges."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

sum_sq = float(np.sum(weights**2))

if total &amp;lt;= 1e-12 or sum_sq &amp;lt;= 1e-12:
```

```
return 0.0

return total**2 / sum_sq

def gini_coefficient(weights: np.ndarray) -> float:

text

"""Compute Gini coefficient."""

x = np.sort(np.asarray(weights, dtype=float))

total = float(np.sum(x))

if total &lt;= 1e-12:

return 0.0

n = len(x)

ranks = np.arange(1, n + 1, dtype=float)

return float(2.0 *[np.dot](http://np.dot)(ranks, x) / (n* total) - (n + 1.0) / n)

def normalized_entropy(weights: np.ndarray) -> float:

text

"""Compute normalized entropy."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))
```

if total <= 1e-12:

return 0.0

probabilities = weights / total

positive = probabilities > 0.0

entropy = -float(np.sum(probabilities[positive] * np.log(probabilities[positive])))

return entropy / float(np.log(len(weights)))

==========
Required Output Schema for Baseline Equivalence
==========
IMPORTANT: We separate FROZEN keys (must exist in historical NPZ) from
EXTENDED keys (new in v1, only validated internally at rho=1).
The frozen NPZ does NOT contain dual-horizon keys, so baseline equivalence
must only check FROZEN keys against historical data.
These are the keys that MUST exist in the frozen NPZ for baseline equivalence

FROZEN_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
These are the NEW keys introduced in v1 for dual-horizon support
They will NOT exist in frozen NPZ, so baseline equivalence skips them

EXTENDED_BRANCH_MATRIX_KEYS = (

text

"B_mean_late_fixed_physical",

"P_full_fixed_physical",

"P_immediate_fixed_physical",

"P_recovery_fixed_physical",

"P_late_fixed_physical",

"B_mean_late_normalized_EMA",

"P_full_normalized_EMA",

"P_immediate_normalized_EMA",

"P_recovery_normalized_EMA",

"P_late_normalized_EMA",

)
All matrix keys (frozen + extended)

REQUIRED_BRANCH_MATRIX_KEYS = FROZEN_BRANCH_MATRIX_KEYS + EXTENDED_BRANCH_MATRIX_KEYS

REQUIRED_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Fixed physical horizon scalars (NEW - PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (NEW - explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

Projection diagnostics (from ProjectionTotals)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

)
==========
====
Baseline Equivalence Test
==========
====

def exact_array_equal_with_nan(a: np.ndarray, b: np.ndarray) -> bool:

text

"""Compare arrays with NaN-aware exact equality."""

a = np.asarray(a)

b = np.asarray(b)

if a.shape != b.shape or a.dtype != b.dtype:

return False

if np.issubdtype(a.dtype, np.floating) or np.issubdtype(a.dtype, np.complexfloating):

a_nan = np.isnan(a)

b_nan = np.isnan(b)

if not np.array_equal(a_nan, b_nan):

return False

finite = ~a_nan

return np.array_equal(a[finite], b[finite])

return np.array_equal(a, b)

def run_baseline_equivalence(

text

seeds: list[int],

config: ModelConfig,

protocol: ProtocolConfig,

frozen_manifest_path: str,

frozen_npz_path: str,

branch_order: tuple[str, ...],

frozen_csv_path: str | None = None,

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for all seeds.

This function validates that the engine produces EXACTLY the same outputs

as the frozen production run for all required keys.

Requires NaN-aware exact equality for ALL required schema keys:

- All prefix keys

- All branch matrix keys

- All branch scalar keys

Returns:

status: "passed" | "failed" | "not_run"

seed_results: per-seed validation details

engine_source_sha256: hash of this engine source

frozen_manifest_hash: hash of frozen manifest

frozen_npz_hash: hash of frozen NPZ

"""

import numpy as np

from pathlib import Path

```
import hashlib



# Helper function to compute source hash

def *source*sha256() -&gt; str | None:

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None



# Record engine source hash

engine_source_hash = *source*sha256()



results = {

"status": "not_run",

"config": {

"N": config.N,

"dt": config.dt,
```

"tau": config.tau,

"mean_omega": config.mean_omega,

"T_window": config.T_window,

"T_star": config.T_star,

"alpha": config.alpha,

"a": config.a,

"q": config.q,

"delta": config.delta,

"eta": config.eta,

"mu": config.mu,

"gamma": config.gamma,

"B_c": config.B_c,

"B_init_scale": config.B_init_scale,

"scale_epsilon": config.scale_epsilon,

"capacity": config.capacity,

"delay_steps": config.delay_steps,

},

```
"protocol": {
"T_sim": protocol.T_sim,
"burn_end_time": protocol.burn_end_time,
"split_time": protocol.split_time,
"future_duration": protocol.future_duration,
"diagnostic_stride": protocol.diagnostic_stride,
"branches": list(protocol.branches),
},
"n_seeds": len(seeds),
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"engine_source_sha256": engine_source_hash,
"seed_results": {},
}


# Load frozen manifest
with open(frozen_manifest_path, "r") as f:
```

```
frozen_manifest = json.load(f)



# Load frozen data

with np.load(frozen_npz_path, allow_pickle=False) as frozen_data:

frozen_files = dict(frozen_data)



# Load frozen scalar CSV if provided

frozen_scalar_csv: dict[str, dict[str, Any]] = {}

if frozen_csv_path is not None and os.path.exists(frozen_csv_path):

try:

import csv

with open(frozen_csv_path, "r", newline="", encoding="utf-8") as f:

reader = csv.DictReader(f)

for row in reader:

seed_id = row.get("seed", "")

branch = row.get("branch", "")

if seed_id and branch:
```

```
key = f"{branch}_seed_{seed_id}"

if key not in frozen_scalar_csv:

frozen_scalar_csv[key] = {}

for k, v in row.items():

if k not in ("seed", "seed_index", "branch", "rho", "T_window"):

try:

frozen_scalar_csv[key][k] = float(v)

except (ValueError, TypeError):

frozen_scalar_csv[key][k] = v

except Exception as e:

print(f"Warning: Could not load frozen CSV {frozen_csv_path}: {e}")



# Record frozen hashes

results["frozen_manifest_hash"] = hashlib.sha256(

json.dumps(frozen_manifest, sort_keys=True).encode()

).hexdigest()

results["frozen_npz_hash"] = file_sha256(frozen_npz_path)
```

```
if frozen_csv_path is not None and os.path.exists(frozen_csv_path):

results["frozen_csv_hash"] = file_sha256(frozen_csv_path)



# Validate config matches frozen manifest model - use EXACT equality

frozen_model = frozen_manifest["model"]



# Check N

if frozen_model["N"] != config.N:

results["status"] = "failed"

results["error"] = f"N mismatch: config={config.N}, frozen={frozen_model['N']}"

return results



# Check omega - use exact equality

frozen_omega_arr = np.array(frozen_model["omega"], dtype=float)

config_omega_arr = np.array(list([config.omega](http://config.omega)), dtype=float)

if not np.array_equal(frozen_omega_arr, config_omega_arr):

results["status"] = "failed"
```

results["error"] = f"omega mismatch: frozen={frozen_model['omega']}, config={list(config.omega)}"

return results

Check all other model params with EXACT equality (rtol=0, atol=0)

model_checks = [

("dt", frozen_model["dt"], config.dt),

("tau", frozen_model["tau"], config.tau),

("a", frozen_model["a"], config.a),

("q", frozen_model["q"], config.q),

("delta", frozen_model["delta"], config.delta),

("eta", frozen_model["eta"], config.eta),

("mu", frozen_model["mu"], config.mu),

("gamma", frozen_model["gamma"], config.gamma),

("B_c", frozen_model["B_c"], config.B_c),

("B_init_scale", frozen_model.get("B_init_scale", 0.0), config.B_init_scale),

("T_window", frozen_model["T_window"], config.T_window),

]

```
for name, frozen_val, config_val in model_checks:

if not np.isclose(frozen_val, config_val, rtol=0.0, atol=0.0):

results["status"] = "failed"

results["error"] = f"{name} mismatch: frozen={frozen_val}, config={config_val}"

return results



# Check derived values

if not np.isclose(frozen_model.get("alpha", 0.0), config.alpha, rtol=0.0, atol=1e-15):

results["status"] = "failed"

results["error"] = f"alpha mismatch: frozen={frozen_model.get('alpha')}, config={config.alpha}"

return results



# Now run actual equivalence test for each seed

print("Running baseline equivalence test...")

all_passed = True
```

for seed in seeds:

seed_str = str(seed)

seed_results = {"status": "passed", "checks": {}, "max_diffs": {}}

try:

Simulate with our engine

split_state, prefix = simulate_prefix(seed, config, protocol)

branch_rngs = make_branch_rngs(seed, branch_order)

Simulate each branch

engine_results = {}

for branch in protocol.branches:

rng = branch_rngs[branch]

engine_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

```
protocol,

rng,

)


# Check prefix state - iterate over REQUIRED_PREFIX_KEYS

for key in REQUIRED_PREFIX_KEYS:

engine_val = prefix.get(key)

frozen_key = f"prefix_seed_{seed_str}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"prefix_{key}_missing"] = True

all_passed = False

continue


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"prefix_{key}_exact"] = True

else:
```

```
seed_results["checks"][f"prefix_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"prefix_{key}"] = float(np.max(np.abs(e - f)))


# Check branch outputs - separate FROZEN vs EXTENDED keys

for branch in protocol.branches:

engine_branch = engine_results[branch]

frozen_prefix = f"{branch}_seed_{seed_str}"


# === FROZEN MATRIX KEYS: Must exist in historical NPZ ===

for key in FROZEN_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)
```

```
if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_frozen"] = True

all_passed = False

continue


if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"
```

```
all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"{branch}_{key}"] = float(np.max(np.abs(e - f)))


# === EXTENDED MATRIX KEYS: Validate internally at rho=1 ===

# These do NOT exist in frozen NPZ. At rho=1, both horizon systems

# should produce identical results (1750 and 8750 steps match).

if rho == 1.0:

for key in EXTENDED_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue
```

```
# Map extended key to its frozen equivalent

frozen_key_map = {

"B_mean_late_fixed_physical": "B_mean_late",

"P_full_fixed_physical": "P_full",

"P_immediate_fixed_physical": "P_immediate",

"P_recovery_fixed_physical": "P_recovery",

"P_late_fixed_physical": "P_late",

"B_mean_late_normalized_EMA": "B_mean_late",

"P_full_normalized_EMA": "P_full",

"P_immediate_normalized_EMA": "P_immediate",

"P_recovery_normalized_EMA": "P_recovery",

"P_late_normalized_EMA": "P_late",

}


frozen_equiv_key = frozen_key_map.get(key)

if frozen_equiv_key:

engine_frozen_equiv = engine_branch.get(frozen_equiv_key)

if engine_frozen_equiv is not None:
```

```
if exact_array_equal_with_nan(engine_val, engine_frozen_equiv):

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True

else:

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(engine_frozen_equiv, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(engine_frozen_equiv), 0.0, engine_frozen_equiv)

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = float(np.max(np.abs(e - f)))


# === SCALAR KEYS ===

# Separate FROZEN vs EXTENDED scalars

# First, define which scalars are frozen (exist in historical data)

FROZEN_BRANCH_SCALAR_KEYS = (

"D_B_final",

"P_mean_full",

"P_mean_immediate",
```

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

)

EXTENDED_BRANCH_SCALAR_KEYS = (

"P_mean_late_fixed_physical",

```
"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

)


for key in FROZEN_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)


# Get from CSV first (production stores scalars in CSV)

frozen_val = None

csv_key = f"{branch}_seed_{seed_str}"

if csv_key in frozen_scalar_csv and key in frozen_scalar_csv[csv_key]:
```

```
frozen_val = frozen_scalar_csv[csv_key][key]

else:

# Fallback to NPZ

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


# FROZEN scalars MUST exist in historical data

if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_frozen"] = True

all_passed = False

continue


if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue
```

```
# For scalars, compare directly

if isinstance(engine_val, (int, float, np.number)):

engine_is_nan = bool(np.isnan(engine_val))

frozen_is_nan = bool(np.isnan(frozen_val)) if isinstance(frozen_val, (int, float, np.number)) else False


if engine_is_nan and frozen_is_nan:

seed_results["checks"][f"{branch}_{key}_exact"] = True

elif np.isclose(engine_val, frozen_val, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}"] = float(abs(engine_val - frozen_val)) if not
(engine_is_nan or frozen_is_nan) else float('nan')


# === EXTENDED SCALAR KEYS: Validate internally at rho=1 ===

if rho == 1.0:
```

```
for key in EXTENDED_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)

if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue


# Map to frozen equivalent

frozen_scalar_map = {

"P_mean_late_fixed_physical": "P_mean_late",

"Weighted_similarity_late_fixed_physical": "Weighted_similarity_late",

"Retention_contrast_late_fixed_physical": "Retention_contrast_late",

"Corr_Rpast_Plate_fixed_physical": "Corr_Rpast_Plate",

"r_mean_late_fixed_physical": "r_mean_late",

"P_mean_late_normalized_EMA": "P_mean_late",

"Weighted_similarity_late_normalized_EMA": "Weighted_similarity_late",

"Retention_contrast_late_normalized_EMA": "Retention_contrast_late",
```

```
"Corr_Rpast_Plate_normalized_EMA": "Corr_Rpast_Plate",

"r_mean_late_normalized_EMA": "r_mean_late",

}


frozen_equiv = frozen_scalar_map.get(key)

if frozen_equiv:

# Get the frozen equivalent value from CSV

csv_key = f"{branch}_seed_{seed_str}"

if csv_key in frozen_scalar_csv and frozen_equiv in frozen_scalar_csv[csv_key]:

frozen_equiv_val = frozen_scalar_csv[csv_key][frozen_equiv]

if isinstance(engine_val, (int, float, np.number)) and isinstance(frozen_equiv_val, (int, float,
np.number)):

engine_is_nan = bool(np.isnan(engine_val))

frozen_is_nan = bool(np.isnan(frozen_equiv_val))


if engine_is_nan and frozen_is_nan:

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True

elif np.isclose(engine_val, frozen_equiv_val, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True
```

```
else:
seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = False
seed_results["status"] = "failed"
all_passed = False
seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = float(abs(engine_val - frozen_equiv_val))

except Exception as e:
seed_results["status"] = "failed"
seed_results["error"] = str(e)
import traceback
seed_results["traceback"] = traceback.format_exc()
all_passed = False

results["seed_results"][seed_str] = seed_results
print(f" Seed {seed_str}: {seed_results['status']}")

if all_passed:
results["status"] = "passed"
```

```
print("Baseline equivalence: PASSED")

else:

results["status"] = "failed"

print("Baseline equivalence: FAILED")

# Print details of failures

for seed_str, seed_results in results["seed_results"].items():

if seed_results.get("status") != "passed":

print(f" Seed {seed_str} failures:")

for check, val in seed_results.get("checks", {}).items():

if not val:

print(f" CHECK FAILED: {check}")

for key, val in seed_results.get("max_diffs", {}).items():

print(f" MAX DIFF: {key} = {val}")

# Save equivalence certificate even on failure

raise RuntimeError("Baseline equivalence test FAILED. Study stopped.")



return results
```

```
============================================================================
====
Main API
============================================================================
====

def run_seed(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

branch_order: tuple[str, ...],

) -> dict[str, Any]:

text

"""Run a single seed through prefix + all branches.



Returns complete seed results with all branches.

"""

# Simulate prefix

split_state, prefix = simulate_prefix(seed, config, protocol)
```

```
# Create branch RNGs using frozen algorithm

branch_rngs = make_branch_rngs(seed, branch_order)



# Use branch_order to determine which branches to run

# All branches in branch_order must have RNGs

branch_results = {}

for branch in branch_order:

if branch not in branch_rngs:

raise ValueError(

f"Branch '{branch}' not found in branch_rngs. "

f"All branches must be registered in branch_order for explicit RNG namespace. "

f"Registered branches: {list(branch_rngs.keys())}"

)

rng = branch_rngs[branch]


branch_results[branch] = simulate_future(

split_state.clone(),
```

```
prefix,

branch,

config,

protocol,

rng,

)



return {

"seed": int(seed),

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,
```

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

if name == "main":

text

Quick validation

config = DEFAULT_MODEL_CONFIG

config.validate()

protocol = DEFAULT_PROTOCOL_CONFIG

protocol.validate(config)

print(f"Engine v1 loaded successfully")

print(f" N={config.N}, dt={config.dt}, tau={config.tau}")

print(f" T_window={config.T_window}, alpha={config.alpha:.6f}")

print(f" mean_omega={config.mean_omega:.6f}, T_*={config.T_star:.6f}")

print(f" delay_steps={config.delay_steps}")

print(f" Protocol: T_sim={protocol.T_sim}, burn={protocol.burn_end_time}, "

f"split={protocol.split_time}, future={protocol.future_duration}")

This version resolves the frozen-versus-extended schema problem correctly and implements dual horizons well. However, the baseline equivalence function currently contains an immediate NameError, and scalar equivalence cannot pass unless the driver supplies the frozen scalar CSV.

The engine is now scientifically close, but it has not yet reached the “run it and trust the result” stage.
1. Immediate runtime error: rho is undefined

Inside run_baseline_equivalence:

Python

if rho == 1.0:

There is no local or argument named rho.

The baseline function will raise:

text

NameError: name 'rho' is not defined

on the first branch.

This conditional is unnecessary. The function is explicitly the baseline-ρ=1ρ=1 equivalence test.

Replace:

Python

```
if rho == 1.0:
for key in EXTENDED_BRANCH_MATRIX_KEYS:
...
```

with:

Python

```
if not math.isclose(
config.T_window,
config.T_star,
rel_tol=0.0,
abs_tol=0.0,
):
raise RuntimeError(
"extended baseline checks require "
"T_window == T_star"
)

for key in EXTENDED_BRANCH_MATRIX_KEYS:
...
```

Apply the same correction to the extended scalar block.
2. Frozen scalar CSV is now mandatory

The new logic correctly treats missing frozen scalar keys as fatal:

Python

if frozen_val is None:
status = failed

Therefore, if the driver does not pass:

Python

frozen_csv_path

the baseline test fails every scalar for every branch.

The T-window driver must add:

text

--frozen-scalar

and call:

Python

engine.run_baseline_equivalence(
...,
frozen_csv_path=args.frozen_scalar,
)

Do not leave it optional if the function claims scalar equivalence.

Add an early explicit failure:

Python

if frozen_csv_path is None:
return {
"status": "failed",

"error": (
"frozen scalar CSV is required "
"for scalar equivalence"
),
}

3. Frozen/extended schema separation is now correct

This is the right architecture:

text

FROZEN_BRANCH_MATRIX_KEYS
EXTENDED_BRANCH_MATRIX_KEYS

Historical keys are compared against the frozen NPZ. New dual-horizon keys are compared internally against the backward-compatible engine fields at baseline.

At ρ=1ρ=1:
fixed immediate=1750=
fixed immediate=1750=

\text{normalized immediate},
]
fixed recovery=8750=
fixed recovery=8750=

\text{normalized recovery}.
]

Therefore exact internal equality is the right validation.

The same split for scalar fields is also appropriate.
4. Dual-horizon accumulation is correctly implemented

The engine now separately accumulates:

fixed physical horizons;
normalized EMA horizons.

It exposes explicit fields rather than silently changing the meaning of legacy names.

This now supports the primary scientific estimand:

text

*_fixed_physical

and the secondary timescale-relative estimand:

text

*_normalized_EMA

The driver and analysis must now use these explicit names. Generic legacy names remain normalized-EMA for backward compatibility.
5. Projection diagnostics are now substantially faithful

The expanded advance_state and ProjectionTotals now track:

proposal violations;
correction activation;
correction norm;
relative correction to state and update;
minimum pre/post gap.

This addresses the earlier scalar-schema mismatch.

The exact baseline scalar comparison will determine whether arithmetic order and thresholds reproduce production.
6. run_seed still has an unsafe branch API

It still loops over:

Python

for branch in branch_order:

rather than:

Python

for branch in protocol.branches:

Therefore a caller cannot use a full frozen RNG namespace while simulating a study subset.

The generic engine API should be:

Python

def run_seed(
seed,
config,
protocol,
rng_branch_order,
):
split_state, prefix = simulate_prefix(
seed,
config,
protocol,
)

branch_rngs = make_branch_rngs(
seed,
rng_branch_order,
)

branch_results = {}

for branch in protocol.branches:
if branch not in branch_rngs:
raise ValueError(...)

branch_results[branch] = (
simulate_future(
split_state,
prefix,
branch,
config,
protocol,
branch_rngs[branch],
)
)

The current study driver bypasses run_seed, so this does not block that path, but it remains a trap in the reusable engine.
7. Baseline certificate persistence remains unresolved

On failure, the function prints detailed results and raises. Unless the driver catches the exception while retaining the result object, the per-seed certificate is lost.

Prefer:

Python

results["status"] = (
"passed"
if all_passed
else "failed"
)

return results

Let the driver:

atomically save the certificate;
check status;
stop if failed.

On success, save:

text

baseline_equivalence_by_seed.csv
baseline_equivalence_manifest.json

8. Manifest hash is still mislabeled

This:

Python

hashlib.sha256(
json.dumps(
frozen_manifest,
sort_keys=True,
).encode()
)

is a canonical-content hash.

Add the actual artifact hash:

Python

results[
"frozen_manifest_file_sha256"
] = file_sha256(
frozen_manifest_path
)

Optionally keep the canonical hash under a distinct name.
9. State validation should compare against config NN

This check:

Python

self.theta.shape == (self.N,)

is tautological because self.N is derived from len(theta).

Add:

Python

```
if self.N != config.N:
raise RuntimeError(
f"state N={self.N}, "
f"config N={config.N}"
)
```

Then check all matrix/history shapes against config.N.
10. New fields should be synchronized downstream

The driver’s primary outcomes must now use:

text

```
P_mean_late_fixed_physical
Weighted_similarity_late_fixed_physical
Retention_contrast_late_fixed_physical
Corr_Rpast_Plate_fixed_physical
r_mean_late_fixed_physical
```

The normalized secondary analysis should use the explicit normalized names.

If the analysis still uses generic:

text

```
P_mean_late
Weighted_similarity_late
```

it is analyzing normalized-EMA horizons, not the fixed-physical primary estimand.
11. Scalar equivalence details

The frozen scalar CSV should contain the historical production scalars. New extended scalar keys should never be required from that CSV.

The current internal extended comparison is correct in concept.

For legacy scalar comparison:

exact finite values;
NaN-aware equality;
missing historical key is fatal.

That policy is now appropriate.
12. Minor cleanup

Define frozen and extended scalar tuples at module scope rather than inside the seed/branch loop.
make_seed_sequence appears unused.
copy, time, and field may be unused.
Jaccard_final_fixed_physical is redundant because final state does not depend on horizon definition.
P_full_fixed_physical and P_full_normalized_EMA are identical by construction; keeping both is harmless but verbose.
The helper's default Jaccard threshold should require an explicit value rather than defaulting to ZERO_TOL.

Readiness assessment
Core step map: 9/10
Frozen RNG reproduction: 9/10
Dual-horizon implementation: 9/10
Projection diagnostics: 9/10
Historical schema handling: 8.5/10
Baseline execution correctness: 5/10 because of the undefined rho
Certificate/provenance: 7/10
Overall current engine: 8/10 design, 7/10 executable
Immediate next action

Make only these changes:

remove the undefined rho condition;
require and pass the frozen scalar CSV;
return/save the equivalence certificate before stopping;
fix run_seed branch subset semantics.

Then run:

Bash

python -m py_compile aief_experiment_engine_v1.py

followed by one frozen seed through all seven baseline branches.

If that passes, run all 40. At this stage, actual equivalence output—not another static rewrite—is the decisive test.